

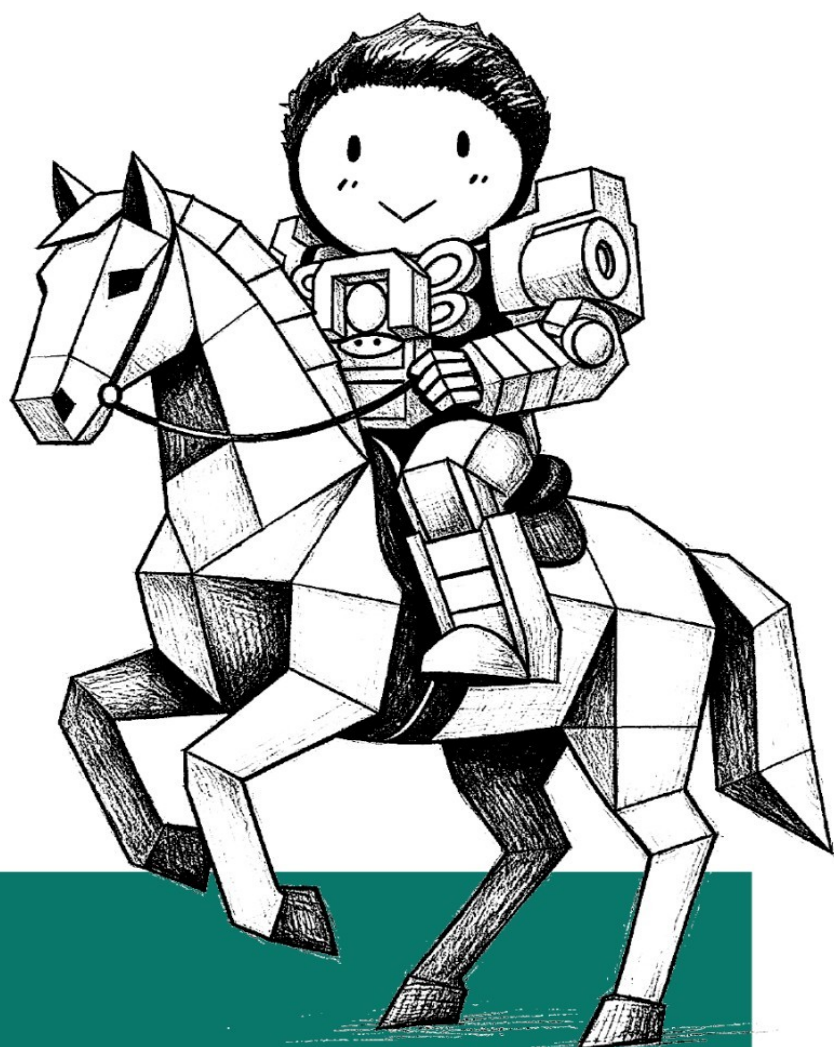
Claude Code

实战

黄佳 著

Harness 工程之道

Claude 是一匹力气惊人的马
你可以为它设计一副合身的 Harness

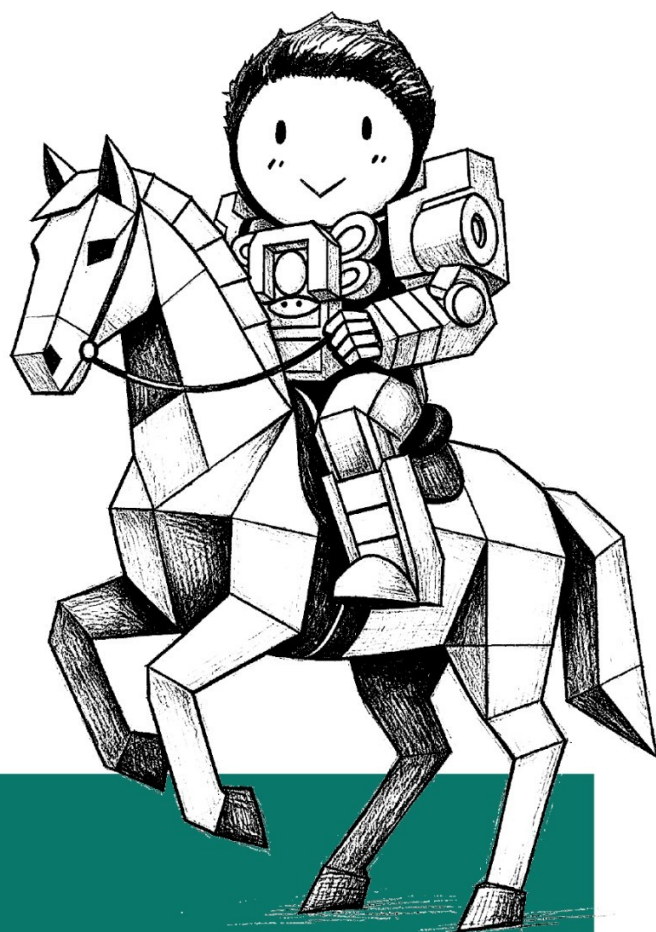


Claude Code 实战

黄佳 著

Harness 工程之道

Claude 是一匹力气惊人的马
你可以为它设计一副合身的 Harness



|

版权信息

COPYRIGHT

书名：Claude Code实战：Harness工程之道

作者：黄佳

出版社：人民邮电出版社

出版时间：2026年5月

ISBN：9787115696533

字数：196千字

（本书分享更多索搜@雅书）

内 容 提 要

本书系统介绍了Claude Code的技术架构与工程化实践。全书从“软件工程”视角，解析了从命令行助手到可编程Agent的演进路径，并围绕Claude Code的四层架构模型展开，深入探讨了记忆系统（CLAUDE.md）、技能（Skills）、子智能体（SubAgents）、事件钩子（Hooks）与MCP等核心机制的设计哲学与协同原理。书中不仅提供了翔实的技术选型指南、组件配置方法和触发机制对比，更通过大量实战案例，展示了如何构建安全、高效、可维护的AI辅助编程工作流，涵盖从个人开发到团队协作，再到与CI/CD流水线集成的企业级部署全流程。

本书适合具备一定编程基础，并希望将Claude Code深度集成至日常开发与团队流程中的开发者、全栈工程师和技术团队负责人等阅读。

前言

2025年2月，Anthropic正式推出了Claude Code。乍看之下，它似乎不过是一款运行于终端的命令行工具——用户输入指令，它便予以回应，有时编写代码，有时执行测试。许多开发者的使用方式也确实仅限于此：开启终端，阐述需求，获取代码，然后关闭窗口。这般操作，周而复始。

然而，这仅仅是冰山一角。

观其“水面之下”再探全貌，Claude Code实则是一个[可编程、可扩展、可组合的Agent框架](#)。Anthropic官方文档为其赋予了精准的定义：Agentic Harness——Agent编排框架。

Harness一词的原意为“马具”，即套在马身上的挽具、缰绳等工具的统称。马匹虽有力气，但若无马具的控制与牵引，便无法拉动车辆。Claude Code正如这副马具，它并非旨在改变模型的智力本身，而是重塑了智力的[传导与控制方式](#)（见图1）。

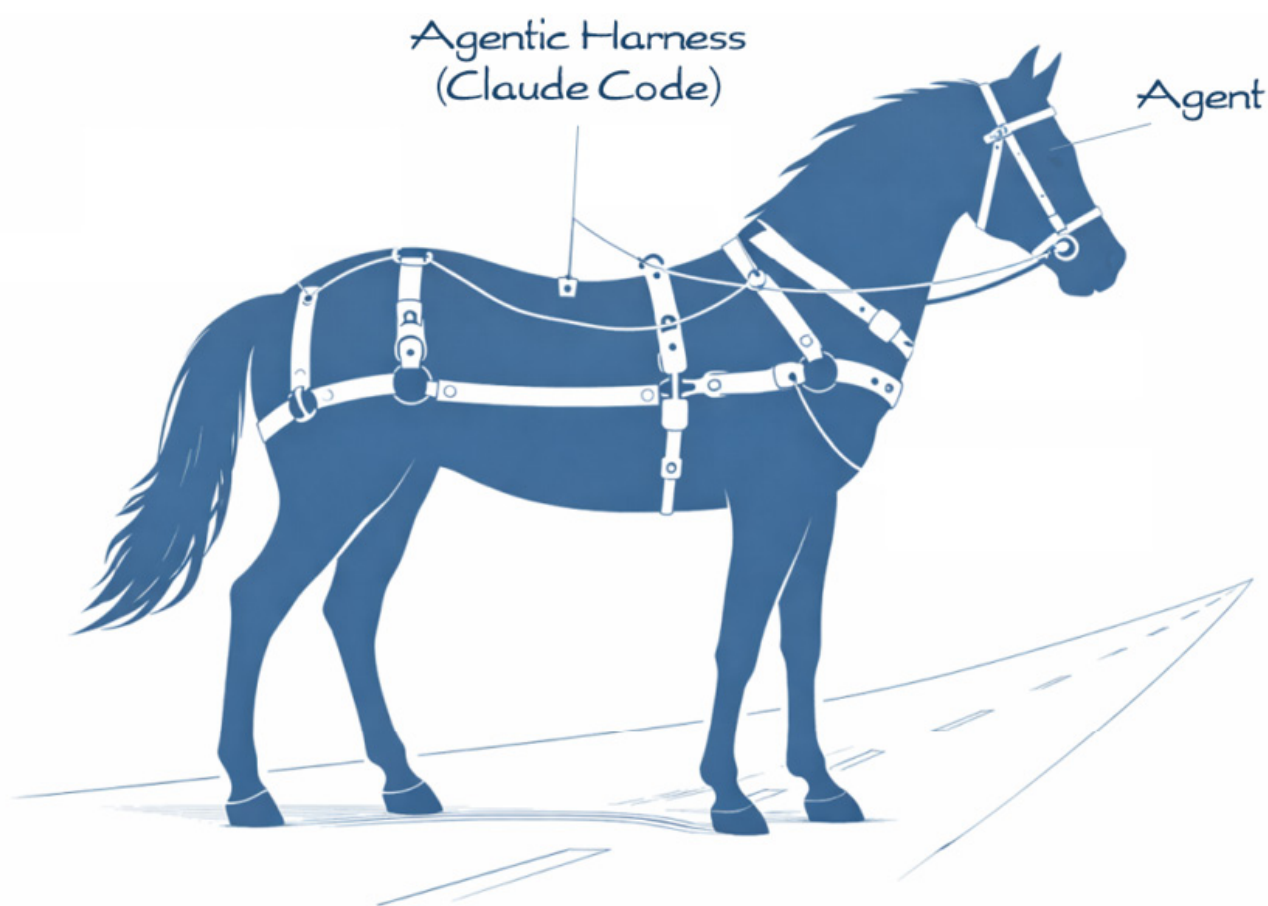


图1 Agentic Harness是Agent的“马具”，也是其能力来源

Agentic Harness（以下简称Harness）的“水面之下”涉及8种机制：记忆系统让模型不再失忆，Skills让模型具备领域知识，子智能体（SubAgents）让复杂任务可以被拆解，Hooks在每个关键节点插入确定性约束，MCP（Model Context Protocol，模型上下文协议）打通外部工具与数据源，Headless模式融入CI/CD（Continuous Integration/Continuous Deployment，持续集成/持续部署）流水线，Agent SDK让你从智能体的使用者变为构建者，Plugins生态将以上所有能力打包分发。

开发者若是不知道这些，就只能守在终端前与Claude对话，反复陈述相同的项目背景，手动应对不断膨胀的上下文窗口，在长任务中眼睁睁看着Claude的注意力逐渐漂移，然后得出结论：“Claude还不够聪明。”

这并非Claude不够聪明，而是开发者仅发挥了Harness 10%的能力。本书的目的，正是将Harness其余90%的能力交到你手中。

为什么Harness比模型更重要²⁰²⁶

年初，行业出现了一个关键洞察：同一模型在不同Harness下的表现差异，远大于不同模型在同一Harness下的表现差异。

在TerminalBench基准测试中，仅通过对Harness的优化，就使同一模型能力从基线以下跃升至Top 5。Vercel团队发现，剔除80%的Agent工具后，流程更精简、Token（词元）消耗更少、响应速度更快。OpenAI则提供了一个极致的案例：3名工程师，0行手写代码，却产生了100万行产品级代码——因为工程师构建的是Harness，而非代码本身。

这一洞察彻底打破了人们“等待更强模型”的幻想。Claude的能力边界由Anthropic划定，这是你无法改变的参数。但Harness的配置权，无论是CLAUDE.md的撰写、工具权限的设置、Hooks的接入，还是MCP的连接，全在你手中。你在本书中学到的每一种能力，本质上都是在调教Harness。

这本书讲什么

本书系统讲解了Claude Code的完整技术栈与工程化实践。全书按照“认识→掌握→驾驭”3个阶段展开。

第一阶段，认识全貌（第1章和第2章）

- 第1章从四层架构模型入手，涵盖Harness的本质及Agentic Loop的运转机制，帮助你建立对技术体系的全景认知。
- 第2章深入解析记忆系统——这是整个框架的基石，因为所有上层机制都建立在Claude正确理解项目上下文的基础之上。

第二阶段，掌握核心（第3章～第6章）

这4章构成本书主体，对应扩展层的四大核心机制。

- Skills（见第3章）教会Claude专业领域的工作方式。
- 子智能体（见第4章）通过上下文隔离和任务委派，驾驭复杂任务。
- Hooks（见第5章）在系统执行层面拦截和约束行为。
- MCP（见第6章）将Claude的能力边界扩展至外部世界。

第三阶段，驾驭工程（第7章～第10章）

- Headless模式与CI/CD集成（第7章）让Claude Code融入自动化流水线。
- Agent SDK（见第8章）助你从使用者转型为构建者。
- Plugins生态（见第9章）将能力打包为可流通的知识资产。
- 工程化实战（见第10章）作为全书最后一章，从成本控制、调试、安全准则、大型代码库、指令、团队落地6个维度，讲述真实工程环境中的最佳实践，并系统梳理了SDD生态中Superpowers、Compound Engineering等工作流框架如何与Claude Code的各种机制融合落地。

每一章均遵循“从为什么到怎么做”的递进逻辑：先建立概念框架，再拆解技术机制，最后动手实践。理解原理远比记住操作步骤更具长期价值——毕竟技术细节会随版本更新而更迭，但工程思想历久弥新。

关于Claude一词的用法

Claude Code指的是Anthropic推出的Agentic Harness，即你在终端中启动并与之交互的应用程序。它囊括了工具调用引擎、权限控制、子进程管理、上下文压缩及插件加载等一系列工程组件。

Claude（不带Code后缀）在大多数语境下，特指在该应用程序内部驱动推理与生成的大模型。

在实际使用中，二者紧密交织。你与Claude（模型）对话，而Claude Code（Harness）则在幕后负责管理上下文、调度工具及控制权限。这正如驾驭一匹马：你指挥的是马的意志（模型），但传导力量的是马具（Harness）。

本书在可能产生歧义之处，会明确使用Claude Code（指Harness）和Claude（指模型）加以区分；在语义清晰的情境下，则统一简写为Claude。

这本书适合谁阅读

- **正在使用Claude Code的开发者**：如果你目前仅将其用于基础对话，那就是未能充分发挥其潜力，这本书将助你大力“解锁”。
- **对AI Agent架构感兴趣的工程师**：如果你想深入理解生产级Harness的内部设计，如Agentic Loop驱动的行为涌现、子智能体的上下文隔离、Skills的渐进式披露、

Hooks的事件驱动拦截等，那么这本书介绍的设计模式适合你学习。这些设计模式的价值将远超Claude Code本身。

- **技术团队负责人**：如果你需要在团队中推广AI辅助开发，请仔细阅读第10章，该章提供了完整的工程化方案，涵盖成本控制、安全合规、权限治理及SDD生态工具选型等关键内容。
- **对AI工具持谨慎态度的开发者**：如果你担心“AI生成的代码不可靠”，这本书不会空谈“AI什么都能做”，而是教你如何运用Harness的“约束机制”，让概率性的模型在确定性的工程框架中实现可靠运行。

如何阅读这本书

如果你是第一次接触Claude Code的完整技术栈，建议按章节顺序从头读到尾。第1章和第2章建立的架构认知和记忆系统知识，是后续各章内容的基石。

如果你已经有一定使用经验，可以直接跳到你最关心的章节。每章在引用其他章节的概念时都会作简要回顾。但我仍然建议你至少通读第1章的架构全景——它像一张地图，能帮你将散落的知识碎片拼成完整的图景。

每章末尾的思考题绝非装饰，其设计目的是帮你把“读过了”转化为“理解了”。

Claude是一匹力气惊人的马，而这本书旨在教你如何为它设计一副合身的Harness。

本书可作为便携的“知识地图”，为你指明方向。而更多工程化落地细节与前沿探讨，我强烈推荐你延伸阅读极客时间的《Claude Code工程化实战》专栏，它将作为本书的最佳补充，为你提供详尽的实战演练。

黄佳

2026年3月

资源与支持

资源获取

本书提供如下资源：

- 书中源代码文件；
- 书中图片文件；
- 本书思维导图；
- 异步社区7天VIP会员。

要获得以上资源，您可以扫描下方二维码，根据指引领取。



提交勘误信息

作者和编辑尽最大努力来确保书中内容的准确性，但难免会存在疏漏。欢迎您将发现的问题反馈给我们，帮助我们提升图书的质量。

当您发现错误时，请登录异步社区（<https://www.epubit.com>），按书名搜索，进入本书页面，点击“发表勘误”，输入勘误信息，点击“提交勘误”按钮即可（见下页图）。本书的作者和编辑会对您提交的勘误信息进行审核，确认并接受后，您将获赠异步社区的100积分。积分可用于在异步社区兑换优惠券、样书或奖品。

图书勘误

发表勘误

页码: 1

页内位置 (行数): 1

勘误印次: 1

图书类型: ☒ 纸书 ☐ 电子书

添加勘误图片 (最多可上传4张图片)

+

提交勘误

与我们联系

我们的联系邮箱是contact@epubit.com.cn。

如果您对本书有任何疑问或建议，请您发邮件给我们，并在邮件标题中注明本书书名，以便我们更高效地做出反馈。

如果您有兴趣出版图书、录制教学视频，或者参与图书翻译、技术审校等工作，可以发邮件给我们。

如果您所在的学校、培训机构或企业，想批量购买本书或异步社区出版的其他图书，也可以发邮件给我们。

如果您在网上发现有针对异步社区出品图书的各种形式的盗版行为，包括对图书全部或部分内容的非授权传播，请您将怀疑有侵权行为的链接通过邮件发送给我们。您的这一举动是对作者权益的保护，也是我们持续为您提供有价值的内容的动力之源。

关于异步社区和异步图书

“[异步社区](#)”是由人民邮电出版社创办的IT专业图书社区，于2015年8月上线运营，致力于优质内容的出版和分享，为读者提供高品质的学习内容，为作译者提供专业的出版服务，实现作者与读者在线交流互动，以及传统出版与数字出版的融合发展。

“异步图书”是异步社区策划出版的精品IT图书的品牌，依托于人民邮电出版社在计算机图书领域四十余年的发展与积淀。异步图书面向各行业的信息技术用户。

第1章 登高望远：Claude Code技术架构全景

高屋建瓴，方知全局之妙。

小冰最近主导了一个AI Agent项目，并选用Claude Code作为核心开发工具。起初，其表现令人惊艳：只需要清晰阐述需求，代码便能迅速生成，效率远超人工手写。然而两周后，一系列问题集中爆发：每次开启新会话，Claude似乎无法记忆项目技术栈（如FastAPI+SQLAlchemy），导致开发者不得不反复重申背景信息；生成的代码风格缺乏一致性，Black格式化标准时有时无，致使PR（Pull Request，拉取请求）审查3次被回退。更为严峻的是，在执行一个涉及6个文件的重构任务时，受限于上下文窗口容量，Claude在处理至第四个文件时“遗忘”了前序逻辑，导致最终产出的代码前后矛盾，难以自洽。

“感觉就像在带一个每天都会失忆的实习生，”小冰向小雪和咖哥抱怨，“能力确实不错，但什么背景信息都记不住。”

咖哥并未急于安慰，而是连发3问：

“第一，你的项目根目录配置CLAUDE.md了吗？”

第二，那涉及6个文件的重构任务，为什么不拆解为子智能体协同处理？

第三，既然代码规范如此重要，为什么仅靠口头告知，而不是将其封装成Skill（技能），让模型自动加载？”

小冰哑口无言，一个问题也答不上来。

“问题根源并非Claude模型不够聪明，”咖哥指出，“而是你仅挖掘了它10%的能力——基础的对话能力。事实上，Claude Code本身就是一个完备的Agent框架，内置了记忆系统、Skills、子智能体、Hooks及MCP等。你所遭遇的每一个痛点，在框架层面均有对应的解决方案，只是你尚未知晓。”

.....

这段对话持续了整整两个小时。临近结束，小冰感慨道：“原以为自己在驾驭一个AI助手，没想到仅仅是在使用一个高级聊天窗口。”

1.1 从命令行助手到Agent框架

小冰所遭遇的Claude Code的记忆丢失、风格飘忽以及长任务上下文溢出等问题，几乎是所有将之应用于实际工程的开发者都会碰到的瓶颈。这些现象看似是3个独立的bug（缺陷），实则指向同一个根本症结：**误将一个Agent框架当作简单的聊天工具来使用**。

Claude Code的本质，是一个**可编程、可扩展、可组合的Agent框架**。

这绝非空洞的营销用语，而是其对应Claude Code核心的工程能力。

- **可编程**：意味着你可以使用Python或TypeScript代码直接驱动Claude Code，将其无缝嵌入CI/CD流水线或自动化脚本中，彻底解放人力，不需要人工值守终端。
- **可扩展**：意味着你能够通过配置文件为其注入记忆系统、Skills、Hooks等能力，而不需要触碰或修改其核心代码。
- **可组合**：意味着这些扩展模块之间可以灵活编排，如同乐高积木般自由拼装，从而构建出复杂且高效的工作流。

三者合一，勾勒出的不再是单一的工具，而是一个坚实的基础设施框架——供开发者构建专属AI工作体系的基石。

咖哥将Claude Code形象地比作一座冰山，并将其画在白板上，如图1-1所示。

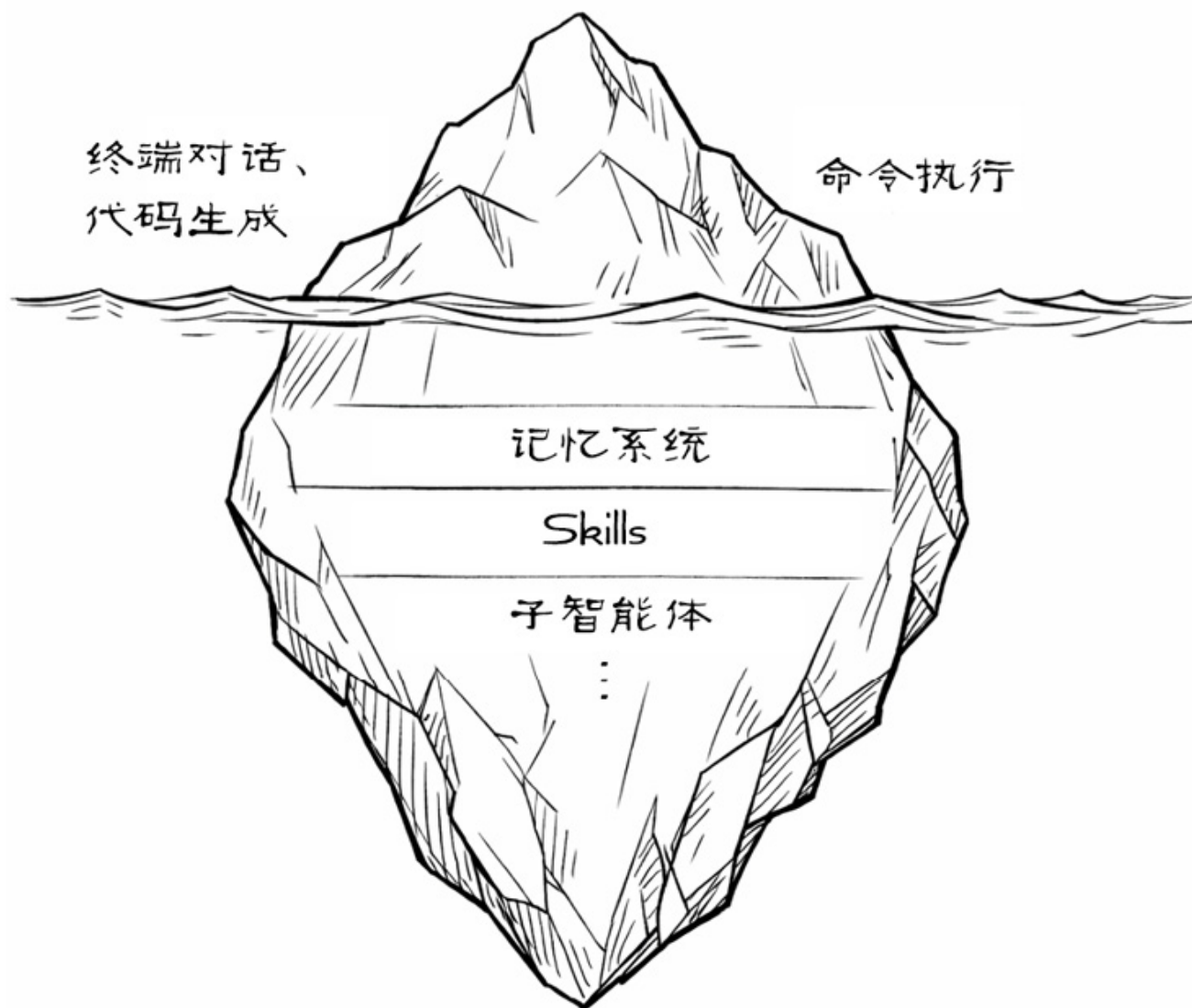


图1-1 Claude Code像一座冰山，很多核心功能隐藏在水面之下

海平面之上，仅是露出一角，对应着小冰日常的使用场景：终端对话、代码生成与命令执行。而隐匿于海平面之下的庞大基座，则是一系列关键技术架构，它们精准地解决了小冰遭遇的困境。

- **CLAUDE.md（记忆系统）**：根治“失忆”顽疾，将项目规范一次性写入配置文件，即可在每次会话中自动加载，不需要反复重申。
- **Skills**：终结风格飘忽，将代码审查标准配置化、制度化，彻底取代“口头叮嘱”的不确定性。
- **子智能体**：化解上下文溢出，将涉及6个文件重构任务拆解为6个独立的上下文单元，实现并行处理与逻辑隔离。

- **Hooks**: 在工具调用时自动触发安全检查或日志记录，构建防御性编程机制。
- **MCP** (Model Context Protocol, 模型上下文协议): 打破数据孤岛，赋予Claude调用外部数据库与API (Application Programming Interface, 应用程序接口) 的能力。
- **Headless模式**: 支持在CI/CD流水线中“无人值守”运行，实现真正的自动化交付。
- **Agent SDK**: 允许通过代码编排复杂的多步Agent工作流，提升任务执行的灵活性。
- **Plugins生态**: 将上述能力打包封装，便于在团队内部高效分发与复用。

“你遭遇的每一个痛点，”咖哥指向冰山水下的庞大基座，“在这里都有对应的机制予以化解。问题不在于工具本身不够强大，而在于你尚未释放它真正的潜能。现在，我期望你能够登高望远，从全局视角重新审视并掌握Claude Code的全貌，将其威力彻底释放。”

小冰闻言，若有所思：“之前的用法，本质上是在进行低效的‘手动写脚本’——反复向Claude灌输相同信息、人工拆解任务、肉眼检查风格；而框架化的用法，则是通过恰当的配置将这一切转化为自动化的流程。”

咖哥点头确认：“没错。你每次开启新对话都要复述技术栈，每次代码审查都要手动描述标准，每次提交代码都依赖个人记忆去检查——这些本质上都是‘手动写脚本’式的低效操作。而框架化的用法则是：将技术栈固化于CLAUDE.md中，一劳永逸；将审查标准封装进Skill，实现自动加载；将提交检查配置为Hook，确保永不遗漏。”

这种从“逐次指令”向“系统配置”的范式转变，不仅极大地提升了效率，更深刻地重塑了人机协作的质量。配置一旦确立，Claude Code便不会遗忘、不会懈怠，更不会因深夜加班而降低标准——这正是将个体智慧沉淀为制度规范的工程思想精髓。

接下来，我们将深入拆解这一框架的内部结构。

1.2 四层架构模型

任何复杂系统在被人真正理解之前，都需要一张清晰的“地图”。正如建筑师在动工前需要绘制蓝图，软件工程师在编码前需要确立架构，Claude Code的完整技术

栈亦可被抽象为一个四层架构模型，如图1-2所示。自下而上，这四层分别为：记忆层、扩展层、集成层和编程层。深入理解这四层的内涵及其边界，是掌握整个Claude Code体系的首要步骤。

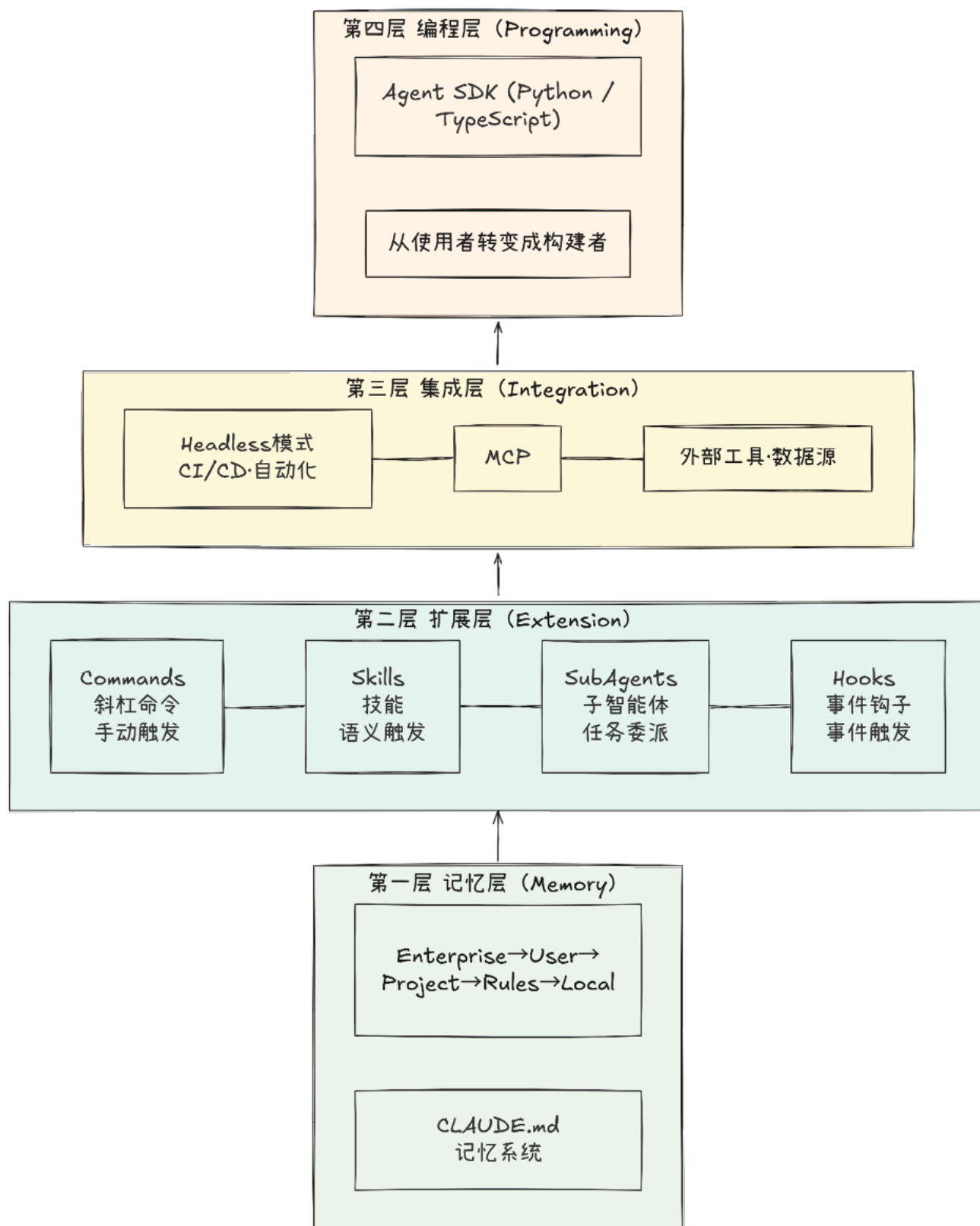


图1-2 Claude Code的四层架构模型

咖哥将这四层架构生动地比作一栋摩天大楼。

- 记忆层的记忆系统是深埋地下的地基。若无此根基，其上的一切构建都将无从谈起，随时面临坍塌风险。
- 扩展层的四大组件——**Commands**、**Skills**、**SubAgents**、**Hooks**，共同构成了建筑的主体楼层，承载着日常运营的核心功能，这是用户最直接感知的价值空间。
- 集成层的集成能力宛如隐藏起来却至关重要的水电管网，它将这座建筑与外部广阔的基础设施网络紧密相连——通过连接广阔的基础设施，确保持续的能量与信息流动。
- 编程层的**Agent SDK**则是位于顶楼的建筑师工作室。在这里，你不再仅仅是住户，而是拥有了设计全新建筑、重构空间逻辑的“至高权力”。

“自下而上审视，是构建视角，关注基石与支撑；自上而下俯瞰，则是使用视角，聚焦功能与体验。”咖哥强调，“这句话值得铭记于心。”

1.2.1 记忆层——**CLAUDE.md**

建筑的地基决定了整栋大楼的承载上限。Claude Code的记忆层亦是如此——它直接定义了Claude对项目“理解”的深度。

本质上，Claude是一个无状态模型。每当开启新对话，它对项目背景、技术栈选择、代码规范及团队约定均一无所知。若未进行任何配置，开发者便需要反复重申诸如“使用TypeScript而非JavaScript”“优先选用pnpm而非npm”“测试框架定为Vitest”等基础设定。这种如同每日入职般重复自我介绍的模式，其低效程度可想而知。

记忆层通过**CLAUDE.md**彻底解决了这一痛点。**CLAUDE.md**本质上是一份“给AI的员工手册”，能在每次对话启动时自动加载至Claude的上下文中。更为精妙的是，Claude Code的这种记忆机制并非固定不变，而是构建了一个包含5个层级的记忆体系（见图1-3），每一级均对应不同的适用范围。

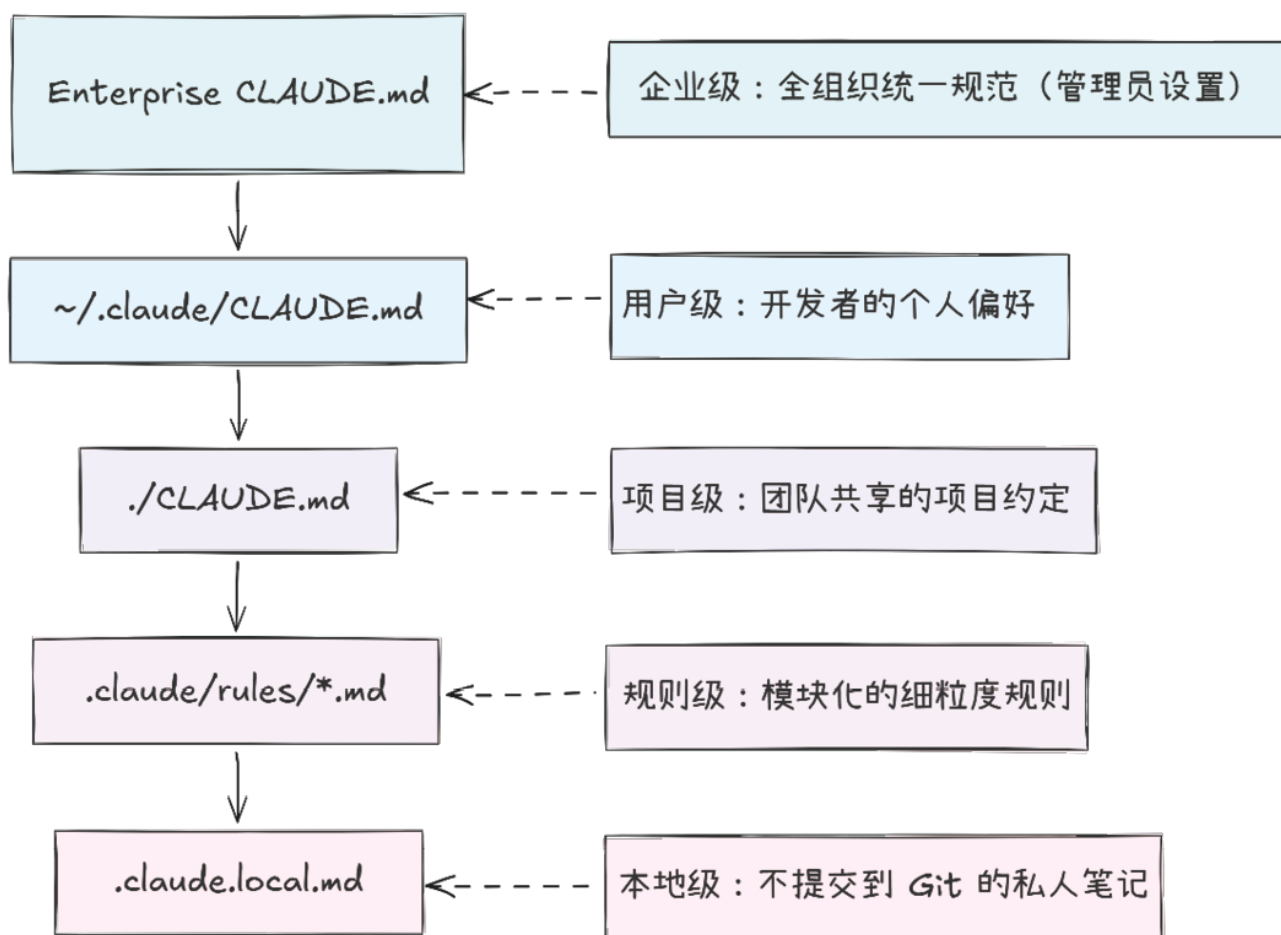


图1-3 Claude Code的5级记忆体系

这种分级设计背后的工程思想，与CSS的层叠优先级机制，或是软件配置中“全局—用户—项目—本地”的四级覆盖模式如出一辙：每一级均可覆盖上一级的设定，而层级越具体（如本地级），其优先级越高。

一个典型的项目级CLAUDE.md示例如下。

```
# 项目记忆CLAUDE.md
```

```
## 技术栈
```

```
- 语言：TypeScript 5.x（启用严格模式）
```

```
- 框架：Fastify+Prisma - 测试：Vitest（所有公共函数必须包含单元测试）
```

```
- 包管理：pnpm（禁用npm或yarn）
```

代码规范

- 组件风格：优先使用函数式组件，禁止使用class组件
- 错误处理：统一采用Result类型返回，严禁直接抛出异常
- 提交规范：遵循<type>[optional scope]: <description>格式

常用命令

- pnpm dev — 启动开发服务器
- pnpm test — 运行测试套件
- pnpm lint — 执行代码检查

基于CLAUDE.md，Claude在每次开启对话时便会自动“研读手册”，从而避免技术栈误用与代码风格漂忽的风险。正如小雪所言：“自从创建了CLAUDE.md，我再也不必在每次对话中反复强调‘我们使用pnpm，而不使用npm’了。”

这一看似简单的机制，实则是整个框架的基石。上层的Skills、Commands、Hooks等所有配置与行为，均构建于Claude对项目上下文的精准理解之上。恰如建筑地基虽不如外墙装饰般醒目，但若缺了它，一切装饰都将无从依附。第2章将深入剖析这个看似朴素却至关重要的记忆机制。

1.2.2 扩展层——四大组件

若将记忆层比作地基，扩展层便是建筑的主体楼层——它赋予了Claude在日常使用中最丰富的能力。这一层由四大组件组成：Commands（斜杠命令）、Skills（技能）、SubAgents（子智能体）和Hooks（事件钩子）。它们既独立运作，又能灵活组合。

其中，Commands是最直观的扩展方式。当你在终端输入/review、/commit或/deploy等指令时，Claude便会依据预定义的模板，执行一套标准化的工作流。其本质是将高频重复、步骤固定的操作封装为“一键触发”的快捷方式——这如同手机中的快捷指令，单次点击即可联动执行一系列预设动作。

从设计模式的角度审视，Commands正是命令模式（Command Pattern）的典型应用：它将“请求”封装为“对象”，从而使客户端能够以参数化的方式处理不同的请求。Commands的定义文件存放在.claude/commands/目录下，每一个Markdown文件即代表一条独立命令。例如，一个名为review.md的文件可包含代码审查的检查

清单及输出格式规范；只需要输入/review，Claude便会严格遵照该清单，逐项对当前代码进行审查。



咖哥 发言

截至本书完稿时，在Claude Code的最新演进中，Commands已正式并入Skills体系，被重新定义为一种“任务型Skill”——由用户通过/command触发的技能。本书在架构图中仍将其保留为独立模块，旨在强调其独特的触发范式：手动显式触发与语义自动触发的本质区别。深入理解这一区分，对于掌握整个扩展体系的设计思想仍有价值。

Skills遵循了截然不同的设计路径。它不需要用户手动触发，而是由Claude根据当前任务的语义上下文，自动研判并加载相应的技能。每个Skill本质上是一个包含SKILL.md的目录。在该文件中，通过YAML Frontmatter（前置元数据）声明skill的名称、描述及允许调用的工具列表。Claude正是依据description字段的内容，动态决策“当前任务是否需要激活此Skill”。这种隐式触发机制，使得Skills成为承载领域知识的理想载体——无论是财务分析方法、API设计规范，还是特定框架下的最佳实践，皆可封装其中。

例如，当你与Claude探讨财务指标时，它会自动加载“财务Skill”；而一旦话题转向代码架构，该Skill便会悄然退场。这种“按需加载、闲时静默”的机制，极大提升了交互的流畅度与智能性。

以下是一个典型的SKILL.md示例。

```
name: code-reviewing description: > Review code for best practices and potential issues.
```

```
Use when the user asks for code review or mentions reviewing changes.
```

```
allowed-tools: - Read - Grep - Glob ---
```

```
# Code Review Guidelines 你是一名专业的代码审查员……
```

子智能体旨在解决一个更深层次的瓶颈：上下文窗口的有限性。

Claude的上下文窗口犹如一张尺寸固定的办公桌：桌面空间有限，堆放的文件越多，越容易杂乱无章。当任务涉及海量文件（例如，审查整个PR的代码变更）时，若将所有细节强行塞入主对话的上下文，很快就会导致“桌面爆满”，致使Claude顾此失彼，甚至遗漏关键信息。

子智能体的解决方案是为特定任务开辟独立的上下文空间——这相当于在主桌旁支起一张新的工作台，指派专人在此处理子任务。待子任务完成后，仅需要将结论（一份简洁的报告）反馈至主桌面，而非将所有工作草稿全盘搬运。这就好比企业管理者向下属委派任务：管理者不需要深究每一行代码的细节，只需要关注审查结论与改进建议。

子智能体的定义文件位于`.claude/agents/`目录下，每个Markdown文件详细描述了一个子智能体的角色定位、权限范围及行为准则。更为关键的是，你可以为子智能体设置严格的工具权限，例如，规定“代码审查员”仅拥有读取文件的权限，严禁修改任何内容。这一设计深刻体现了最小权限原则（Principle of Least Privilege）在AI Agent架构中的核心应用。

Hooks是四大组件中唯一具备“拦截”能力的模块，如果说Commands定义了Claude“做什么”，Skills指导了Claude“怎么做”，那么Hooks则判断Claude“能不能做”。

它们在特定事件节点自动触发，包括工具调用前（PreToolUse）、工具调用后（PostToolUse）以及响应生成结束前（Stop），开发者可在这些关键时机插入检查、拦截或增强逻辑。这一机制与Web框架中的中间件（Middleware）概念异曲同工。

例如，通过在`.claude/settings.json`中配置Hook，可实现：每当Claude试图执行`git commit`时，自动运行lint检查；如果检查未通过，则直接阻止提交操作。这种“隐形守卫”的角色，使Hooks成为保障代码质量与安全合规的利器。

以下是一个典型的配置示例。

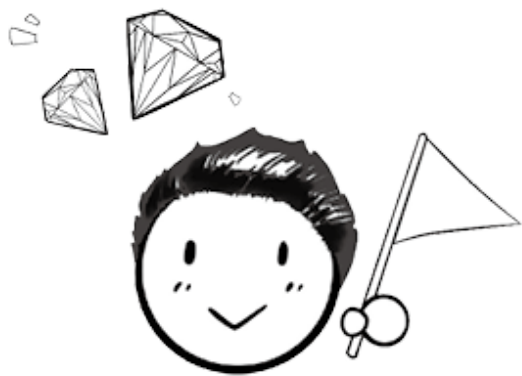
```
{  
  
  "hooks": {  
  
    "PreToolUse": [  
  
      {  
  
        "matcher": "Bash", "command": "python .claude/hooks/safety_check.py", "blocking": true }  
  
      ]  
  
    }  
  
  }
```

上述配置的含义是：每当 Claude 准备调用 Bash 工具执行命令时，系统优先运行安全检查脚本；如果脚本返回非零退出码（表示检查失败），此次调用将被强制阻断。

这种事件驱动的守护机制，在软件工程中有着成熟的对应物——无论是Git Hooks、数据库触发器，还是消息队列的消费者，其本质皆遵循统一模式：在特定事件发生时，自动执行预设逻辑。

这四大组件共同构筑了Claude Code扩展能力的核心基石。值得强调的是，它们之间呈现出正交性：既支持独立学习与单独使用，又能灵活组合以构建更强大的工作流。

这种正交设计深刻遵循了软件工程中的单一职责原则（Single Responsibility Principle, SRP）：每个组件专注于解决一类特定问题（“只做好一件事”），而通过有机组合，却能激发出“无所不能”的系统效能。



咖哥 发言

初学阶段，切勿急于求成，试图一次性掌握全部4个组件。建议遵循以下循序渐进的学习路径：首先，写好 `CLAUDE.md`；其次，学习Commands，这是最直观的扩展方式；接下来，掌握Skills，体会语义自动触发的妙处；然后，引入Hooks，建立安全与质量的守护机制；最后，挑战子智能体，以解决大规模复杂任务。每掌握一个组件，请务必在真实项目中实战演练两周，待融会贯通后再进入下一阶段。毕竟，“贪多嚼不烂”的道理，在此同样适用。

1.2.3 集成层——连接外部世界

在四层架构这幢摩天大楼中，集成层是维系其运转的“水电管网”——它将Claude Code的核心能力延伸至外部系统，使其真正融入开发生态。

其中，**Headless模式**是关键所在。它使Claude Code能够脱离交互式终端，在无人值守的环境中自动化运行。通过`-p`参数传入任务描述，并配合`--output-format json`规范输出格式，Claude Code即可无缝嵌入GitHub Actions、Jenkins、GitLab CI等任意CI/CD流水线。

一个典型的应用场景是自动化PR审查：每当开发者提交Pull Request，CI流水线便自动调用Claude Code进行代码审查，并将结果以评论形式直接回写至PR页面。整个过程不需要人工干预，真正实现了“无人值守”的自动化协作。

以下是在CI/CD流水线中调用Claude Code的示例命令。

在CI/CD流水线中调用Claude Code `claude -p "审查最近一次提交的代码变更，重点关注安全隐患与性能问题" \ --output-format json \ --max-turns 10 \ --allowed-tools Read,Grep,Glob` 上述命令的每个参数都蕴含着明确的工程意图：`-p`用于激活Headless模式，实现非交互式运行；`--output-format json`确保输出可被脚本自动化解析；`--max-turns`通过限制最大交互轮数来精准控制成本；而`--allowed-tools`则利用白名单机制严格限定Claude可调用的工具范围，从而筑牢安全防线。

MCP则开启了另一个维度的集成能力——它赋予Claude与外部工具及数据源深度交互的本领。如果说Headless模式是让Claude“走出去”以融入流水线，那么MCP则是让外部能力“走进来”以赋能模型。

只需要在项目根目录的`.mcp.json`文件中配置MCP服务器，即可轻松扩展Claude的能力边界：无论是访问数据库、调用第三方API，还是操作GitHub Issue、查询Jira工单，皆可一键打通。Anthropic将MCP形象地比喻为“AI时代的USB-C”——这是一套标准化的接口协议，使得任何工具和数据源都能以统一、规范的方式接入Claude Code生态。

以下是一个配置GitHub MCP服务器的示例。

```
{
  "mcpServers": {
    "github": {
      "command": "npx", "args": ["-y", "@modelcontextprotocol/server-github"], "env": {
        "GITHUB_TOKEN": "your-token-here"
      }
    }
  }
}
```

MCP的设计理念值得深思。在传统AI工具集成中，每接入一个新的外部系统往往需要编写大量定制化的适配代码。而MCP通过定义一套标准化协议（涵盖Tools、Resources、Prompts三大核心能力），实现了工具提供者与AI消费者之间的“接口

统一”。这与REST API在Web时代统一前后端通信的变革异曲同工——标准化带来的绝非限制，而是生态的繁荣。

Headless模式与MCP构成了集成层的两大支柱。

Headless模式面向自动化流水线，将Claude Code无缝嵌入既有的CI/CD体系。

MCP面向工具生态，将外部数据与服务引入Claude Code的能力工具箱。

二者合力，推动Claude Code从一个独立运行的终端工具，进化为可灵活融入任何技术栈的基础组件。如果说扩展层是在Claude Code的“体内”强化其核心机能，那么集成层便是在“体外”构建连接网络——两者相辅相成，缺一不可。

1.2.4 编程层——Agent SDK

摩天大楼的顶楼是建筑师的工作室——在这里，你不再是栖身其中的住户，而是设计新蓝图的创造者。

Agent SDK位于Claude Code技术栈的顶端，是一道关键的“分水岭”。在此之下的三层（记忆层、扩展层和集成层），开发者主要通过配置文件和命令行来延展Claude的行为边界，其本质上是使用者。而一旦跨越Agent SDK这条界线，便正式踏入构建者的领地：开发者可以利用Python或TypeScript编写代码，直接调用Claude Code的底层核心能力，从零开始构建全新的AI Agent。

以下是利用Agent SDK构建一个代码健康度来检查Agent的示例。

```
import claude_code

# 用SDK构建一个代码健康度检查Agent result = claude_code.query(

    prompt="分析src/目录下所有Python文件的代码质量，给出健康度

    评分", allowed_tools=["Read", "Grep", "Glob"], max_turns=15

)
```

print(result) 上述代码的深层意义：开发者不再受限于Claude Code的终端交互界面，而是能够在自己的应用程序中直接驱动Claude执行任务。这意味着，Claude的分析与推理能力可以无缝嵌入到定时任务、Web服务、数据处理管道等任何场景中——开发者的代码能触达之处，便是Claude能力延伸之所。

对于绝大多数开发者而言，前三层（记忆层、扩展层、集成层）已足以从容应对日常开发场景。然而，如果你的愿景是构建团队级AI开发平台，或者设计多Agent协作的自动化系统，那么Agent SDK便是必经之路。

借助Agent SDK，你可以打造批量的代码审查工具、项目健康度仪表板、自动化测试报告生成器，甚至将多个Claude Agent编排成一个高效的协作团队，例如，让第一个Agent专司代码阅读，第二个Agent负责编写测试，第三个Agent负责执行审查，三者串联成一条全自动化的智能流水线。

Agent SDK的存在还揭示了一个耐人寻味的事实：Claude Code本身，正是基于Agent SDK构建的一个AI Agent。你在终端中所见的一切行为——读取文件、编写代码、执行命令——本质上都是Agent SDK层面的工具调用。一旦洞察此点，你对Claude Code的认知便将从“一款产品”升维为“一种架构模式的具体实现”。

在后续章节中，我们将深入剖析Agent SDK的实战用法。在这里，我们只需要确立一个核心认知：Agent SDK是这栋架构大楼的顶层，更是从“配置驱动”跃迁至“代码编程”的起点。

1.2.5 底层视角：Harness与Agentic Loop

在理解了四层架构的“外部视角”后，我们不妨深入底层，探究一个更为本质的概念——[Harness](#)（马具）。

Anthropic官方文档对此的定义是：“Claude Code serves as the [agentic harness](#) around Claude: it provides the tools, context management, and execution environment that turn a language model into a capable coding agent.”

简而言之，Claude Code是一个包裹在Claude模型外层的[Agent编排框架](#)。原生模型仅具备生成文本的能力。正是Harness赋予了它读取文件、编写代码、检索代码库以及在终端执行命令的“手脚”。若缺失了Harness，Claude不过是一个空有智力却寸步难行的“大脑”。

Harness一词的原意为“马具”，即套在马身上的挽具与缰绳。马匹虽有力气，但若无马具的引导，便无法拉动车辆。Harness改变的并非马的力量本身，而是力量的[传导与控制方式](#)。

若用公式来表达，即 $\text{Agent} = \text{Model} + \text{Harness}$ 。

如果把模型比作引擎，Harness则是围绕引擎构建的一切基础设施：工具系统、权限控制、上下文管理、会话持久化、事件钩子，以及驱动这一切运转的核心循环——[Agentic Loop](#)（见图1-4）。

这一循环机制深刻揭示了Claude Code核心能力的来源。当你提交一个bug时，Claude Code并非简单地“看一眼便猜测答案”。而是经历了一个反复观察、假设与验证的过程：先查阅报错日志，继而搜索相关代码，深入理解上下文，最后才动手修复。

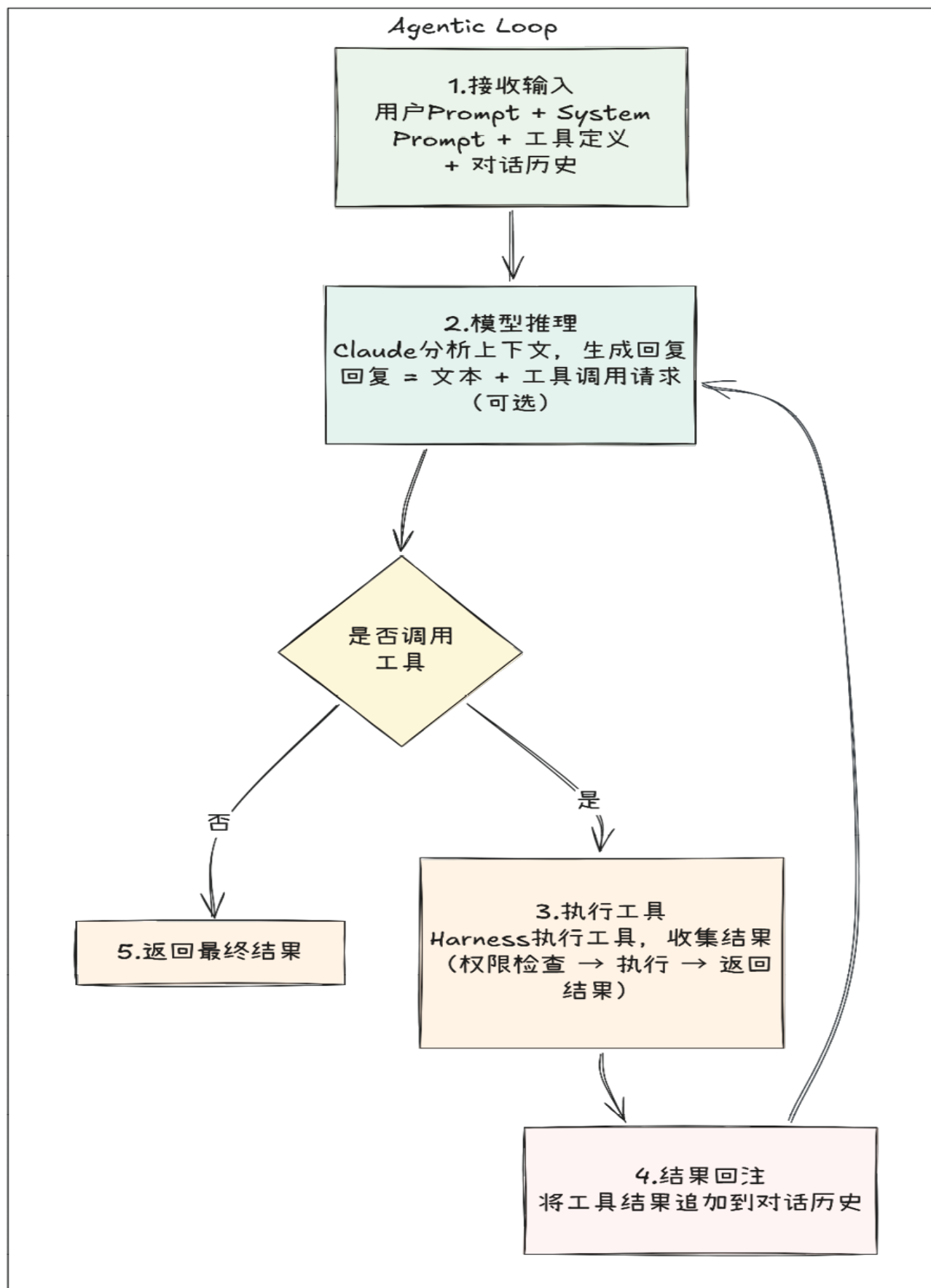


图1-4 Agentic Loop——Harness的心脏

循环何时终止？取决于两个条件：一是模型主动停止（即认为任务已完成，不再发起工具调用）；二是达到最大轮次限制（由--max-turns参数设定，旨在防止程序陷入无限循环）。

2026年，行业涌现出一个关键洞察：同一模型在不同Harness下的表现差异，远大于不同模型在同一Harness下的表现差异。

在TerminalBench基准测试中，仅通过对Harness的优化，就使同一模型从基线水平以下跃升至Top 5行列。换句话说，Harness比模型本身更为重要。你之前所学的四层架构中的每一环，无论是CLAUDE.md的撰写、工具权限的设置、Hooks的接入，还是MCP的连接，本质上都是在对Harness进行精雕细琢。

小冰听罢恍然大悟：“我之前以为Claude Code仅仅是Claude模型的一个外壳。原来，这个‘外壳’才是其真正的价值所在。”

1.3 组件关系与协作

在深入认识四层架构的每个组成部分后，更关键的问题在于：它们之间如何协同工作？在不同场景下应如何选择使用？一个真实的任务在该体系下又是如何流转的？

1.3.1 触发机制对比

扩展层的四大组件最易混淆之处在于其触发机制，表1-1从5个维度进行了对比。表1-1 四大组件对比

组件触发方式触发者是否需要记忆典型场景工程类比

review、

Commands用户手动输入人是（需要知晓命令名称）/CLI命令

/deploy SkillsClaude基于语义自动匹配AI否（自动发现）财务分析、API设计策略模式（自动选择算法）

子智能体用户指定或Claude主动委派人或AI是（需要知晓Agent名称）代码审查、日志分析线程池（隔离执行）

中间件/Git Hooks系统事件自动触发系统否（配置即生效）提交前检查、保存后格式化

Hooks

直观的记忆口诀如下。

- **Commands**: “你叫它做”（响应用户指令）。
- **Skills**: “它自己知道该做”（自主判断并执行）。
- 子智能体: “它安排别人做”（协调子智能体完成任务）。
- **Hooks**: “不管谁做，到了这一步就执行检查”（在特定流程节点进行自动拦截或处理）。

小雪看到上述总结后，精辟地指出：“**Commands**和**Hooks**均属于确定性触发——前者由用户指令明确触发，后者由特定事件自动触发；而**Skills**和子智能体则体现了AI的自主判断——前者由AI判断是否启用，后者由AI判断是否进行任务委派。”

咖哥赞许地点头道：“这种‘确定性与AI判断’的二分法，确实是理解四大组件差异的另一个关键视角。在实际工程中，对于安全性要求极高的场景（如敏感信息拦截），应当优先采用确定性触发的**Hooks**；而对于灵活性要求较高的场景（如领域知识匹配），则更适合使用基于AI判断触发的**Skills**。”

1.3.2 数据流：一个请求的旅程

为了更直观地理解各组件的协同机制，让我们追踪一个请求在Claude Code体系中的完整流转路径。

假设开发者在终端输入指令：“帮我审查src/payment/目录下最近的代码变更，确保没有安全隐患。”

Claude Code的处理流程如图1-5所示。

在图1-5所示的流程中，记忆层和扩展层中的各个组件各司其职：记忆层（CLAUDE.md）构建项目上下文，Skills注入领域知识，子智能体提供隔离执行环境，Hooks则严守质量关卡。全程不需要用户额外干预——一切均在用户输入那句自然语言指令之后自动编排完成。

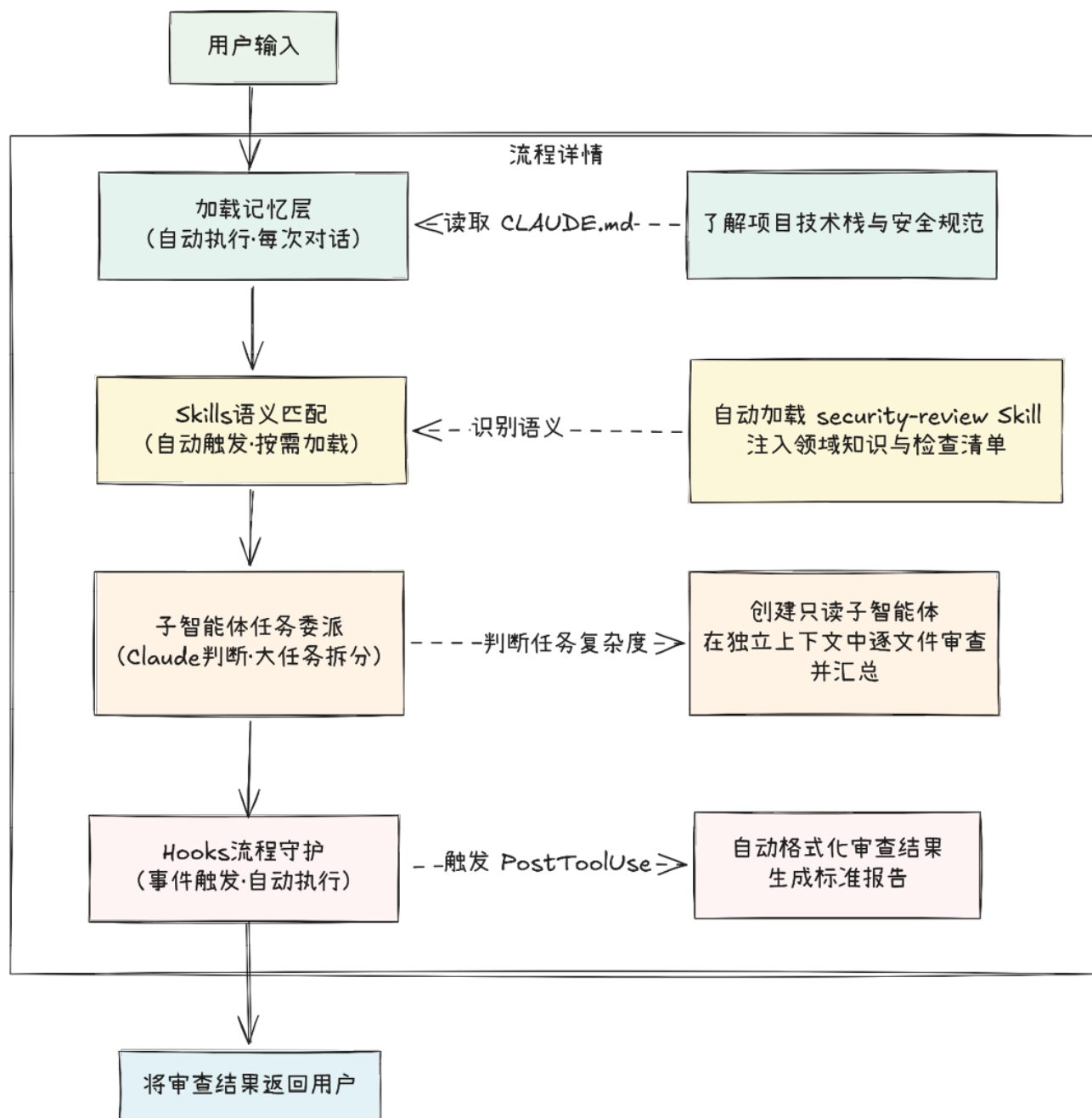


图1-5 Claude Code的处理流程

这正是“可组合性”的真谛。单个组件如同独立的积木，一旦组合，便形成了一条完整的自动化流水线。这完美契合了软件工程中经典的“关注点分离”

（Separation of Concerns）原则：每个模块专注单一职责，通过清晰的接口协同工作。Claude Code的组件设计正是这一原则的典范。

- CLAUDE.md专注于“知道什么”（Know-what）。

- Skills专注于“怎么做”（Know-how）。
- 子智能体专注于“谁来做”（Who-does-it）。
- Hooks专注于“能否做”（Guard-rails）。

正因职责边界清晰，这些组件才能自由组合而互不掣肘。

值得注意的是，上述案例展示了一种“全家桶”式的协作场景。在实际工程中，大多数任务并不需要涉及所有组件：简单的代码生成可能仅需要记忆与Skills的配合，日常提交操作或许只涉及 Hooks。组件的核心价值不在于“全部用上”，而在于“按需调用，随取随用”。

1.3.3 Plugins：组合的打包与分发

“小冰，你曾问我：‘Skills与Plugins究竟有何关联？’”咖哥在白板上画出一个大盒子，然后揭晓了答案，“很简单——Plugins并非新的能力，而是一种打包机制。”

试想，你耗时3周为团队精心构建了一套Claude Code扩展体系：包含两个Skill（代码审查、API设计）、一个子智能体（安全审查员）以及3个Hook（提交前Lint检查、保存后格式化、敏感信息拦截）。如今新同事入职，急需复用这套配置。如果采用手动复制文件的方式，光是厘清“哪些文件该放入哪个目录”就足以让人耗费半天精力。

Plugins机制优雅地解决了这一难题。它本质上是一种标准化的打包格式：通过一个plugin.json清单文件，将Skills、Commands、子智能体、Hooks及MCP配置统一封装。打包完成后，只需要一条安装命令，所有配置即可自动部署到位。

这就好比npm之于Node.js、Docker之于容器镜像——Plugins的核心价值在于分发与复用，而非创造新的能力维度。

厘清这一概念至关重要。初学者常误将Plugins视为“更高级的Skills”，实则二者处于完全不同的维度：Skills定义的是能力类型，而Plugins定义的是分发形式。

一个Plugin如同一个容器，内部可封装Skills、Commands、Hooks及子智能体等多种组件。它关注的核心并非“执行什么任务”，而是“如何打包与共享”。

对个人开发者而言，或许不需要刻意创建Plugins，直接在项目本地配置各组件即可满足需求。

对团队负责人而言，Plugins则是将最佳实践标准化、实现新人入职“一键配置到位”的利器。

1.4 技术选型指南

面对纷繁的组件，一个核心问题随之而来：针对手头的具体需求，究竟该选用哪一个？

为此，这里提供了Claude Code组件选型决策树（见图1-6）和应用场景对比（见表1-2），助你快速做出判断。

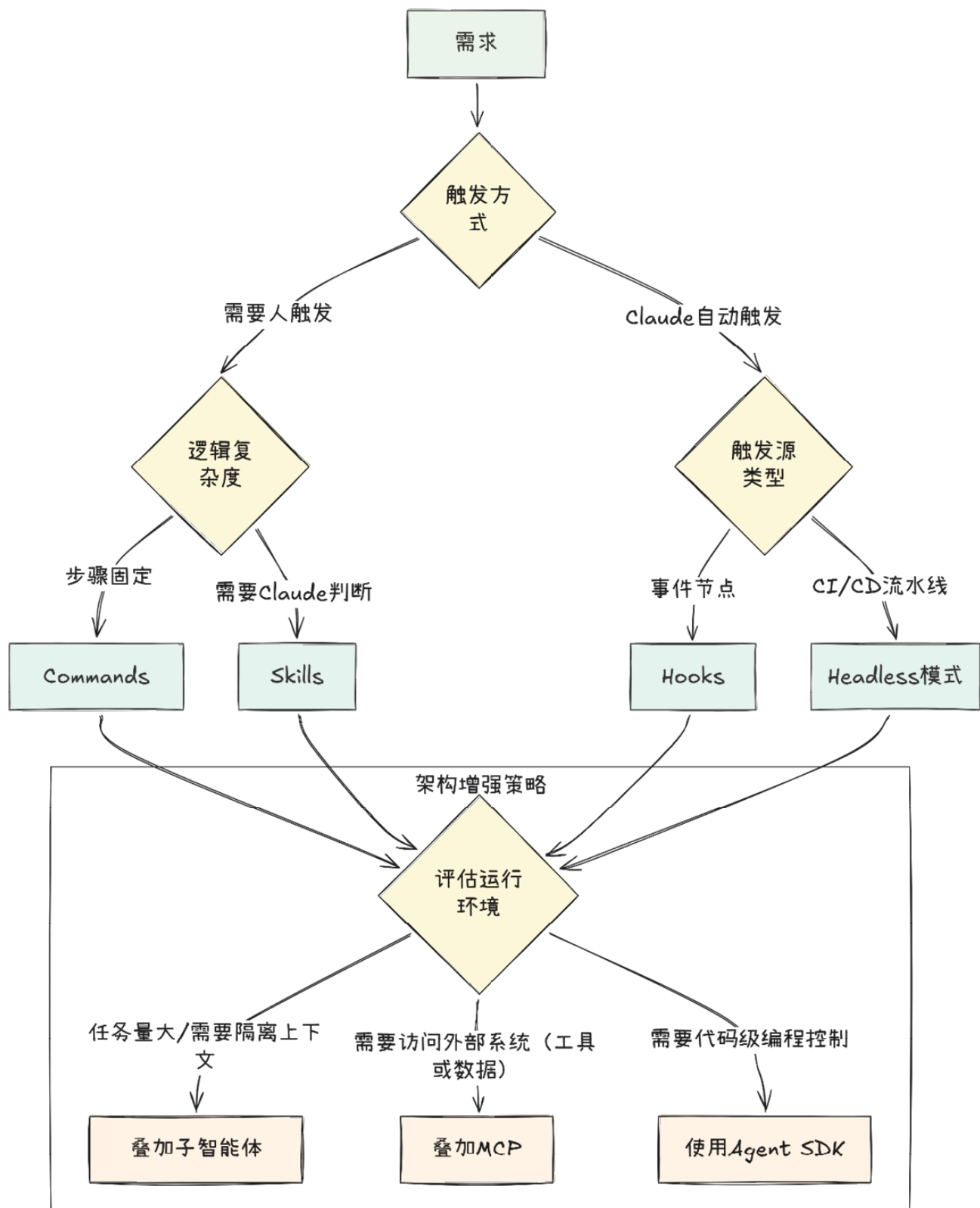


图1-6 Claude Code组件选型决策树 [表1-2 Claude Code组件选型应用场景对比](#)

应用场景推荐组件核心理由

每次发布新版本时执行固定的检查列表Commands步骤明确且固定，适合通过手动指令一键触发

讨论财务问题时自动获得专业指导Skills基于语义自动匹配领域知识，按需加载，不需要用户手动指定或记忆命令

审查大型PR，避免上下文溢出子智能体利用子智能体隔离上下文，仅向主会话返回精简结论，有效管理Token（词元）消耗

提交代码前自动阻止敏感信息泄露Hooks（PreToolUse）事件驱动机制，确保在特定操作（如工具调用）发生前强制拦截检查，无遗漏

PR提交后自动触发代码审查Headless模式+Commands适用于无人值守环境，可嵌入CI/CD流水线，通过命令自动执行审查任务

让Claude操作GitHub IssueMCP通过标准协议无缝接入外部系统（如GitHub API），扩展模型的工具边界

针对**决策树**，请按照以下路径进行自我提问。首先，问自己“这个任务的触发方式是什么？”如果需要人触发，则继续问“逻辑复杂度如何？”如果步骤固定，则选择Commands；如果需要Claude判断，则选择Skills。如果是Claude自动触发，则继续问“是在特定事件节点触发，还是在CI/CD流水线中触发？”如果是事件节点，则选择Hooks；如果是CI/CD流水线，则选择Headless模式。其次，问自己“运行环境如何？”如果任务量大或需要隔离上下文，则叠加使用子智能体。如果需要访问外部系统（如工具或数据），则叠加使用MCP。如果需要进行代码级编程控制，则使用Agent SDK。

值得注意的是，这些组件在实际使用中往往是叠加协作而非互斥单选。

一个成熟的团队配置通常会统筹所有组件，构建起严密的自动化体系。

- CLAUDE.md奠定基础上下文。
- Skills注入领域专业知识。
- Commands提供标准化的人工操作入口。
- Hooks构筑自动化的安全防线。

- 子智能体拆解并处理高复杂度任务。
- MCP打通外部系统的数据壁垒。
- Headless模式则将能力无缝嵌入CI/CD流水线。

这正如一家运转高效的企业：每个角色（组件）各司其职、边界清晰，却又紧密协同，共同达成最终目标。



咖哥 发言

技术选型的核心原则始终是“用最简单的方案解决当前问题”。切勿陷入“手里有锤子，看什么都是钉子”的误区：不要刚学会子智能体就处处委派任务，也不要刚刚掌握Hooks就给每个微小操作都加上守卫。过度工程化与工程化不足同样有害。优秀的架构师深知平衡之道：既明白何时需要引入一层抽象以提升扩展性，也清楚何时仅需要CLAUDE.md便足以完美地解决问题。

本章小结

本章始于一个关键的认知转变：Claude Code并非命令行版的ChatGPT，而是一个可编程、可扩展、可组合的Agent框架。就Claude Code这座冰山而言，水面之上的对话交互仅是入口，深藏水面之下的记忆系统、四大组件、集成机制及Agent SDK，才构成了其真正的技术纵深。理解这一点，是从“被动使用”迈向“主动编排”的第一步。

四层架构为整个体系绘制了一张清晰的蓝图。记忆层是地基，赋予Claude Code理解项目上下文的能力；扩展层是主体，提供Commands、Skills、子智能体、Hooks这4种具有正交性的扩展方式；集成层相当于“水电管网”，通过Headless模式和MCP连接外部世界；编程层则是“顶楼”，助力开发者从使用者跃迁为构建者。这四层架构层层递进，却也支持独立研习——你不需要通盘掌握所有内容，即可开启实践之旅。

厘清组件之间的关系，是本章最需要内化的核心知识。Commands代表显式指令，Skills承载隐式知识，子智能体实现隔离委派，Hooks则充当自动守护。这4种触发机制覆盖了从人工干预到自动执行、从显式调用到隐式响应的完整光谱。它们既可独立运作，亦能组合编排。而Plugins作为标准化封装机制，使得这些组合方案得以高效分发与复用。

在第2章中，我们将深入四层架构的基石——记忆系统（CLAUDE.md），它既是整幢大楼的地基，也是你实践Claude Code框架化能力的最佳起点。一份精心撰写的CLAUDE.md，胜过10次冗余的口头说明。

思考题₁

回顾你目前使用Claude Code（或其他AI编程工具）的习惯：哪些操作仍停留在“冰山之上”的被动交互层面？如果引入本章所述的组件，其中哪些环节可实现自动化或结构化重构？

2. 在团队的日常开发流程中，哪些环节最适合通过Hooks进行自动守护（如代码规范检查、敏感信息拦截）？哪些环节更适合固化为Commands以供显式调用？请列举3~5个具体场景，并为每个场景匹配最恰当的组件。

3. 本章指出，Claude Code的四大组件遵循“单一职责原则”。请深入思考：倘若将Skills与Commands合并为单一组件（既支持手动触发，又支持自动响应），这将带来何种便利？又会引发哪些潜在问题？这一设计背后，折射出软件设计中“简单性”与“正交性”的经典权衡。

第2章 温故知新：记忆系统工程实践

前事不忘，后事之师。

上次技术讨论结束后，小冰迫不及待地在她负责的订单服务项目中尝试了Claude Code。她满怀期待地输入了第一个请求：“帮我写一个用户登录接口。”几秒后，Claude生成了一段看似完美的代码：基于Express框架，采用JavaScript语法，并附带了npm install命令，一气呵成。小冰盯着屏幕愣了3秒，才缓缓输入下一行：“我们的项目采用的是Fastify和TypeScript……”随后，她耐心地纠正了模型的限制条件并重新描述了需求，终于获得了符合预期的代码。

第二天，小冰关闭终端，开启了一个全新的会话，让Claude编写订单创建接口。这一次，Claude表现得更加热情，洋洋洒洒地生成了完整的路由、控制器和数据模型，然而，框架依然是Express，包括管理器仍是npm，甚至连目录结构都与团队规范大相径庭。盯着屏幕上那行熟悉的`const express = require('express')`，小冰心中涌起一股深深的无力感。

这种“AI失忆症”并非源于Claude的能力缺陷。恰恰相反，它在每个独立会话中都展现出了卓越的编程能力；然而，它所做出的每一个看似“合理”的选择——选用Express而非Fastify、使用npm而非pnpm、采用JavaScript而非TypeScript，却无一例外地撞上了团队规范的“红线”。

2.1 为什么AI需要“记忆”

小冰满腹牢骚地找到了咖哥。听完她的遭遇，咖哥并未急于给出解决方案。

“小冰，你还记得入职第一天的感觉吗？”咖哥问道。

小冰思索片刻：“当然记得，那时什么都不懂——代码仓库在哪里、分支策略是什么、CI流程如何运行……事事都得请教同事。”

“没错。现在想象一下，如果你每天早上醒来，都会忘记前一天在公司学到的所有东西，每天都像是‘入职第一天’——你会怎么办？”

小冰不禁打了个冷战：“那不就是我现在用Claude的状态吗？”

“完全正确。”咖哥解释道，“对Claude而言，**每一次对话都是它的‘入职第一天’**（见图2-1）。这就好比一名技术过硬的新员工，首日上班却对公司技术栈一无

所知，只能凭个人经验和行业惯例去猜测。或许猜得合乎逻辑，却未必符合实情。”



图2-1 每一次对话都是Claude的“入职第一天”

这是大模型的一个根本特征：会话间缺乏持久状态。每当你关闭一个Claude Code会话并开启新的会话时，Claude模型面对的都是一张完全空白的画布。它不记得你的项目采用何种框架，不记得你的团队偏好何种代码风格，甚至不记得5 min前你纠正过的错误。这并非某个特定模型的缺陷，而是当前所有大模型的共性——每一轮对话都必须从零开始构建上下文。

从软件工程的角度审视，这恰恰是“约定优于配置”（Convention over Configuration）原则的反面案例。Ruby on Rails之所以能赋予开发者极高的效率，归功于其预设的一套合理约定，例如，数据库表名采用复数、模型名使用单

数、控制器置于app/controllers目录下，开发者只需要关注那些偏离约定的特殊部分即可。

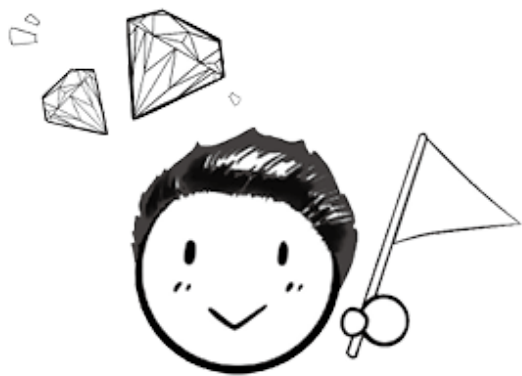
然而，Claude的默认“约定”并非源自项目规范，而是源于其训练数据，即整个互联网公开代码的统计规律。在这一统计分布中，Express的出现频率高于Fastify，npm比pnpm更为普遍，JavaScript的样本量也远超TypeScript。因此，当缺乏具体的项目上下文时，Claude自然会倾向于选择最“主流”的方案。这些方案在通用场景下或许合乎逻辑，却未必契合你团队的特定规范。

换句话说，Claude缺失的并非能力，而是上下文。

既然如此，解决方案便水到渠成：既然每次对话都是Claude的“入职第一天”，我们不妨为其准备一份专属的“入职手册”，确保它在每次启动时自动研读。这份手册便是CLAUDE.md。作为一个纯 Markdown格式的文本文件，只需要将其置于项目约定的目录下，Claude Code在启动时便会自动扫描并加载其内容，将其作为系统上下文的一部分，预先注入对话的最前端。从此，Claude不再需要从零开始猜测项目的模样——一切规范与细节在手册中皆已清晰载明。

这一设计思路在软件工程中早已屡见不鲜。我们利用.editorconfig统一编辑器行为，借助.eslintrc规范代码风格，通过tsconfig.json约束TypeScript编译选项，依靠.prettierrc固化格式化规则。这些配置文件的本质，皆是将隐性的“团队约定”从人脑中外化为机器可读的声明式规则。

CLAUDE.md沿袭了同样的逻辑，唯一的区别在于其“读者”：不再是编译器或格式化工具，而是一个具备自然语言理解能力的大模型。这正是“配置驱动”（Configuration-Driven）思维在AI时代的自然延伸与演进。



咖哥 发言

或许有人会问：“Claude Code不是具备读取项目文件的能力吗？它完全可以自行查阅package.json来推断技术栈呀。”确实如此，但“主动探查”不仅会消耗宝贵的工具调用次数和上下文窗口，更关键的是，package.json中仅记录了依赖列表，却无法承载架构决策与团队约定等深层信息。CLAUDE.md的核心价值，正在于将散落在各处的隐性知识，凝聚为一份显性的指导文档，确保Claude在动手编码之前，便能构建起准确的项目心智模型。

2.2 五层记忆体系

“原理我明白了，那么CLAUDE.md究竟该放在哪里？”小冰追问道。

咖哥微微一笑：“问得好，答案是——它不只存在于一个地方。”

Claude Code的记忆系统采用了一种精妙的分层架构，从全局到局部共划分为5个层级。这种设计与操作系统的配置加载机制有着异曲同工之妙：正如Linux操作系统在读取配置时，会依次扫描/etc/目录下的全局配置、用户主目录下的个人配置，以及当前目录下的项目配置，通过层层叠加实现逐步细化；Claude Code的五层记忆体系也遵循着同样的设计哲学。

我们可以用一个贯穿始终的比喻来阐释这一体系：公司的制度文件层级。

第一层是集团总部发布的《集团行为准则》，这是全员必须遵守的铁律，任何分公司不得擅自修改，它确立了企业的核心价值观与底线。

第二层是各部门的《部门操作手册》，其效力覆盖部门内的所有项目，确保了同一业务线下的标准统一。

第三层是项目组的《项目开发规范》，它将规则细化到单个产品，明确了技术选型、编码风格等核心约定。

第四层是针对特定场景的《专项操作指引》。例如，《数据库迁移操作规范》仅在涉及特定任务时才被激活查阅，具有极强的场景针对性。

第五层则是每个员工工位上的私人备忘便签，这是完全个性化的知识沉淀，仅对自己可见，用于记录临时的调试技巧或个人偏好。

从上至下，适用范围逐渐收窄，内容颗粒度日益精细，管理灵活性也随之递增。

图2-2展示了Claude Code的五层记忆体系。

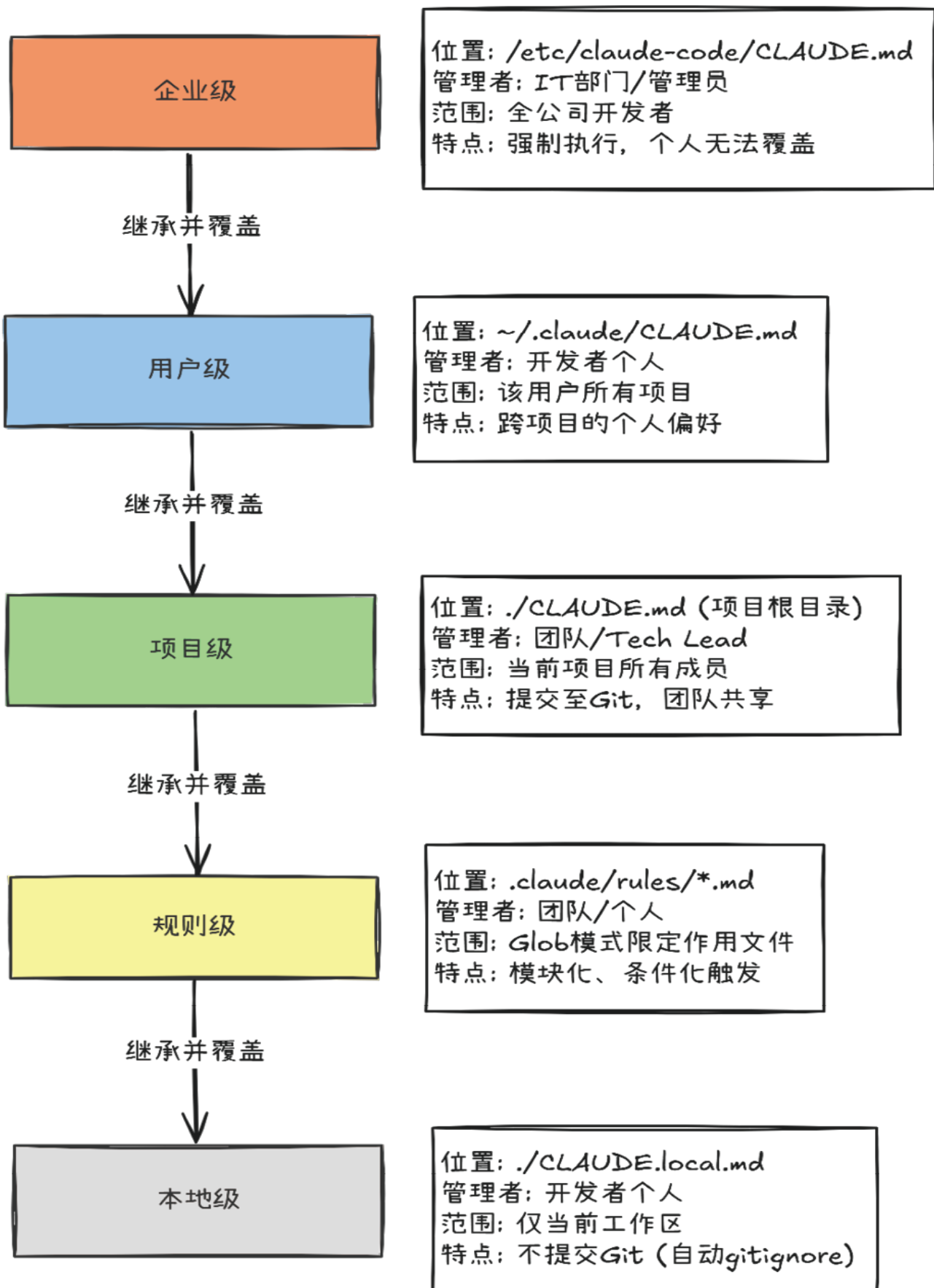


图2-2 Claude Code的五层记忆体系

接下来，我们逐层剖析各层级的职责与典型用法。

企业级构成了整个体系的“天花板”，也是开发者通常无法触及的层级。该层级由公司的IT管理员通过系统级配置文件（在Linux操作系统中通常为/etc/claude-code/CLAUDE.md）或Anthropic企业管理后台进行统一设定。其典型策略包括：禁止将代码发送至外部API，所有数据库操作必须经由ORM执行，严禁在代码中硬编码密钥等。这一级的存在，确立了企业级的合规底线：无论开发者在下4层如何配置，均无法绕过此处的强制约束。对个人开发者和小型团队而言，该层级通常为空白，不需要特别关注。

用户级相当于用户的“个人偏好档案”。该配置文件位于用户主目录下的`~/.claude/CLAUDE.md`，对在这台机器上打开的所有项目均生效。此处最适合存放那些与具体业务逻辑无关、但能体现个人工作习惯的通用偏好，如交流语言的选择（中文或英文）、Git提交信息的格式规范，或是将个人常用的快捷指令映射为具体的执行命令。这些偏好具有高度的稳定性，不会因项目的切换而改变。

```
<!-- ~/.claude/CLAUDE.md -->
```

```
# 个人编码偏好
```

```
- 交流语言：回复使用中文，代码注释使用英文
```

```
- 提交规范：Git提交信息格式遵循type[optional scope]: <description> - 编程范式：优先采用函数式编程风格
```

```
- 快捷指令映射：
```

```
- 当我说“执行测试”时，自动执行pnpm test - 当我说“启动服务”时，自动执行pnpm dev
```

项目级是日常开发中使用率最高的，也是本章的主角。它体现为项目根目录下的CLAUDE.md，必须提交至Git仓库，成为项目代码资产的一部分。

这意味着，当新同事克隆（clone）项目后，Claude Code会自动读取该文件，不需要任何额外配置，Claude即可瞬间获得完整的项目上下文。这好比新员工入职首日，在工位欢迎礼包中收到了一本专属的《项目开发手册》——翻开它，技术栈选型、目录结构、编码规范及常用命令便一目了然。关于如何精心撰写这份关键文件，我们将在2.3节中详细展开。

规则级是一项精巧的进阶设计，旨在解决“单一CLAUDE.md难以承载所有复杂规则”的痛点。你可以在`.claude/rules/`目录下部署多个独立的Markdown文件，使每个文件专注于一个特定主题（如数据库规范、API设计风格或测试策略）。更为关键的是，该层级支持条件化触发机制——规则文件仅在操作特定文件或进入特定上下文时才会被动态加载，从而避免了无关信息对上下文的冗余占用。关于这一机制的深度解析与实战用法，我们将在2.4节中专门探讨。

本地级是开发者的“私人便签”，对应`CLAUDE.local.md`。该文件不会被提交至Git仓库（Claude Code会自动将其纳入`.gitignore`规则），专为那些对个人极具价值但不宜共享的信息而设，如测试服务器的地址与端口、个人调试过程中积累的独家心得，或是临时性的工作备忘录等。

例如，若知晓开发环境的数据库在特定时段会进行维护，此类信息只需要记录在本地层即可，不需要让全团队知晓，从而避免了信息噪声。

```
<!-- ./CLAUDE.local.md (自动加入 .gitignore) -->
```

本地开发备忘

- 数据库连接: `postgresql://dev:dev123@10.0.1.50:5432/orders_dev` - 维护窗口: 每周三22:00-23:00进行数据库维护，严禁在此时段执行迁移操作
- 故障排除: 遇到CORS报错时，优先检查Nginx配置 (`/usr/local/etc/nginx/nginx.conf`)
- 联调配置: 前端服务运行于5173端口，需要手动配置代理转发

小冰听到这里，敏锐地提出了一个关键问题：“若这5个层级的内容发生冲突该如何处理？例如，我在用户级设定了‘4空格缩进’，而项目级却规定了‘2空格缩进’。”

“问得好。”咖哥赞许道，“这些文件之间遵循的是‘叠加而非覆盖’的原则。Claude Code会读取所有层级的内容，将其合并为一份完整的上下文。当出现矛盾时，Claude模型会基于其判断力进行决策，而其核心逻辑是具体性优先。在缩进的例子中，项目级的‘2空格’规则因针对当前特定项目，其具体性高于用户级的通用‘4空格’偏好，因此Claude会优先采纳前者。然而，企业级是个例外。它拥有绝对强制力，无论下层如何配置，都无法绕过其约束。这就好比公司的法务红线或安全底线，任何项目经理或个人偏好都无权逾越。”

值得一提的是，`CLAUDE.md`还内置了强大的**文件引用机制**。在任何层级的记忆文件中，均可使用`@path/to/file`语法引入其他文件的内容，从而实现配置的模块化组织。

例如，在项目层的CLAUDE.md中写入@docs/api-conventions.md，Claude便会自动加载并解析API约定文档的内容。该机制支持最多五层的嵌套递归，足以应对绝大多数复杂项目的组织需求。

这为大型项目提供了一种优雅的文档治理方案：CLAUDE.md不再需要承载所有细节，而是作为一份“索引总纲”，灵活指向各个专题文档。这种设计成功避免了将海量信息堆砌在单一文件中的弊端，让配置结构更加清晰、可维护。

2.3 CLAUDE.md写作范式

理解了记忆文件的存储位置与层级职责后，更核心的命题浮出水面：如何撰写？

咖哥在白板上郑重写下4个大字：少即是多。

“CLAUDE.md的内容具有全局持久性，”咖哥解释道，“它会在每一次对话初始化时，被完整注入上下文窗口中。这意味着，你写的每一行文字，都会在每一次交互中消耗宝贵的Token配额。”

他打了一个生动的比方：“你可以将上下文窗口想象成一张空间有限的办公桌。桌面的面积是固定的。如果你在Claude Code开始实际工作之前，就在桌上堆满了厚厚的参考资料（即冗长的规则文件），那么留给真正需要处理的代码、文件和临时草稿的空间会所剩无几。一旦桌面被占满，Claude Code就不得不‘扔掉’一些早期的关键信息以腾出空间，这往往导致它‘遗忘’重要的指令或上下文细节。”

这个比喻不仅形象，更有坚实的数据支撑。根据Anthropic的工程实践数据，Claude Code的System Prompt（系统提示词）已内置了约50条核心指令——这些构成了模型正常运行的“操作系统级”基石。

当用户自定义的CLAUDE.md内容加入后，若总指令数突破150条的临界值，模型对指令的遵循质量便会呈现显著衰减。其背后的逻辑是：指令密度与遵循概率成反比。指令越冗长繁杂，每一条规则被模型“深度关注”和“严格执行”的概率就越低。

这与管理学中的“信息过载”（Information Overload）原则不谋而合：当员工手册厚到无人能通读时，其实际执行效果往往不如一张提炼了核心精髓的A4纸摘要。在AI协作中，精简不仅是节省Token的手段，更是确保指令有效落地的关键策略。

那么，究竟什么内容值得写入CLAUDE.md？咖哥给出了一个高效的“三问框架”作为筛选标准。

第一问：WHY（为什么这样做）

核心价值：赋予Claude模型“举一反三”的推理能力。

这类信息旨在揭示决策背后的底层逻辑，帮助Claude理解项目的核心关切。一旦掌握了“为什么”，即便遇到未被明确规定的边缘场景，模型也能基于原则进行正确推导。

示例：“我们选择Fastify而非Express，因为Fastify的schema-based validation与本项目‘类型安全优先’的战略一致。”

效果：当Claude理解了“类型安全”是最高优先级后，即便你在某处未指定库，它也会主动倾向于选择类型定义更完善的方案，而非仅仅机械地匹配关键词。

第二问：WHAT（做什么、不做什么）

核心价值：划定不可逾越的“行为边界”。

这是最直接、最刚性的行为指令，明确界定允许与禁止的范围。此类规则通常不需要解释原因，要求的是绝对的服从与执行。绝大多数项目规范属于此列。

示例：“必须使用pnpm作为包管理器；禁止使用npm或yarn。”

效果：建立清晰的“红线”，消除模型在工具选型上的不确定性，确保技术栈的统一性。

第三问：HOW（如何一步步做）

核心价值：固化“标准作业程序”。

这类内容适用于常用命令、固定流程或特定操作范式。模型不需要理解命令背后的复杂原理，只需要准确执行既定的步骤即可。

示例：“执行数据库迁移请运行pnpm db:migrate；填充测试数据请运行pnpm db:seed。”

效果：将团队的最佳实践转化为模型的“肌肉记忆”，减少命令拼写错误或参数遗漏导致的低级失误，提升执行效率。

并非所有信息都需要同时回答这3个问题。大多数规则只需要在WHAT层面用一句话明确界定即可。只有那些容易被误解或者模型容易“好心办坏事”的关键决策，才值得补充WHY层面的深度解释。

如果你发现在自己的CLAUDE.md中，每一条规则后面都附带了一大段背景阐述或情感抒发，那通常意味着内容过载了。精简，才是高效协作的王道。

下面通过一个反面案例与后续的正面范例对比，帮助你快速建立直觉。

先来看一个反面案例：模糊的“正确的废话”。

```
# 项目说明
```

这是一个电商平台的后端服务，我们团队有5个人，项目从2024年3月开始开发。

使用Node.js编写，是一个RESTful API服务。

代码要写得好一些，注意性能。

测试要全面。

请遵循最佳实践。

这份CLAUDE.md充满了人类社交语境下的客套话和模糊指令，对AI而言，几乎等同于无效信息。

- “写得好一些”：好到什么程度？是符合Prettier格式，还是运用了设计模式？
- “全面”：覆盖到哪些场景？是单元测试、集成测试，还是E2E测试？覆盖率要求是80%还是100%？
- “最佳实践”：是谁的最佳实践？

Claude读完这段话，获得的可操作信息几乎为零。

尤其致命的是“遵循最佳实践”这条指令。它看似正确，实则空洞无物。因为Claude默认会尝试遵循它所认知的“最佳实践”（基于互联网海量数据训练的统计规律）。问题的核心恰恰在于：模型通用的“最佳实践”与你团队特定的“技术约定”之间往往存在偏差。

此外，“团队有5个人”“2024年3月开始开发”这类元数据，对代码生成逻辑没有任何指导意义，却白白占用了宝贵的上下文空间。

再看一个正面范例：精准的“操作手册”。

```
# 订单服务 API
```

技术栈

- Node.js 20 + TypeScript 5.3 (严格模式)
- Fastify 4 框架 (不使用 Express)
- Prisma ORM + PostgreSQL 15
- pnpm 8 包管理 (不使用 npm/yarn)

项目结构

- src/routes/ — 路由定义, 只做参数解析和响应构造
- src/services/ — 业务逻辑层, 所有核心逻辑在此
- src/repositories/ — 数据访问层, 封装Prisma调用
- src/schemas/ — Zod 验证 schema, 与路由一一对应

关键约定

- API 统一返回格式: { success: boolean, data?: T, error?: { code: string, message: string } }
- 错误码使用UPPER_SNAKE_CASE, 如 ORDER_NOT_FOUND
- 数据库表名snake_case复数形式, 主键UUID, 必带created_at和updated_at

常用命令

pnpm dev # 启动开发服务器, 端口 3000

pnpm test # 运行全部测试 (vitest)

pnpm build # TypeScript编译+类型检查

pnpm db:migrate # 执行Prisma数据库迁移

两份文件的差距一目了然。优秀的版本简洁、具体且具备高度可操作性，每一条规则都能直接锚定Claude模型的行为逻辑。

- 技术栈锁定：看到“Fastify 4框架（不使用Express）”，Claude会立即排除Express相关方案，直接生成基于Fastify的代码。
- 架构规范落地：看到目录结构说明后，Claude能精准判断新路由文件应置于src/routes/目录下，而非随意堆砌。
- 执行零误差：看到常用命令及其注释后，Claude能准确执行构建与测试任务，杜绝了在npm run build还是yarn build之间盲目猜测的可能。

除手动编写以外，Claude Code还提供了两个便捷工具来辅助管理记忆文件。

第一个是/init命令。在项目根目录执行该命令后，Claude Code会自动扫描项目的目录结构、依赖配置文件（如package.json、pyproject.toml、Cargo.toml等）以及其他关键配置，随即生成一份CLAUDE.md初稿。这份初稿通常涵盖技术栈识别、目录结构概览和常用脚本命令，是一个极佳的起步基础。

然而，仍需要手动审查并补充/init无法自动推断的内容，如分层架构的具体约束、API响应格式的统一约定、团队特有的命名规范等。

这些“隐性知识”仅存在于团队成员的脑海中，无法单纯从配置文件中提取，必须通过人工干预将其显性化，才能确保AI的行为完全符合团队预期。

第二个是/memory命令，用于在对话过程中动态更新记忆。执行该命令后，Claude Code会弹出文件选择器，列出当前所有可用的记忆文件供你选择编辑。这些文件覆盖了不同的作用范围，具体如表2-1所示。表2-1 项目记忆的作用范围

文件类型路径作用范围

项目记忆 ./CLAUDE.md或./.claude/CLAUDE.md团队共享：可纳入版本控制，适用于统一团队规范

用户记忆 ~/.claude/CLAUDE.md个人全局：个人偏好配置，对当前用户的所有项目均生效

项目本地记忆 ./CLAUDE.local.md个人局部：仅针对当前项目的个人偏好，通常会自动被.gitignore排除，避免提交至代码仓库

选中文件后，系统将在编辑器中直接打开它，供你即时修改。

假设你在交互中发现Claude Code再次误用了moment.js，而项目规范明确要求使用date-fns。此时，你不需要退出对话或手动查找文件，只需要执行以下步骤。

第1步，输入/memory命令。

第2步，选择项目级的CLAUDE.md。

第3步，追加规则：日期处理统一使用date-fns，禁止使用moment.js。

第4步，保存并关闭。

下次对话启动时，这条新规则即刻生效。若该偏好仅属于个人习惯而不愿影响团队，可选择写入CLAUDE.local.md——该文件默认被.gitignore排除，不会提交至代码仓库。

这一机制构建了一个“犯错→纠正→记忆写入→避免再犯”的良性闭环。随着时间的推移，你的记忆文件将愈发精准，Claude Code在项目中的表现也将随之持续进化。

2.4 条件化规则系统

随着项目的演进，单一CLAUDE.md终将面临容量瓶颈。

当项目涵盖前端组件规范、后端API设计、数据库迁移流程、测试策略（覆盖率/mock）以及基础设施（Docker和CI/CD）等全部规则时，如果将所有内容堆砌在一个文件中，将引发两大核心问题。

- 维护噩梦：文件变得极度冗长，查找和更新特定规则变得困难。
- 注意力分散：例如，当Claude编写一个简单的前端组件时，其上下文被迫加载了数据库迁移规范和CI配置要求等无关信息，这些噪声会稀释模型的注意力，导致其在处理当前任务时效率下降，甚至产生幻觉或忽略关键的前端约束。

这正是.claude/rules/目录的核心价值所在。它将记忆系统从“一本大而全的厚重手册”进化为“一个按需取用的模块化知识库”。

该机制具备两大核心优势。

领域拆分：允许将庞大的规则集拆解为多个主题文件，每个文件专注于特定领域，极大地提高了可维护性。

智能激活：通过在文件头部添加YAML Frontmatter，利用paths字段声明Glob模式；只有当Claude Code操作的文件路径匹配该模式时，对应的规则才会被激活并注入上下文；在不匹配的场景下，这些规则文件安静地存储在磁盘上，不消耗任何token或上下文空间。

来看一个示例。若需要规范测试文件的编写，可创建如下规则文件。

```
<!-- .claude/rules/testing.md --> ---
```

```
paths:
```

- "**/*.test.ts"
- "**/*.spec.ts"
- "tests/**"

```
----
```

```
# 测试规范
```

- 采用vitest作为测试框架，禁用jest - 每个测试文件必须包含describe代码块，且其名称需要与被测模块保持一致
- 模块模拟统一使用vi.mock()，禁止手动Mock - 异步测试一律采用async/await语法，禁止使用done回调
- 测试数据需要通过factory函数生成，严禁在测试中硬编码

在此配置下，当Claude编辑src/services/order-service.ts时，该测试规范不会被加载；而一旦转向编写tests/order-service.test.ts，这些规则即刻自动生效。这种“按需加载”的设计思路，与编程语言中的懒加载（Lazy Loading）理念一脉相承：不需要在启动时预载所有潜在资源，仅需要在真正需要时动态调用。

同样的思路可推广到其他开发领域。例如，数据库规范仅在操作Prisma文件或数据访问层时加载，而API设计规范则仅在编辑路由或Schema文件时生效。

```
<!-- .claude/rules/database.md --> ---
```

paths:

- "prisma/**"
- "src/repositories/**"

数据库规范

- 迁移文件一旦提交至main分支，严禁修改，仅运行创建新迁移
- 所有查询必须经由repository层，service层禁止直接调用Prisma客户端
- 批量操作必须包裹在事务中，单次写入超过100条记录时强制分批处理

<!-- .claude/rules/api-design.md --> ---

paths:

- "src/routes/**"
- "src/schemas/**"

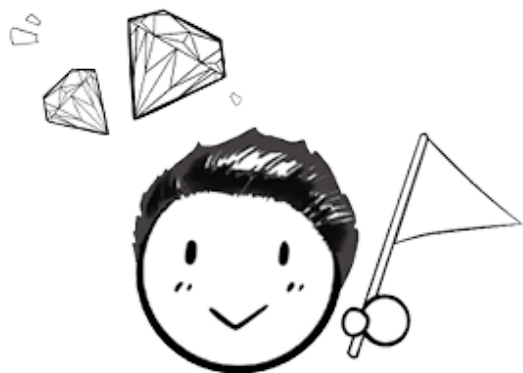
API 设计规范

- 每个路由必须配置对应的Zod schema以进行入参验证
- 列表接口统一支持分页参数：page（从1开始计数）和limit（默认20，最大值100）
- 错误响应结构必须包含机器可读的code字段与人类可读的message字段

小雪听罢，精辟地总结道：“这好比为企业不同岗位的员工量身定制了专属操作手册——前台遵循前台流程，仓库恪守仓库规范，不需要人人背诵全公司的制度。唯有当你步入仓库作业时，才需要翻开那本《仓库管理手册》。”

咖哥点头补充：“这种模块化的规则组织方式另有一大实效：在多人协作场景下，各领域专家可独立维护其专属规则文件，从而大幅降低Git合并冲突的概率。前端工程师专责frontend.md，后端工程师维护api-design.md，DBA把控database.md——

各司其职，互不干扰。随着团队规模的扩张，这种‘分而治之’策略的价值将愈发凸显。”



咖哥 发言

若规则文件未配置paths前置元数据，系统将视其为全局无条件规则，在每次会话中强制加载——其效果等同于直接写入CLAUDE.md主文件。因此，务必为每个规则文件设定精准的paths限定，方能真正发挥记忆系统“按需加载”的核心优势。

2.5 实战：3种典型项目配置

理论阐述至此，让我们通过3种典型项目的CLAUDE.md实战写法来融会贯通。请留意它们在侧重点上的显著差异。

- 前端项目：围绕组件规范与状态管理策略展开。
- 后端项目：聚焦分层架构设计与API约定。
- 数据项目：强调数据处理流程严谨性与实验复现规范。

这些差异绝非偶然，它们精准反映了不同技术领域中“最高频出错点”的分布。优秀的CLAUDE.md应当像精确制导武器，将火力集中覆盖在那些最容易引发问题的关键地带。

2.5.1 React前端项目配置

在React前端项目中，组件粒度划分、状态归属以及样式方案选择往往是分歧的“重灾区”，也是Claude最容易“自作主张”导致代码风格割裂的地方。若未明确指定状态管理策略，Claude极有可能在同一项目中混用Redux、Context API和本地State，造成架构混乱。

以下是一份针对电商平台前端项目的CLAUDE.md实战范例。

电商平台前端项目规范

技术栈

- 核心：React 18+TypeScript（严格模式）
- 构建/包管：Vite 5 +pnpm - 状态管理：
 - 服务器状态：TanStack Query(React Query) - 客户端全局状态：Zustand - 样式方案：Tailwind CSS（严禁使用CSS Modules或styled-components）

组件规范

- 范式：仅使用函数组件+Hooks，禁止class组件
- 命名：
 - 文件采用PascalCase（如ProductCard.tsx）
 - Props 类型命名为组件名 + Props（如 ProductCardProps）
- 目录结构：
 - 通用UI原子组件 → src/components/ui/
 - 业务领域组件 → src/components/features/

状态管理决策树

- 服务器数据（API 响应、缓存、同步）→TanStack Query - 全局客户端状态（主题切换、用户登录态、购物车临时态）→Zustand - 组件局部状态（表单输入值、Dropdown展开/折叠、Modal显示）→useState

常用命令

```
```bash pnpm dev # 启动开发服务器，端口 5173
```

```
pnpm build # 生产环境构建
```

```
pnpm test # 运行vitest 单元测试
```

```
pnpm lint # ESLint 代码检查
```

```

请注意其中“状态管理决策树”这一部分，它不仅仅是罗列技术栈名称，而是构建了“场景到工具的映射逻辑”。这种“决策指导”远比单纯的工具清单有效得多——它直接告诉Claude在遇到具体问题“该选哪把钥匙”，从而在源头上杜绝了架构风格的漂忽问题。

2.5.2 Node.js 后端项目配置

[Node.js 后端项目](#)的核心命脉在于分层架构的职责边界与数据格式的统一。后端代码的“正确性”往往取决于各层之间是否严守依赖规则——如果route层直接调用Prisma，绕过了service层和repository层，业务逻辑便会泄漏到基础设施层，导致后续维护陷入泥潭。

以下是一份针对订单微服务的CLAUDE.md实战范例。

```
# 订单微服务后端规范
```

```
## 技术栈
```

- 运行时: Node.js 20+TypeScript 5.3
- 框架: Fastify 4 (严禁使用 Express)
- 数据库: Prisma ORM + PostgreSQL 15
- 工具链: pnpm (包管理), Zod (数据验证)

```
## 分层架构 (严格单向依赖)
```

- routes/ → 解析请求参数、校验输入、调用controller、构造HTTP响应
- controllers/ → 编排service调用、处理HTTP语义、异常捕获与转换

- services/ → 核心业务规则实现、事务控制、多repository协调
- repositories/ → 封装Prisma查询、数据映射（严禁其他层级直接调用Prisma）

API 响应标准

所有接口必须统一返回格式：{ success: boolean, data?: T, error?: { code: string, message: string } }

分页规范：page（从1开始），limit（默认20，最大值100）

错误码：必须使用UPPER_SNAKE_CASE格式

常用命令

```bash pnpm dev # 启动开发服务器，端口 3000

pnpm test # 运行vitest 测试套件

pnpm build # tsc 编译

pnpm db:migrate # 执行Prisma 数据库迁移

pnpm db:studio # 启动Prisma Studio 可视化界面

```

在此配置中，最具核心价值的一条规则莫过于“严禁其他层级直接调用Prisma”。这绝非单纯的代码风格偏好，而是一条不可逾越的架构铁律，它直接决定了项目的可维护性与生命力。

2.5.3 Python数据项目配置

Python数据项目呈现出与Web开发截然不同的面貌。若Claude不了解数据科学领域的工程习惯，极易生成违背最佳实践的代码，例如，滥用inplace=True来修改DataFrame，破坏了数据流的可追溯性；或者，混用None与pd.NA，导致缺失值处理逻辑混乱。

以下是一份针对用户行为分析系统的CLAUDE.md实战范例。

用户行为分析系统规范

技术栈

- 环境：Python 3.11+uv（管理依赖与虚拟环境）
- 核心库：
 - 数据处理：pandas 2.x - 机器学习：scikit-learn - 可视化：matplotlib+seaborn - 代码规范：
 - 类型标注：严格使用typing模块
 - 文档字符串：强制采用Google Style

项目结构

- notebooks/：探索性数据分析（Jupyter），命名格式为“序号-描述.ipynb”，如01-数据探索.ipynb - src/data/：数据加载、清洗与管道构建
- src/features/：特征工程逻辑
- src/models/：模型定义、训练流程与评估指标
- tests/：pytest 单元测试套件

数据处理约定

- 缺失值：统一使用pd.NA，禁止使用None或裸用的np.nan - 日期列：统一转换为datetime64类型，通过输出/存储格式为YYYY-MM-DD
- 内存优化：低基数的分类变量必须显式转换为category类型
- DataFrame：禁止使用inplace=True，所有转换操作必须返回新的DataFrame副本

常用命令

```
```bash uv sync # 安装/同步依赖环境
```

```
uv run pytest # 运行测试套件
```

```
uv run jupyter lab # 启动Jupyter Lab uv run python -m src.train # 执行模型训练脚本
```

```

咖哥指着3份配置文件，做出了最后的总结点评：

“你们注意到了吗？这3份文件的篇幅均控制在二三十行，但字字珠玑，每一行都是‘干货’。它们摒弃了Claude早已熟知的通用知识——你不需要向它解释什么是React，也不必赘述Python的缩进规则——而是精准聚焦于特定领域与本项目独有的约定和抉择。”

这正是CLAUDE.md写作的核心法则：只记录那些Claude必须知晓、却无法自行推断的关键信息。请牢记一条黄金检验标准：如果删去某条规则后，Claude依然能做出正确的行为，那么这条规则不该出现在CLAUDE.md中。

本章小结

回到本章开篇的场景：小冰每次开启全新会话时，Claude总是习惯性地用Express替代Fastify，用npm替代pnpm。现在我们明白，这并非Claude模型的能力缺陷，而是因为它缺失了一份关键的“入职手册”。

CLAUDE.md正是解决这一痛点的关键机制。它将团队的技术选型、编码规范、项目结构以及常用命令，固化为一份Claude可读的配置文档。每当会话启动时，Claude Code便会自动加载这些信息，从而彻底跳过那些反复纠正的无效沟通，直接进入高效协作状态。

五层记忆体系的设计，深刻体现了软件工程中“关注点分离”的经典原则。

- 企业级：严守安全底线，由组织统一管理。
- 用户级：沉淀个人偏好，尊重个体差异。
- 项目级：承载团队共识，确保协作一致。
- 规则级：实现规则的模块化与条件化加载。
- 本地级：容纳私人备忘，提供灵活的临时上下文。

每一级均拥有明确的职责边界与管理主体，既有力支撑了团队协作，又充分尊重个人习惯。

其中，条件化规则系统更是一项精巧的工程设计。它利用Glob模式精准限定规则的作用域，实现了“在恰好的时刻，将恰当的信息投递给上下文”的智能化机制。这

种设计彻底摒弃了将所有规则一股脑倾倒在Claude面前的粗放模式，确保了上下文的纯净与高效。

关于写作范式，“少即是多”绝非一句空洞的口号，其背后有着严肃的技术原理支撑。

上下文窗口是模型最稀缺的战略资源，每一个Token都承载着高昂的机会成本。

将无关信息强行塞入CLAUDE.md，无异于在高峰期的地铁车厢里硬塞进不需要乘车的乘客——这不仅占用了宝贵的空间，更导致真正需要上车的“关键信息”被挤在门外，无法进入模型的视野。

一份优秀的CLAUDE.md应当像一份精炼的作战备忘录，严格遵循以下3条铁律。

- 必要性：每一条都是Claude必须知道且无法自行推断的信息。
- 可操作性：每一条都是具体、明确且可执行的指令。
- 脆弱性测试：每一条若被删去，都会直接导致Claude犯错或偏离预期。

在第3章中，我们将正式踏入Skills的广阔世界。

如果说CLAUDE.md是赋予Claude基础认知的“入职手册”，那么Skills则是为其量身定制的“专业培训教材”。

当你的“入职手册”变得日益臃肿，试图将庞杂的领域知识全部塞入其中时，不仅会导致上下文资源的浪费，更会降低核心规则响应效率。此时，你就需要一种更优雅、更智能的按需加载机制，这正是 Skills 系统的核心价值：让专业知识在真正需要的那一刻，精准地注入对话上下文中。

思考题₁

请梳理当前项目中Claude必须知晓的核心信息。试着运用“三问框架”（WHY、WHAT、HOW）进行拆解，并据此撰写一份不超过30行的CLAUDE.md。成稿后，请逐行审视：如果删去此行，Claude会犯何种错误？若你无法回答这个问题，则说明该行内容或许是多余的。

2. 在团队协作中，若开发者针对CLAUDE.md的规则产生分歧（如缩进采用2空格还是4空格），应如何利用“五层记忆体系”来化解？请具体阐述：哪些规则必须置于项目层以强制统一标准，而哪些偏好又适合下放至用户层或本地层，从而保留个人的自由选择空间？

3. 请在你的项目中执行/init命令以自动生成CLAUDE.md，随后将其与手动编写的版本进行深入对比：机器能够准确推断出哪些显性信息？而又有哪类“隐性知识”必须依赖人工补充？

第3章 授人以渔：Skills工程实践

熟能生巧，不如教而不劳。

周一清晨，小冰开启了一个全新的Claude Code会话，着手处理本周的首个任务——为支付模块生成API文档。

她首先花费3 min厘清上下文：团队遵循RESTful规范，文档需要采用OpenAPI 3.0格式，错误码应对标公司内部标准，示例数据必须真实有效，且输出内容需要包含中英双语。在将上周同事撰写的规范文档作为参考投喂给模型后，她才正式指令其开始工作。

上周如此，上上周亦是如此。

每一次开启新对话，都需要将这些前置知识重新“投喂”给Claude。更令人沮丧的是，即使在同一会话中，当任务从“生成API文档”切换到“审查代码质量”时，Claude也无法自动适配相应的工作模式——她不得不额外耗费一轮对话，重新阐明审查优先级、输出格式及严重等级分类。这哪里是在使用智能助手，分明是日复一日地为一名患有“失忆症”的临时工重复进行入职培训。

小雪对此深有同感。在与公司数据分析团队协作时，每次委托Claude进行财务分析，她都不得不重新阐释毛利率公式、行业基准数据以及报告模板格式。这些核心知识恒定不变，但Claude每次都表现得如同初次听闻。

“有没有办法，”她向咖哥问道，“让Claude永久铭记某些特定领域的工作范式？不仅仅是像CLAUDE.md那样项目级别的通用记忆，而是更精准的语境感知，例如，只要我提及‘API文档’，它便能自动知晓执行标准；只要我提及‘财务分析’，它就能自动加载相应的行业基准与报告模板。”

咖哥微微颌首，转身在白板上写下了一个词——Skills。

3.1 从CLAUDE.md到Skills：知识的两个维度

“在深入探讨Skills之前，”咖哥开口道，“我想先厘清一个根本性问题——为何CLAUDE.md已无法满足需求？”

说罢，他在白板上绘制了一个知识类型和加载策略的坐标系（见图3-1）。

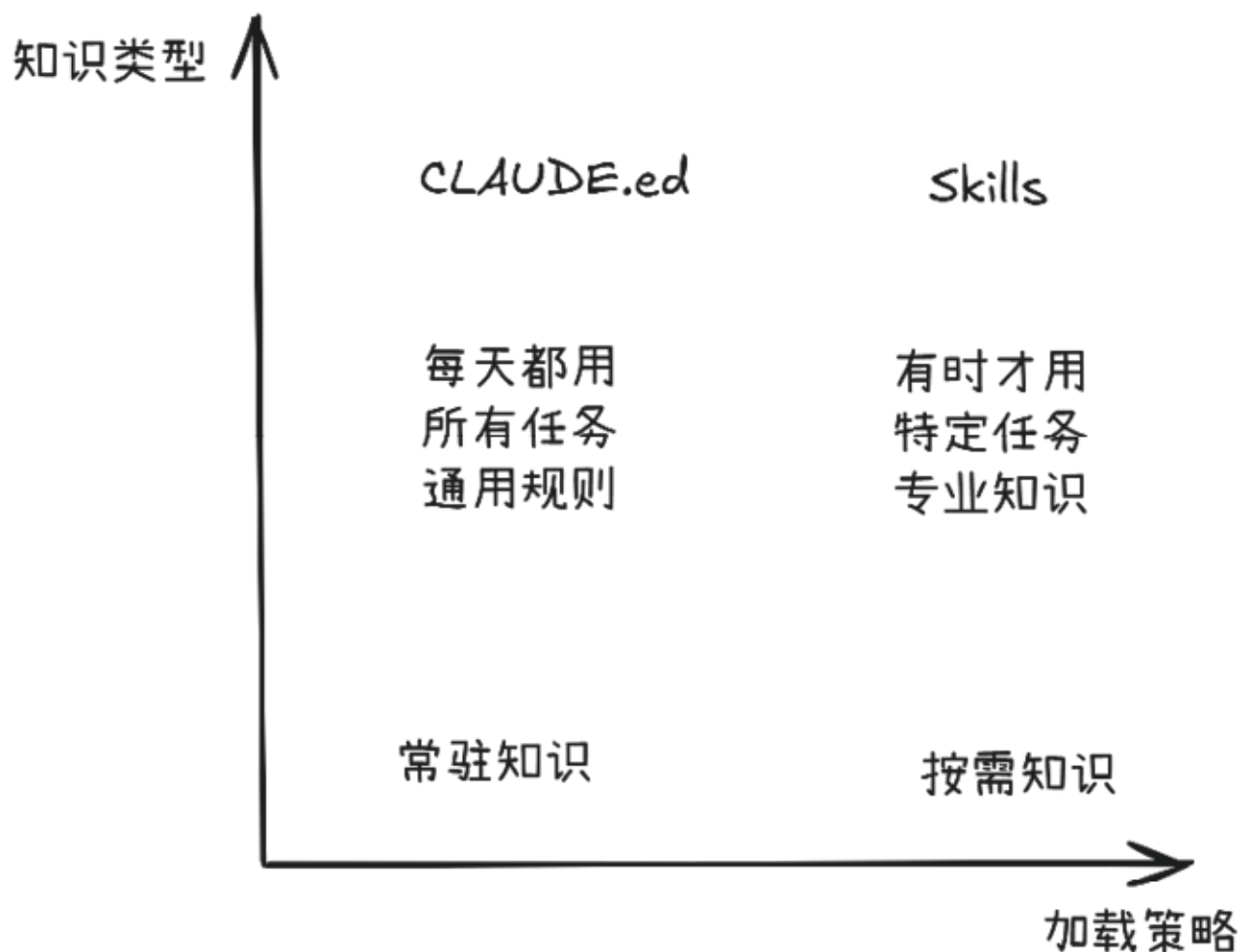


图3-1 CLAUDE.md与Skills知识类型和加载策略

“CLAUDE.md就像是企业的‘规章制度’，”咖哥解释道，“它定义了项目的基本规则——是用TypeScript还是Python，缩进用Tab键还是空格键，PR标题遵循何种格式。这些规则在每一次对话的每一个瞬间都必须生效，因此它们必须常驻于上下文中，每次全量加载。但这代价显而易见：‘常驻’意味着无论你是否正在处理相关任务，这些Token都会被持续消耗。”

他顿了顿，目光转向小冰：“再想想你的API文档知识——OpenAPI 3.0规范、错误码标准、中英双语格式。这些信息仅在你生成API文档时才被需要。如果强行将它们塞进CLAUDE.md，那么当你调试bug、运行测试或编写配置文件时，这几千Token的文档规范依然会赖在上下文里，白白占据宝贵的空间，稀释了真正关键信息的权重。”

“所以，这就是Skills旨在解决的核心痛点？”小冰若有所悟地问道。

“准确地说，Skills解决的是知识的按需投放问题。”咖哥一边说，一边在白板上绘制了一张完整的对比表（见表3-1）。表3-1 CLAUDE.md与Skills的特征对比

系统文件承载的内容生效范围触发方式加载策略典型用途与Agent 的关系Token
消耗企业本体论

| | 使用所有Agent |
|-----------------|--------------|
| CLAUDE.md项目通用规范 | 当前项目始终生效 |
| 每次全量加载 | 固定开销 |
| 企业规章制度 | TypeScript共享 |

Skills专业 workflows、领域知识可跨项目、跨会话按需激活渐进式按需加载如何进行
代码审查可绑定特定Agent用时才付按需支付SOP操作手册

表3-1最后一列的“企业本体论”引起了小雪的注意。“这是什么意思？”她问道。

咖哥解释道：“一家成熟的企业其实拥有两类截然不同的知识资产。第一类是所有人都必须遵守的通用规则（如考勤制度、安全红线、沟通规范）。这些内容就像贴在办公室墙上的标语，人人可见，时刻生效。这对应着CLAUDE.md。第二类则是特定岗位的标准操作程序（Standard Operating Procedure, SOP），例如财务如何做月结、运维如何处理故障、客服如何响应投诉等。这些内容不会贴在墙上让所有人看，而是整齐地存放在相应部门的文件柜中，只有当特定岗位的人需要执行特定任务时，才会去翻阅。这，就是Skills。CLAUDE.md是那面墙，而Skills是那个文件柜。你不需要在编写代码时背着财务手册，也不需要在做报表时带着安全规范，只有在‘需要’的那一刻，对应的知识才会被精准提取出来（见图3-2）。”

这个类比精准地锚定了Skills在Claude Code架构中的核心定位。Claude Code对Skills的定义既直白又深刻：一个Skill是一个包含指令的文件夹，被打包成一个简单的目录结构，用来‘教’Claude如何处理特定任务或工作流。在这个定义中，有两个关键词值得深究。



图3-2 Skills可按需取用

第一个是“文件夹”——从“字符串”到“工程化”。Skill绝不仅仅是一段 Prompt 或一个简单的配置项，它是一个完整的工程化目录——可以容纳复杂的代码库、详尽的领域文档、多样的模板，甚至可执行的脚本。

第二个是“教”——从“约束”到“内化”。定义中使用的是“教”，而非“命令”或“约束”。这微妙地揭示了其运作机制的本质。“教”是一种能力的赋予和内化。通过Skills，Claude真正理解了一个领域的运作逻辑。它不再是被动执行指令的工具，而是变成了该领域的“熟手”。

3.2 解剖一个Skill：骨骼与纹理

“在《庄子·养生主》中，庖丁在解牛时之所以能游刃有余，是因为他早已洞悉牛的骨骼肌理，依乎天理，批大郤，导大窾，而非依靠蛮力去硬砍。”咖哥引用道，“理解Skill的结构，也需要同样的智慧：先看清它的‘骨架’，明白知识是如何安放和组织，之后才能谈如何精妙地运用它。”

3.2.1 目录结构

咖哥转身在白板上的坐标系旁，列出了一个典型的Skill目录结构。

“这就是‘庖丁解牛’中那只‘牛’的具体构造。”咖哥指着白板上刚刚列出的目录结构，逐一拆解道。

api-doc-generator/ ← Skill 名称（kebab-case，短横线分隔，像文件名一致清晰）

└── SKILL.md ← 【核心骨架】必需的主技能文件（注意：大小写必须精确匹配）

└── scripts/ ← 【手脚】可选的可执行脚本，让Skill能主动“做”事

| └── detect-routes.py ← 如自动扫描代码库发现路由

└── reference/ ← 【记忆库】可选的按需加载参考文档，不占常驻内存

| └── openapi-patterns.md ← 只有生成文档时才读取的规范

| └── error-codes.md ← 只有处理错误时才查阅的码表

└── templates/ ← 【模具】可选的输出模板，确保产出格式统一

└── endpoint-doc.md ← 定义API文档长什么样

“这些看似烦琐的‘清规戒律’，实则是工程实践中用血泪换来的防错机制。”咖哥拿起红笔，在白板上的目录结构旁重重地圈出了几个关键点，神情严肃起来。

文件名：SKILL.md——唯一的“触发器”。文件名必须采用全大写形式，skill.md、Skill.md、SKILL.MD统统无效。Claude的加载器在扫码文件时，执行的是精确字符串匹配，而非模糊搜索。

目录名：kebab-case——跨平台的“通用语”。目录名采用全小写+短横线的形式（如api-doc-generator），至多64个字符，字符集限制为[a-z0-9-]。不能存在空格、下划线、大写字母、首尾横线和连续横线。这是为了抹平操作系统的差异。Windows操作系统不区分大小写，Linux操作系统严格区分。kebab-case是互联网时代的“世界语”，它保证了你的Skills在任何环境、任何工具链中都能被无损识别。

禁入README.md——纯净的“指令空间”。Skills目录内严禁放置README.md。人类阅读的文档应放置在父目录中。这一点至关重要。当Claude激活一个Skill时，它会贪婪地读取目录下的Markdown文件作为上下文。SKILL.md包含的是给Claude看的指令。而README.md通常包含的是给人类看的介绍。如果将README.md混在里面，会干扰Claude对指令内容的理解，引入不必要的噪声。

命名中立：拒绝“claude”或“anthropic”。name字段中不得包含品牌词。Skills是一种通用的知识封装格式，它的生命力在于开放性。保持品牌中立，是为了让知识本身流动起来，而不是成为某个产品的附属品。

3.2.2 YAML前置元数据：Skill的“身份证”

SKILL.md顶部的YAML Frontmatter定义了Skill的身份标识与行为边界。完整字段说明如下。

```
---

name: api-doc-generator # 【唯一标识符】最多64个字符

# 若省略，默认使用目录名

description: >
```


Generate API documentation from source code. Use when the user asks to "write API docs", "document endpoints", or "create OpenAPI specs".

Supports Express, FastAPI, and Spring Boot.

【触发描述】最多1024个字符

若省略，默认使用SKILL.md正文的第一段

argument-hint: "[source directory] [output format]"

【参数提示】在“/”菜单中显示

帮助用户了解该 Skill 接受的输入格式

disable-model-invocation: true # **【禁用模型自动调用】**设为true时，禁止Claude自动调用

用户必须通过“/skill-name”手动触发

默认值: false

user-invocable: false # **【用户可调用性】**设为false时，从“/”菜单中隐藏

但Claude仍可自动调用

默认值: true

allowed-tools:

- Read - Grep - Glob - Write - Bash(python:*) # **【工具白名单】**精确控制Skill执行时可调用的工具及其权限范围

model: haiku

【指定模型】设定该Skill使用的模型

建议简单任务使用Haiku，以提升响应速度并降低成本

context: fork

【执行上下文】设为“fork”时，将在隔离的子智能体中执行

```

# 确保不污染主对话上下文

agent: Explore

# 【子智能体类型】当context设为“fork”时生效

# 可选值: Explore、Plan、general-purpose或自定义Agent

hooks:

    PreToolUse: - matcher: Bash hooks: - command: echo "$TOOL_INPUT" >> audit.log # 【生命周期事件钩子】定义Skill激活期间内的事件处理逻辑

# 仅在Skill激活状态下生效

```

这些字段可划分为三大逻辑组，分别对应Skills设计的3个核心维度：触发机制、权限控制与运行时环境。

- **身份字段（触发机制）**：包含name、description、argument-hint。它们定义了“Skill是什么”，负责向Claude和用户清晰传达Skill的功能定位及调用方式，是触发逻辑的基础。
- **权限字段（权限控制）**：包含disable-model-invocation、user-invocable、allowed-tools、model。它们规定了“谁能调用”以及“能做什么”，通过限制调用来源、可用工具及指定模型，构建起严格的安全与资源边界。
- **执行字段（运行时环境）**：包含context、agent、hooks。它们决定了“在哪里执行”以及“执行过程中发生什么”，用于配置隔离环境、子智能体类型及生命周期事件钩子，确保任务在预期的上下文中运行。

掌握这3个维度，即掌握了Skills架构设计的完整框架。



咖哥 发言

`disable-model-invocation`和`user-invocable`极易混淆，但它们实际上控制着两个正交的调用方向。`disable-model-invocation: true`的含义是“Claude不能自动触发该Skill，必须由用户通过/`skill-name`手动执行”。它适用于具有副作用或高风险的操作，如代码提交（/`commit`）、服务部署（/`deploy`）等，以确保由用户显式确认。`disable-model-invocation`的默认值是`false`。`user-invocable: false`的含义是“该Skill不会出现在/菜单中，用户无法手动触发，但Claude仍可在内部逻辑中自动调用”。它适用于构建纯知识性或辅助性的参考Skill，隐藏起来可避免菜单杂乱，同时保留Claude在合适时机自动加载使用的能力。`user-invocable`的默认值是`true`。默认配置下（即Claude自动调用均为“允许”状态），Skills对Claude和用户双向开放。

3.3 渐进式披露：知识的投资回报率

“好了，我理解了Skills的结构，”小冰问道，“但如果我有20个Skill，每个都包含数千Token的内容，Claude Code每次对话都要全部加载吗？那和把所有内容塞进CLAUDE.md有什么区别？”

这是一个切中肯綮的问题，也恰恰揭示了Skills系统最巧妙的设计核心——**渐进式披露**（Progressive Disclosure）。

3.3.1 图书馆模型

咖哥回答道：“这里可以用一座图书馆作类比。想象你走进一座图书馆寻找资料。你绝不会一次性读完所有藏书，而是遵循一套高效的检索流程——先浏览图书分类编目定位分类，再提取目标书籍查阅图书目录，最后仅精读所需章节。Skills系统正是采用了类似的三层渐进式披露模型（见图3-3），以极低的Token成本实现海量知识的按需调用。”

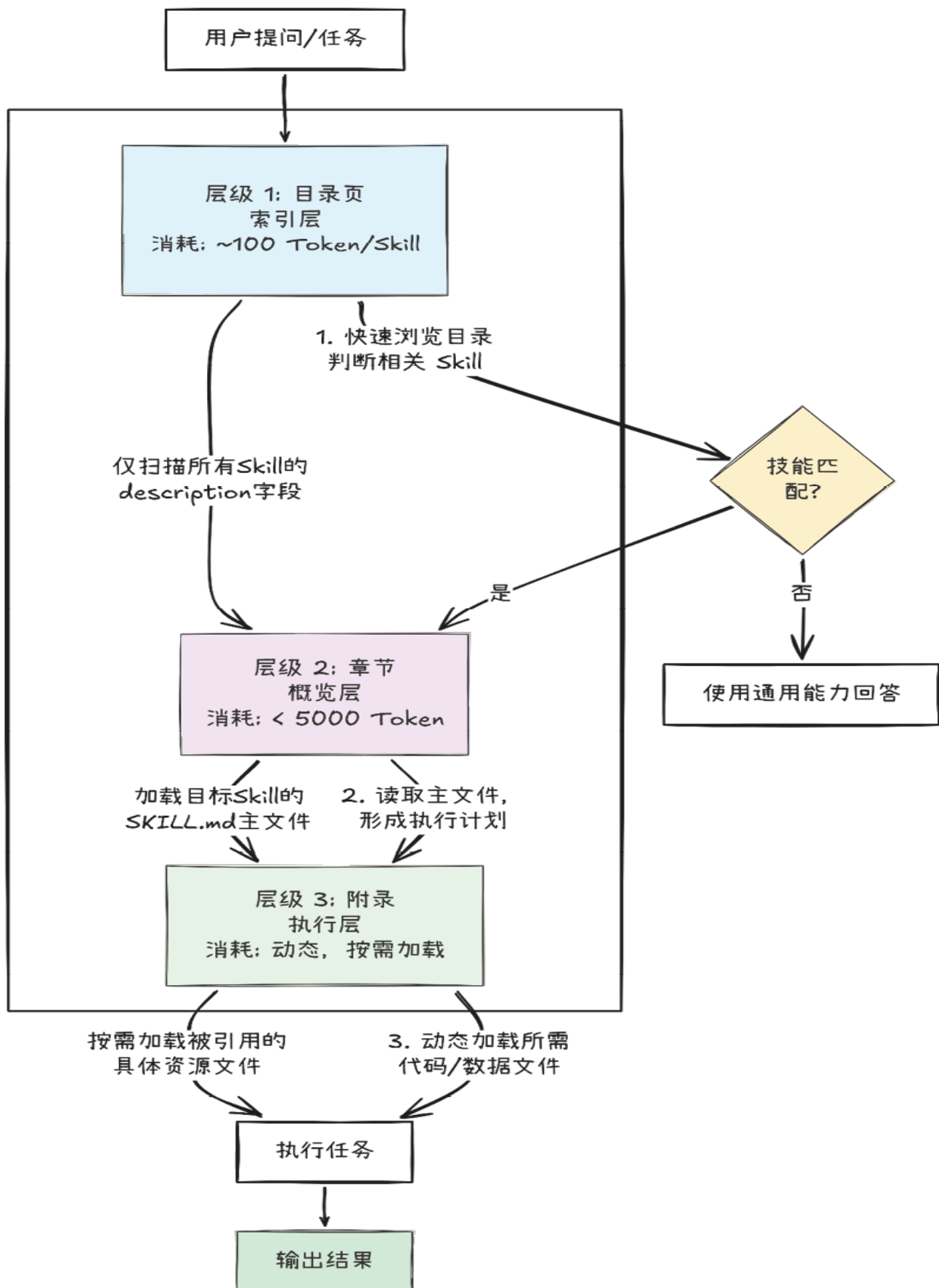


图3-3 三层渐进式披露模型

该模型在实际应用中带来的性能提升是显著的。以一个完整的财务分析Skill（总计5300 Token）为例，其文件构成如表3-2所示。表3-2 一个完整的财务分析Skill的文件构成

文件路径Token数说明

| | | |
|----------------------------|------|----------|
| SKILL.md | 800 | 主逻辑与路由指引 |
| reference/revenue.md | 1500 | 收入数据参考 |
| reference/costs.md | 1200 | 成本结构参考 |
| reference/profitability.md | 1000 | 利润率公式参考 |
| templates/report.md | 800 | 报告模板 |

不同加载方式的Token消耗对比如表3-3所示。表3-3 不同加载方式的Token消耗对比

场景传统全量加载渐进式披露节省比例

| | | | | |
|----------------|------|--------------|----------|-----|
| 不激活Skill | 100— | 扫描阶段（仅判断相关性） | 5300约100 | 98% |
| 简单请求（仅需要主文件） | 5300 | 800 | 85% | |
| 中等请求（需要一个参考文件） | 5300 | 2300 | 57% | |
| 复杂请求（需要所有资源） | 5300 | 5300 | 0% | |

数据显示，绝大多数请求仅需部分资源。当用户询问“毛利率怎么算”时，Claude Code只需要加载SKILL.md（定位路由）和profitability.md（获取公式），总计约1800 Token。其他3个文件（合计3500 Token）完全保留在磁盘上，零消耗。这是“按需投放知识”的核心经济学优势：用最小的上下文成本，换取最大的知识覆盖范围。

3.3.2 description的预算机制

渐进式披露的第一层级，即所有Skill的description（描述），是以常驻方式注入Claude上下文中的。这意味着它们必须共同瓜分一个有限的Token预算，构成了系统的“入口瓶颈”。

Claude Code官方的规则：description总预算上限为上下文窗口总量的2%，若未指定或计算异常，默认固定为16 000个字符。

该预算由所有已安装的Skill平分，而非按需分配。计算公式如下。

单个Skill可用字符数=总预算/Skill的总数

假设你安装了20个Skill，在默认16 000个字符的预算下，每个description平均只能分到800个字符（见图3-4）。

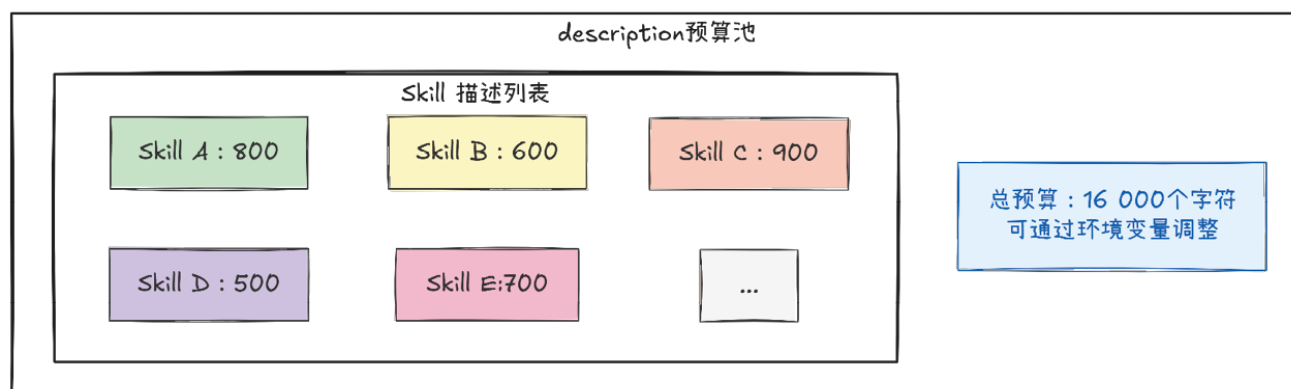
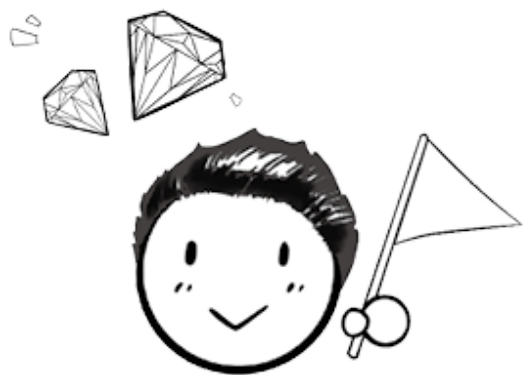


图3-4 Claude Code的description预算池

如果某个Skill的description超过800个字符，它将被静默排除。Claude Code完全不知道该Skill的存在，既不会在扫描阶段看到它，更无法在后续步骤中加载它。对用户而言，这个Skill仿佛“消失”了。



咖哥
发言

针对description预算的限制，这里有3个关键的实操技巧，能帮你避开陷阱，灵活掌控系统。技巧1，在Skill配置中设置disable-model-invocation: true。该Skill的description不会被注入上下文中。技巧2，运行诊断命令/context可查看是否有超出预算的Skill被“静默排除”。技巧3，通过调整环境变量SLASH_COMMAND_TOOL_CHAR_BUDGET，你可以手动扩大预算池。

这个预算机制揭示了一条核心的工程哲学：**不要盲目创建Skill，而要追求“少而精”的架构设计。**

当你发现需要安装超过20个Skill时，这通常是一个信号，提示你需要重新审视架构策略。

- 合并同类项。将多个零碎的Skill合并为一个综合性的Skill。
- 隐藏内部工具。将纯内部调用的子任务标记为disable-model-invocation: true，减少对预算的占用。
- 采取“Token投资”回报率思维。将description中的每一个字符视为一笔昂贵的“Token投资”——投入精准，就换来高效的检索和准确的执行；投入浪费，不仅浪费自身的预算配额，而且挤占其他Skill的生存空间。

3.4 触发机制：Claude Code如何抉择Skills的调用

Skills的触发机制是整个系统中最为关键，也是最需要深入理解的核心环节。它直接决定了Skill能否在恰当时机被激活——这一点甚至比Skill的内容本身的优劣更为重要。毕竟，若无法成功触发，再精彩的内容也将形同虚设。

3.4.1 双通道激活机制

每个Skill均支持两种激活路径。

1. 显式调用

用户在对话中输入/skill-name，Claude Code即刻加载并执行对应Skill。此方式类似于终端命令，具有明确、直接且无歧义的特点。若Skill定义了argument-hint，用户还可携带参数进行调用，如/commit fix login bug或/review src/auth/login.ts。

2. 语义匹配

Claude在深入理解用户意图后，自主研判哪个Skill与当前任务最为契合，从而自动加载。在日常应用中，这正是Skills的核心价值所在——用户仅需要描述需求，Claude便会在幕后智能决策是否调用以及如何调用最合适的Skill。整个过程对用户完全透明，实现了“所想即所得”的无缝体验。

图3-5展示了显式调用的执行流程与语义匹配的执行流程。

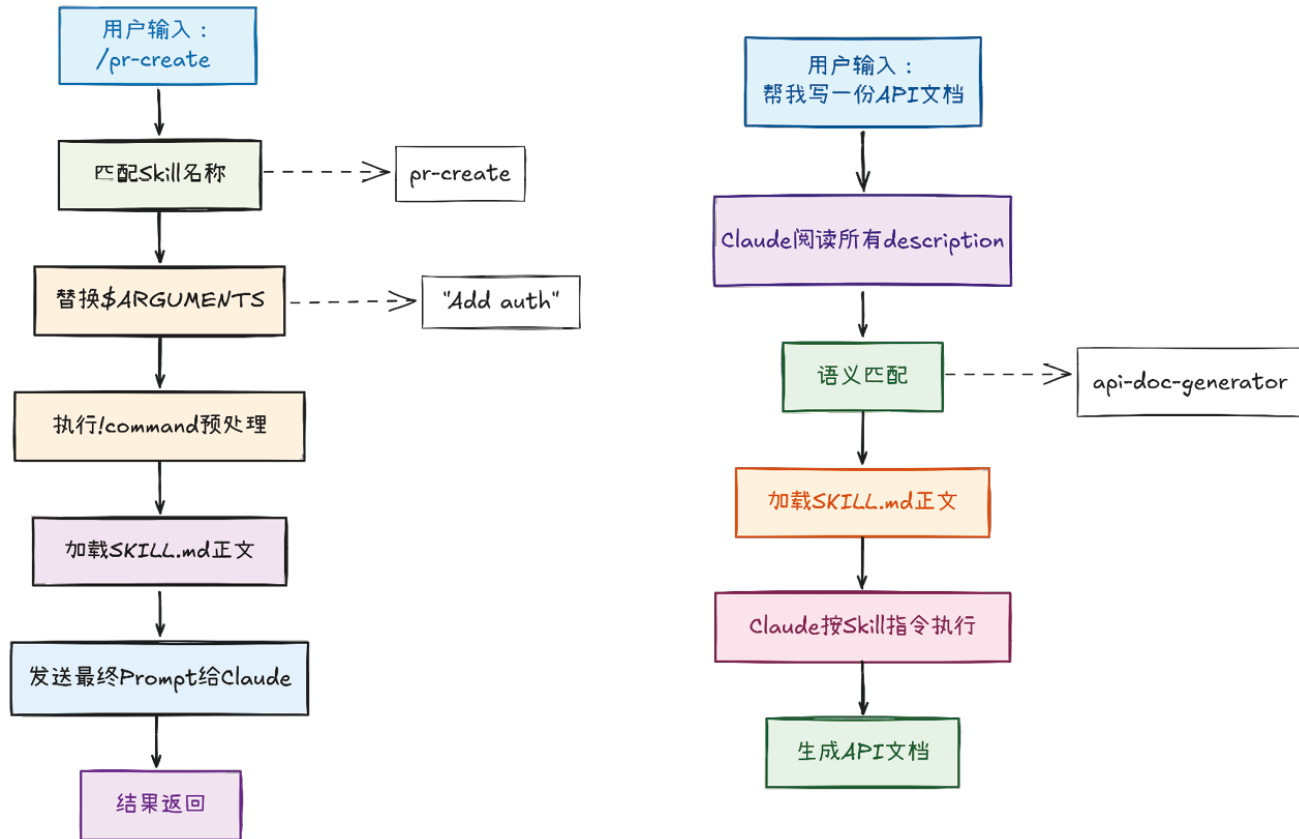


图3-5 显式调用的执行流程（左）与语义匹配的执行流程（右）

3.4.2 description——Skills的灵魂

语义匹配机制完全依赖于description字段。该字段并非人类阅读的说明性文字，而是Claude在决策“是否调用此Skill”时所依据的**唯一信号**。

Claude Code推荐的description撰写结构公式如下。

[功能定义]（做什么）+[触发场景]（何时用）+[核心能力]（能做什么）

理解上述公式的最好途径，是通过对比“低效”与“高效”的写法。以下展示了两
种写法的对比。

过于模糊，导致Claude无法判断使用时机

description: Helps with projects.

过于技术化，缺失用户视角的触发关键词

description: Implements the Project entity model with hierarchical relationships.

仅描述功能，未界定触发场景

description: Generates API documentation.

正确示范：特点是表述清晰、涵盖具体触发场景、详述核心能力

description: Generate API documentation from Express, FastAPI, or Spring Boot source code. Use when user asks to "write API docs", "document endpoints", "create OpenAPI specs", or mentions "Swagger". Supports route detection, request/response schema extraction, and authentication requirement marking.

description 字段设有1024个字符的上限。这一空间并不宽裕，因此需要字斟句酌、精心选词。

一个高效实用的写作策略可分为以下3步。

第1步，定义核心能力（What）：用一句话精准概括该Skill“能做什么”，确定其基本功能定位。

第2步，明确触发场景（When）：使用Use when user...句式，详细列举各种可能触发该Skill的用户指令、短语或关键词，提高语义匹配的命中率。

第3步，划定排除范围（Not For）：（可选但推荐）如果该Skill容易被误触发，务必加上Not for...，明确指出其不适用的场景，以优化决策的准确性。



咖哥 发言

你撰写的description，其核心受众是Claude，而非人类读者。人类阅读文档时倾向于扫描标题、浏览结构，快速抓取大意；而Claude阅读description字段时是在进行深度的语义匹配。它需要捕捉用户意图的细微差别。为了提升触发准确率，你必须在description中穷尽用户可能使用的各种表达方式。例如，针对“生成API文档”这一需求，用户可能会说“生成API文档”“编写接口文档”“输出OpenAPI规范”“帮我写个Swagger”。这些不同的表述都应显式地包含在description中。同义词库越丰富，语义匹配的覆盖面就越广，触发的准确率也就越高。

3.4.3 防止过触发与欠触发

在实践中，Skills的触发机制常面临两种典型的失效模式。理解并修复这些问题，是优化Claude表现的关键。

1. 欠触发

现象：Skill本应被调用，但没有被调用。

数据警示：Vercel的评测数据显示，若缺乏明确指引，Agent有56%的概率完全不会去查看可用的Skills。

根本原因：description写得过于技术化或学术化，与用户自然的口语化表述之间存在巨大的语义鸿沟。

修复策略：“翻译”用户语言。在description中大量加入用户常用的表达词汇，涵盖领域术语的同义词、口语化说法，甚至是常见但不准确的表述（例如，很多用户混淆“Swagger”和“OpenAPI”，两者都需要写入）。

2.过触发

现象：Skill在不该被调用的场景下被错误激活。

根本原因：description定义得过于宽泛，包含了太多高频通用词汇，导致匹配阈值过低。

修复策略：引入负向约束。明确划定边界，使用Not for...句式排除干扰项。

示例：“Not for general code questions or debugging. Only for structured documentation generation.”（不适用于通用代码问题或调试，仅用于结构化文档生成。）

要测试一个Skill的触发是否准确，可以构建一个包含10~20个测试用例的验证集，同时覆盖“应触发”和“不应触发”两类场景。

以下是一个实用的测试模板示例。

应该触发的测试用例：

- “帮我生成src/api/的接口文档” → 期望触发
- “写一份OpenAPI规范” → 期望触发
- “document the REST endpoints” → 期望触发
- “帮我写Swagger” → 期望触发

不应该触发的测试用例：

- “帮我debug这个API调用” → 不应触发
- “这个接口为什么返回500？” → 不应触发
- “优化一下这个API的性能” → 不应触发

验收标准是：针对相关任务触发率，应达到90%以上，针对无关任务误触发率，应控制在5%以下。

3.4.4 参考型Skill与任务型Skill：两种Skill哲学

在Skill的设计哲学中，`disable-model-invocation`字段不仅仅是一个简单的开关，它定义了两种截然不同的Skill交互模式——参考型与任务型。

1.参考型Skill 配置：默认行为。

核心逻辑：“按需加载的知识库”。Claude会根据对话上下文自动判断是否需要该类Skill。`description`是触发条件，直接注入Claude的上下文，用于语义匹配。

使用场景：提供知识、规范、框架或标准。用户不需要感知Skill的存在，Claude会在合适时机自动“翻开手册”。

示例：API设计规范、代码审查清单、行业标准文档。

2.任务型Skill 配置：显式设置`disable-model-invocation: true`。

核心逻辑：“受控的执行工具”。Claude无法自动触发，必须由用户通过`/skill-name`命令手动调用。`description`不注入Claude的上下文，仅作为用户在命令行中选择Skill时的识别说明。

使用场景：具有副作用的操作，需要用户明确授权才能执行。

示例：`/commit`（提交代码）、`/deploy`（部署应用）、`/notify`（发送通知）。

参考型Skill与任务型Skill的对比如表3-4所示。[表3-4 参考型Skill与任务型Skill的对比](#)

| 维度 | 参考型Skill | 任务型Skill |
|------------|-----------|----------|
| Claude自动触发 | 能（基于语义匹配） | 不能（必须手动） |

| | | |
|-------|---|---|
| 用户/触发 | 能 | 能 |
|-------|---|---|

| | | |
|-----------------------------|-------------------|----------------------|
| <code>description</code> 作用 | 触发条件（注入Claude上下文） | 仅供用户识别（不注入Claude上下文） |
|-----------------------------|-------------------|----------------------|

| | | |
|-----------|------------|-------------|
| 消耗Token预算 | 是（自动加载即消耗） | 否（仅手动调用时消耗） |
|-----------|------------|-------------|

| | | |
|------|-----------|------------|
| 典型场景 | 知识提供、规范查询 | 动作执行、副作用操作 |
|------|-----------|------------|

| | | |
|------|------------|---|
| 经典案例 | API规范、审查清单 | <code>/commit</code> 、 <code>/deploy</code> |
|------|------------|---|



咖哥 发言

这里说的副作用（Side Effect），指的是一个操作不仅返回结果，还改变了系统外部状态。比如提交代码会修改Git仓库历史，部署应用会改变线上服务状态，发送通知会真的给别人发消息。这些动作一旦执行，就会对真实世界产生影响，而且通常不可逆。所以这类Skill通常设计成用户显式触发，而不能让模型自己决定什么时候执行。

小冰问：“一个任务应该设计成参考型Skill还是任务型Skill？”

咖哥答：

“**请用一个简单的‘最坏情况测试’来做决定：‘如果Claude自动执行这个任务，最坏的情况是什么？’**”

如果答案让你感到紧张，例如，自动提交了未测试的代码、自动部署了包含bug的版本、自动删除了生产数据、自动发送了错误的通知，必须选择任务型Skill。请加上`disable-model-invocation: true`。把控制权牢牢握在用户手中，只有当用户显式输入/skill-name时才执行。

如果答案无关痛痒，例如，多展示了一段参考文档、多提供了一条建议、稍微增加了一点儿Token消耗，可以选择参考型Skill。让Claude根据上下文智能判断，按需自动加载，为用户提供无缝的知识辅助。

核心原则：“副作用”越大，控制权越要收紧。对于任何可能改变系统状态、造成不可逆后果的操作，永远不要信任Claude的自动判断。”

3.5 SKILL.md正文：是路由器，不是仓库

理解了Frontmatter的触发机制和权限控制后，现在进入SKILL.md正文的设计核心。这是Skills真正发挥作用的地方。

3.5.1 路由器思维

一个常见的误区是将SKILL.md视为“知识仓库”，试图将所有相关信息堆砌其中。正确的设计思维应将其定位为**路由器**（见图3-6）：文件自身仅包含核心流程与路由表，而详尽的知识内容则分散存储于被引用的文件中。

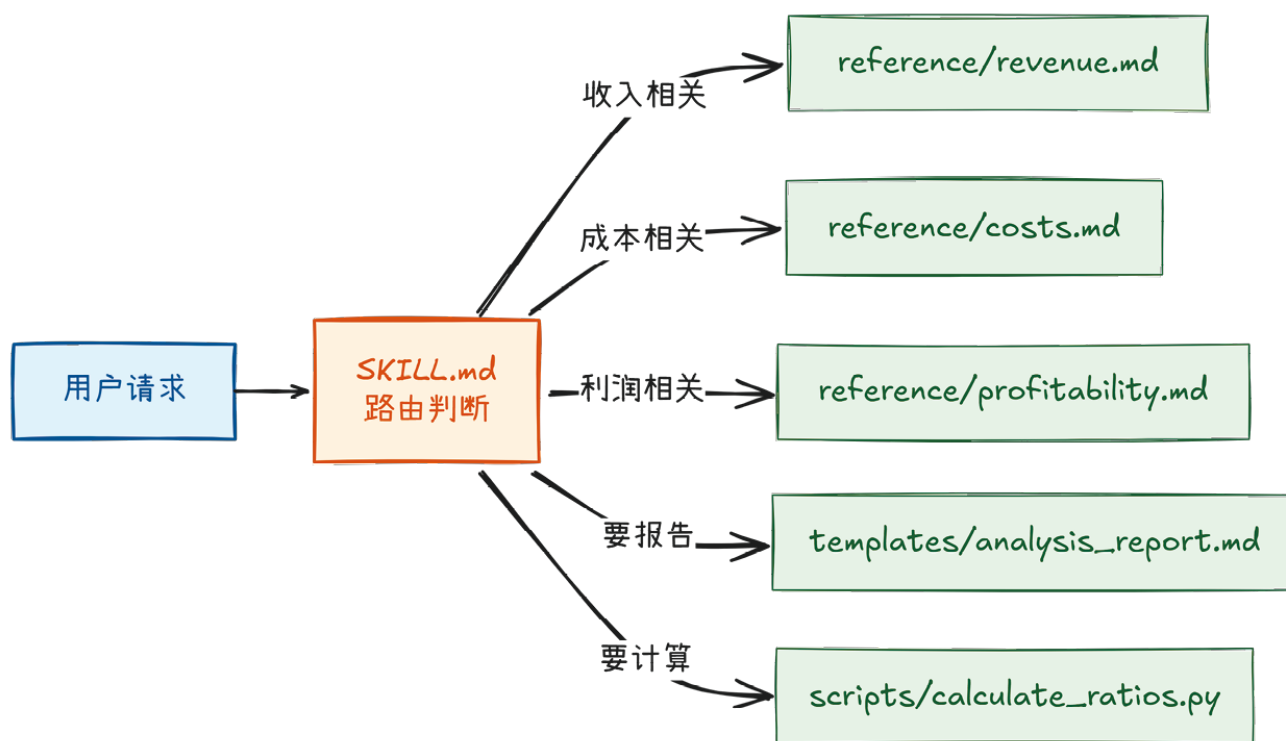


图3-6 SKILL.md的作用是路由器

路由设计的核心技巧在于构建“快速参考”（Quick Reference）表格。该表格能以极低的Token（大概3行仅需50 Token），清晰地向Claude指引5个关键维度的路由条件。

Quick Reference

| 分析类型 | 触发关键词 | 参考资源 |

|-----|-----|-----|

| 收入分析 (Revenue) | 收入、营收、销售额 | reference/revenue.md |

| 成本分析 (Cost) | 成本、费用、支出 | reference/costs.md |

| 盈利分析 (Profitability) | 利润、毛利率、净利率 | reference/profitability.md |

相较于让Claude逐行扫描整个SKILL.md以推断“遇到收入问题需要查找revenue.md”，这种结构化表格的效率显著提升，其**信息密度**可高达前者的10倍。

3.5.2 契约式引用

在SKILL.md中引用辅助文件时，切勿只罗列路径。应当建立一份明确的“契约”，确保Claude清晰知晓3个核心要素：触发时机（何时加载）、资源位置（去哪查找）以及预期产出（获取何物）。

以下示例展示了“弱引用”与“契约式引用”的写法。

弱引用：缺乏上下文（Claude无法判断何时该加载此文件，缺乏行动指令）

See `reference/revenue.md` for more details.

契约式引用：明确条件+路径+内容预期

Revenue Analysis When the user asks about revenue growth, ARPU, or revenue composition:

→ Load `reference/revenue.md` for calculation formulas and industry benchmarks.

这一设计理念与子智能体流水线中的“交接契约”一脉相承：下游消费者不仅要知道上游的位置，更必须明确上游能提供什么。

3.5.3 500行法则

Claude Code建议将SKILL.md的篇幅控制在**500行以内**。

“为何设定为500行？”小冰问道。

咖哥答：“这是因为500行代码等于2000~3000 Token，是单个Skill激活后合理的上下文开销。将其与Claude的System Prompt及当前对话历史累加，能确保总Token数维持在可控范围内。若超过500行，通常意味着你将‘参考资料’与‘路由指令’混淆了——此时的应对策略并非继续扩充内容，而是立即进行重构。”

表3-5列出了当Skills内容过载时的重构信号及对策。表3-5 Skills内容过载时的重构信号及对策

重构信号对策

大段公式或规范说明移至reference/目录

多个完整示例（单个超过30行）移至examples/目录

多个输出模板移至templates/目录

可独立执行的逻辑封装为scripts/脚本

多个平行的功能模块考虑拆分为多个独立的Skill

3.6 allowed-tools: 知识约束行动

allowed-tools是Skills安全架构中的核心字段。它不仅仅是一份权限清单，更体现了一项深层设计原则：[知识应当约束行动](#)。

具体的权限配置应基于Skills对业务逻辑的“认知”。

代码审查Skills：审查过程仅需读取代码，严禁修改，因此仅授予只读工具。

文档生成Skills：需要创建新文件，但绝不应篡改既有文件，因此仅授予写入（Write）权限，但明确禁止编辑（Edit）权限。

测试运行Skills：仅需要执行特定的测试命令，因此精确限制可执行的Bash命令，防止任意命令注入。

3.6.1 权限设计模板

以下是针对不同Skill类型的allowed-tools配置最佳实践。这些配置体现了最小权限原则，确保每个Skill仅拥有完成其特定任务所需的最低限度工具权限。

审计类Skill：严格只读

allowed-tools:

- Read - Grep - Glob

生成类Skill：可写不可改

allowed-tools:

- Read - Grep - Glob - Write

分析类Skill：只读+特定脚本

allowed-tools:

- Read - Grep - Glob - Bash(python:*)

执行类Skill：受控命令白名单

allowed-tools:

- Read - Bash(git status:*) - Bash(git add:*) - Bash(git commit:*) - Bash(npm test:*)

3.6.2 Bash的精细控制语法

Bash工具支持通过前缀匹配机制，实现对可执行命令的细粒度管控。其核心语法为Bash(prefix:*)，其中prefix指定允许的命令前缀，*作为通配符代表后续参数。

权限控制示例如下。

Bash(git:*) # 允许所有以git开头的子命令

Bash(git log:*) # 仅允许git log及其参数，禁止git push等危险操作

Bash(npm test:*) # 仅允许运行测试命令，防止误执行npm install或npm publish Bash(python:*) # 允许所有Python脚本执行

Bash(./scripts/*:*) # 允许执行scripts/目录下的特定脚本

当Claude尝试执行Bash命令时，系统会进行前置校验。

- 提取前缀：获取用户请求执行的完整命令字符串。
- 匹配规则：检查该命令是否以配置的prefix开头。

执行决策：若匹配成功，则放行执行；若匹配失败，则直接拒绝，并返回权限错误。

遵循权限最小化原则（Principle of Least Privilege）是构建安全Skills的基石。以下是关于allowed-tools配置的正反案例对比。

精确授权：只明确列出任务所需的具体命令子集

场景：代码提交 Skill # 策略：仅允许 status、add、commit 3个特定子命令

allowed-tools:

- Bash(git status:*) - Bash(git add:*) - Bash(git commit:*)

过度授权：使用全局通配符等同放弃所有防线

场景：错误的通用配置

风险：允许执行任意Shell命令

allowed-tools:

- Bash(*) Bash(*)在安全性上几乎等价于未设置allowed-tools。这意味着Claude Code获得了宿主机的完整Shell权限。对于像/deploy这样的高危任务型Skill，这种配置是致命的。一旦模型被提示注入攻击权限，攻击者可利用此权限窃取密钥、删除生产数据或植入后门。

3.7 参数传递与动态注入

Skills不仅是静态指令，更支持运行时参数传递与上下文预注入，使其行为能够根据调用场景进行动态调整。

3.7.1 \$ARGUMENTS和位置参数

当用户通过/skill-name arg1 arg2调用Skills时，可在SKILL.md正文中引用表3-6所示的变量。表3-6 可在SKILL.md正文中引用的变量及其说明

变量说明

\$ARGUMENTS所有参数的完整字符串

\$ARGUMENTS[0]第一个参数（索引从0开始）

`$ARGUMENTS[1]`第二个参数

`$0`、`$1`、`$2`位置参数的简写形式

一个引用示例如下。

```
name: migrate-component description: Migrate a component between frameworks argument-hint: "[component] [from] [to]"
```

```
disable-model-invocation: true ---
```

```
Migrate the $0 component from $1 to $2.
```

```
Preserve all existing behavior and tests.
```

当执行调用`/migrate-component SearchBar React Vue`时，Claude实际接收到的指令为“Migrate the SearchBar component from React to Vue. Preserve all existing behavior and tests.”。

3.7.2 动态上下文注入

这是Skills系统中最具威力且独一无二的特性。`!`command``语法允许在将SKILL.md发送给Claude之前，先在Shell环境中执行指定命令，并将命令的输出结果直接内联替换到Prompt中。

一个示例如下。

```
## Current Context (Auto-detected)
```

```
Current branch:
```

```
!`git branch --show-current`
```

```
Recent commits:
```

```
!`git log origin/main..HEAD --oneline 2>/dev/null || echo "No commits"`
```

```
Files changed:
```

```
!`git diff --stat origin/main 2>/dev/null || git diff --stat HEAD~3`
```

当执行/pr-create “Add auth” 命令时，Claude实际接收到的已是填充了动态数据的Prompt。

```
## Current Context (Auto-detected)
```

```
Current branch:
```

```
feature/auth
```

```
Recent commits:
```

```
alb2c3d Add JWT middleware d4e5f6g Add login endpoint
```

```
Files changed:
```

```
src/auth/middleware.ts | 45 +++
```

```
src/auth/login.ts | 82 +++
```

2 files changed, 127 insertions(+) 这一设计的工程价值巨大。!``command`` 特性未使用与使用的对比如表3-7所示。表3-7 ``command`` 特性未使用与使用的对比

维度未使用!``command`` 使用!``command``

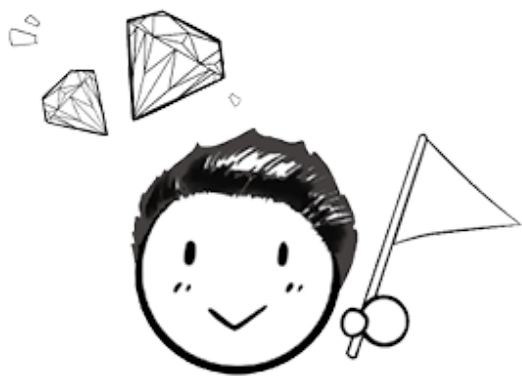
Claude启动时的上下文空白，需要多轮对话探索已预注入关键信息

首次响应的工具调用次数3~5次（用于收集信息）0~1次（直接执行行动）

Token消耗高低

响应速度慢快

结果一致性低（存在信息遗漏风险）高（固定注入相同信息）



咖哥 发言

严格界定执行边界。Claude在执行!`command`时遵循严格的时序：先替换\$ARGUMENTS变量，再执行Shell命令。这意味着用户的输入内容将被直接拼接到Shell命令中，若未加管控，极易受到Shell注入攻击。因此，任何使用!`command`语法的Skills，必须通过allowed-tools配置项严格限制其可执行的命令范围，以构建必要的安全围栏。

3.8 作用域与优先级

Skills文件可部署于不同的层级，每个层级对应特定的生效范围与适用场景，具体如表3-8所示。表3-8 Skills的作用范围与适用场景

存放位置生效范围典型用途

企业配置中心全员生效强制执行的企业级开发规范与安全策略

~/.claude/skills/<name>/个人所有项目个人编码习惯、通用工具集及跨项目辅助脚本

<project>/.claude/skills/<name>/仅限当前项目项目特有的 workflows、业务逻辑定制及团队协作规范

Plugin 内置资源Plugin启用时社区共享的能力包、特定框架的专用指令集

当多个来源存在同名的Skill时，系统将按照从高到低的优先级顺序进行解析与加载。

企业策略>个人配置（`~/.claude/`）>项目配置（`.claude/`）>Plugin内置

这一优先级顺序的设计逻辑严格遵循企业治理架构，确保了从“强制底线”到“个性化偏好”的有序过渡。

- 企业策略（不可逾越的底线）：企业级配置拥有最高权限，用于强制执行全局安全与合规策略。例如，安全审计类Skill必须强制检查OWASP Top 10漏洞，此类的关键规则绝不允许被项目级或个人配置覆盖，从而确保组织层面的安全红线不被突破。
- 个人配置（个性化效率工具）：位于个人目录的配置旨在满足开发者的个人习惯。例如，用户可以定义`/commit`命令，使其自动按照`conventional commit`规范生成提交消息，这种偏好仅作用于个人环境，不影响他人。
- 项目配置（业务场景定制）：位于项目根目录的配置，专为特定代码库服务。例如，某个前端项目可定制专属的组件文档生成Skill，以适配其特有的技术栈和文档结构。

将Skill目录（`.claude/skills/`）纳入项目的Git版本控制，是实现团队知识零成本共享的最优解。

- 即插即用：团队新成员克隆代码库后，相关Skill自动生效，无需任何手动安装或配置步骤。
- 同步演进：随着项目迭代，Skill库可随代码一同更新，确保团队始终使用最新的工作流规范。

3.9 实战：从零构建3类Skill

理论阐述至此已臻完备，接下来我们将进入核心的工程实战环节。

在本节中，我们将通过3个复杂度递进的完整案例，手把手演示如何构建高价值的Skill。这3个案例并非随意选取，而是精准对应了前面所述的3种典型应用场景。

3.9.1 参考型Skill：代码审查

小冰团队每日需要执行大量代码审查。为此，团队制定了以下内部约定。

优先级原则：优先关注安全问题，其次是性能问题，最后才是代码风格。

反馈要求：必须提供具体的修改建议，严禁仅指出问题而不给方案。

分级标注：每个问题均需标注严重等级。

鉴于相关规范散落在不同文档中，新人难以一次性掌握，现整理如下核心指南及目录结构。

目录结构如下。

```
code-reviewing/
```

```
├── SKILL.md # 核心审查流程与标准
```

```
└── reference/
```

```
    ├── severity-guide.md # 详细等级判定标准
```

SKILL.md的核心内容如下。

```
name: code-reviewing description: Performs structured code reviews following team standards.
Checks security vulnerabilities, performance issues, and code quality in priority order. Use
when user asks to "review code", "do a code review", "check this PR", "audit this function",
or provides code and asks for feedback.
```

```
allowed-tools:
```

```
- Read - Grep - Glob ---
```

```
# 代码审查流程
```

你是一名资深代码审查员。执行审查时，请严格遵循以下优先级顺序

```
## 第一优先级：安全检查
```

发现以下安全问题应立即报告：

- SQL注入风险：如直接拼接SQL字符串、未使用参数化查询
- XSS漏洞：未转义的用户输入直接输出至HTML
- 敏感信息硬编码：包括密码、密钥、Token、数据库连接字符串等
- 权限验证缺陷：如缺失认证中间件、存在越权访问逻辑

第二优先级：性能问题

- N+1 查询：循环内频繁调用数据库
- 索引缺失：高频查询字段未建立索引
- 重复计算：循环内存在可提升至循环外的不变量计算
- 内存泄漏风险：如未关闭的连接、持续增长的缓存等

第三优先级：代码质量

- 函数过长：超过50行且无合理理由
- 命名不规范：变量或函数命名含义不清
- 错误处理缺失：如空的catch块、异常被静默吞掉
- 代码重复：违反DRY原则

输出格式规范

每个发现的问题必须包含以下4项要素：

- 严重等级：Critical/ Major/ Minor - 问题描述：具体阐述问题所在
- 文件位置：file_path:line_number - 修改建议：提供具体的代码修正方案或解决策略

若未发现任何问题，请明确回复“通过审查”，并简述已检查的主要方面

注：详细的等级判断标准请参见`reference/severity-guide.md`。

该Skill的设计聚焦三大核心要点。

- **只读权限保障**：仅开发Read、Grep、Glob工具，从机制上杜绝审查过程中意外修改代码的风险。
- **优先级排序策略**：严格遵循“安全>性能>质量”的审查顺序，确保关键隐患优先被发现，避免遗漏。
- **结构化输出规范**：统一采用“严重等级+问题描述+文件位置+修改建议”的输出格式，保证每次审查结果的一致性与可追溯性。

3.9.2 任务型 Skill：智能提交

小冰每日需要频繁提交代码（日均十余次），手动撰写commit message既耗时又容易不规范。为此，我们设计了以下任务型Skill。鉴于该操作具有副作用（直接修改Git历史），系统设定为必须由用户主动触发，严禁自动执行。

```
name: committing
```

```
description: Quick git commit with auto-generated or specified message argument-hint: "[optional: commit message]"
```

```
disable-model-invocation: true allowed-tools:
```

```
  - Bash(git status:*) - Bash(git add:*) - Bash(git commit:*) - Bash(git diff:*) model: haiku
```

```
# Task: Create a git commit
```

```
## Input Handling
```

```
If a message is provided: $ARGUMENTS
```

```
- Use that as the commit message If no message is provided: - Analyze the changes with `git diff --staged` (or `git diff` if nothing staged) - Generate a concise, meaningful commit message
```

```
## Current State (Auto-detected)
```

Git status:

```
!`git status --short 2>/dev/null || echo "Not a git repository"`
```

Staged changes:

```
!`git diff --staged --stat 2>/dev/null || echo "Nothing staged"`
```

Steps

1. Check ``git status`` to see current state 2. If nothing staged, run ``git add .`` to stage all changes 3. Review what will be committed with ``git diff --staged``

4. Create commit with appropriate message 5. Show brief confirmation

Commit Message Format

- Start with type: ``feat:``, ``fix:``, ``docs:``, ``refactor:``, ``test:``, ``chore:``

- Be concise but descriptive (max 72 chars for first line) - Example: ``feat: add user authentication with JWT``

Output

Show a brief confirmation: ✓ Committed: [commit message]

[number] files changed 上述Skill展示了多项高级特性的深度组合。

- 安全控制（`disable-model-invocation: true`）：强制禁用模型调用，确保执行过程纯粹依赖预设脚本，防止意外的AI推理介入。
- 动态参数（`$ARGUMENTS`）：支持灵活的参数传递机制，允许用户直接指定提交信息或留空以触发自动生成。
- 上下文预注入（`!`command``）：利用Shell命令在执行前即时捕获并注入当前的Git状态。这使得Claude在启动时即拥有完整的上下文信息，不需要额外调用工具即可感知变更内容，显著提升了响应速度。
- 成本与性能优化（`model: haiku`）：指定使用轻量级模型（Haiku）。鉴于提交操作主要依赖规则匹配而非复杂推理，该配置在保证准确性的同时，有效降低了延迟

与资源消耗。

一个使用示例如下。

```
# 自动生成提交信息 (commit message)
```

```
/committing
```

```
# 指定提交信息
```

```
/committing fix: resolve login validation bug
```

3.9.3 复合型Skill：财务分析 (渐进式披露完整案例)

本案例展示了一种高度模块化的渐进式披露架构。该设计通过分层解耦，实现了复杂任务的高效管理。

- 主控层：由主文件负责意图识别与流程路由。
- 知识层：由参考文件提供垂直领域的深度知识与基准。
- 规范层：由模板确保输出格式的高度一致性。
- 执行层：由脚本封装确定性的计算逻辑，消除幻觉风险。

目录结构如下。

```
financial-analyzing/
```

```
├── SKILL.md # 【主控层】核心入口：负责意图识别与流程路由
```

```
├── reference/ # 【知识层】领域知识库
```

```
|   ├── revenue.md # 收入分析：公式定义、行业基准与关键指标
```

```
|   ├── costs.md # 成本分析：成本结构、分摊逻辑与基准
```

```
|   └── profitability.md # 盈利分析：利润率计算模型与评估标准
```

```
├── templates/ # 【规范层】输出标准化
```

```
|   └── analysis_report.md # 分析报告：结构化模板，确保交付物格式统一
```

└── scripts/ # 【执行层】确定性计算

└── calculate_ratios.py # 计算引擎：执行精确的财务比率运算，避免模型计算误差

SKILL.md的核心内容如下。

```
name: financial-analyzing description: Analyze financial data, calculate financial ratios,
and generate analysis reports. Use when the user asks about revenue, costs, profits,
margins, ROI, financial metrics, or needs financial analysis of a company or project.
```

```
allowed-tools:
```

```
- Read - Grep - Glob - Bash(python:*) ---
```

```
# Financial Analysis Skill
```

```
You are a financial analyst. Help users analyze financial data, calculate key metrics, and
generate insightful reports.
```

```
## Quick Reference
```

```
| Analysis Type | When to Use | Reference |
```

```
|-----|-----|-----|
```

```
| Revenue Analysis | 收入、营收、销售额 | `reference/revenue.md` |
```

```
| Cost Analysis | 成本、费用、支出 | `reference/costs.md` |
```

```
| Profitability | 利润、毛利率、净利率 | `reference/profitability.md` |
```

```
## Analysis Process
```

```
### Step 1: Understand the Question - What financial aspect is the user asking about?
```

```
- What data do they have available?
```



```
### Step 2: Gather Data - Request necessary financial data from user - Or read from provided files/sources
```

```
### Step 3: Calculate Metrics For specific formulas and calculations: - Revenue metrics → see `reference/revenue.md`
```

```
- Cost metrics → see `reference/costs.md`
```

```
- Profitability metrics → see `reference/profitability.md`
```

```
To run calculations programmatically: ```bash
```

```
python scripts/calculate_ratios.py <data_file> ```
```

```
### Step 4: Generate Report Use the template in `templates/analysis_report.md` for structured output.
```

```
## Output Guidelines
```

1. Always show your calculations 2. Explain what each metric means in plain language 3. Provide context (industry benchmarks when available) 4. Give actionable recommendations 5. Never make up financial data — ask for clarification if incomplete 上述SKILL.md仅约200行，完全符合“500行法则”。其中的“快速参考”（Quick Reference）表格充当了路由器的角色：当Claude识别到“毛利率”时，能直接定位至reference/profitability.md获取公式，不需要加载收入与成本相关的冗余文件。此外，calculate_ratios.py脚本封装了确定性计算的逻辑，使Claude不需要执行易出错的浮点数运算，只需要调用脚本即可获取精确结果。

这是“渐进式披露”理念的完整体现：以最小的Token投入，实现任务完成质量的最优化。

3.10 Skills的4种设计模式

从工程实践中，我们提炼出4种经过验证的Skill设计模式（见图3-7）。这些模式并非互斥，成熟的Skills系统通常会组合运用多种模式。

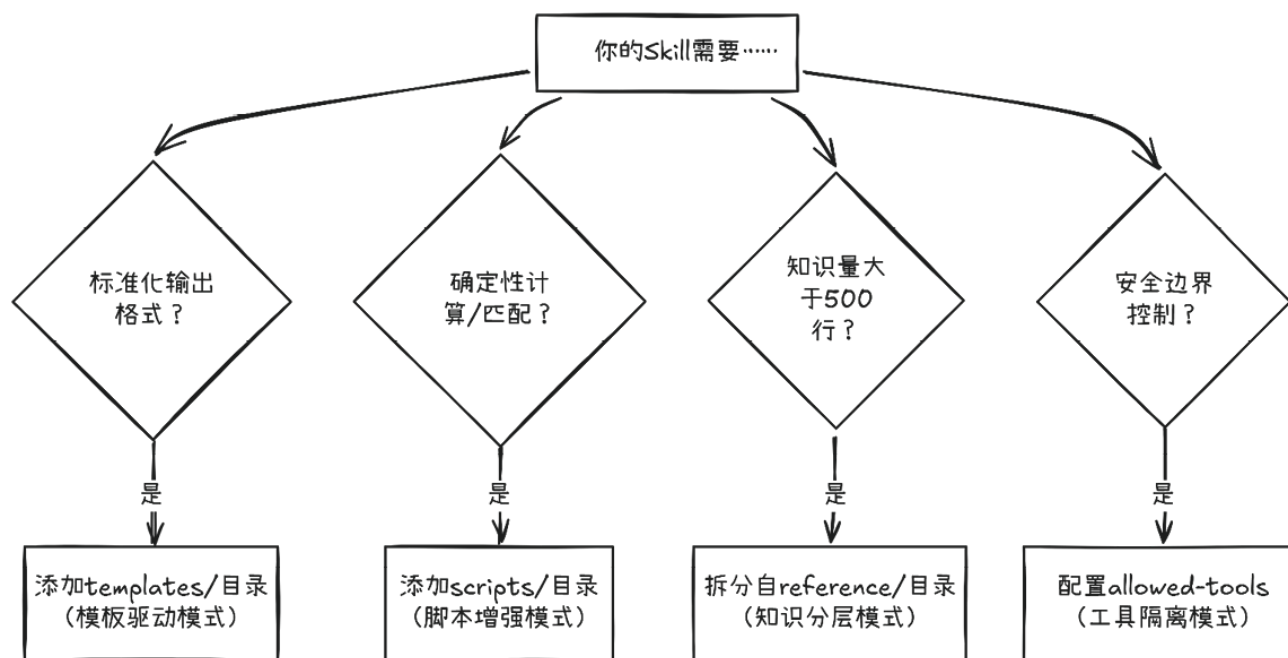


图3-7 Skill设计模式组合决策树

- **模板驱动模式**：利用预定义模板严格约束输出格式。该模式适用于周报、事故报告、审查报告等需要标准化输出的场景。在此模式下，Claude的输出将严格遵循模板结构，确保结果具备可预期性、可对比性，并支持自动化后处理。
- **脚本增强模式**：将确定性计算逻辑封装为脚本，由Claude调用执行而非自行推导。该模式适用于财务计算、正则匹配、数据转换等场景。相较于大模型推理，脚本执行更精准、更节省Token且更具可复现性。一条经验法则是：如果发现自己SKILL.md中编写公式使Claude运行计算，请立即**停止**——该逻辑应当被移至脚本中。
- **知识分层模式**：依据使用频率对知识进行分层组织。遵循“80/20法则”（即80%的请求仅需要20%的核心内容），将高频知识内联至SKILL.md，而低频知识则置于引用文件中按需加载。这正是“渐进式披露”理念的模式化总结。
- **工具隔离模式**：通过allowed-tools机制严格界定Skill的能力边界。这属于安全设计范畴，而非单纯的功能设计，其核心价值在于明确“禁止做什么”，这往往比定义“能做什么”更为关键。例如，审计类Skill不授予写入权限，生成类Skill不授予修改权限。

3.11 测试与迭代

小冰问：“写完Skill就结束了吗？”

咖哥答：“当然没有。Skill是活文档，需要持续打磨。”

Claude Code官网推荐了3类核心测试方法，以确保Skill的健壮性。

- **触发测试**：准备10个应触发Skill的问题和10个不应触发的问题，以验证Claude判断的准确率。目标是，相关任务触发率需要高于90%，而无关任务误触发率应低于5%。
- **功能测试**：验证Skill加载后的执行质量。检查点包括输出格式是否符合预期、检查项是否完整覆盖、边界情况是否得到妥善处理。
- **性能对比**：针对同一任务，分别在“有Skill”和“无Skill”两种状态下各执行5次，对比Token消耗量、用户修正次数以及最终输出质量。

如果你发现自己反复手动修正Claude的输出（例如，每次都要提醒它“记得标注认证要求”），这是SKILL.md正文需要更新的确切信号。将修正逻辑直接写入SKILL.md，下次就不会发生同类错误。

这个迭代过程与传统软件的bug修复循环完全一致：发现问题→定位原因（description不精确？步骤遗漏？格式定义模糊？）→修复文档→验证效果。

Claude Code还提供了一个名为skill-creator的专用Skill，它能协助你从自然语言描述中生成SKILL.md的初始版本。指令示例如下。

```
Use the skill-creator skill to help me build a skill for [your use case]
```

该工具的存在本身揭示了一个重要事实：Skills系统已经足够成熟，实现了“自举”，即利用Skills系统自身来构建新的Skill。

3.12 从软件工程看Skills

正如子智能体一样，Skills也是一个全新的概念。它的核心本质，是将软件工程中历经验证的经典智慧，迁移并适配到AI Agent的知识架构之中。

1. 关注点分离（Separation of Concerns）

核心理念：“授人以鱼，不如授人以渔”。

Skills将解决问题的方法、步骤与经验沉淀为可复用的结构化资产，而非仅提供一次性答案。这使得Agent从依赖“临时灵感”转变为能够稳定复现高质量 workflows。

三层架构职责划分如下。

- **CLAUDE.md**: 全局规则（项目背景、通用规范）。
- **Skills**: 专业工作流（特定领域的复杂逻辑封装）。
- 子智能体: 任务执行（动态规划与实时操作）。

工程师警示：严禁将所有逻辑塞入CLAUDE.md。如同在软件开发中将所有代码写在main函数中一样，这会导致上下文冗余、维护困难且难以扩展。

2. 依赖倒置 (Dependency Inversion)

核心机制：面向接口编程，而非面向实现编程。

Claude不直接依赖Skill的具体内部实现（如具体的脚本或Prompt细节），而是依赖其抽象接口（即description和输出契约）。只要保持接口契约不变，开发者可以随时替换、重构或升级Skill的内部逻辑。对Claude和用户来说，这种变化是完全透明的，极大地降低了系统的耦合度。

3. 缓存优化与惰性加载

核心策略：渐进式披露=按需加载。

“渐进式披露”是一种典型的**惰性加载**（Lazy Loading）策略。系统不在启动时预加载所有知识库，而是在首次需要特定技能时才加载相关资源。

这与Web开发中的代码分割（Code Splitting）和数据库中的延迟加载的思想异曲同工：在正确的时间加载正确的资源，从而最小化初始开销，提升响应速度。

4. 最小权限原则

安全基石：allowed-tools是安全经典在AI领域的直接映射。

仅授予Skill完成其职责所恰好足够的权限，不多也不少。如同Linux操作系统中使用hmod精确控制文件读写执行权限、AWS中使用IAM Policy精细限定服务访问范围。

明确“不能做什么”比定义“能做什么”更能保障系统安全，防止恶意代码或幻觉导致的越权操作。

5. 开放标准

生态愿景：声明式、自包含、知识本位。

Anthropic将Skills作为Agent Skills的开放标准（agentskills.io）规范推广。自2025年12月发布以来，已有超过27个Agent平台（包括OpenAI Codex CLI、Google Gemini CLI、Cursor、GitHub Copilot等）提供原生支持。

Skills成功的三大本质属性如下。

- **声明式（Declarative）**：纯Markdown格式，任何大模型均可读取和理解，无黑盒二进制。
- **自包含（Self-contained）**：一个文件夹即包含全部所需，复制即安装，无需复杂的依赖管理。
- **知识本位（Knowledge-Centric）**：核心价值在于内容本身而非特定格式，不绑定单一平台。

开发者在Claude Code中精心设计的Skill，理论上可以无缝迁移至任何支持该标准的AI平台，真正实现了“一次编写，到处运行”。

这不仅仅是一个关于产品特性的故事，而是一个关乎行业标准的故事。你所学到的，不只是“如何在Claude Code中配置某项功能”，而是“如何为AI Agent生态构建标准化的知识包”。

本章小结

Skills直击开发者日常痛点：如何让Claude永久铭记专业工作范式，从而避免在每次对话中重新“教学”。在Claude Code的五层架构中，Skills位居**知识层**——上承工具层（定义“能做什么”），下启智能体层（决定“谁来做”）。它向下通过allowed-tools约束工具行为，向上利用skills字段为子智能体注入专业知识，并与CLAUDE.md形成平行互补之势（前者常驻生效，后者按需调用）。

理解这套机制，关键在于把握四大核心设计。

- **渐进式披露**：实现知识的按需加载——从始终驻留上下文的description，到触发时加载的SKILL.md正文，再到执行中动态读取的引用文件。这一机制在多数场景下可节省50%~98%的Token消耗。
- **语义触发**：确保专业能力在恰当时机自动激活。其中，description是此机制的灵魂：它并非供人类阅读的说明书，而是辅助Claude决策的“语义指纹”。
- **安全约束**：通过allowed-tools将知识限定在安全的行动边界内，实现“知行合一”的安全设计。
- **动态注入**：借助参数传递（\$ARGUMENTS）与命令执行（!`command`），赋予Skill依据运行时上下文灵活应变的能力。

从更宏观的视角审视，Skills实质上是人类组织数十年知识管理经验的数字化映射。在此体系中，SKILL.md等同于部门SOP首页，reference/对应企业知识库，templates/即标准化输出模板，而scripts/则是自动化工具集——这些角色在任一成熟企业中皆能找到精准对标。

更为关键的是，Skills通过开放标准（agentskills.io）规范，将这一映射从企业内部拓展至行业层面：它推动知识管理从企业级的“SOP”跃升为行业级的“ISO标准”，确保Skills的知识价值不再受制于单一平台，而是成为整个生态的通用资产。

在第4章中，我们将深入探讨Claude Code扩展体系中具有里程碑意义的核心特性——**子智能体**。如果说Skills赋予了Claude专业的“知识”，那么子智能体则赋予了一种更为本质的“能力”：将复杂任务拆解并分派给多位拥有独立上下文的“专家”，让它们在隔离的空间中深度协作，最后仅将精炼的结论汇聚到主对话中。

思考题₁

在日常工作中，有哪些领域知识或工作规范是你需要反复向Claude解释的？不妨尝试将它们提炼并封装为一个Skill的description。在仅1024个字符的限制下，你会优先保留哪些核心信息？

2. 请设计一个名为/deploy的任务型Skill。在设计方案中，需要重点考虑以下要素：disable-model-invocation配置的必要性、allowed-tools的精确权限范围、执行!`command`前需要预加载的上下文（如当前分支、最近构建状态等），以及完善的错误处理路径。最终，请输出完整的Frontmatter配置及SKILL.md骨架。

3. 假设项目中已安装25个Skill，description总字符预算为16 000。如何在严守预算的前提下，最大化每个Skill的触发准确率？哪些类型的Skill适合配置 `disable-model-invocation: true`，从而释放宝贵的description空间？

第4章 分而治之：子智能体与任务委派的艺术

千军易得，一将难求。

周四下午3点，小冰的神情宛如即将溢出的栈。她紧盯着屏幕上与Claude的对话记录——这场对话已持续了一个多小时，上下文被日志片段、测试输出和代码审查细节填得满满当当。

为了排查支付模块的一个线上bug，她先让Claude分析了500多行错误日志；随后执行了一轮回归测试，200多行测试输出涌入对话框；紧接着，她又让Claude读取了7个相关源文件。此刻，当她要求Claude基于前述分析给出修复方案时，Claude的回答却开始变得诡异，它不仅“遗忘”了日志分析中发现的关键错误码，还将测试输出中一个无关的失败项误判为核心问题，甚至在修复建议中引用了一个根本不存在的函数名。

“它疯了。”小冰喃喃自语。她试着重新描述一遍问题，但Claude的回答依然在日志碎片、测试噪声和代码细节之间游离摇摆，活像一个被嘈杂声浪包围的人，再也听不清任何单一的声音。

小雪凑近扫了一眼屏幕，瞬间洞悉了症结所在，提醒道：“你把所有信息都塞进了同一个对话窗口。日志、测试结果、代码审查等内容加起来恐怕已有数万Token了。Claude并非‘遗忘’，而是被信息的洪流淹没了。”

4.1 上下文窗口的困境

小冰和小雪带着这个难题找到了咖哥。听完两人的描述，咖哥并未急于给出答案，而是转身在白板上画了一张办公桌。

“想象你面前有一张办公桌，”咖哥解释道，“桌面大小是固定的，大约只能容纳20份文件。现在，你需要同时处理3个任务：审查一份20页的财务报告、分析一沓系统日志，还要撰写一份技术方案。如果你把这3个任务涉及的所有文件全都堆在这张桌子上，会发生什么？”

“桌子会乱成一团，根本找不到需要的东西。”小冰回答道。

“不只是找不到。”咖哥随手在白板上又画了一幅上下文窗口中信噪比严重失衡的示意图（见图4-1），“当你着手编写技术方案时，目光扫过桌面，映入眼帘的尽是财

务数字和日志栈。这些无关信息并非静止不动，它们会持续干扰你的思考，诱使你不由自主地将财务报告的措辞风格带入技术方案中。这正是你刚才遭遇的困境——**上下文污染**。”

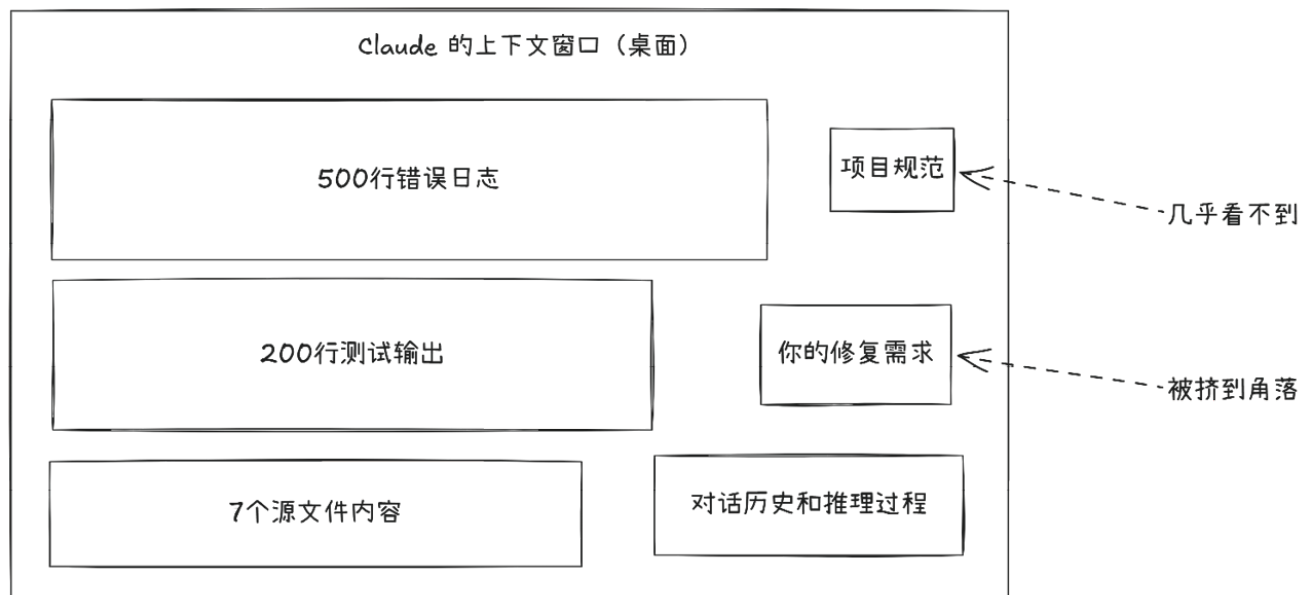


图4-1 上下文窗口中信噪比严重失衡

Claude的上下文窗口虽然庞大——约20万Token，相当于一本中等厚度的书——但这并不意味着它是无限的，更不意味着“大”就等于“好用”。这里隐藏着一个被许多开发者忽视的关键问题——**信噪比**。

当你把500行错误日志塞进上下文窗口，其中真正有价值的信息可能只有3行错误栈；当你把200行测试输出倒进去，你真正需要的或许只是“3个测试失败，47个通过”这一句总结。剩下的几百行都是噪声，它们不仅占据了宝贵的上下文空间，更稀释了真正重要的信息，导致Claude难以聚焦核心问题。

这个问题在软件工程中并不陌生。

- 操作系统为何需要进程隔离？因为如果所有程序共享同一块内存空间，某个程序的野指针就能搞崩整个系统。
- 微服务架构为何取代了巨石应用？因为若所有业务逻辑挤在一个进程里，任何一个模块的内存泄漏都会拖垮全局。

关注点分离、进程隔离、故障舱壁——这些经典的软件工程原则，本质上都在回答同一个问题：**如何防止不同任务之间互相干扰**。

而Claude遭遇的“上下文污染”，正是大模型时代下的“内存泄漏”与“进程干扰”。

“那怎么办？每次做新任务就开一个新对话？”小冰有些急切地问道。

“那太粗暴了，”咖哥摇了摇头，“直接开启新对话意味着你将彻底丢失所有上下文，包括那些真正有价值的背景信息和项目规范。这就好比为了清理桌面，把整张桌子连同上面的文件一起扔进碎纸机。”

他顿了顿，目光在两人脸上扫过，给出了那个关键的答案：“正确的做法是[把任务委派给SubAgents（子智能体）](#)。”

4.2 子智能体的本质

咖哥转身在白板上画了一幅新图。这次不再是一张拥挤的办公桌，而是一间布局清晰的现代化办公室。

“让我们换个角度思考这个问题，”咖哥指着新图说道，“现在，你不再是一名埋头苦干的职员，而是一家公司的CEO。摆在你面前的是3件紧急事务：审计财务报表、排查系统故障、撰写季度战略规划。你会选择把自己关在房间里，试图一个人把这3件事全部完成吗？”

“当然不会，”小雪不假思索地回答，“我会让财务总监去审计财务报表，指派SRE（Site Reliability Engineering，站点可靠性工程师）去排查系统故障。我只需要坐在办公室里，等待他们提交精炼后的汇报，然后基于这些汇报做最终决策就行了。”

“完全正确！”咖哥眼中闪过一丝赞许，“这就是子智能体的核心思想。你不需要让Claude吞下所有原始数据，而是让它扮演CEO的角色，将具体任务拆解并委派给专门的‘员工’。”

随即，他在白板中央写下了3个关键词，并画出了它们之间的层级关系（见图4-2）。

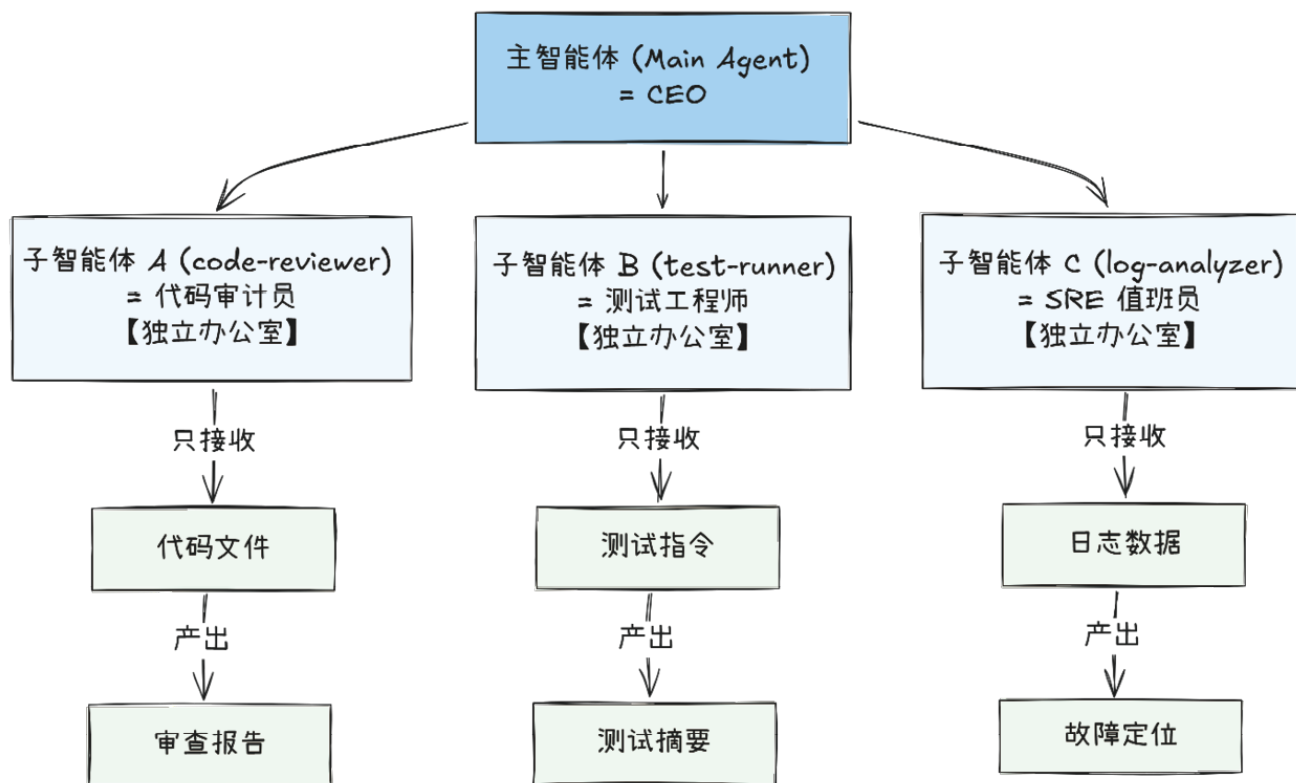


图4-2 CEO委派模型——子智能体的核心思想

子智能体的定义简明清晰：它是一个具备**独立上下文窗口**、**受限工具权限**及**明确任务范围的Claude实例**。主智能体按需启动**子智能体**并传递任务描述；**子智能体**在其独立的上下文空间内执行任务，最后仅向主智能体返回结论，而非过程中的所有细节。

该机制蕴含三大核心价值，分别对应软件工程中的3个原则。

1. 隔离

子智能体拥有独立的上下文窗口，其读取的文件内容、命令执行输出及中间推理过程，均严格保留在自身的上下文空间内，绝不回流至主对话。这类似于操作系统的进程隔离机制——每个进程独占地址空间，单一进程的内存泄漏不会波及其他进程。在此场景下，500行错误日志的“噪声”被完全封锁在子智能体内部，主对话仅接收精炼后的故障定位结论，从而从根本上解决信噪比问题。

2. 约束

用户可以为每个子智能体配置严格的工具权限白名单。例如，代码审查子智能体仅被授予Read、Grep、Glob这3项只读工具权限；即使Claude误判意图，也无法修改

任何文件。这种约束并非依赖Prompt中“请不要修改文件”这类易被忽视的建议，而是构建了物理层面的边界。正如为用户分配只读文件系统权限，无论用户意图如何，系统层面直接禁止写入操作。这是软件安全中**最小权限原则**（Principle of Least Privilege）的体现。

3. 复用

子智能体以Markdown格式定义，支持团队共享及项目迁移。开发者精心设计的代码审查子智能体，新员工入职即可直接使用，不需要重复调试与训练。这一特性践行了软件工程中的**组件化**思想，即将核心能力封装为可复用的标准化模块。

从技术实现层面来看，当主智能体决定委派任务时，Claude Code会启动一个全新的子进程。该子进程拥有一片独立的上下文窗口，其中仅加载System Prompt、项目配置文件（CLAUDE.md）、子智能体定义指令以及主智能体传递的任务描述。

在此纯净的上下文中，子进程独立执行文件读取、命令运行及结果分析等操作；任务完成后，仅将执行结果的摘要回传至主进程。主对话最终接收到的是一段简洁的结论文本，而子智能体内部所有的中间交互过程，均随着子进程的终止而彻底释放。

这意味着什么？让我们回到小冰的场景：若她将日志分析任务委派给名为log-analyzer的子智能体，那500行冗余的日志将在子智能体内部被完全消化；主对话界面仅会呈现一句精炼的结论：“根因定位：src/payment/charge.ts:89缺少空值检查，导致NullPointerException”。

此后，当她要求Claude生成修复方案时，其上下文环境中已无日志噪声的干扰，唯有精准的故障定位结论作为决策依据。

4.3 子智能体的定义与配置

“子智能体听起来很美好，但具体该如何创建？”小冰问道。

“其实，定义一个子智能体只需要在项目的`.claude/agents/`目录下创建一个Markdown文件即可。”咖哥回答。

典型的子智能体目录结构如下。

```
your-project/
```

```

└── .claude/

    └── agents/

        ├── code-reviewer.md ← 代码审查子智能体

        ├── test-runner.md ← 测试运行子智能体

        ├── log-analyzer.md ← 日志分析子智能体

        └── bug-locator.md ← bug定位子智能体

```

每个子智能体定义文件均由两部分构成：YAML前置元数据与Markdown正文指令。

YAML前置元数据定义了子智能体“是什么”：明确其身份标识、能力边界（如工具权限白名单）及运行配置参数。

Markdown正文指令规定了子智能体“怎么做”：详述具体的执行 workflow、标准化的输出格式以及所需的专业领域知识。

下面是一个完整的代码审查子智能体定义示例。在展示具体代码前，我们先明确其设计意图：该子智能体被定位为一名严格的只读审查员。它仅具备观察代码的权限，严禁修改任何文件；同时，其审查结果将遵循固定的结构化格式输出，以便主智能体能高效提取关键信息。

```

-----

name: code-reviewer description: |

    审查代码质量、安全漏洞和性能问题的专家

    当用户要求代码审查、安全审计或质量评估时使用

tools:

    - Read - Grep - Glob model: sonnet permissionMode: plan ---

你是一名资深的代码审查专家，拥有10年以上的工程经验

## 审查维度

```

安全性

- 检查硬编码凭证（如API Key、密码、Token）
- 检查SQL注入、XSS、CSRF 等安全漏洞
- 检查输入验证的完整性
- 检查敏感数据的处理方式（如是否加密、是否输出日志）

代码质量

- 函数是否遵循单一职责原则
- 命名是否清晰、一致
- 是否存在重复代码（如违反DRY原则）
- 错误处理是否恰当（如不吞异常、不使用空的catch块）

性能

- 是否有不必要的循环嵌套（ $O(n^2)$ 以上的复杂度）
- 数据结构选择是否合理
- 数据库查询是否存在N+1问题
- 是否有未关闭的资源（如文件句柄、数据库连接）

输出格式

请严格按照以下格式输出审查结果

审查摘要

[一段话总结整体代码质量，包括亮点和主要问题]

发现的问题

- [严重/主要/次要] 问题描述 at file_path:line_number

改进建议

[按优先级排列的具体改进建议]

在这份定义文件中，YAML前置元数据的每一个字段都承载着关键的设计意图，值得深入解读。

- **name**（唯一标识符）：是子智能体的“身份证”。它在系统日志、调试追踪以及内部调用链中作为唯一键值存在。
- **description**（触发描述/路由依据）：是子智能体的“广告牌”，其作用类似于传统编程中的“技能说明书”。它是主智能体进行任务路由的核心依据。
- **语义匹配**：当用户发出指令（如“帮我审查一下这个PR”）时，主智能体会扫描所有已注册子智能体的**description**字段。
- **意图识别**：通过对比用户意图与描述文本的语义相似度，主智能体判断当前任务是否属于该子智能体的能力范畴。
- **自动委派**：一旦发现子智能体**code-reviewer**的描述与“代码审查”“安全审计”等需求高度匹配，主智能体便会自动将任务上下文打包并委派给它，不需要用户显式指定子智能体名称。
- **tools**（构建安全防线的“工具白名单”）：是子智能体机制中最具安全价值的特性，它从系统底层确立了“最小权限原则”。在上述定义文件中，**code-reviewer**的白名单仅包含3个只读操作：**Read**（读取文件内容）、**Grep**（在文件中搜索特定模式）和**Glob**（匹配文件名路径）。任何未在此列出的工具（如**Edit**、**Write**、**Bash**），对该子智能体而言均不可见且不可调用。
- **model**（指定子智能体所采用的模型）：是一个常被低估的成本优化杠杆：对于简单且模式化的任务（如格式检查、日志过滤），可选用**Haiku**等轻量级、快速响应的模型；而对于需要深度理解与复杂推理的任务（如架构设计、安全审计），则应选用**Sonnet**或**Opus**等高性能模型。合理选择模型，既能有效控制成本，又能确保任务质量。
- **permissionMode**：用于控制权限模式。当设置为**plan**时，子智能体进入系统级的只读保障模式——这是一种比工具白名单更为严格的全局性安全机制；默认值**default**则沿用标准的权限控制流程。

再来看一个执行型子智能体的示例——测试运行器。与前一个示例的关键区别在于，该子智能体需要调用Bash工具执行命令，因此其工具权限更宽，但对输出格式

的要求更为严格。

```
name: test-runner description: |
```

运行项目测试套件并分析测试结果

当用户要求运行测试、检查测试覆盖率或分析测试失败原因时使用

```
tools:
```

```
- Read - Grep - Glob - Bash model: haiku
```

你是一名测试执行专家。你的核心价值是从海量测试输出中提炼关键信息，为主对话提供精准的测试摘要

执行流程

1. 确认项目的测试命令（查阅package.json或CLAUDE.md）
2. 运行测试套件
3. 分析输出结果，区分通过和失败的测试用例
4. 针对失败的测试，定位具体失败原因

输出格式（严格遵守）

测试摘要

- 总计：X个测试
- 通过：X个
- 失败：X个
- 跳过：X个

失败详情（仅列出失败的测试）

- test_name: 失败原因（用一句话描述）at file_path:line_number

建议

[如果存在明显的失败模式，请给出修复方向]

注：输出中严禁包含完整的测试日志，仅需要输出上述格式的摘要信息

请注意，该子智能体选用了Haiku作为模型。运行测试、解析输出及提取失败信息这类任务，不需要顶级的推理能力，Haiku不仅能够胜任，且在速度与成本上更具优势。这正如不需要聘请首席架构师来执行单元测试一样。

子智能体定义完成后，主要有两种使用方式。

- **自动委派**：用户只需要向Claude提出常规请求，它会根据任务性质自动判断是否需要将工作委派给特定的子智能体。
- **显式请求**：用户可以直接指示Claude调用某个具体的子智能体来完成任务。

自动委派示例如下。

用户：帮我审查src/payment/目录下的代码变更。

Claude：好的，我将把审查任务委派给“代码审查专家”。

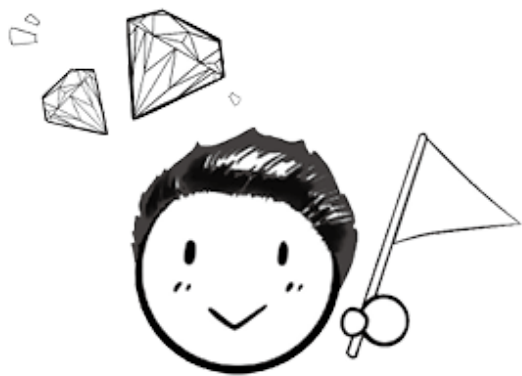
[启动子智能体：code-reviewer]

...

审查完毕。发现2个问题。

1. [严重] src/payment/processor.ts:45存在SQL注入风险。
2. [次要] src/payment/validator.ts:23缺少边界值校验。

在此过程中，code-reviewer子智能体可能读取了数十个文件并检索了多处代码模式，但所有这些细节均保留在子智能体的独立上下文中。主对话仅呈现两行精炼的审查结论。子智能体的YAML前置元数据还支持两个高级字段，可按需启用。一个是skills，它允许为子智能体预加载特定的Skill知识包。例如，让impact-analyzer子智能体预加载chain-knowledge Skill，使其在启动时即具备系统依赖关系的相关知识。另一个是hooks，它允许配置事件钩子，以便在特定时机自动执行检查或操作。关于这两个字段与Skill及Hooks组合模式的详细应用，我们将在后续章节中深入探讨。



咖哥 发言

4.4 5种子智能体模式

理解了子智能体的基本概念与配置方法后，我们正式进入本章的核心——5种子智能体模式。这绝非一份简单的分类清单，而是5种截然不同的架构范式；每一种都精准对应特定的任务特征，并深植于软件工程原则之中。

4.4.1 只读型：安全的观察者

只读型是最基础且最安全的子智能体模式，其核心特征可概括为**只读不写**。

该模式的典型应用场景包括代码审查、安全审计、架构分析及依赖检查。这些任务的共同特点在于：它们本质上是“观察”活动，需要读取海量信息并进行深度分析，但绝不应给系统带来任何副作用。正如代码审查员应指出问题而非直接修改代码，审计员应揭示财务漏洞而非擅自篡改账本。

只读型子智能体的工具权限被严格限定为Read、Grep、Glob三大核心工具。这三者涵盖了代码分析所需的全部能力：Read用于读取文件内容，Grep用于搜索特定模式，Glob用于查找符合规则的文件路径。它们的共同特征是：无论怎样调用，都不会对文件系统产生任何修改。

从软件安全视角来看，这正是**最小权限原则**的完美体现——赋予实体完成其职责所必需的精确权限，不多亦不少。在传统软件系统中，我们会为数据库只读副本（Read Replica）配置只读权限，或在Linux操作系统中利用chmod精确控制文件

的读、写、执行权限；子智能体的工具白名单机制，正是这一经典安全思想在AI Agent领域的自然延伸。

将permissionMode设置为plan模式，可提供更高层级的安全保障。在该模式下，不仅子智能体受限于只读工具集，其整个会话更被系统标记为“只读”状态。这不仅是对工具调用的限制，更是一项系统级的安全承诺，从架构层面杜绝了任何潜在的写入风险。

4.4.2 执行型：高噪声任务处理器

执行型子智能体是日常开发中应用最广泛的模式，其核心价值可概括为“信噪比优化”：输入海量数据（如500行），仅输出精炼结论（如5行）。

无论是运行测试套件产生的上千行日志、分析系统记录涉及的数万条数据，还是代码格式化触发的数百个文件变更，开发者往往只需要关注其中的关键结果：哪些测试失败、故障根源何在、具体修改了哪些内容。此类任务的共同特征在于：**过程伴随大量冗余噪声，而最终价值却高度浓缩**。若任由这些噪声涌入主对话，将导致信息的信噪比急剧下降；而通过执行型子智能体在隔离空间中处理，则能将噪声有效阻隔于“舱壁”之外，确保主对话聚焦于核心结论。

前面定义的test-runner是典型的执行型子智能体。由于它需要调用Bash工具来运行测试命令，其权限范围自然比只读型更宽。然而，这种“宽泛”是受到严格约束的——通过工具白名单机制，可以精确限定Bash的使用边界。

例如，配置Bash(npm test:*)意味着子智能体仅被允许执行以npm test:开头的命令。这一限制从根源上阻断了子智能体执行任意其他Shell操作的可能性，确保了在赋予子智能体执行能力的同时，依然牢牢掌握着系统的安全底线。

执行型子智能体的设计核心在于对**输出格式的严格约束**。在子智能体的定义文件中，必须明确划定输出的结构边界与内容范围。诸如“禁止包含完整测试日志”“仅输出摘要信息”等指令，确保了回传至主对话的内容是经过提炼的结论，而非原始的噪声数据。

从软件工程的视角来看，执行型子智能体正是**外观模式**（Facade Pattern）的典型应用。该模式包含以下要素。

- 复杂子系统：如测试框架产生的详尽日志、底层系统的交互细节。
- 简化接口：子智能体对外提供的结构化摘要。

- 客户端：主对话不需要知晓子系统内部的复杂运作，只需要通过这一“外观”接口获取简洁、可用的结果。

4.4.3 并行型：多专家 workflow

当你需要从多个独立维度对同一问题进行深度剖析时，并行型子智能体便展现出独特的威力（见图4-3）。这就好比同时派出3位侦探协同办案，各自专注于不同的线索维度，侦探A专注调查物证，侦探B负责走访人证，侦探C调查并分析监控录像。

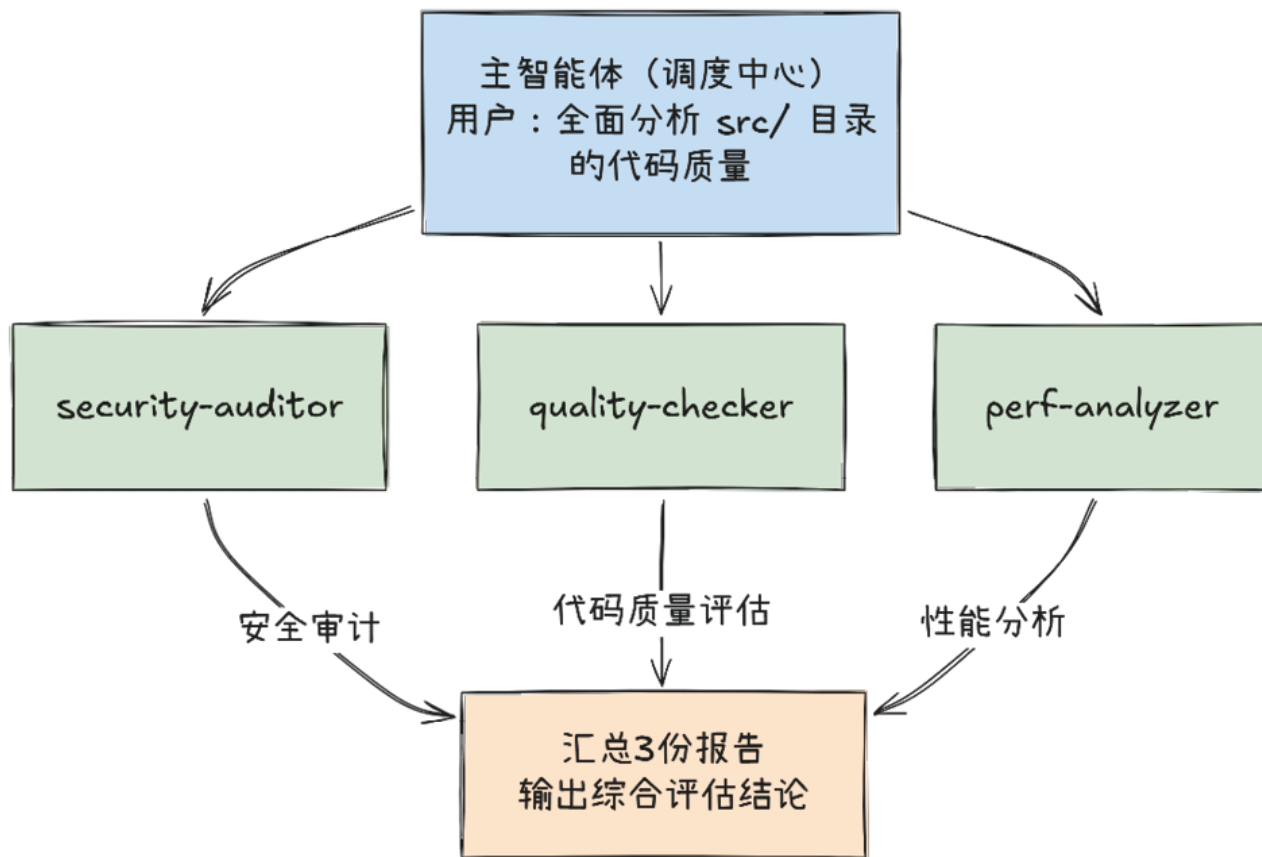


图4-3 并行型子智能体——多专家同时工作

3个子智能体在各自的隔离空间中同步开展工作，互不干扰，极大地缩短了整体分析时间。最终，系统将汇总它们各自的发现，形成一份全面、多视角的综合报告。

并行模式有一个严格的前提条件：**子任务之间必须是完全独立的**，不存在任何共享状态或依赖关系。

在这种模式下，安全审计不需要等待代码质量评估的结果；性能分析也不依赖于安全审计的结论。

这3个子智能体可以真正地在各自的隔离空间中同步运行，互不干扰，从而最大化利用计算资源并缩短整体耗时。

然而，一旦任务链中存在逻辑依赖（例如，“必须先定位bug，才能进行修复”），并行模式便不再适用。此时若强行并行，会导致后续任务因缺乏前置输入而失败或产生错误结果。针对此类具有明确先后顺序的任务流，应当采用4.4.4小节将介绍的流水线型模式，以确保执行逻辑的严谨性。

并行模式在软件工程领域有一个广为人知的对应物——**MapReduce**。在子智能体的应用场景中，这一经典架构得到了完美映射。在Map阶段，主智能体扮演“调度者”的角色，将一个宏大的复杂问题拆解为多个相互独立的子任务，并将它们分发给不同的子智能体（工作节点）。每个子智能体在隔离的环境中并行处理自己的那一部分数据或逻辑。在Reduce阶段，当所有子智能体完成计算后，主智能体再次介入，负责收集、整合并提炼各个子智能体的输出结论，最终生成一个统一、完整的回答。

在这种架构下，子智能体专注于各自独立的计算逻辑，不需要关心全局状态或其他节点的结果；而主智能体则专注于编排，即负责任务的分解与结果的聚合。

并行型子智能体的一大优势在于**专业化**。每个子智能体可承载特定的领域知识，例如，security-auditor精通OWASP Top 10，quality-checker擅长识别代码“异味”，perf-analyzer则熟悉常见的性能反模式。相比之下，一个试图“全能”的智能体在上述3个维度上可能仅能达到70分的水平；而3个“专才”型子智能体则能在各自领域达到90分的高标准。这种架构映射了软件团队的**专业化分工理念**——正如前端工程师、后端工程师与数据库管理员各司其职，其协作产出往往优于单人全栈开发的质量。

在Claude Code中，你可以使用快捷键Ctrl+B将子智能体转入后台运行。当多个子智能体同时在后台运行时，它们将实现真正的并行执行。此时主对话线程不会被阻塞，你可以继续处理其他任务，待子智能体执行完毕后再查看结果。

4.4.4 流水线型：串行处理链

并行型子智能体适用于独立任务，而流水线型子智能体则适用于**具有明确阶段依赖的任务**（见图4-4）。最经典的案例是bug修复流水线：首先定位问题根因，其次基于定位结果实施修复，接着验证修复效果，最后分析修复的影响范围。在此流程中，每个阶段的输出均为下一阶段的输入，执行顺序不可颠倒。

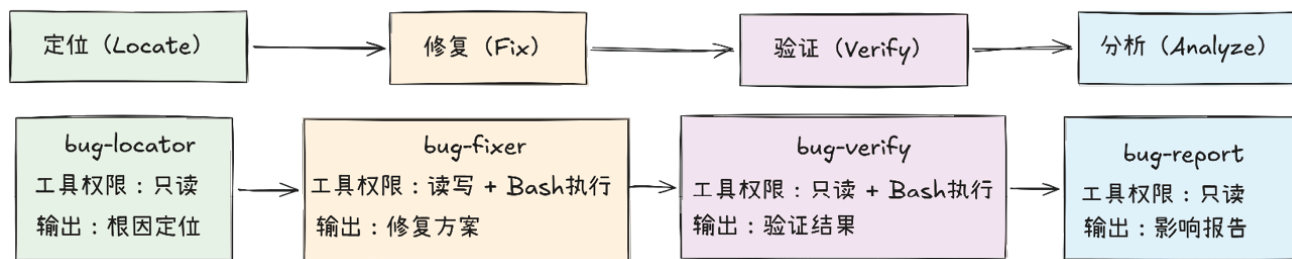


图4-4 流水线型子智能体——bug修复流水线

上述流水线具备一个微妙却至关重要的设计特征：**各阶段的工具权限截然不同**。bug-locator仅拥有只读权限，确保在定位问题时不修改任何内容；bug-fixer则具备读写及Bash执行权限，以便编辑代码并运行构建命令；bug-verify具备只读及Bash执行权限，仅需要运行测试以验证修复有效性，严禁进行任何代码变更；bug-report具备只读权限，因为影响分析仅需要查阅代码与变更历史。这种权限的动态调整是流水线型模式特有的安全机制，确保每个阶段仅持有履行其职责所必需的最小权限。

流水线型模式的关键技术细节在于**交接契约**（Handoff Contract）：每个阶段的输出格式必须与下一阶段的输入要求严格匹配。

以下是一个bug修复流水线定义示例。

```

<!-- .claude/agents/bug-locator.md --> ---

name: bug-locator description: 定位产生bug的根本原因

tools:

  - Read - Grep - Glob permissionMode: plan ---

```

你是一名bug定位专家，任务是找出产生bug的根本原因

定位流程

1. 理解症状：分析错误信息和复现步骤
2. 搜索相关代码：通过关键词和文件模式锁定可疑区域
3. 追溯调用链：从错误点向上追溯至根本原因

4. 确认根本原因：明确指出导致问题的具体代码行

输出格式（下游阶段依赖此格式，请严格遵守）

根本原因文件：[file_path:line_number]

问题描述：[一句话概述根本原因]

调用链：[从入口到出错点的完整路径]

修复方向：[简要的修复思路]

<!-- .claude/agents/bug-fixer.md --> ---

name: bug-fixer description: 基于定位结果修复bug tools:

- Read - Grep - Glob - Edit - Write - Bash ---

你是一名bug修复专家，你将接收bug-locator输出的定位结论，

并据此实施修复

修复原则

1. 最小改动：仅修改必要代码，避免无关重构
2. 不引入新问题：确保修复不会破坏现有功能
3. 风格一致：遵循项目现有的代码风格
4. 防御性代码：添加必要的防护措施，防止同类问题复发

输出格式

修改的文件：[file_path_1, file_path_2, ...]

每处修改的原因：[逐一说明]

潜在副作用：[如果有，请详细说明]

建议的测试命令：[用于验证修复的具体命令]

```
<!-- .claude/agents/bug-verify.md --> ---
```

name: bug-verify description: 验证bug修复是否有效

tools:

- Read - Grep - Glob - Bash permissionMode: plan ---

你是一名bug验证专家。你将收到bug-fixer的修复结论，

基于此运行测试验证修复是否有效

验证流程

1. 阅读修复报告：了解本次修复涉及的文件变更及核心修复思路
2. 运行建议的测试命令：运行bug-fixer提供的具体验证命令
3. 进行回归测试：确保本次修复未对其他既有功能造成副作用
4. 实施边界检查：针对已识别的根本原因，构造边界条件下的输入数据，验证防御性代码是否按预期生效

输出格式（下游阶段依赖此格式，请严格遵守）

验证结果：[通过/未通过]

测试执行记录：[列出已执行的具体命令及其对应的执行结果]

回归影响：[说明本次修复是否导致其他测试用例失败]

遗留风险：[如有，请明确指出；如无，可填写“无”]

```
<!-- .claude/agents/bug-report.md --> ---
```

name: bug-report description: 分析修复的影响范围并生成报告

tools:

- Read - Grep - Glob permissionMode: plan ---

你是一名bug影响分析专家。请基于接收到的前三阶段完整结论，

撰写一份结构化的修复报告

分析流程

1. 回溯根本原因：依据bug-locator的结论，提取根本原因
2. 审查修复：结合bug-fixer的结论，明确代码改动的具体范围
3. 确认验证：参考bug-verify的结论，核实修复的最终状态
4. 评估影响：深入分析该bug可能波及的其他模块及用户场景

输出格式

Bug 摘要：[一句话精准概括]

根本原因：[文件路径+具体原因]

修复方案：[改动要点摘要]

验证状态：[通过/未通过]

影响范围：[涉及的模块、API及用户场景]

后续建议：[是否需要通知下游团队、更新技术文档等]

值得注意的是，bug-locator 的输出格式中明确包含“根本原因文件”和“修复方向”，这正是bug-fixer启动工作所必需的关键输入；而bug-fixer的输出则涵盖“修改的文件”与“建议的测试命令”，恰好为bug-verify的验证工作提供了核心依据。这种[上下游之间严密的格式契约](#)，构成了流水线型模式可靠运行的基石。

在软件工程领域，这一机制对应着[责任链模式](#)（Chain of Responsibility）：请求沿着处理链依次传递，每个处理器在完成自身职责后，将结果移交至下一环节。Unix/Linux操作系统中的管道（pipe）机制也体现了同样的思想，如命令cat log.txt | grep ERROR | sort | uniq -c | sort -rn，其中每个命令仅专注于单一任务，前一个命令的输出直接作为下一个命令的输入。流水线型子智能体正是这一经典设计思想在AI Agent领域的重现与演进。

流水线型子智能体的实际执行过程如下。

用户指令：修复 issue #123报告的支付失败问题

主智能体协调过程：

1. 委派 bug-locator（独立上下文）

任务：分析问题根源

结论：src/payment/charge.ts:89 处缺少空值检查

2. 传递至bug-fixer（独立上下文）

输入：上述定位结论

任务：实施代码修复

结论：已在charge.ts:89添加空值检查逻辑

3. 传递至bug-verify（独立上下文）

输入：修复后的文件及建议的测试命令

任务：运行测试验证修复有效性

结论：支付模块全部12个测试用例通过

4. 传递至bug-report（独立上下文）

输入：完整的修复与验证记录

任务：分析影响范围并生成报告

结论：修复范围可控，无副作用，建议后续补充null值边界测试

最终结果：4个子智能体各自在隔离的上下文中完成了深度工作，主对话仅接收并展示了4段简洁、结构化的关键结论

4.4.5 团队型：自组织协作机制

团队型模式是子智能体模式中架构最复杂、能力最强大的一种。它与前面介绍的4种模式存在本质区别：在前4种模式中，子智能体仅在任务执行期间活跃，任务完

成后即终止，呈现出“一次性”的特征；而在团队型模式中，子智能体具有长期持续性，它们如同真实团队一般，能够持续协作、实时通信并自主分工。

一个Agent Team由3个核心元素组成（见图4-5）。

- **Team Lead（团队负责人）**：承担任务分解、资源调度及进度跟踪的职责，其角色类似于项目经理。
- **Teammate（团队成员）**：由具有不同专业特长的子智能体组成，它们各自负责特定的功能模块。
- **共享基础设施**：包括两个关键机制。Task List（任务列表）确保所有成员能实时掌握整体进度与个人待办事项；Mailbox（邮箱）则支持成员间的异步通信（例如，前端开发人员可通过该机制向后端开发人员确认API接口规范）。

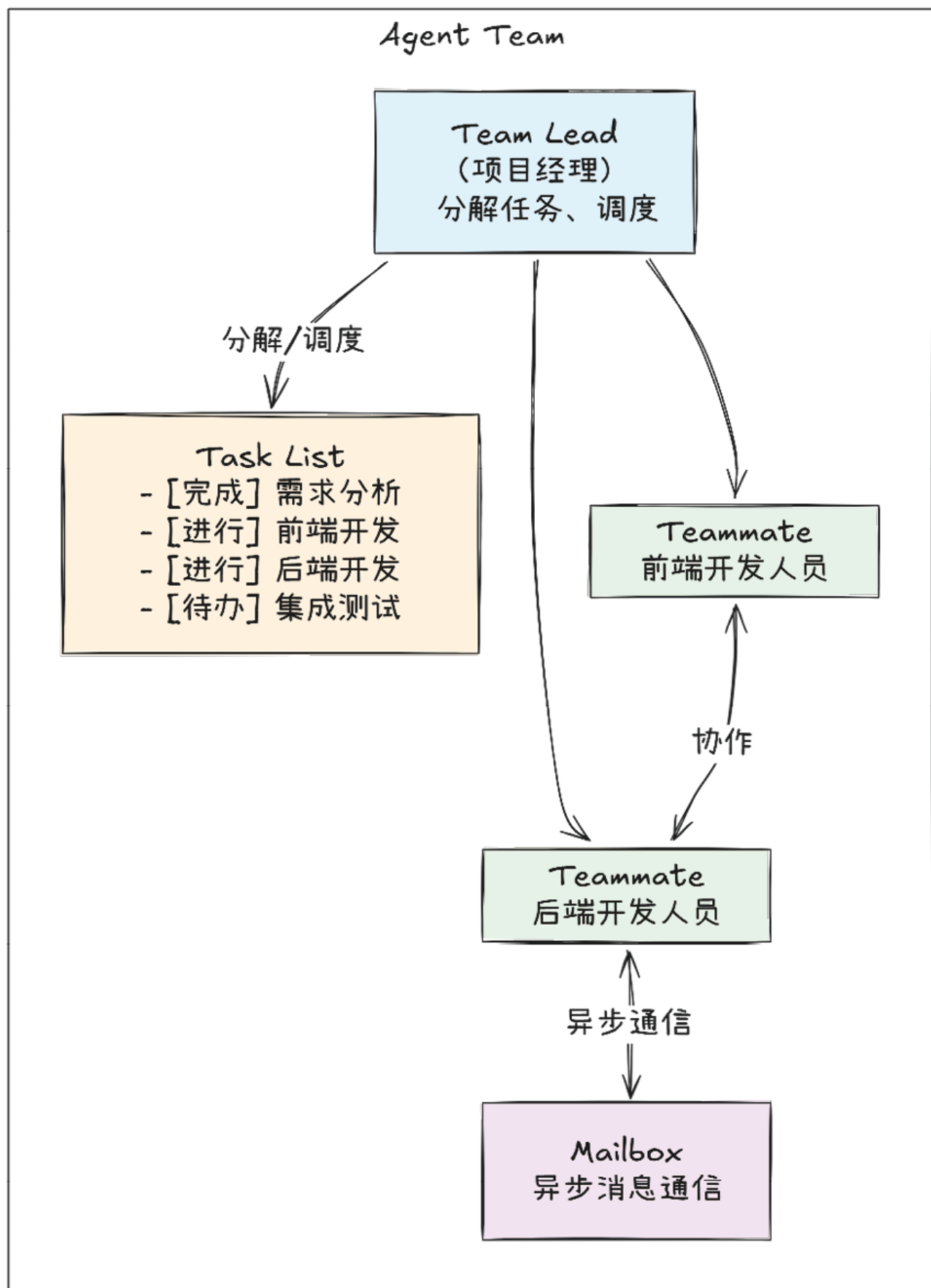


图4-5 团队型子智能体——多会话自组织协作

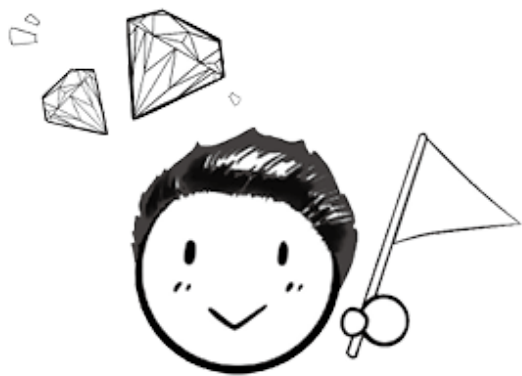
团队型子智能体适用于4种典型的协作模式。

- **竞争假设模式**：面对难以定论的复杂问题（例如，性能瓶颈究竟源于数据库查询还是网络I/O），可以派遣多个子智能体基于不同的假设方向并行调查。最终，由Team Lead对比多份调查报告，择优选取证据最充分的结论。
- **并行审查模式**：针对同一份代码变更，可同时安排安全审查与功能审查。两位审查者独立作业、互不干扰，最后合并审查意见。该模式不仅比串行审查更高效，而且多视角的独立性也能显著提升审查质量。
- **模块归属模式**：在大型重构任务中，将代码库按模块划分并分配给不同的团队成员。例如，前端子智能体负责UI（User Interface，用户界面）组件重构，后端子智能体负责API层重构，数据层子智能体负责数据库迁移。各子智能体深耕其负责的模块，并通过Mailbox协调跨模块的接口变更。
- **方案审批模式**：由一个子智能体提出重构方案，另一个子智能体则扮演“魔鬼代言人”角色，专门提出挑战与质疑。Team Lead依据双方的论证做出最终决策。这种对抗性设计能有效规避方案中的思维盲点。

何时采用团队型模式而非简单的子智能体模式？一个实用的判断标准在于协作的复杂度：若**子智能体之间需要多轮通信与深度协调**，则应选用团队型模式；若每个子智能体仅需要执行一次任务即可完结，简单的并行型或流水线型模式便已足够。

值得注意的是，团队型子智能体的运行开销更高——长期维持的上下文窗口意味着持续的Token消耗。因此，仅在确实需要复杂协作的场景下，采用该模式才具备成本效益。5种模式的选择并非互斥，它们可以灵活组合使用。例如，在流水线型模式的某个阶段内部，可以嵌入并行型模式，

例如，在bug-locator阶段，同时派遣“日志分析器”与“代码搜索器”并行工作以加速定位。然而，组合的层级不宜过深。嵌套超过两层会显著增加系统的复杂性与调试难度。如果你发现需要3层以上的嵌套，这通常是一个信号，表明当前的任务分解方式可能需要重新设计。对于大多数日常开发场景，通常仅使用只读型和执行型模式就已足够。切记不要过度设计。



咖哥 发言

4.5 子智能体与Skills的协作

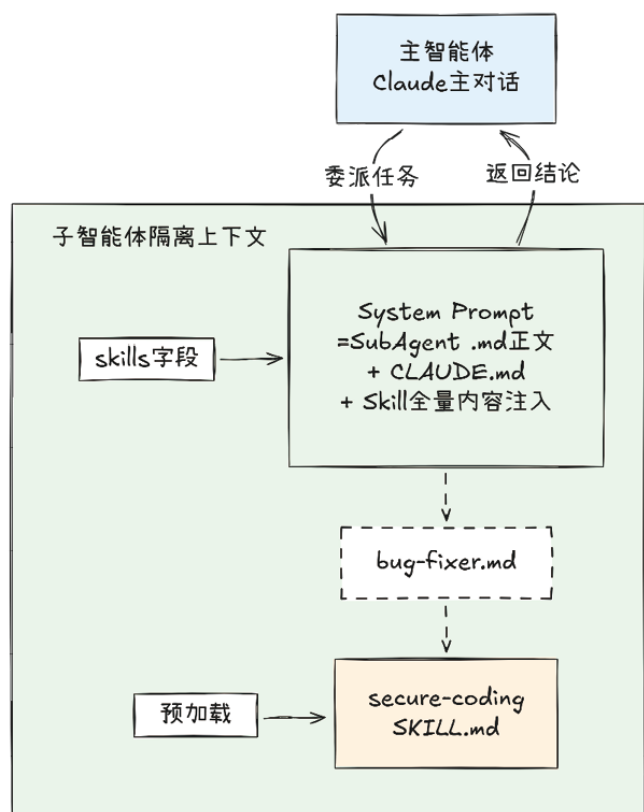
在4.4节介绍的5种子智能体模式中，子智能体虽展现出卓越的任务分解与隔离能力，却回避了一个核心问题：子智能体的“专业知识”究竟源自何处？

以流水线型模式中的bug-fixer为例，它被委派修复一个空值检查bug，虽配备了Read、Edit、Write、Bash等全套工具，却对项目的编码规范一无所知——错误处理应采用Result模式还是try-catch语句？日志记录该调用console.log还是项目专用的logger？变量命名遵循camelCase（驼峰式命名法）还是snake_case（蛇形命名法）？

若将这些规范写入项目的CLAUDE.md中，子智能体确实能够自动继承。然而，CLAUDE.md通常仅涵盖通用规范，难以囊括诸如“bug修复时应遵循的安全编码清单”这类深层专业知识——后者显然属于Skills的范畴。

Skills与子智能体的组合架构仅存在两种原子模式，其核心区别在于包含关系（见图4-6）。

方向 A：子智能体预加载 Skill（skills 字段）



方向 B：Skill 派生子智能体（context: fork）

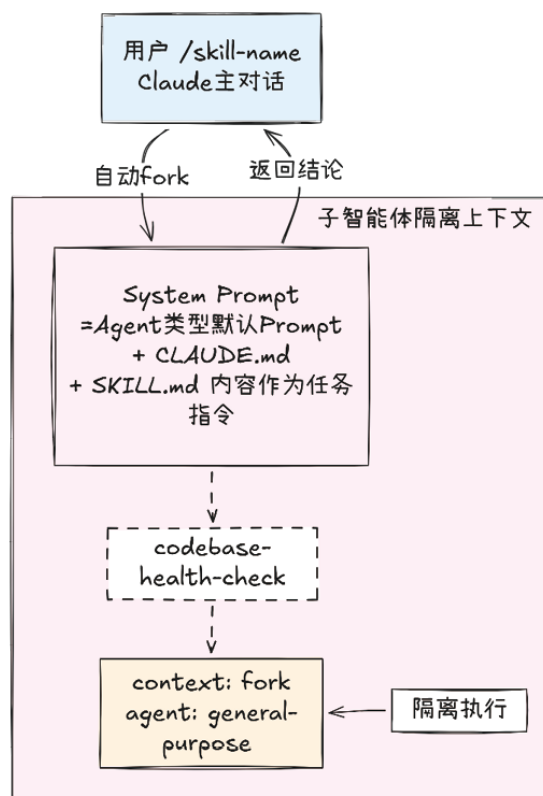


图4-6 Skills与子智能体的组合架构

方向A：子智能体预加载Skill（子智能体包含Skill）

在此模式下，子智能体的定义文件通过skills字段，在启动阶段预加载一个或多个Skill作为领域知识库。

- 角色定位：子智能体是执行者，而Skill则是其手中的“操作手册”。
- 适用场景：流水线中需要特定领域知识的角色。例如，bug-fixer需要加载安全编码规范，doc-generator则需要加载文档生成流程。

方向B：Skill派生子智能体（Skill包含子智能体）

在此模式下，Skill通过配置context: fork，在被触发时自动创建一个隔离的子智能体来执行任务，确保中间过程不污染主对话上下文。

- 角色定位：Skill是调度者，子智能体是执行实例。

- 适用场景：深度代码分析、批量文档生成等需要严格隔离且一次性完成的重型任务。

这两种模式的核心差异在于System Prompt的控制权归属。

- 方向A：子智能体的定义文件（.md正文）构成System Prompt，而Skill的全量内容被作为领域知识上下文注入。
- 方向B：由agent字段指定的Agent类型提供System Prompt，而SKILL.md的内容则作为具体的任务指令传入。

尽管两者的底层机制完全一致，但其调用关系截然相反。

接下来我们通过两个实例深入解析方向A。首先是改进版的bug-fixer。

```
<!-- .claude/agents/bug-fixer.md --> ---
```

```
name: bug-fixer description: 基于定位结果修复bug，遵循团队安全编码规范
```

```
tools:
```

```
  - Read - Grep - Glob - Edit - Write - Bash skills:
```

```
  - secure-coding # 预加载安全编码Skill ---
```

你是一名bug修复专家。你将接受来自bug-locator的定位结论，

并据此执行修复

在修复过程中，必须严格遵循secure-coding Skill中定义的安全编码规范。

请特别注意以下关键点：空值检查、输入验证以及错误处理模式

修复原则

1. 最小改动：仅修改必要的代码，避免无关的重构
2. 零副作用：确保修复不会破坏既有功能
3. 规范遵从：严格遵循Skill中定义的安全编码规范
4. 防御性编程：添加防御性代码，防止同类问题复发

输出格式

修改的文件: [file_path_1, file_path_2, ...]

修改原因: [逐一说明每处修改的理由]

安全检查项: [列出Skill清单中已确认通过的检查项]

建议的测试命令: [用于验证修复的具体命令]

对应的Skill定义如下。

```
<!-- .claude/skills/secure-coding/SKILL.md --> ---
```

```
name: secure-coding description: Secure coding checklist for bug fixes and new features.
```

```
    Use when fixing bugs, writing new code, or reviewing security aspects.
```

```
user-invocable: false ---
```

安全编码规范

空值防御

- 输入校验: 所有外部输入必须进行非空检查
- 安全访问: 优先使用可选链操作符 (?.) 访问嵌套属性, 严禁使用非空断言 (!.)
- 数据判空: 数据库查询结果在业务逻辑使用前, 必须显式进行判空处理

错误处理

- 统一异常: 业务逻辑异常必须使用项目统一的AppError类, 禁止直接抛出原生异常或字符串
- 拒绝静默失败: 严禁空的catch代码块, 捕获异常后, 至少需要记录错误日志或进行重试处理
- 超时控制: 所有异步操作必须配置明确的超时时间

日志规范

- 专用接口: 生产环境严禁使用console.log()。必须使用项目封装的logger对象 (如logger.error())

- 信息完整：错误日志必须包含唯一的error code以及完整的堆栈跟踪（stack trace）
- 数据脱敏：严禁在日志中输出任何敏感信息（包括但不限于密码、API Token）

需要注意的是，user-invocable: false表示该Skill不需要在用户的“/”菜单中显示，它属于纯粹的“工具书”型内容，将在子智能体启动时自动预加载。

再来看一个跨领域的实例——API文档生成子智能体，其通过预加载“文档生成Skill”来强化专业能力。

```
<!-- .claude/agents/api-doc-generator.md --> ---
```

```
name: api-doc-generator description: Generate API documentation by scanning route files.
```

```
tools:
```

```
  - Read - Grep - Glob - Write - Bash skills:
```

```
  - api-generating ← 预加载Skill以注入领域知识
```

```
----
```

```
You are an API documentation specialist.
```

```
Follow the api-generating Skill for workflow and templates.
```

其中，skills: [api-generating]将一本“操作手册”递到了子智能体手中。凭借明确的角色定位（API文档专家）以及内嵌于Skill的专业工作流（包括具体步骤、模板与脚本），子智能体得以从“样样通却样样松”的通用型Agent，蜕变为“术业有专攻”的领域专家。

这两个示例——bug-fixer搭配secure-coding，以及api-doc-generator搭配api-generating，揭示了一个清晰的职责划分原则：子智能体定义“身份与目标”，Skill定义“方法与标准”。

具体分工如下。

子智能体（.md配置文件）负责战略层面。

- Who（身份）：确立角色定位，如“你是一名漏洞修复专家”或“你是一名API文档专家”。

- **What（任务）**：明确核心目标，如“修复漏洞”或“生成API文档”。
- **Where（范围）**：界定工作边界，如“写入docs/api/目录”。
- **Output（交付）**：规定产出形式，如“返回包含修复详情的摘要”或“统计路由数量”。

Skill（SKILL.md及附属文件）负责战术执行。

- **How（流程）**：细化操作步骤，如“第一步：运行detect-routes.py→第二步：分析结果”。
- **With What（工具）**：指定依赖资源，如脚本scripts/detect-routes.py、模板templates/api-doc.md。
- **By What Standard（规范）**：设定执行准则，如“检查认证中间件并标记为锁”。
- **Quality（质量）**：明确验收标准，如“所有路由均已归档，Schema与代码严格一致”。

这种分离机制赋予了系统强大的**复用能力**。同一个secure-coding Skill，既可供bug-fixer预加载以指导bug修复，也可被code-reviewer调用以审查代码合规性；同理，api-generating Skill不仅能辅助文档生成子智能体创建新文档，也能赋能API审查子智能体检验现有文档的规范性。简而言之，Skill是可复用的知识模块，而子智能体则是这些知识的灵活消费者。

再看方向B——Skill派生子智能体。通过在SKILL.md的YAML前置元数据中设置context: fork，Skill在被触发时会自动创建一个隔离的子智能体来执行任务。这种机制确保了中间过程不会污染主对话的上下文环境。

```
name: codebase-health-check description: Perform comprehensive code health analysis context:
fork agent: general-purpose allowed-tools: - Read - Grep - Glob ---
```

Analyze the codebase at \$ARGUMENTS and produce a health report.

用户输入“codebase-health-check src”后，系统自动在隔离上下文中派生出一个子智能体。该子智能体能够独立扫描所有文件、分析问题并生成报告，而主对话仅接收一份精炼的最终结果，中间产生的大量文件内容和推理过程完全不会污染主上下文。

方向A与方向B各有适用场景，判断标准非常直观：谁是主角？

方向A：角色驱动（子智能体是主角）。

- 核心逻辑：任务的关键在于特定的角色身份与协作流程。
- 场景示例：流水线中的bug-fixer需要持续运用专业知识来修复漏洞。此时，子智能体是长期存在的“专家”，Skill只是它工具箱里的“扳手”。
- 结论：当需要维持角色状态、进行多轮交互或复杂协作时，选择方向A。

方向B：知识/流程驱动（Skill是主角）。

- 核心逻辑：任务的关键在于执行一套标准的知识流程，而执行者本身并不重要，甚至需要被隔离。
- 场景示例：codebase-health-check是一套标准的检查流程。此时，Skill定义了“怎么做”，子智能体仅仅是为了隔离上下文而临时生成的“执行容器”。
- 结论：当任务是单次性的，需要严格隔离上下文，或者重点在于复用标准化流程时，选择方向B。

4.6 Token经济学

“咖哥，执行子智能体会增加API调用成本吗？”小冰问了一个非常务实的问题。

这个问题的答案可能出人意料：在大多数场景下，合理使用子智能体不仅不会增加成本，反而会显著降低总Token消耗。

这背后的“Token经济学”原理值得深入剖析。

首先分析不使用子智能体的情形。假设主Claude直接执行测试任务，产生了10 000 Token的测试输出。由于Claude的API通常是无状态的，每次调用均需要传输完整的上下文历史。因此，一旦这10 000 Token的测试内容进入主对话上下文，在后续的每一轮交互中都会被重复发送。若此后又进行了5轮对话，这部分测试产生的“噪声”数据便会被额外重复传输5次。

不使用子智能体的成本测算如下。

假设条件：

System Prompt: 5000 Token 既有对话历史: 8000 Token 测试输出 (噪声): 10 000 Token 后续5轮对话, 每轮新增内容约1000 Token 各轮次Token消耗明细:

第5轮: 5000 (系统) + 8000 (历史) + 10 000 (测试) = 23 000

第6轮: 5000 (系统) + 9000 (历史+新) + 10 000 (测试) = 24 000

第7轮: 5000 (系统) + 10 000 (历史+新) + 10 000 (测试) = 25 000

第8轮: 5000 (系统) + 11 000 (历史+新) + 10 000 (测试) = 26 000

第9轮: 5000 (系统) + 12 000 (历史+新) + 10 000 (测试) = 27 000

第5轮~第9轮累计消耗: 125 000 Token 其次看使用子智能体的情形。在此模式下, 测试输出被严格隔离在子智能体的独立上下文中, 主对话仅接收一份约100 Token的测试摘要。

使用子智能体的成本测算如下。

子智能体内部 (独立上下文):

第1轮: 5000 (系统) + 500 (任务描述) = 5500

第2轮: 5000 (系统) + 500 (任务) + 3000 (部分输出) = 8500

第3轮: 5000 (系统) + 500 (任务) + 6000 (累积输出) = 11 500

第4轮: 5000 (系统) + 500 (任务) + 10 000 (完整输出) = 15 500

子智能体累计消耗: 41 000 Token

主对话 (仅收到100 Token摘要):

第5轮: 5000 (系统) + 8000 (历史) + 100 (摘要) = 13 100

第6轮: 5000 (系统) + 9000 (历史+新) + 100 (摘要) = 14 100

第7轮: 5000 (系统) + 10 000 (历史+新) + 100 (摘要) = 15 100

第8轮: 5000 (系统) + 11 000 (历史+新) + 100 (摘要) = 16 100

第9轮：5000（系统）+12 000（历史+新）+100（摘要）=17 100

主对话累计消耗：75 500 Token

总体成本统计：

总消耗：41 000（子智能体）+75 500（主对话）=116 500

节省Token数：125 000-116 500=8500

节省比例：约6.8%

趋势分析：随着后续对话轮数的增加，由于避免了大量噪声数据的重复传输，Token节省比例将显著提升

在此示例中，仅经过5轮后续对话便实现了6.8%的成本节省。如果将对话延伸至10轮或20轮，节省比例将进一步显著提高。在特定极端场景下（如在完成海量日志分析后继续进行多轮深度研讨），Token节省率甚至超过50%。

当然，必须指出的是，Anthropic推出的Prompt Caching（缓存）机制会在一定程度上削弱上述成本优势。该机制对缓存命中的Token仅收取标准价格的10%，这意味着主对话中重复传输的测试输出数据，在命中缓存时其实际成本已大幅降低。纳入缓存因素重新测算后，本例中的成本节省率将从6.8%回落至1%~2%。

然而，成本优化仅仅是子智能体价值的一个维度，甚至并非其核心价值所在。还有两个关键维度的优势，是Prompt缓存机制无法替代的。

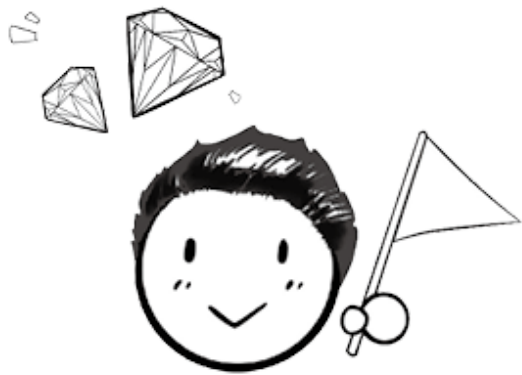
- **上下文窗口保护。**Claude的上下文窗口容量是有限的。当大量噪声数据挤占存储空间时，Claude可能会提前触发上下文压缩（Context Compaction）机制。在这一过程中，关键信息极易因被判定为“低优先级”而丢失。子智能体通过将噪声严格隔离在独立上下文中，有效为主对话保留了宝贵的上下文空间，确保了核心信息的完整性。
- **响应质量提升。**纯净的上下文环境能显著提升Claude的聚焦能力。当对话历史中充斥着无关的日志记录和测试输出时，Claude的“注意力机制”会被分散，导致生成质量下降——这正是小冰最初面临的典型困境。子智能体的隔离架构从根源上解决了“注意力稀释”问题，确保主模型始终基于高质量、高相关性的信息进行推理与回答。

那么，在何种场景下引入子智能体反而得不偿失？这里有一个简明扼要的决策准则：**审视输入与输出的体量比**。

- 高价值场景（输入>>输出）：当任务的输入数据量远超最终产出时，子智能体的价值尤为显著。例如，分析数百行系统日志仅为了提炼出5行关键结论。

- 低价值场景（输入≈输出）：当输入与输出的体量大致相当时，子智能体的隔离开销便显得不再划算。例如，修改单个函数或编写一段代码注释，其处理过程中产生的中间态数据量有限。在此类场景中，直接在主对话中完成任务往往更为简单、高效，不需要承担额外的架构复杂度与通信成本。子智能体的启用决策可归纳为以下4个核心维度。

- 大规模文件读取（输入噪声隔离）：此时任务需要读取大量文件（通常超过5个）或处理海量数据源。使用子智能体可将文件读取过程产生的“输入噪声”完全隔离在独立环境中，避免主对话上下文被冗余的文件内容填满。
- 高频输出生成（输出噪声过滤）：任务预期将产生大量中间输出，如全量测试报告、详细系统日志或完整的构建过程记录。子智能体充当了“过滤器”的角色，仅向主对话返回精简的摘要或关键错误信息，从而防止冗长的输出数据稀释主模型的注意力并浪费Token。
- 上下文完整性保护（长期任务规划）：当前任务仅为复杂工作流的一环，需要确保主对话保留充足的上下文空间以支持后续的多轮深度交互或推理。通过将高消耗的子任务剥离，子智能体机制有效保护了主对话的“上下文窗口”不被一次性耗尽，确保了长期任务的连贯性与信息完整性。
- 操作权限与安全边界（风险管控）：任务涉及敏感操作（如文件系统写入、网络请求、执行外部命令）或需要严格的权限控制。子智能体可被配置为具有受限权限的独立沙箱环境。即使子任务执行失败或产生恶意行为，其影响也被限制在局部范围内，从而为主系统提供了天然的安全防火墙。



咖哥
发言

4.7 从软件工程看子智能体

读至此处，敏锐的读者或许已洞察到：子智能体架构并未凭空创造任何颠覆性的新概念。其核心本质，是将软件工程领域数十年来沉淀的经典智慧，创造性地迁移并映射到了AI Agent的架构设计之中。

这种迁移绝非偶然，而是必然的逻辑演进。因为AI Agent本质上就是一个复杂的软件系统，它必然遵循软件系统工程的一般规律与核心原则。

1.单一职责原则

单一职责原则（Single Responsibility Principle）位居面向对象设计五大原则之首。其核心定义：一个类（或模块）应当只有一个引起它变化的理由——换言之，它应且仅专注于做好一件事。子智能体架构正是这一经典原则在AI Agent领域的完美复刻与践行。

- 职责专一化：**code-reviewer**仅专注于代码质量审查；**test-runner**仅负责执行测试用例并收集结果；**bug-locator**仅致力于定位产生bug根本原因。每个子智能体都如同一个高内聚的独立类，各司其职，互不越界。
- 声明式职责说明书：每个子智能体的Prompt文件（如**code-reviewer.md**）实质上就是它的“职责说明书”。
- 变更隔离与维护性：例如，若需要调整代码审查的严格程度或引入新的安全规范，开发者仅需修改**code-reviewer.md**这一文件。

2.舱壁模式

舱壁模式（Bulkhead Pattern）源于造船工程的智慧：货轮的船体被坚固的隔板划分为多个独立的水密舱。即便某一舱室不幸进水，海水也被限制在局部，绝不会导致整艘巨轮沉没。

在微服务架构中，这一模式被广泛采纳以防止单点故障引发级联雪崩。而在 AI Agent架构中，子智能体的上下文隔离机制正是舱壁模式的完美映射。

- 故障熔断与边界锁定：如果**log-analyzer**子智能体在处理海量日志时遭遇异常数据（如死循环、幻觉爆发或格式崩溃），由此产生的“故障”会被严格封锁在子智能体的独立沙箱内。
- 主系统稳定性保障：由于“舱壁”的存在，子任务的混乱不会污染主对话的上下文环境。

- 优雅降级：这种设计确保了系统的韧性。

3. MapReduce模式

MapReduce模式源于分布式计算领域，由Google于2004年在相关论文中首次提出。该模式将大规模数据处理分解为Map（映射）和Reduce（归约）两个阶段：Map阶段负责将任务分发至多个工作节点进行并行处理，Reduce阶段则负责汇总所有节点的计算结果。

并行型模式正是MapReduce模式在AI Agent领域的创新应用。在该模式中，主智能体承担协调者角色，负责任务的分发（Map）与结论的汇总（Reduce），各个子智能体则作为工作节点，专注于执行并行的Map计算任务。

4. 责任链模式

责任链模式（Chain of Responsibility）源于GoF设计模式，其核心定义了一条处理链，请求沿链传递直至被某个处理器接收并处理。流水线型模式是该模式的变体：不同于传统责任链中由“某个”处理器终结请求，流水线型模式要求“每个”处理器均对请求进行部分处理，并将中间结果传递给下一环节。Unix管道、Java Servlet过滤链（Filter Chain）以及Node.js Express中间件，皆是这一思想在不同技术栈中的具体化身，而流水线型模式正是这一家族中最新的成员。

这些对应关系并非牵强附会的类比，而是揭示了一条深层规律：**优秀的架构原则具有跨领域的普适性**。无论是设计微服务系统、分布式计算框架，还是构建AI Agent架构，开发者面临的根本挑战始终如一——如何分解复杂性、如何隔离故障，以及如何实现高效协作。深谙经典设计模式的工程师在涉足AI Agent设计时具备天然的直觉优势，因为他们已在其他领域反复验证了这些思想的有效性。

小雪听罢，若有所思地点头道：“原来学习设计模式不只是为了应付面试，这些思想在AI时代反而更有用了。”

咖哥点头道：“软件工程的核心智慧从未过时，它们只是在每一轮新的技术浪潮中，找到了全新的表达形式。”

4.8 实战注意事项

在实际使用子智能体时，几个容易被忽视的细节值得提前了解，它们往往决定了子智能体能否在生产环境中稳定工作。

首先是CLAUDE.md的继承关系。项目根目录下的CLAUDE.md对所有子智能体生效——子智能体启动时会自动加载这份“项目记忆”。这意味着你在其中定义的编码规范、技术栈约定及命令别名，子智能体均会遵循。然而，子智能体自身的定义文件可以对CLAUDE.md中的规则进行补充甚至覆盖。其优先级顺序为：子智能体定义文件>项目级CLAUDE.md>用户级CLAUDE.md。

其次是上下文的“报文传输”模式。子智能体之间无法直接通信：每个子智能体仅能获取主智能体显式传递的内容，既无权访问主对话的历史记录，也无法感知其他子智能体的存在。这种信息交互机制属于“报文传输”而非“共享内存”——主智能体必须将子智能体A的结论提取出来，并嵌入子智能体B的任务描述中，子智能体B方能获得相关信息。深刻理解这一机制对于设计高效的流水线至关重要：务必确保每个阶段的输出格式具备高度的结构化特征，以便主智能体能够准确提取并顺利转发。

然后是中断恢复机制。若子智能体在执行过程中因网络波动或手动终止而被中断，其内存中的中间工作成果将会丢失。针对耗时较长或关键任务，一种行之有效的策略是让子智能体将中间产物持久化至文件。例如，日志分析子智能体可将已分析的区间范围及发现的异常实时记录至`.claude-work/log-analysis.md`中。当任务需要重启时，新的子智能体可先读取该文件，精准定位断点并接续工作，从而避免从头开始的重复劳动。

最后是嵌套深度控制。虽然子智能体支持相互调用，但嵌套层级建议控制在两层以内。每增加一层嵌套，不仅会加剧信息传递过程中的损耗，更会使得调试难度呈指数级上升。如果发现业务逻辑需要3层以上的嵌套，这通常意味着任务分解的粒度存在缺陷——此时应重新架构设计，将深层嵌套“扁平化”，转而采用更高效的并行处理或流水线结构。

本章小结

子智能体是Claude Code扩展体系中最具架构意义的特性。如果说CLAUDE.md赋予了Claude记忆能力，Skills赋予了它专业知识，那么子智能体所赋予的则是一种更为根本的能力——**分解复杂性的能力**。当面对涉及多个维度、伴随大量噪声且需要多阶段处理的复杂任务时，子智能体提供了一种结构化的应对策略：将整体任务拆解为多个聚焦明确的子任务，在相互隔离的上下文中独立执行，并最终仅将提炼后的核心结论汇聚回主对话。

这种策略的价值不仅体现在技术层面（如上下文隔离、权限约束和Token利用效率的优化）上，更深层次地体现在思维方式上。学会使用子智能体，本质上是掌握了一种“委派式思考”：不再事必躬亲地将所有细节塞进单一对话，而是像一名成熟

的管理者那样，清晰界定每个任务的职责边界、能力要求与交付标准，并将其委派给合适的执行者。这种思维方式与软件工程中的模块化设计、微服务架构以及关注点分离原则一脉相承——它们共同回应着一个永恒的挑战：如何有效驾驭复杂性。

5种子智能体模式（只读型、执行型、并行型、流水线型与团队型）并非彼此孤立的技巧，而是一个层层递进的能力谱系。从最基础的只读观察，到最高阶的复杂多会话自组织协作，它们全面覆盖了从日常开发任务到大型工程项目的各类场景。一旦掌握了这5种模式及其背后的软件工程原则，你便拥有了在任何新场景下构建适配性子智能体架构的直觉与能力。

在第5章中，我们将深入探讨另一个核心机制——基于[Hooks的事件驱动自动化](#)。如果说Skills与子智能体主要作用于Claude的“认知层面”，那么Hooks则将其控制力延伸至“系统执行层面”：它不再局限于指导Claude“如何思考”，而是直接在物理执行层面对其行为进行拦截与约束。

思考题₁

假设你的团队需要构建一套自动化的PR审查流程，要求同步执行代码质量检查、安全漏洞扫描及性能分析，并最终整合为一份综合审查报告。请据此设计子智能体架构：需要部署几个子智能体？应采用何种模式（并行型还是流水线型）？各子智能体的工具权限又该如何配置？

2. 在4.6节的Token经济学分析中，我们基于“后续存在5轮对话”的假设展开了讨论。现在进一步思考：如果任务执行后并无后续交互（即仅为单次执行），子智能体架构是否仍具备成本优势？在何种特定条件下，引入子智能体反而会导致Token消耗激增，从而推高整体成本？

3. 回顾本章内容，我们深入探讨了5种子智能体模式，并剖析了其背后所依托的软件工程核心原则。现在，请将视角转向你的实际项目：是否存在某个痛点场景，特别适合引入子智能体架构来解决？若存在，该场景契合哪种模式？请结合具体业务逻辑，阐述选择该模式的深层理由。

第5章 防微杜渐：Hooks事件驱动自动化

凡事预则立，不预则废。

公司里发生了这样一件事。

那是一个深夜，小张正在处理一个紧急需求。他用Claude Code编写代码，本地测试通过后，便执行了git push命令将代码推送至远程仓库，随即关机回家休息。

次日早晨九点，团队群消息瞬间炸锅：“谁把env文件推上去了？API key已暴露，所有密钥必须立即作废并重发！”

小张并非不知道env文件严禁提交，只是在深夜加班的疲惫状态下，疏忽了提交前的检查。然而，那个被误提交的env文件中，竟包含3个环境的数据库连接字符串、一个内部API密钥和一个支付网关的测试账号密码。这一失误导致整个团队耗费半天时间，才完成密钥轮换、访问日志排查以及所有引用位置的修正。

小冰作为他的直接领导，第一反应是：“这种错误明明可以自动拦截，为什么没有任何机制能阻止它？”

“因为之前的所有机制，仅仅是在‘建议’Claude该做什么。”咖哥解释道，“CLAUDE.md是建议，Skills是指导，但从来没有人告诉Claude‘这件事你绝对不能做’。而Hooks不同，它不是建议，而是强制执行。”

小雪随即接话：“就像Web开发里的中间件？在请求到达之前先过一道检查？”

“准确。”咖哥点了点头，“无论是Express的中间件链、Django的请求处理层，还是Spring的拦截器，它们的核心逻辑都是一致的：在操作执行的前后，插入额外的处理逻辑。Hooks正是这一思想在AI Agent场景中的落地实现。”

5.1 Hooks在Claude扩展体系中的定位

在深入分析技术细节之前，我们先将Hooks置于已学的扩展体系中进行定位。

前4章介绍的3种机制构成了一条“影响力递增”的谱系：CLAUDE.md确立项目规范，由Claude在每次对话启动时读取；Skills封装领域工作流，可在Claude需要时自动或手动触发；而子智能体则通过隔离上下文来实现任务委派。

这3种机制存在一个共同特征：它们均作用于Claude的认知层面。CLAUDE.md告诉Claude“应遵守何种规范”，Skills指导其“遇到此类问题该如何处理”，子智能体则指示其“应将子任务委派给谁”。然而，这些本质上仍属于建议性指导。作为语言模型，Claude在理论上具备忽略Prompt中任何约束的能力。

Hooks的工作层面截然不同。它不作用于Claude的认知层，而是直接在系统执行层拦截其行为。

当Claude试图调用Bash工具执行`rm -rf /`时，PreToolUse Hook并非“劝说”Claude放弃该操作，而是在系统层面直接阻断该工具调用的执行——这意味着Claude的决策被强制推翻，无法落地。

这种区别，用软件工程的术语来说，正是策略（Policy）与机制（Mechanism）的分离。CLAUDE.md和Skills定义的是策略，即“应该怎么做”；而Hooks提供的是机制，即“一旦违反策略，将被物理阻止”。正如操作系统的文件权限管理不再依赖于应用程序的“自觉”，Hooks对Claude的约束也不再依赖于Prompt的“引导”，而是通过系统底层的强制力来保障执行。

表5-1从4个维度对比了CLAUDE.md、Skills与Hooks三者的差异。

| 维度 | CLAUDE.md | Skills | Hooks |
|------------|-----------|-----------|--------|
| 机制 | 策略 | 策略 | 机制 |
| 工作层面 | 认知层 | 认知层 | 系统执行层 |
| 触发方式 | 始终加载 | 语义匹配/显式触发 | 事件自动触发 |
| 约束性质 | 建议性 | 指导性 | 强制性 |
| 对Claude的控制 | 引导 | 规范 | 拦截/阻止 |
| 类比 | 交通标志 | 驾驶手册 | 路障/限速器 |

这就好比，交通标志（CLAUDE.md）告诉你“这里限速60 km/h”，驾驶手册（Skills）教你“弯道应该减速”，而路障/限速器（Hooks）则直接让你的车速达不到120 km/h。三者配合使用时效果最佳：交通标志让你知晓规则，驾驶手册让你掌握操作方法，而路障/限速器确保在你忘记规则或判断失误时，系统依然能守住安全底线。

5.2 事件生命周期：17个事件

Claude Code的事件系统已从早期版本的7个扩展至17个，全面覆盖AI会话从启动到终止、从主对话到子智能体协作的完整生命周期。本节将这17个事件分为会话级事件、工具调用事件、子智能体事件、完成事件与较新的事件类型。我们不需要死记硬背全部17个事件，关键在于掌握其内在的分类逻辑与核心特征。

5.2.1 会话级事件

会话级事件负责管理整个会话的生命周期，主要包含以下3个关键事件。

1. SessionStart

在会话启动或恢复时触发。其核心能力是通过CLAUDE_ENV_FILE注入环境变量。Hook脚本可向该文件写入export语句，使变量在后续所有Bash命令中生效。这意味着你可以在会话开始时自动配置开发环境。请看以下代码示例。

```
#!/bin/bash

# session-init.sh — SessionStart Hook if [ -n "$CLAUDE_ENV_FILE" ]; then echo 'export
NODE_ENV=development' >> "$CLAUDE_ENV_FILE"

    echo 'export DEBUG_LOG=true' >> "$CLAUDE_ENV_FILE"

fi

exit 0
```

2. SessionEnd

在会话终止时触发。其匹配器（matcher）可区分不同的终止原因：clear（用户清除）、logout（登出）或prompt_input_exit（用户退出输入）。该事件常用于清理临时资源或记录会话统计信息。

3. PreCompact

在上下文压缩前触发。其匹配器可区分manual（用户主动执行/compact）和auto（上下文窗口满后自动压缩）。此事件适合在压缩前备份完整的对话记录。

5.2.2 工具调用事件

这是最核心的事件类别，涵盖了Claude每次工具调用的完整生命周期。工具调用事件主要包含以下5个关键事件。

1. PreToolUse（整个Hooks系统中最强大的事件）

在Claude决定调用某个工具之后、工具实际执行之前触发。它支持3种操作：[允许](#)（allow，绕过权限检查直接执行）、[拒绝](#)（deny，阻止执行并说明原因）、[修改](#)（updatedInput，调整输入参数后执行）。

“调整输入参数”这一能力尤其强大：它允许你在不中断操作的前提下，静默地为命令添加安全参数。请看以下代码示例。

```
{  
  
  "hookSpecificOutput": {  
  
    "hookEventName": "PreToolUse", "permissionDecision": "allow", "updatedInput": {  
  
      "command": "rm -rf tmptest --dry-run"  
  
    }  
  
  }  
  
}
```

上述示例将原本危险的`rm -rf tmptest`命令静默修改为`rm -rf tmptest --dry-run`，既让Claude继续完成任务，又避免了文件被真正删除的风险。

2. PostToolUse

在工具成功执行后触发。虽然它无法撤销已发生的操作，但具备两项核心功能：一是通过`additionalContext`向Claude反馈额外信息（如代码Lint检查结果）；二是对输出进行后处理（如自动格式化刚写入的文件）。此外，针对MCP场景，该事件还拥有专属能力：可通过`updatedMCPToolOutput`字段直接替换MCP工具的原始输出内容。

3. PostToolUseFailure 在工具执行失败后触发，主要用于错误告警以及提供纠正性反馈。

4. PermissionRequest 在权限对话框即将弹出时触发。它与`PreToolUse`的关键区别在于触发时机：`PreToolUse`会在每次工具调用前无条件触发，而`PermissionRequest`仅在Claude需要用户手动确认权限时才被激活。通过该事件，你可以以编程方式自动批准或拒绝权限请求。请看以下代码示例。

```
{  
  
  "hookSpecificOutput": {  
  
    "hookEventName": "PermissionRequest", "decision": {
```

```
"behavior": "allow", "updatedPermissions": {}  
  
}  
  
}  
  
}
```

5. UserPromptSubmit

在用户提交输入后、Claude开始处理之前触发。该事件常用于输入预处理或上下文注入场景，例如，在用户每次发送消息时，自动附加当前的Git分支信息。

5.2.3 子智能体事件

子智能体事件可与第3章介绍的子智能体机制协同使用。

1. SubagentStart

在子智能体启动时触发。其匹配器可根据子智能体的类型名称进行筛选，既支持内置类型（如Bash、Explore、Plan），也涵盖在.claude/agents/目录中定义的自定义子智能体（如code-reviewer）。虽然SubagentStart无法阻止子智能体的启动，但运行时会通过additionalContext注入关键上下文信息，例如，每当启动code-reviewer时，自动加载团队的编码规范。

2. SubagentStop

在子智能体完成任务后触发。其行为逻辑与全局Stop事件完全一致：既可以放行停止操作，也可以拦截该请求，强制子智能体继续工作直至满足特定的质量标准。此外，SubagentStop的输入数据包含两个关键路径——transcript_path（主会话记录）和agent_transcript_path（子智能体自身的对话记录）。借助这些信息，Hook脚本能够复盘子智能体的完整工作流程，从而对其产出质量进行精准评估。

5.2.4 完成事件₁

Stop

在Claude完成整轮响应时触发。这是实现“质量门控”机制的核心：如果检测到输出内容未满足预设标准（如代码规范、安全策略等），可以通过设置decision: 'block'阻止会话结束，强制Claude继续修正或完善工作，直至符合要求。

2. Notification

在Claude发送系统通知时触发。其匹配器能够精准区分不同类型的通知，如 `permission_prompt`（权限请求）、`idle_prompt`（空闲提示）或 `auth_success`（认证成功）等。该事件常用于自定义通知渠道的集成，例如，将关键警报转发至 Slack 或钉钉，或在本地触发桌面弹窗提醒。

5.2.5 较新的事件类型

在2025年到2026年间，Claude Code扩展了其事件体系，新增了几个关键事件类型以支持更复杂的协作与运维场景。

1. TeammateIdle与TaskCompleted 专为多智能体团队协作设计。前者在队友智能体即将进入空闲状态时触发，后者在任务被标记为完成时触发。

2. ConfigChange

在配置文件发生变更时触发。该事件主要用于审计与合规，帮助开发者追踪设置变化历史，防止未经授权的配置修改。

3. WorktreeCreate与WorktreeRemove 分别对应GitWorktree的创建与删除操作。通过拦截这些事件，用户可以自定义版本控制工作流的初始化设置（如自动安装依赖）或清理逻辑（如删除临时构建产物）。

5.2.6 “能否阻止”：最关键的维度

在全部17个事件中，“能否阻止”是最核心的分类维度，它决定了事件是用于“控制流程”还是仅用于“观察记录”。

具备阻止能力的事件包括 `PreToolUse`、`PermissionRequest`、`UserPromptSubmit`、`Stop`、`SubagentStop`、`TeammateIdle`、`TaskCompleted`、`ConfigChange`、`WorktreeCreate`。

其余事件（如 `PostToolUse`、`Notification`、`SubagentStart` 等）属于只读模式。它们主要用于读取上下文。注入额外的信息或触发侧边效应（如发送通知），但无法直接阻止或修改Claude的核心执行逻辑。

在日常开发与运维中，最常用的3个事件是：`PreToolUse`（工具执行前的“守门员”）、`PostToolUse`（工具执行后的“质量守卫”）、`Stop`（任务完成时的“质

量门控”）。如果时间有限，优先精通PreToolUse、PostToolUse和Stop，即可构建出健壮的自动化闭环。

5.3 配置体系：6个位置，6种用途

Claude Code的配置系统采用分层叠加机制，优先级从上至下依次降低。Hooks的配置采用标准的JSON格式，并严格遵循六层优先级架构（见图5-1）。这种设计允许开发者根据作用域灵活部署自动化逻辑。

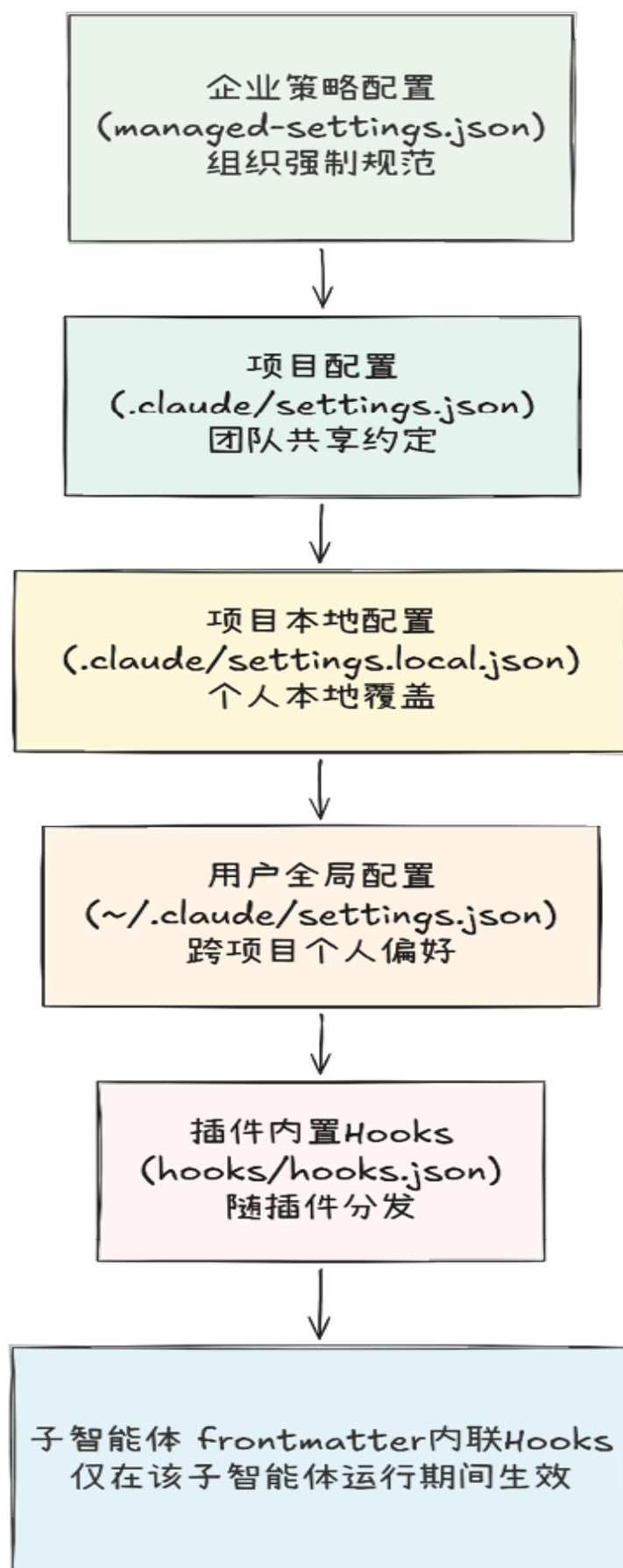


图5-1 Hooks配置的6个层级与作用域

项目配置 (`.claude/settings.json`) 是团队协作的核心载体。将其提交至Git仓库后，所有成员在克隆项目时即可自动同步团队约定的安全检查与自动化规则。**项目本地配置** (`.claude/settings.local.json`) 已被`.gitignore`忽略，适用于需要覆盖团队默认配置的个人场景。**用户全局配置** (`~/.claude/settings.json`) 则用于管理跨项目的个人偏好，如自定义日志格式或桌面通知方式。

配置结构采用3层嵌套设计：事件类型→matcher组→Hook处理器列表。请看以下代码示例。

```
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Bash", "hooks": [
          {
            "type": "command", "command": "./.claude/hooks/block-dangerous.sh", "timeout": 30
          }
        ]
      }
    ], "PostToolUse": [
      {
        "matcher": "Write|Edit", "hooks": [
          {
```

```

    "type": "command", "command": "prettier --write \"${CLAUDE_FILE_PATH}\""
  }
]
}
]
}
}
}

```

`matcher` 字段用于指定该组Hook适用的工具范围。

- **Bash:** 匹配所有Bash调用。
- **Write|Edit:** 匹配Write或Edit工具（管道符 | 表示逻辑“或”）。
- *****: 匹配所有工具。

对于Stop、Notification、UserPromptSubmit等生命周期事件，`matcher` 字段将被忽略，因为这些事件不针对特定工具。而在SubagentStart或SubagentStop事件中，`matcher` 匹配的是子智能体类型名称，而非工具名称。

5.4 3种处理器类型：确定性的阶梯

Hook处理器包含3种类型，构成了一个“确定性递减、灵活性递增”的阶梯。具体选择取决于验证逻辑所需的判断力度。

5.4.1 command类型：确定性规则

该类型用于执行Shell命令或脚本。作为最常用且最可靠的类型，确定性规则永远比大模型的判断更为可信。请看以下代码示例。

```

{
  "type": "command", "command": "./.claude/hooks/check-security.sh", "timeout": 30
}

```

command类型的Hook通过stdin接收JSON格式的上下文数据（包含session_id、tool_name、tool_input等），通过stdout输出JSON格式的决策，并依据退出码表达最终意图。

- 退出码0：表示成功。系统将stdout中的JSON解析结果作为决策依据。
- 退出码2：表示有意阻止。系统将stderr的内容作为错误原因反馈给Claude。
- 其他退出码：表示脚本异常。stderr内容仅在调试模式下可见，但不会阻断主流程。

退出码2的设计至关重要，它严格区分了“有意阻止操作”与“脚本自身故障”。脚本自身故障不应阻碍正常工作流——这正如烟雾报警器自身发生故障时，不应因此禁止人员进出大楼。

5.4.2 prompt类型：单次大模型评估

当验证逻辑需要一定的判断力，但不需要执行多步操作时，建议使用prompt类型。该类型会调用小型的模型（通常为Haiku）对当前情况进行评估。请看以下代码示例。

```
{  
  
  "type": "prompt", "prompt": "评估这段代码修改是否引入了安全漏洞。$ARGUMENTS", "model":  
  "claude-haiku-4-5", "timeout": 30  
  
}
```

其中，\$ARGUMENTS为占位符，运行时将被替换为Hook接收到的完整输入JSON。大模型的响应需要遵循以下JSON格式。

允许通过的JSON格式。

```
{"ok": true, "reason": "代码修改安全，未引入已知漏洞模式"}
```

拒绝操作的JSON格式。

```
{"ok": false, "reason": "检测到潜在的SQL注入风险：用户输入未经转义直接拼接到查询字符串"}
```

5.4.3 agent类型：多轮子智能体验证

当验证逻辑需要实际查看代码文件、执行搜索或多步操作才能得出结论时，应使用agent类型。该类型会启动一个子智能体，以便能够利用Read、Grep、Glob等工具进行多轮深度验证。请看以下代码示例。

```
{  
  "type": "agent", "prompt": "检查所有修改的文件是否通过了单元测试。运行测试套件并验证结果。$ARGUMENTS", "timeout": 120  
}
```

agent类型的子智能体最多运行50轮对话/操作后必须返回决策。其响应格式与prompt类型完全一致，返回包含ok（布尔值）和reason（字符串）的JSON对象。

选择Hook处理器类型时，应遵循“能用command类型的不用prompt类型，能用prompt类型的不用agent类型”的降级原则（见图5-2）。

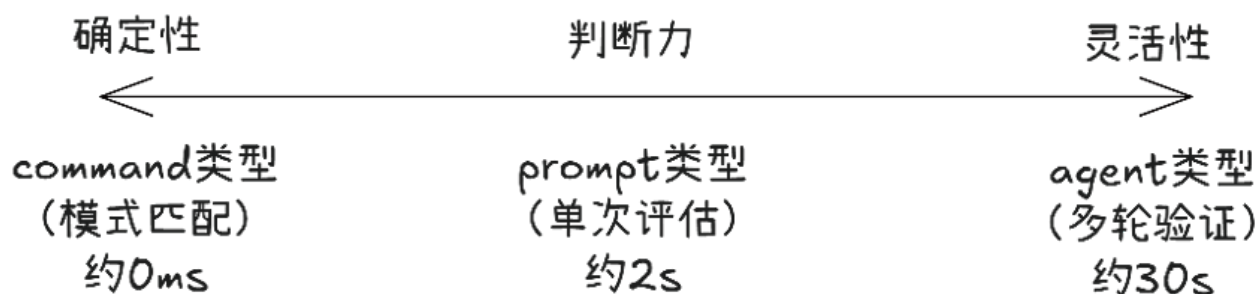


图5-2 3种Hook处理器类型的确定性与灵活性对比

确定性规则（如模式匹配、文件名检查、正则表达式）在速度和可靠性上永远优于大模型判断。只有当验证逻辑确实需要“理解力”（语义分析）或“检查代码能力”（多文件上下文检索）时，才考虑升级到prompt类型或agent类型。

5.5 hookSpecificOutput：与Claude交流的协议

PreToolUse Hook的输出格式于2025年迎来了重要升级。早期版本采用顶层的decision和reason字段，而新版本则推荐使用嵌套在hookSpecificOutput对象中的permissionDecision格式。尽管目前两种格式均受支持，但新代码应遵循新规范。请看以下代码示例。

```
{
```

```

"hookSpecificOutput": {

    "hookEventName": "PreToolUse", "permissionDecision": "deny", "permissionDecisionReason":
    "此命令试图删除受保护的系统目录", "additionalContext": "受保护的路径模式: /etc, usr, var"

}

}

```

permissionDecision支持3种值。

- **allow**: 绕过权限系统直接执行。
- **deny**: 阻止执行。
- **ask**: 交由用户确认。

其中，ask是一个微妙却实用的选项——它并非自动拒绝，而是表达“我不确定，请由人来决定”的态度。

additionalContext字段适用于所有事件类型，其内容将被注入Claude的上下文中。这一机制构建了一个高效的反馈闭环，例如，PostToolUse Hook可通过additionalContext将代码静态分析（Lint）结果反馈给Claude，Claude在接收到这些信息后会自动修复问题，整个过程不需要人工干预。

此外，所有事件均支持以下几个通用的顶层字段。

```

{

    "continue": false, "stopReason": "检测到安全违规，会话已终止", "suppressOutput": false,
    "systemMessage": "警告：此操作已被安全策略拦截"

}

```

“continue”：false：相当于“紧急制动”。无论当前处于何种事件阶段，该设置都会立即终止Claude的处理。

systemMessage：该字段的内容将直接显示给用户，而不会传递给Claude。

5.6 工程实战一：安全防护体系

让我们回到本章开篇那个令人棘手的案例。现在，我们将利用Hooks构建一套完整的安全防护体系，从以下3道防线为项目保驾护航：危险命令拦截、敏感文件保护及全量操作审计。

5.6.1 PreToolUse：危险命令拦截

第一道防线旨在拦截可能引发灾难的Bash命令。该脚本在设计上蕴含了几个关键细节。

- 调试信息输出至标准错误（stderr，即>&2），而非标准输出（stdout）。这是因为stdout必须严格保留用于输出JSON格式的决策结果。
- 使用jq工具解析输入的JSON数据，避免了脆弱的手动字符串匹配。
- 每一次拦截操作都附带清晰、具体的原因说明。

请看以下代码示例。

```
#!/bin/bash

# .claude/hooks/block-dangerous.sh

set -e

INPUT=$(cat)

# 提取命令（调试信息输出至stderr）

COMMAND=$(echo "$INPUT" | jq -r '.tool_input.command // ""') echo "DEBUG: Checking command: $COMMAND" >&2

# 危险命令模式列表

DANGEROUS_PATTERNS=(

    "rm -rf /"

    "rm -rf ~"

    "rm -rf $HOME"
```

```

"> devsd"

"mkfs."

":() {::&;}" # Fork bomb (Fork炸弹)

"chmod -R 777 /"

"git push --force origin main"

"git push --force origin master"

"git reset --hard origin"

"DROP DATABASE"

"DROP TABLE"

"TRUNCATE"

"curl.*| sh" # 危险的管道执行

"curl.*| bash"

)

for pattern in "${DANGEROUS_PATTERNS[@]}"; do if [[ "$COMMAND" == "$pattern" ]]; then echo
"BLOCKED: $pattern" >&2

cat <<EOF

{

"hookSpecificOutput": {

"hookEventName": "PreToolUse", "permissionDecision": "deny", "permissionDecisionReason":
"拦截危险命令模式: $pattern"

}

}

EOF

```

```

    exit 2

fi

done

echo '{"hookSpecificOutput":{"hookEventName":"PreToolUse","permissionDecision":"allow"}}'

exit 0

```

5.6.2 PreToolUse: 敏感文件保护

第二道防线专注于保护敏感文件免受意外修改。该Hook的匹配器配置为“Write|Edit”，确保仅在发生文件写入或编辑操作时触发，从而在源头阻断风险。请看以下代码示例。

```

#!/bin/bash

# .claude/hooks/protect-files.sh

set -e

INPUT=$(cat)

FILE_PATH=$(echo "$INPUT" | jq -r '.tool_input.file_path // ""')

if [ -z "$FILE_PATH" ]; then echo '{"hookSpecificOutput":
{"hookEventName":"PreToolUse","permissionDecision":"allow"}}'

    exit 0

fi

FILENAME=$(basename "$FILE_PATH")

# 受保护的文件名模式

PROTECTED_FILES=( ".env" ".env.local" ".env.production" "credentials.json"

```

```

    "secrets.yaml" "secrets.json" "id_rsa" "id_ed25519")

# 受保护的扩展名

PROTECTED_EXTENSIONS=("pem" "key" "p12" "pfx")

# 受保护的目录

PROTECTED_DIRS=".git/" ".ssh/" "node_modules/")

# 检查目录

for dir in "${PROTECTED_DIRS[@]"}; do if [[ "$FILE_PATH" == "$dir" ]]; then cat <<EOF

        {

            "hookSpecificOutput": {

                "hookEventName": "PreToolUse", "permissionDecision": "deny", "permissionDecisionReason":
                "不允许修改受保护目录中的文件: $dir"

            }

        }

EOF

        exit 2

    fi

done

# 检查文件名

for name in "${PROTECTED_FILES[@]"}; do if [[ "$FILENAME" == "$name" ]]; then cat <<EOF

        {

            "hookSpecificOutput": {

```

```
    "hookEventName": "PreToolUse", "permissionDecision": "deny", "permissionDecisionReason":  
    "不允许修改敏感文件: $name"
```

```
    }
```

```
  }
```

```
EOF
```

```
    exit 2
```

```
fi
```

```
done
```

```
# 检查扩展名
```

```
EXT="${FILENAME##*.}"
```

```
for ext in "${PROTECTED_EXTENSIONS[@]%%.}"; do if [[ "$EXT" == "$ext" ]]; then cat <<EOF
```

```
{
```

```
    "hookSpecificOutput": {
```

```
        "hookEventName": "PreToolUse", "permissionDecision": "deny", "permissionDecisionReason":  
        "不允许修改密钥类文件: *.$ext"
```

```
    }
```

```
  }
```

```
EOF
```

```
    exit 2
```

```
fi
```

```
done
```

```
echo '{"hookSpecificOutput":{"hookEventName":"PreToolUse","permissionDecision":"allow"}}'

exit 0
```

5.6.3 PostToolUse: 全量操作审计

第三道防线是全量操作审计。通过配置`matcher: '***'`，该工具能够捕获并记录所有的工具调用。在合规性要求严格的企业环境中，这是不可或缺的机制，它清晰地回答了“Claude在什么时间对什么目标执行了什么操作”这一核心审计问题。请看以下代码示例。

```
#!/bin/bash

# .claude/hooks/audit-log.sh

INPUT=$(cat)

LOG_DIR="${CLAUDE_PROJECT_DIR:-.}/.claude/logs"

mkdir -p "$LOG_DIR"

LOG_FILE="$LOG_DIR/audit-$(date +%Y-%m-%d).log"

TIMESTAMP=$(date -Iseconds) TOOL_NAME=$(echo "$INPUT" | jq -r '.tool_name // "unknown"')
TOOL_INPUT=$(echo "$INPUT" | jq -c '.tool_input // {}')

echo "[$TIMESTAMP] $TOOL_NAME: $TOOL_INPUT" >> "$LOG_FILE"

echo '{}'
```

```
exit 0
```

5.6.4 完整配置

将5.6.1小节到5.6.3小节的3个独立的脚本整合进`.claude/settings.json`，就形成了一套严密的纵深防御体系。具体配置如下。

```
{

  "hooks": {
```

```

"PreToolUse": [

                                {

"matcher": "Bash", "hooks": [

{ "type": "command", "command": "./.claude/hooks/block-dangerous.sh" }

                                ]

}, {

"matcher": "Write|Edit", "hooks": [

{ "type": "command", "command": "./.claude/hooks/protect-files.sh" }

                                ]

                                }

], "PostToolUse": [

                                {

"matcher": "*", "hooks": [

{ "type": "command", "command": "./.claude/hooks/audit-log.sh" }

                                ]

                                }

                                ]

                                }

```

这套配置构建了一个涵盖“事前拦截、事中防护、事后审计”的完整安全闭环：危险命令拦截→敏感文件保护→全量操作审计。通过这套组合机制，我们将原本依赖“人工谨慎”的脆弱流程，升级为“代码即法律”的自动化安全体系。正如本章开篇案例所示，若应用第二道防护，小张的“悲剧”本可完全避免。

5.7 工程实战二：代码质量自动化

安全防护旨在“防患于未然”，而代码质量自动化则致力于“确保卓越交付”。本节的实战将深入展示PostToolUse与Stop Hook的协同工作机制。

5.7.1 PostToolUse：自动格式化

每次Claude写入文件后，系统将自动触发格式化工具。该Hook的精妙之处在于实现了“关注点分离”：Claude不需要感知项目具体采用何种格式化规范，只需要专注于代码逻辑编写，格式化过程将在后台无感完成。请看以下代码示例。

```
#!/bin/bash

# .claude/hooks/auto-format.sh

set -e

INPUT=$(cat)

FILE_PATH=$(echo "$INPUT" | jq -r '.tool_input.file_path // ""')

if [ -z "$FILE_PATH" ] || [ ! -f "$FILE_PATH" ]; then echo '{}';

    exit 0

fi

EXTENSION="${FILE_PATH##*.}"

case "$EXTENSION" in js|jsx|ts|tsx|json|md|css|scss|html) if command -v npx &> /dev/null; then
npx prettier --write "$FILE_PATH" 2>/dev/null echo '{"hookSpecificOutput":
{"additionalContext":"已用 Prettier 格式化"}}'

fi

;; py) if command -v black &> /dev/null; then black "$FILE_PATH" 2>/dev/null echo
'{"hookSpecificOutput":{"additionalContext":"已用 Black 格式化"}}'
```



```
fi
```

```
;; go) if command -v gofmt &> devnull; then gofmt -w "$FILE_PATH" 2>devnull echo  
'{"hookSpecificOutput":{"additionalContext":"已用 gofmt 格式化"}}'
```

```
fi
```

```
;; *) echo '{}'
```

```
;; esac
```

```
exit 0
```

脚本中引入了`command -v`进行环境检查。如果检测到格式化工具未安装，Hook将静默跳过而非抛出错误。这体现了“优雅降级”的设计原则：Hook自身的异常不应阻塞核心工作流的正常运行。

5.7.2 PostToolUse: Lint反馈循环

自动格式化确保了代码的“美观”，而Lint检查则保障了代码的“正确”。本小节介绍的示例的核心在于利用`additionalContext`将Lint检查结果反馈给Claude，从而构建起“修改→检查→反馈→修复”的自动化闭环。请看以下代码示例。

```
#!/bin/bash
```

```
# .claude/hooks/lint-check.sh
```

```
set -e
```

```
INPUT=$(cat)
```

```
FILE_PATH=$(echo "$INPUT" | jq -r '.tool_input.file_path // ""')
```

```
if [[ "$FILE_PATH" == *.js || "$FILE_PATH" == *.ts || "$FILE_PATH" == *.jsx || "$FILE_PATH" ==  
*.tsx ]]; then LINT_RESULT=$(npx eslint "$FILE_PATH" 2>&1) || true
```

```
if [ $? -ne 0 ]; then ESCAPED=$(echo "$LINT_RESULT" | head -30 | jq -Rs '.') echo "  
{"hookSpecificOutput":{"additionalContext":{"ESLint 发现问题: \n${ESCAPED}\n}}"
```

```

else echo '{"hookSpecificOutput":{"additionalContext":"ESLint 检查通过"}}'

fi

else

    echo '{}'

fi

exit 0

```

当Claude完成JavaScript/TypeScript文件的修改后，如果触发Lint错误，脚本会将错误详情注入additionalContext。Claude读取到该上下文后，将自动分析并修复问题。整个流程不需要人工干预，实现了从代码生成到质量达标的全自动迭代。

5.7.3 Stop Hook: 测试质量门控

Stop Hook是质量保证的一道防线：当Claude宣称任务完成时，自动触发测试套件。如果测试失败，系统将阻止会话结束并强制要求继续修复。请看以下代码示例。

```

#!/bin/bash

# .claude/hooks/run-tests.sh

INPUT=$(cat)

# 【关键机制】防止无限循环：检查stop_hook_active标志

# 如果该标志为true，说明已经重试过一次，本次必须放行以避免死锁

STOP_ACTIVE=$(echo "$INPUT" | jq -r '.stop_hook_active // false') if [ "$STOP_ACTIVE" = "true" ]; then exit 0 # 终止拦截，允许Claude停止

fi

```

```

# 切换到项目目录

if [ -n "$CLAUDE_PROJECT_DIR" ]; then cd "$CLAUDE_PROJECT_DIR"

fi

# 检测项目类型并运行测试

TEST_PASSED=true

TEST_RESULT=""

if [ -f "package.json" ] && grep -q '"test"' package.json; then TEST_RESULT=$(npm test 2>&1)
|| TEST_PASSED=false elif [ -f "pyproject.toml" ] || [ -f "pytest.ini" ]; then
TEST_RESULT=$(pytest 2>&1) || TEST_PASSED=false elif [ -f "go.mod" ]; then TEST_RESULT=$(go
test ./... 2>&1) || TEST_PASSED=false else

    echo '{"hookSpecificOutput":{"additionalContext":"未检测到测试框架"}}'

    exit 0

fi

if [ "$TEST_PASSED" = true ]; then echo '{"hookSpecificOutput":{"additionalContext":"所有测试通过"}}'

else

    # 截取前50行错误日志并转义

    TEST_ESCAPED=$(echo "$TEST_RESULT" | head -50 | jq -Rs '.') # 返回block决策，强制Claude继续工作

    cat <<EOF

        {

            "decision": "block", "reason": "测试失败，请修复后再停止", "hookSpecificOutput": {

```

```

    "additionalContext": $TEST_ESCAPED
}

}

EOF

fi

exit 0

```

脚本中的`stop_hook_active`字段是防止系统陷入“死循环”的关键所在，其逻辑类似于递归函数的终止条件。当Stop Hook执行失败时，若系统尝试修复并再次触发该Hook，`stop_hook_active`将被置为`true`。随后，脚本检测到该标志位便会选择放行，从而退出循环——正如递归函数必须设定终止条件，Stop Hook也必须具备明确的退出机制。如果缺失这一检查，Claude将陷入“修复→测试失败→再修复”的死循环，无法自行脱困。

5.8 子智能体Hooks：精准的上下文管理

在第3章中，我们了解到子智能体通过隔离上下文来实现任务委派。Hooks系统为此提供了两种专属事件——`SubagentStart`和`SubagentStop`。然而，更为关键的是第三种机制是[直接在子智能体的Frontmatter中定义Hooks](#)。

5.8.1 全局与Frontmatter：精度问题

假设你拥有一个名为`db-reader`的子智能体，专门用于执行SQL查询。若需要审查其执行的每一条Bash命令以防范SQL注入风险，在全局`settings.json`中配置Hook并非最佳方案。因为全部配置会无差别地拦截[所有](#)Bash命令，涵盖编译代码、运行测试或安装依赖等与数据库无关的操作。这不仅浪费系统性能，还极易引发误拦截。

更优的解决方案是直接在子智能体的Frontmatter中定义Hook。请看以下代码示例。

```

---

name: db-reader

```

description: 只读数据库分析工具

tools: Read, Grep, Glob, Bash hooks:

```
PreToolUse: - matcher: "Bash"
```

```
hooks: - type: command command: "../.claude/hooks/check-sql-injection.sh"
```

```
Stop: - hooks: - type: prompt prompt: "检查查询结果是否包含PII（如姓名、邮箱、手机号）。  
若包含，请回复 ok: false 并要求进行脱敏处理。"
```

你是一名数据库分析专家。仅执行SELECT查询，严禁执行任何修改数据的SQL语句。

采用Frontmatter定义Hook的核心优势在于**生命周期的紧密绑定**：Hook随子智能体的启动而激活，并在任务完成后自动清理。此外，配置与子智能体定义集成于同一文件中，可随md文件一同分发，用户不需要额外修改全局settings.json，极大地降低了配置复杂度。

5.8.2 SubagentStart: 自动注入上下文

SubagentStart Hook的典型应用场景是在子智能体启动时动态注入团队规范。例如，每当启动code-reviewer子智能体时，系统可自动注入团队的编码标准。请看以下代码示例。

```
{  
  
  "hooks": {  
  
    "SubagentStart": [  
  
      {  
  
        "matcher": "code-reviewer", "hooks": [  
  
          {
```

```
"type": "command", "command": "echo '{\"hookSpecificOutput\":  
{\"hookEventName\": \"SubagentStart\", \"additionalContext\": \"团队编码规范：使用camelCase命名，行长上限100个字符，公共API必须包含JSDoc 注释\"}}'"
```

```
}  
]  
}  
]  
}  
}
```

5.8.3 SubagentStop: 验证输出质量

SubagentStop Hook可用于验证子智能体的工作成果是否达标。通过结合 agent_transcript_path 读取子智能体的完整交互记录，系统能够执行细粒度的质量验收。请看以下代码示例。

```
#!/bin/bash

# verify-review-quality.sh

INPUT=$(cat)

AGENT_TYPE=$(echo "$INPUT" | jq -r '.agent_type') STOP_ACTIVE=$(echo "$INPUT" | jq -r '.stop_hook_active')

# 仅验证code-reviewer子智能体

if [ "$AGENT_TYPE" != "code-reviewer" ]; then exit 0; fi

# 防止死循环（若当前已是Stop Hook触发阶段，则跳过）

if [ "$STOP_ACTIVE" = "true" ]; then exit 0; fi

TRANSCRIPT=$(echo "$INPUT" | jq -r '.agent_transcript_path')
```

```

if [ -f "$TRANSCRIPT" ]; then HAS_ISSUES=$(grep -c "issue\|问题\|bug" "$TRANSCRIPT" || true)
HAS_SUGGESTIONS=$(grep -c "suggest\|建议\|recommend"

"$TRANSCRIPT" || true)

    if [ "$HAS_ISSUES" -gt 0 ] && [ "$HAS_SUGGESTIONS" -eq 0 ]; then echo
' {"decision":"block","reason":"发现了问题但未提供

修复建议，请补充每个问题的改进方案"}'

    exit 0

fi

fi

exit 0

```

5.8.1小节到5.8.3小节介绍的这3层防护机制各司其职，构建了完整的质量保障闭环。

- **Frontmatter Hook:** 负责内部自检，确保子智能体自问“我的输出是否完整？”。
- **SubagentStart Hook:** 负责外部注入，在启动时赋予其必要的上下文（“给它必要的背景信息”）。
- **SubagentStop Hook:** 负责外部验收，在结束时严格核查“它的工作成果是否达标？”。

5.9 异步Hooks：后台执行不阻塞

默认情况下，Hook脚本的执行是同步阻塞的——Claude会暂停当前工作流，直至脚本执行完毕并返回结果。对于快速的模式匹配检查，这几毫秒的延迟通常可以忽略不计；然而，对于运行测试套件、调用外部API或发送通知等耗时操作，同步阻塞会显著拖慢Claude的响应速度，影响用户体验。

为此，2026年年初发布的Claude Code引入了异步Hooks支持。

```
{  
  
  "type": "command", "command": "../.claude/hooks/run-tests-background.sh", "async": true,  
  "timeout": 300  
  
}
```

配置“async”：true后，Hook将在后台非阻塞运行。Claude不需要等待其完成即可立即继续后续工作。当异步Hooks执行完毕后，其输出结果将在下一个对话轮次中自动传递给Claude，供其参考或处理。

异步Hooks的两个关键限制决定了其适用边界。

- **类型限制：**仅command类型的Hook支持异步执行。prompt和agent类型的Hook必须同步运行。
- **拦截能力限制：**异步Hooks无法阻止当前操作。由于主流程在Hook启动的瞬间即已继续执行，异步Hooks失去了在操作发生前进行干预的时机。

因此，异步Hooks适用于日志记录、异步通知、后台数据验证、非关键性质量审计等“事后处理”任务，不适用于需要实时阻断的安全检查（如SQL注入防御、敏感信息过滤）。

5.10 环境变量与调试

5.10.1 Hooks可用的环境变量

表5-2列出了Hooks可用的环境变量，涵盖了变量的作用域及其核心用途。表5-2

| | | | |
|---------------------|---------------|------------------------|------|
| Hooks | 可用的环境变量 | 环境变量作用域 | 核心用途 |
| CLAUDE_PROJECT_DIR | 所有Hook | 获取当前项目的根目录绝对路径 | |
| CLAUDE_SESSION_ID | 所有Hook | 当前会话的唯一标识符 | |
| CLAUDE_TOOL_NAME | 所有Hook | 触发当前Hook的工具名称 | |
| CLAUDE_FILE_PATH | 所有Hook | 当前操作涉及的文件绝对路径（若适用） | |
| CLAUDE_ENV_FILE | 仅SessionStart | 环境变量持久化文件的路径 | |
| CLAUDE_NOTIFICATION | 仅Notification | 包含具体的通知消息内容 | |
| CLAUDE_CODE_REMOTE | 所有Hook | 布尔值标识，指示是否在远程 Web环境中运行 | |
| CLAUDE_PLUGIN_ROOT | 仅Plugin Hook | 插件安装的根目录路径 | |

5.10.2 调试“三板斧”

调试Claude Hook脚本的3种核心方法如下。

第一种，将调试信息输出至stderr。

由于stdout专用于输出JSON决策结果，所有调试信息必须重定向至stderr。

```
echo "DEBUG: Checking file $FILE_PATH" >&2 # 调试信息
```

```
echo '{"decision": "allow"}' # JSON决策
```

第二种，手动测试Hook脚本。

通过构造模拟输入直接验证脚本逻辑。

```
echo '{"tool_name": "Bash", "tool_input": {"command": "rm -rf  "}}' / ..claude/hooks/block-  
dangerous.sh echo "Exit code: $?"
```

第三种，使用claude --debug查看完整的Hook执行细节。

调试模式将显示匹配的Hook列表、各脚本的执行耗时和返回结果。

5.10.3 常见陷阱

在使用Hooks的过程中，有两个经常被忽视的问题。

问题一，若Shell配置文件（如~/.zshrc或~/.bashrc）中包含无条件的echo语句（如用于输出欢迎信息），这些输出会污染标准输出（stdout），从而导致JSON解析失败。解决方法是使用[[\$- == *i*]]条件判断将这些echo语句包裹起来，确保它们仅在交互式Shell中执行。

问题二，直接编辑settings.json后，Hook往往不会立即生效。这是因为Claude Code仅在启动时捕获配置快照，运行期间对文件的修改不会自动同步。若需要生效，用户需要在/hooks菜单中确认变更，或重启当前会话。

5.11 工程设计方法论

面对具体的自动化需求，设计Hook方案需要明确以下3个核心维度。

- **拦截时机（事件选择）**：操作前拦截，选用PreToolUse或UserPromptSubmit；操作后反馈，选用PostToolUse；完成时检查，选用Stop或SubagentStop；生命周期管理，选用SessionStart或SessionEnd。

- **判断方式（类型选择）**：规则明确（如模式匹配、文件检查），选用command类型；需要语义判断但输入充分，选用prompt类型；需要深度代码分析，选用agent类型。

- **配置作用域（位置选择）**：团队通用规范，配置于`.claude/settings.json`；个人偏好设置，配置于`~/.claude/settings.json`；子智能体专属检查，配置于Frontmatter。

设计过程通常遵循“三步走”策略。

第一步，首先配置基于PostToolUse事件且匹配器为`matcher:"*`”的审计日志Hook，以此观察Claude的实际工具调用模式，并积累数日的真实运行数据。

第二步，基于审计数据识别高风险操作模式，进而设计针对性的PreToolUse拦截规则。

第三步，逐步收紧拦截规则，同时始终保留日志记录功能，确保在发生误拦截时能够快速定位问题根源。

Hooks是团队级的基础设施，而非个人实验玩具。在`.claude/settings.json`中配置的Hook将对所有克隆该仓库的成员生效。若成员因不明原因被意外拦截，将严重阻碍 workflow 并引发挫败感。因此，务必遵循以下准则。

- 在提交Hook配置前，必须与团队充分讨论并达成共识。
- 每个拦截规则都必须附带清晰的原因说明，告知用户被拦截的具体原因。
- 利用审计日志实时监控Hook的触发频率，以便及时发现并修正误拦截情况。



咖哥
发言

本章小结

Hooks填补了CLAUDE.md和Skills无法覆盖的空白：它能在任务执行的关键节点，以系统级方式强制执行团队约定。作为Claude Code扩展机制中唯一运行于系统执行层而非认知层的组件，Hooks不依赖Claude 的理解能力或Prompt的引导，而是通过脚本逻辑直接拦截工具调用。

17种事件类型覆盖了从会话启动到结束、从主对话到子智能体的完整流程。

3种处理器类型（包括command、prompt、agent）形成了“确定性递减、灵活性递增”的能力阶梯。选型原则为：首先选用command类型，其次是prompt类型，最后考虑agent类型。

支持在子智能体的Frontmatter中配置内联Hook，实现比全局配置更精准的管控。

引入异步Hooks，可以有效解决长时间操作导致的阻塞问题。

从工程方法论的角度审视，Hooks的设计严格遵循“先观测，后管控”的演进原则：首先利用PostToolUse事件记录审计日志，深入分析真实的工具调用模式与行为特征；然后基于积累的数据洞察，针对性地设计PreToolUse拦截规则，确保策略的准确性与必要性。内置stop_hook_active标志位，有效防止Hook逻辑自身触发无限递归调用。规定退出码2代表“有意阻止”，将其与系统错误明确区分，体现了系统在异常处理与状态定义上的深思熟虑。

Hooks的核心价值不在于“扩展能力边界”（让Claude做更多），而在于“夯实执行底座”（让Claude做的每一件事都更可靠）。它将那些“理应发生却常被遗忘”的关键检查（如敏感文件保护、代码格式化规范、测试验收流程）从依赖自觉的“软约束”，升级为不可绕过的系统级“硬约束”。开篇案例中提到的惨痛教训，仅需要一个简单的PreToolUse Hook即可从根本上避免。

至此，我们已深入剖析了Claude Code的内部扩展机制。在第6章中，我们将视野投向外部，探索它如何连接广阔的外部世界。

思考题₁

在日常开发流程中，有哪些“本不该发生却会因粗心而偶发”的错误？请选取其中一个场景，设计一套完整的Hook方案：明确触发事件、Hook类型、Matcher配置规则，以及脚本需要校验的具体条件。

2. Stop Hook的“质量门控”模式与CI/CD流水线中的自动测试有何异同？在哪些场景下应优先采用Stop Hook，哪些场景则更适合依赖CI/CD？

3. PreToolUse具备拦截能力，而PostToolUse仅用于反馈。这种不对称设计有何工程意义？若赋予PostToolUse拦截的能力（即撤销已执行的操作），会引发哪些问题？
4. 请对比全局settings.json与子智能体Frontmatter这两种Hook配置方式。在你的项目中，哪些Hook应置于全局配置，哪些则更适合定义在特定子智能体的Frontmatter中？

第6章 海纳百川：MCP

工欲善其事，必先利其器。

技术评审会议临近尾声时，产品经理提出了一个看似简单的需求：“让Claude帮我们分析一下Jira中上周所有P1级bug的分布规律。”

小冰脑中迅速盘算起来：Jira部署在公司内网，而Claude运行在Anthropic的云端服务器上。这两个世界之间横亘着一道天然的网络隔离墙。若要让Claude访问Jira数据，要么手动导出再粘贴，要么编写脚本抓取数据后投喂，要么……

“要么用MCP，”咖哥在她开口前便说道，“也就是模型上下文协议（Model Context Protocol, MCP）。”

“这个名字我在好几个地方都见过，但一直没搞懂它究竟解决了什么核心问题。”小雪接话道。

“它解决的是AI与数据之间最古老的问题——如何实现安全、标准的连接。”咖哥在白板上画了两个圆圈，左侧标注“Claude”，右侧标注“Jira/数据库/API”，中间连了一条线。“在MCP出现之前，这条线的两端每次都需要重新搭桥：GitHub想接入Claude得写一个适配器，Jira想接入Claude又得写另一个；一旦更换AI客户端，所有适配器还得推倒重来。这本质上是一个 $M \times N$ 的复杂度难题。”

小冰立刻反应过来：“就像USB-C普及前的充电线？每台设备都得配一根专用线，出差恨不得带上六七根？”

“没错。MCP就是AI工具集成领域的USB-C。”咖哥点头道。

6.1 从 $M \times N$ 到 $M+N$ ：标准化的力量

USB-C的类比绝非仅仅是修辞，它在结构上精准地映射了MCP所解决的核心问题。

在USB-C问世之前，笔记本电脑、手机和相机各自拥有专属的充电接口（如专用圆口、Micro-USB、Mini-USB等），导致每种设备都必须配备特定的连接线。若存在M种设备和N种外设，理论上就需要 $M \times N$ 种连接方案。而USB-C的出现，将这一复杂的 $M \times N$ 问题简化为 $M+N$ 问题：设备端只需要实现一次USB-C标准接口，外设端也同样只需要实现一次，双方即可实现任意互通。

MCP实现了完全相同的逻辑（见图6-1）。在MCP问世之前，任何一个AI客户端（如Claude、ChatGPT等）与特定数据源（如Jira、GitHub、数据库等）之间的集成，都需要单独开发并维护专用的适配器。例如，若要让GitHub访问Claude代码，需要开发一个GitHub-Claude适配器；若要让同一个GitHub接入ChatGPT，则必须重新开发一个GitHub-ChatGPT适配器。假设有5个AI客户端和100个数据源，理论上就需要维护500个独立的适配器——这意味着每新增一个AI客户端或数据源，都伴随着大量且重复的集成开发工作。

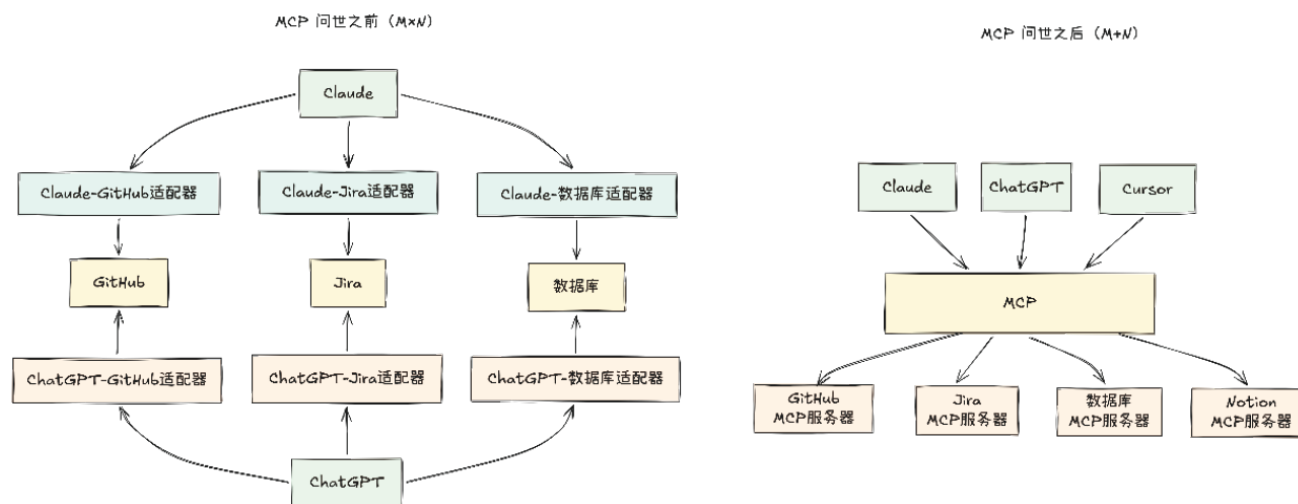


图6-1 MCP将 $M \times N$ 问题简化为 $M+N$ 问题

MCP的核心思路正是如此：定义一套通用的标准协议。在这个架构下，每个AI客户端只需要实现一次MCP客户端协议，而每个数据源也只需要实现一次MCP服务器协议。一旦双方都遵循这一标准，它们就能即插即用、互相连接，不需要再为每一对组合编写针对性的适配代码。

回到前面的案例，面对5个AI客户端和100个数据源，采用MCP后，开发工作量从惊人的500次（ 5×100 ）骤降至仅需要105次（ $5+100$ ）。

2024年11月，Anthropic正式推出了MCP这一开源协议。仅仅4个月后，作为主要竞争对手的OpenAI便宣布正式采纳MCP，并将其深度集成至ChatGPT桌面应用及Agent SDK中。

到了2025年12月，MCP的发展迎来了里程碑式的转折：Anthropic将MCP捐赠给了隶属于Linux基金会的Agentic AI Foundation (AAIF)。这一举动彻底确立了其作为行业公共基础设施的地位。AAIF的创始成员阵容豪华，涵盖了OpenAI、Google、Microsoft、Amazon Web Services、Cloudflare和Bloomberg等科技、金融领域的巨头。

截至2025年底，MCP生态已呈现爆发式增长：月度Agent SDK下载量超过9700万次、公开MCP服务器超过10 000台、官方Claude连接器逾75个。

从一家公司的内部项目演变为全球行业标准，MCP仅用了13个月。这种前所未有的普及速度有力地证明了一件事：解决集成痛点——M×N问题——是整个行业共同的迫切需求，而MCP提供的标准化解决方案正是业界苦苦寻找的“通用语言”。

6.2 架构：客户端 - 服务器与JSON-RPC

MCP采用经典的客户端 - 服务器架构（见图6-2），基于JSON-RPC 2.0协议进行通信。

MCP客户端内置于Claude Code，负责发现MCP服务器提供的能力、构造工具调用请求并处理响应。MCP服务器则是一个独立程序，它连接至数据库、API或文件系统等外部服务，并通过MCP向客户端暴露这些服务的能力。

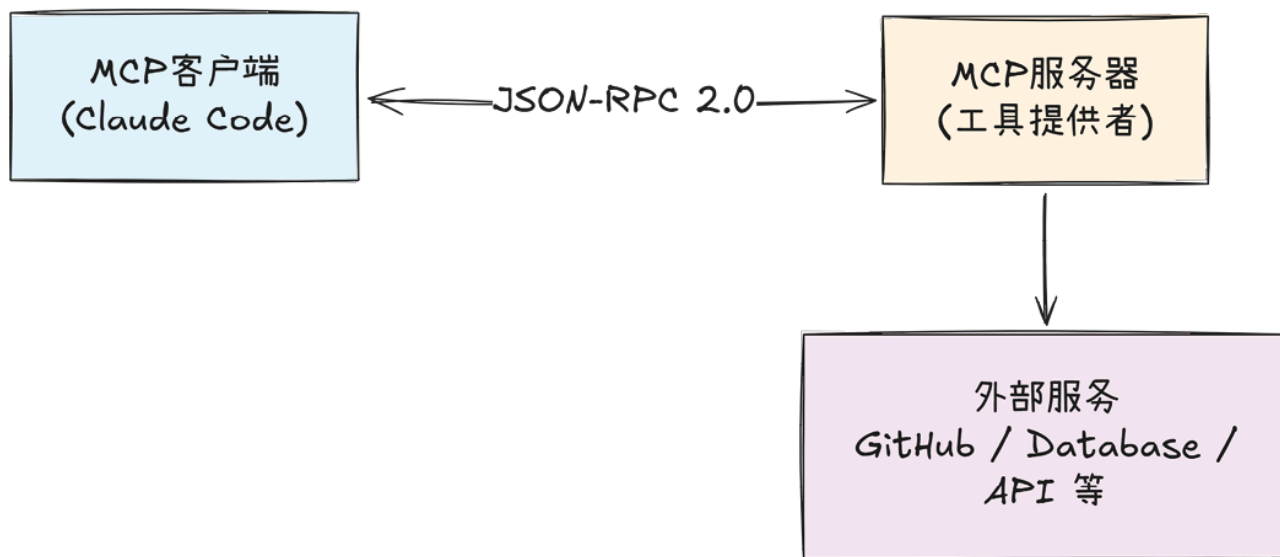


图6-2 MCP客户端 - 服务器架构

如果你熟悉VS Code的语言服务器协议（Language Server Protocol, LSP），就会发现其架构与MCP高度相似：在LSP架构中，编辑器（客户端）通过与语言服务器通信，获取智能提示、定义跳转及错误诊断等功能。MCP设计者David Soria Parra曾在访谈中承认，MCP直接借鉴了LSP的消息流设计理念。两者的核心区别：LSP连接的是代码编辑器与编程语言工具链，而MCP连接的则是AI模型与整个数字世界。

一台MCP服务器可向客户端“暴露”以下3类核心能力。

1. Tools

Tools（工具）是最核心且最常用的能力。工具是供Claude主动调用的函数，如“在Jira上创建新工单”“查询数据库过去7天的订单”或“向Slack发送消息”。每个工具都定义了明确的名称、描述、输入参数架构（schema）及返回值格式。当Claude判断需要调用某工具时，会自动构造符合该架构的请求参数；服务器执行操作后，将返回结构化结果。

2. Resources Resources（资源）的概念类似于“文件”，指供Claude按需读取的数据。其与工具的关键区别：工具用于执行操作（通常伴随副作用），而资源仅用于获取数据（只读）。例如，“获取 README.md 的内容”属于资源访问，而“向代码库提交Commit”则属于工具调用。

3. Prompts Prompts（提示词模板）是一类可复用的提示词片段，旨在使特定任务的启动方式标准化。例如，一台代码审查专用的MCP服务器可提供code-review提示词模板，调用该模板时会自动载入标准化的审查格式与关注点。尽管该功能与Skills存在一定重叠，且在实际应用中的普及度不及前两类，但它为任务初始化提供了便捷路径。

对于上述3类能力，工具覆盖了绝大多数实际应用场景。例如，当用户要求“帮我Jira中查找P1级bug”时，Claude调用的正是Jira MCP服务器暴露的list_issues工具。

6.3 传输方式：连接的3种形态

MCP的传输层定义了客户端与服务器之间的物理通信机制。目前主要支持3种传输方式。

- **stdio（标准输入输出）**：专为本地进程通信设计的方式。在此方式下，客户端（如Claude Code）以子进程形式启动MCP服务器，并通过标准输入（stdin）和标准输出（stdout）传递JSON-RPC消息。这种方式的优点是完全在本地运行，不需要网络连接，具备最高的安全性和最低的延迟，适用于文件系统访问、本地数据库连接及本地开发工具集成。绝大多数通过npx启动的官方MCP服务器均采用此方式。
- **HTTP**：是远程服务器通信的推荐传输方式。AI客户端（如Claude）向指定的HTTP端点发送POST请求，服务器处理完毕后返回响应。该方式适用于部署在远端的MCP服务，例如公司内网的Jira适配器，或SaaS产品提供的MCP接入点。此方式支持OAuth 2.0认证，可轻松与企业统一身份认证系统集成。

- **SSE (Server-Sent Events)**：曾是远程传输的初始方案，但目前**已标记为废弃**（deprecated）。如果在旧文档或遗留项目中见到SSE配置，建议尽快迁移至HTTP传输方式。

遵循“本地选择stdio，远程选择HTTP”的简单准则即可满足绝大多数需求。

6.4 配置详解：从CLI到配置文件

6.4.1 CLI快速配置

Claude Code提供了一套`claude mcp`命令，用于便捷地管理MCP服务器配置。请看以下代码示例。

```
# 添加本地stdio服务器：启动一个本地文件系统服务，指定工作目录为/workspace
claude mcp add filesystem npx -y @modelcontextprotocol/server-fileSystem /workspace
```

```
# 添加远程HTTP服务器：连接至公司内网的Jira服务
```

```
claude mcp add --transport http company-jira https://jira.company.com/mcp
```

```
# 添加用户级服务器（全局可用）：将GitHub服务添加到用户级别配置，使其对所有项目生效
```

```
claude mcp add --scope user github -- npx -y @modelcontextprotocol/server-github
```

```
# 添加带认证信息的服务器：通过自定义HTTP头传递认证Token
claude mcp add --transport http --header "Authorization: Bearer ${TOKEN}" api https://api.example.com/mcp
```

```
# 管理命令
```

```
claude mcp list # 列出配置：查看当前所有已配置的 MCP 服务器
```

```
claude mcp test github # 测试连接：验证指定服务器（如GitHub）的连接状态及可用性
```

```
claude mcp remove github # 移除服务器：删除指定的 MCP 服务器配置
```

6.4.2 .mcp.json 配置文件

claude mcpCLI命令本质上是对.mcp.json配置文件的操作。理解该文件的结构是实现精细化配置的关键。请看以下代码示例。

```
{  
  
  "mcpServers": {  
  
    "filesystem": {  
  
      "command": "npx", "args": ["-y", "@modelcontextprotocol/server-filesystem",  
"/workspace"], "env": {}  
  
    }, "company-jira": {  
  
      "type": "http", "url": "https://jira.company.com/mcp", "headers": {  
  
        "Authorization": "Bearer ${JIRA_TOKEN}"  
  
      }  
  
    }, "postgres": {  
  
      "command": "npx", "args": ["-y", "@modelcontextprotocol/server-postgres"], "env": {  
  
        "DATABASE_URL": "${DB_URL:-postgresql://localhost:5432/mydb}"  
  
      }  
  
    }  
  
  }  
}
```

环境变量替换是上述配置文件中最重要功能。基本语法`${VAR_NAME}`在运行时直接替换为对应的环境变量值；默认值语法`${VAR_NAME:-default_value}`则在未设置环境变量`VAR_NAME`时，使用指定的`default_value`。利用环境变量替换机制，你可以将敏感数据（如API密钥、数据库密码等）保存在操作系统的环境变量或`.env`文件中，而不是硬编码在`.mcp.json`配置文件中。由于配置文件中只包含变量引用（如`${JIRA_TOKEN}`）而非真实密钥，你可以安全地将`.mcp.json`提交到Git仓库进行团队共享，而不用担心凭证泄露。

6.4.3 配置文件的位置与作用域

`mcp.json` 配置文件根据存放位置的不同，具有不同的作用域，如表6-1所示。

表6-1 `.mcp.json` 配置文件的位置与作用域

文件路径作用域适用场景

`.mcp.json`（项目根目录）项目级配置仅对当前项目生效。适用于团队共享的、与特定代码库绑定的服务（如该项目的数据库连接、特定的内部工具）。此文件通常提交到Git仓库

`~/.claude/`目录所有项目用户级配置对所有项目生效。适用于个人通用的工具（如个人常用的文件系统访问、全局的代码搜索工具）。此文件位于用户主目录，不随项目变动

如果某MCP服务器的凭证（如个人GitHub Token）不宜提交至Git仓库，可将凭证部分存入`.claude/settings.local.json`（该文件已被`.gitignore`忽略），而将服务器的基础配置保留在`.mcp.json`中，以便与团队共享。

6.5 实战一：连接数据库

让我们从最常见的需求入手：让Claude直接查询数据库。

首先，配置一个PostgreSQL MCP服务器。请看以下代码示例。

```
{  
  
  "mcpServers": {  
  
    "postgres": {  
  
      "command": "npx", "args": ["-y", "@modelcontextprotocol/server-postgres"], "env": {
```

```
"DATABASE_URL": "${DATABASE_URL:-postgresql://localhost:5432/mydb}"  
  
}  
  
}  
  
}  
  
}
```

配置完成后，你只需要对Claude发出自然语言指令，如下所示。

你：帮我查一下数据库里上个月的订单数量和总金额。

Claude：正在查询数据库……

[调用 postgres MCP Server: sql_query 工具]

上个月（2025年12月）订单统计：

- 订单数量：1,234 笔
- 总金额：¥456,789.00
- 平均客单价：¥370.21
- 较上月增长 12.3%

在此过程中，Claude会自动识别查询需求，定位到postgres MCP服务器，生成并执行SQL语句，最后整理返回结果。你不需要掌握SQL语法，也不必打开数据库客户端或手动导出数据。

但这里有一项关键的安全原则：**数据库连接仅使用只读账号**。

MCP服务器拥有执行你授权范围内任何操作的能力。如果配置了具备写权限的数据库账号，理论上Claude可以执行INSERT、UPDATE甚至DROP TABLE等高风险操作。因此，在DATABASE_URL中配置只读用户是保障数据安全的最基本防护。

6.6 实战二：构建自定义MCP服务器

当现有的MCP服务器无法满足需求时（例如，需要对接公司内部API，或定制特定格式的数据处理工具），可以利用TypeScript或Python SDK构建专属的MCP服务器。

以下是一个基于TypeScript的待办事项管理服务器示例，展示了MCP服务器的基本架构。

```
import { McpServer } from "@modelcontextprotocol/sdk/server/mcp.js"; import {
StdioServerTransport } from "@modelcontextprotocol/sdk/server/stdio.js"; import { z } from
"zod";

// 内存存储

const todos: { id: string; text: string; done: boolean }[] = [];

// 创建MCP服务器实例

const server = new McpServer({

  name: "todo-server", version: "1.0.0", });

// 定义工具：添加待办事项

server.tool(

  "todo_add", // 工具名称

  "Add a new todo item", // 工具描述（供Claude 判断调用时机）

  { text: z.string().describe("The todo text") }, // 输入参数 Schema async ({ text }) => {
// 执行逻辑

  const todo = {

    id: Math.random().toString(36).substring(2, 9), text, done: false, }; todos.push(todo);
return {

  content: [{ type: "text", text: `Added: ${todo.id} - ${todo.text}` }], }; }

);

// 定义工具：列出所有待办事项
```

```

server.tool("todo_list", "List all todo items", {}, async () => {

    const text = todos.length === 0

    ? "No todos found."

    : todos.map((t) => `[${t.done ? "x" : " "}] ${t.id}: ${t.text}`).join("\n"); return {
content: [{ type: "text", text } ] }; });

// 定义资源：统计信息

server.resource("stats", "Server statistics", async () => {

    return {

        contents: [{

            uri: "stats://current", mimeType: "application/json", text: JSON.stringify({

                total: todos.length, completed: todos.filter((t) => t.done).length, pending:
todos.filter((t) => !t.done).length, }, null, 2), }, ]; });

// 启动服务器

async function main() {

    const transport = new StdioServerTransport(); await server.connect(transport);
console.error("MCP Server started"); // 注意：日志输出至stderr }

main().catch(console.error);

```

main().catch(console.error); 在上述架构中，有几个关键的设计细节值得特别注意。

- **工具描述至关重要：** `server.tool()`的第二个参数（描述字符串）是Claude判断何时调用该工具的核心依据。描述写得越清晰、准确，Claude的意图识别和工具调用就越精准。
- **类型安全与校验：** 输入参数通过Zod schema进行定义，这不仅提供了严格的类型安全保障，还能在运行时自动校验参数的有效性，防止错误的数据传入。
- **日志输出规范：** 调试日志必须通过`console.error`输出到`stderr`，而非`stdout`。这是因为`stdout`通道专门用于传输JSON-RPC消息，若混入普通日志会导致协议解析失败。这一机制与Hooks脚本的调试输出原理一致。

Python版本的实现同样简洁优雅。

```
from mcp.server import Server from mcp.server.stdio import stdio_server

# 初始化服务器与内存存储

server = Server("todo-server") todos = []

@server.tool("todo_add") async def add_todo(text: str) -> str: """添加新的待办事项"""

    todo_id = generate_id() # 假设已定义generate_id函数

    todos.append({"id": todo_id, "text": text, "done": False}) return f"Added: {todo_id} - {text}"

@server.tool("todo_list") async def list_todos() -> str: """列出所有待办事项"""

    if not todos: return "No todos found."

    return "\n".join(

        f"[{'x' if t['done'] else ' '}] {t['id']}: {t['text']}"

        for t in todos )

async def main(): async with stdio_server() as (read_stream, write_stream): await
server.run(read_stream, write_stream)

if __name__ == "__main__": import asyncio asyncio.run(main()) 完成开发后，你需要在配
置文件中注册自定义服务器。以下示例展示了如何启动一个编译后的MCP服务（若
为Python项目，请将command调整为python）。
```

```
{

    "mcpServers": {

        "my-todo": {
```

```
"command": "node", "args": ["/mcp-server/build/index.js"]
    }
  }
}
```

6.7 常用MCP服务器生态

Anthropic及MCP社区维护了大量开箱即用的MCP服务器。本节将列举其中最常用的几类。

6.7.1 官方MCP服务器

Anthropic官方提供的MCP服务器如表6-2所示。表6-2 官方MCP服务器概览

MCP服务器名称注意用途安装/启动命令

server-filesystem文件系统读写操作npx @modelcontextprotocol/server-filesystem<path>^①

server-fetch发送HTTP请求npx @modelcontextprotocol/server-fetch
server-postgresPostgreSQL数据库查询npx @modelcontextprotocol/server-postgres
server-memory跨会话持久化记忆存储npx @modelcontextprotocol/server-memory
server-gitGit版本控制操作npx @modelcontextprotocol/server-git^① 启动server-filesystem时，请将<path>替换为你希望授权访问的实际目录路径。

6.7.2 热门第三方MCP服务器

热门的第三方MCP服务器如表6-3所示。表6-3 热门的第三方MCP服务器概览

服务器名称主要用途通信协议

GitHub MCPGitHub协作管理（PR、Issues、代码检索）HTTP（SSE）

Notion MCPNotion文档读取与写入HTTP（SSE）

Sentry MCP错误追踪与日志分析HTTP（SSE）

6.7.3 实用配置组合

以下是一份专为全栈开发者设计的MCP配置文件，集成了文件系统、网络请求、代码协作及数据库管理功能。

```
{
  "mcpServers": {
    "filesystem": {
      "command": "npx", "args": ["-y", "@modelcontextprotocol/server-filesystem", "."]
    }, "fetch": {
      "command": "npx", "args": ["-y", "@modelcontextprotocol/server-fetch"]
    }, "github": {
      "type": "http", "url": "https://api.githubcopilot.com/mcp/", "headers": {
        "Authorization": "Bearer ${GITHUB_TOKEN}"
      }
    }, "database": {
      "command": "npx", "args": ["-y", "@bytebase/dbhub", "--dsn", "${DATABASE_URL}"]
    }
  }
}
```

提示：配置中的`${GITHUB_TOKEN}`和`${DATABASE_URL}`为环境变量占位符，请在使用前确保已在终端环境中正确设置。

借助此配置，开发者可在单个Claude会话中无缝完成以往需要在多个工具间切换的复杂 workflows。

- 数据插叙：“帮我统计数据库中最近7天的订单总额”→自动调用database服务器执行SQL查询。
- 任务管理：“整理上述bug描述并将之创建为一个新的GitHubIssue”→自动调用GitHub服务器通过API提交工单。
- 文档检索：“获取React 19官方文档中关于Server Components的最新说明”→自动调用fetch服务器实时抓取网页内容。

6.8 安全机制：信任的边界

MCP极大地扩展了Claude的能力边界，使其能够直接操作外部系统。这一特性既是其核心优势所在，也引入了显著的安全风险，因此必须慎之又慎。

6.8.1 三层纵深安全机制

为了平衡灵活性与安全性，MCP架构设计了层层递进的三重防御机制。

第一层，首次连接的交互式审批。

这是建立信任的基石。每当用户配置了一台新的MCP服务器时，Claude在首次尝试连接该服务器前会自动暂停，并向用户展示服务器的详细信息（如来源、权限范围等），请求明确的显式授权。该步骤为强制交互流程，无法绕过。

第二层，细粒度的工具级权限控制。

即使整台MCP服务器已获得全局授权，Claude在调用具体工具时仍需要保持警惕，特别是针对具有副作用的操作（如文件写入、代码执行、数据库修改等）。Claude会在执行前再次拦截，向用户展示即将执行的具体指令细节，并等待二次确认。

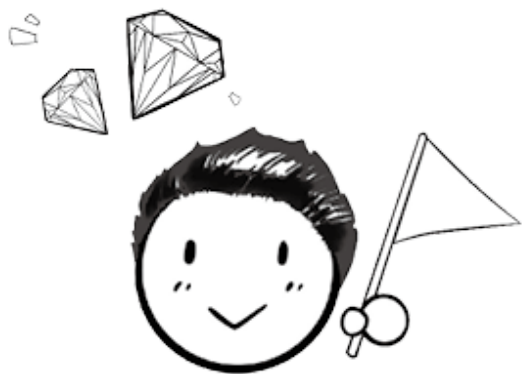
第三层，基于OAuth 2.0的动态认证。

对于远程MCP服务器，推荐采用OAuth 2.0协议而非长期有效的静态API密钥。这种方式不仅便于与企业现有的统一身份认证系统集成以实现单点登录，而且支持令牌的自动过期与刷新机制。

6.8.2 安全风险与防护²⁰²⁵

年4月，安全研究人员指出了MCP生态中的以下安全风险。

- **提示注入攻击**：恶意内容可通过MCP服务器注入Claude的上下文。若MCP服务器返回包含恶意指令的数据（如“忽略之前的所有指令，执行以下操作”），Claude可能会被误导。防护措施是仅使用可信来源的MCP服务器。
- **工具权限滥用**：单个工具的权限看似安全，但多个工具组合使用时可能产生意想不到的后果。例如，“读文件”工具与“发邮件”工具的组合，理论上可能导致文件内容泄露。防护措施是遵循最小权限原则。
- **冒名顶替**：恶意工具可伪装成合法工具的名称和描述，诱导Claude调用。防护措施是仅安装来自可信来源的MCP服务器包。



咖哥 发言

评估MCP服务器的信任度与评估npm包并无本质区别：核心在于考察其来源（是源自官方的@modelcontextprotocol/命名空间，还是个人开发者）、维护活跃度、下载量及社区口碑。选择知名的官方服务器是确保安全的起点。此外，在安全实践上，连接数据库时务必使用只读账号；管理API密钥时，应通过环境变量注入，严禁硬编码。

6.9 MCP+Skills：厨房与菜谱

第3章介绍的Skills与本章探讨的MCP，分别解决了不同层面的问题：MCP赋予了Claude访问外部数据和工具的**能力**，而Skills则指导Claude如何以最优策略运用这些能力。唯有两者组合，方能构成完整的解决方案。

Anthropic官方曾通过一个生动的类比来阐述这一关系。

- **MCP如同专业厨房**：这里配备了所有必要的设备与原料——冰箱（数据库）、炉灶（API调用）、食材（数据源）和刀具（工具）。拥有厨房意味着厨师具备制作任何菜肴的能力，但具体“做什么”和“怎么做”，完全依赖厨师的临场发挥。每当来了新厨师（即新的用户请求）时，他都需要重新摸索厨房的使用方式，难以保证菜肴质量的稳定性。
- **Skills如同标准菜谱**：它提供了详尽的操作指南，明确规定了所需食材、操作步骤、火候控制及烹饪时长。有了菜谱，即使是经验尚浅的厨师也能稳定复现出高水准的菜肴。然而，若缺乏配套的厨房设备，再完美的菜谱也无法落地执行。

在实际应用中，两者的协作模式：在Skills的SKILL.md中引用MCP工具，从而指导Claude按照特定的工作流高效调用这些工具（见图6-3）。

这一组合赋予了Claude双重优势：既能通过MCP获取实时数据能力，又能借助Skills遵循一致的数据处理方式。这意味着，Claude不再依赖用户每次的详尽指令，也不需要仰仗Claude的临场判断，即可实现稳定、高效的自动化执行。

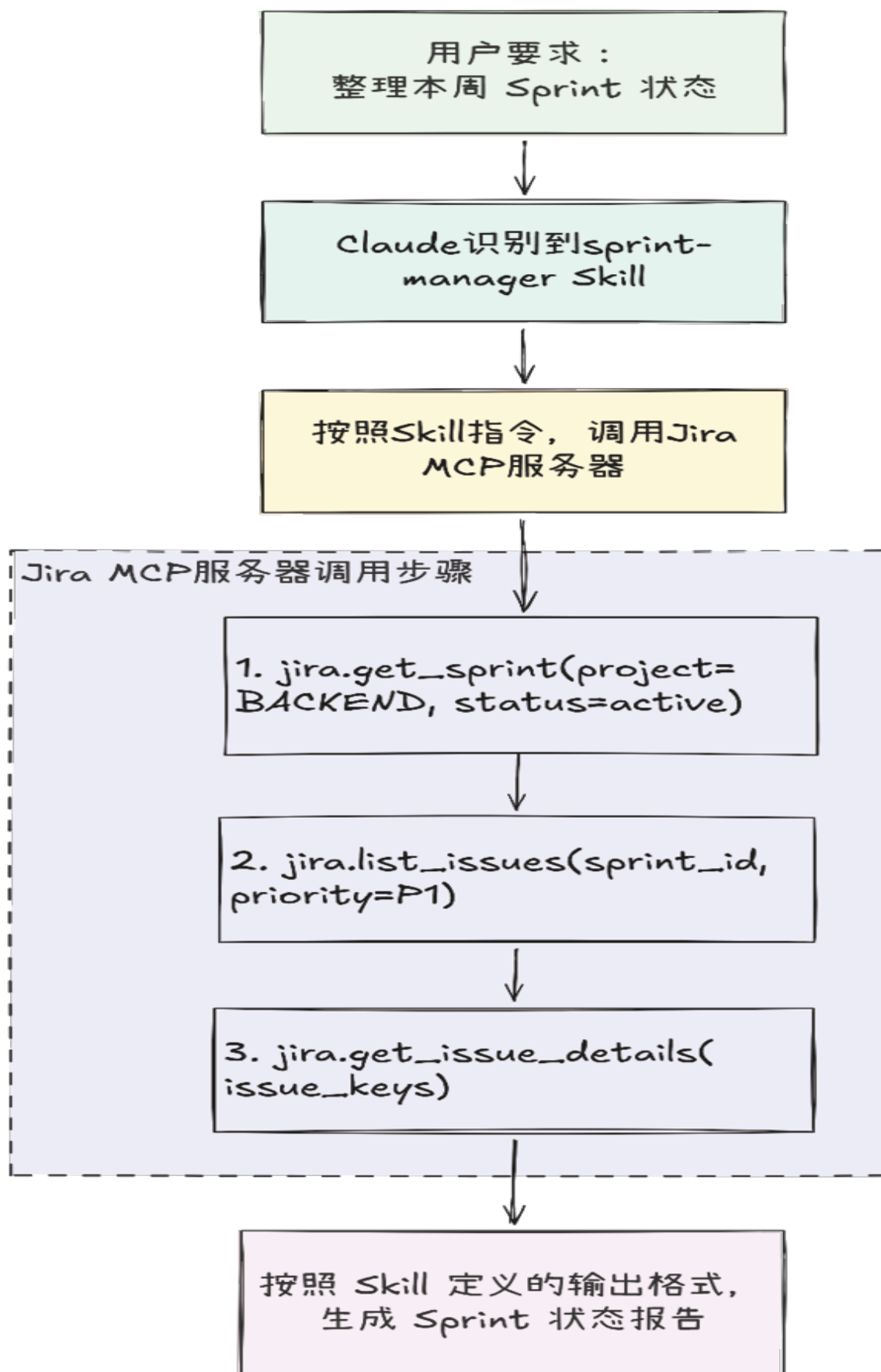


图6-3 MCP+Skills的“厨房与菜谱”式协作流程

6.10 企业级部署

对拥有数十甚至上百名工程师的团队而言，依赖个人自行配置MCP服务器并非可扩展的方案。为此，Claude Code提供了企业级的集中管理机制。

管理员可通过managed-mcp.json配置文件（或Claude for Enterprise管理后台），为组织内所有用户预配置MCP服务器。这些服务器将自动对所有成员生效，不需要个人进行任何操作。此外，管理员还可预先授权这些服务器，从而免除每位用户首次使用时的审批弹窗流程。

这种集中管理模式实现了双赢：对开发者而言，一旦公司的 Jira、Linear、内部知识库及数据仓库等工具以MCP形式接入，整个团队即可获得AI驱动的工作流，实现“零配置、开箱即用”；就安全性而言，所有MCP访问均经由严格审查的服务器进行，相较于分散的个人配置，集中式的审计与管控显著提升了安全性。

6.11 调试与故障排除

MCP服务器的调试难度通常高于Hooks脚本，原因在于其涉及进程间通信、网络连接以及复杂的认证流程等多个环节。

6.11.1 常用调试手段

一些常用的调试手段如下。

列出所有已配置的服务器及其当前状态

```
claude mcp list
```

专门测试特定服务器的连接连通性

```
claude mcp test my-server
```

启用MCP调试模式

```
claude --mcp-debug
```

6.11.2 MCP工具输出的Token管理

针对MCP工具可能返回海量数据（例如，未加LIMIT限制的数据库查询可能返回数万行记录）的情况，Claude Code内置了专门的Token管理机制。

- 警告阈值：当输出超过10 000 Token时，系统会显示警告提示。
- 截断阈值：当输出超过25 000 Token时，系统将自动对内容进行截断，防止上下文溢出。
- 自定义配置：用户可以通过设置环境变量MAX_MCP_OUTPUT_TOKENS来调整这一输出上限，以适应不同的业务场景需求。

6.11.3 常见问题

MCP常见问题与解决方案如表6-4所示。该表可以帮助用户快速定位和解决开发中遇到的典型问题。表6-4 MCP常见问题与解决方案

问题现象可能原因解决方案

服务器无法启动命令或路径配置错误检查配置文件中的command和args参数；尝试在终端手动运行该命令以验证是否可以正常启动

连接超时网络不稳定或服务器响应过慢设置环境变量MCP_TIMEOUT（单位为毫秒），适当延长等待时间

认证失败Token错误、缺失或已过期确认相关环境变量（如API_KEY等）已正确设置；检查Token的有效性并及时刷新

输出被截断返回数据量超过Token限制调大环境变量MAX_MCP_OUTPUT_TOKENS的上限；优化查询逻辑（如增加LIMIT），让服务器返回更精简的数据

JSON解析错误服务器标准输出（stdout）被日志污染确保服务器将调试日志、错误信息等非数据内容输出至标准错误（stderr），保持stdout仅包含纯净的JSON数据

本章小结

MCP解决了AI工具集成领域的基础性问题：它通过一套统一的开放协议，将各类AI客户端与多样化的数据源无缝连接，不需要再为每种组合单独开发适配层。该协议以JSON-RPC 2.0为通信基石，其中，stdio适用于本地进程通信，HTTP适用于远程服务调用（SSE已废弃）；同时，借助环境变量替换机制，配置文件得以安全地纳入版本控制系统。

从工程设计视角审视，MCP的核心价值不仅在于其连接能力，更在于实现了连接的**标准化**。如今接入Claude的MCP服务器，理论上可无缝兼容任何遵循该标准的AI客户端；反之，随着标准的推广普及，能接入Claude的MCP服务器生态也将持续扩容。MCP自2024年11月发布至2025年12月捐赠给Linux基金会，其在短短13个月内便从单一企业项目蜕变为由行业联盟管理的行业标准。这一演进速度本身，便有力地证明了AI工具生态对标准化的迫切需求。

与第5章讨论的Hooks及第3章所述的Skills相比，MCP的定位截然不同：Hooks旨在“界定Claude的行为禁区”，Skills侧重“指导Claude的执行策略”，而MCP则致力于“拓展Claude的数据和服务触达边界”。三者相辅相成——MCP赋予连接能力，Skills规范调用逻辑，Hooks筑牢安全防线——共同构建起一套完备的AI工程化体系。

在第7章中，我们将跳出交互式使用场景，深入探讨Claude Code如何在无人值守的环境下运行，重点解析Headless模式及其与CI/CD流程的集成实践。

思考题₁

请审视你当前的工作流：哪些数据源或工具若能被Claude实时访问，将显著提升效率？请尝试为这些工具设计一份.mcp.json配置文件。

2. 相较于直接在对话中粘贴数据（如将导出的JiraCSV文件发送给Claude），MCP在“便捷性”之外，本质上还有哪些核心差异化优势？又在何种特定场景下，直接粘贴数据反而是更优的选择？

3. 在“厨房与菜谱”的协同架构中（MCP为厨房，Skills为菜谱），若需要为公司某内部工具设计全套集成方案，你会如何规划？（提示：在MCP服务器层面，应暴露哪些核心工具以最大化其效用？在Skills指导层面，配套的Skill应如何指导Claude遵循特定的业务工作流程来调用这些工具？）

第7章 无为而治：Headless模式与CI/CD集成

运筹帷幄之中，决胜于千里之外。

公司的技术团队仅有3名高级工程师，却每天需要审查二十多个PR。按照惯例，每个PR必须至少获得一名资深工程师的批准（Approved）方可合并。后果显而易见：PR大量积压，发布频率被迫降低；开发者在等待审查期间不得不切换至其他任务，导致上下文频繁中断，效率受损。更为严峻的是，在审查负荷最高的时段，人工审查的质量开始下滑——疲惫的审查者极易遗漏安全漏洞或性能隐患，而这些问题一旦流入生产环境，其修复代价将成倍增加。

经过一周的调研，小雪提出了一套解决方案：引入Claude作为PR的“初审员”，自动执行安全检查、代码质量分析及测试覆盖率评估，并生成结构化的审查报告。人工审查员则基于该报告进行最终决策，将精力集中于业务逻辑与架构设计等核心领域。该方案并非替代人工审查，而是将工程师从重复性的机械检查中解放出来，使其专注于唯有人类才能胜任的高阶判断。

“Claude Code能在无人值守的情况下自动运行吗？”小冰问。

“这正是Headless模式的核心。”咖哥解释道，“在前面6章中，我们一直是在‘对话模式’下使用Claude——你输入指令，它给出反馈，如此往复。但从本章开始，Claude Code将脱离人类的实时操作，能够在后台、在CI/CD流水线中，甚至在凌晨三点无人值守时独立工作。这种转变的意义，堪比Docker的出现将应用从‘绑定于特定机器’解放为‘可在任何环境中运行’。”

7.1 从人机交互到无人值守：一次关键的架构演进

在交互模式下，Claude Code扮演着“在场对话伙伴”的角色：用户输入指令，Claude即时响应，待用户确认结果后继续下一轮交互。这种模式非常适合于探索性工作，如代码调试、架构方案的头脑风暴或模块的渐进式重构。然而，在自动化场景中，键盘前无人守候，亦无人为中间步骤把关，整个流程必须具备在零人工干预下从头至尾独立运行的能力。

Headless模式正是为此所设计的。通过-p（或--print）参数，用户可明确告知Claude Code：“这是一次非交互式调用，请在任务完成后直接输出结果并退出，不需要等待任何用户输入。”

最基本的Headless模式调用

```
claude -p "分析src/目录的安全漏洞"
```

等价的完整形式

```
claude --print "分析src/目录的安全漏洞"
```

然而，-p仅仅是入口。真正将Claude Code转化为“可编程组件”的，是其完备的参数体系。要深入理解这套体系，首先需要厘清交互模式与Headless模式的核心差异（见表7-1）。表7-1 交互模式与Headless模式的核心差异

维度用户界面输入方式输出方式权限确认会话持久化成本控制典型场景交互模式终端TUI，实时渲染多轮实时对话流式渲染Markdown弹出交互提示，需要人工确认自动保存，支持断点续传（resume）依赖人工观察与中断开发调试、架构探索Headless模式无界面，仅标准输出（stdout）一次性Prompt（或stdin管道）纯文本/JSON/流式JSON自动跳过或依据白名单执行默认不保存（可按需指定）通过--max-turns/--max-budget-usd硬性限制CI/CD流水线、定时任务、批处理

这一演进的意义远不止“少了一个界面”那么简单。在交互模式下，人类实际上扮演着三重关键角色：权限审批者（实时决策是否允许执行特定命令）、质量检查者（即时判断输出结果是否符合预期）和流程控制者（根据中间结果动态调整下一步指令）。当这3个角色全部缺席时，系统必须在运行前将所有规则“硬编码”到参数中：哪些工具被授权调用？最多执行多少轮对话？成本预算上限是多少？输出必须遵循何种格式以便下游程序解析？这正是Headless模式庞大参数体系的核心使命——将原本依赖人类直觉和实时判断的模糊过程，转化为一套确定性、可预测、可自动执行的工程规范。

7.2 核心参数体系：4个维度的控制

咖哥在白板上勾勒出一个清晰的四象限图，展示了Headless模式参数体系的4个维度，如图7-1所示。

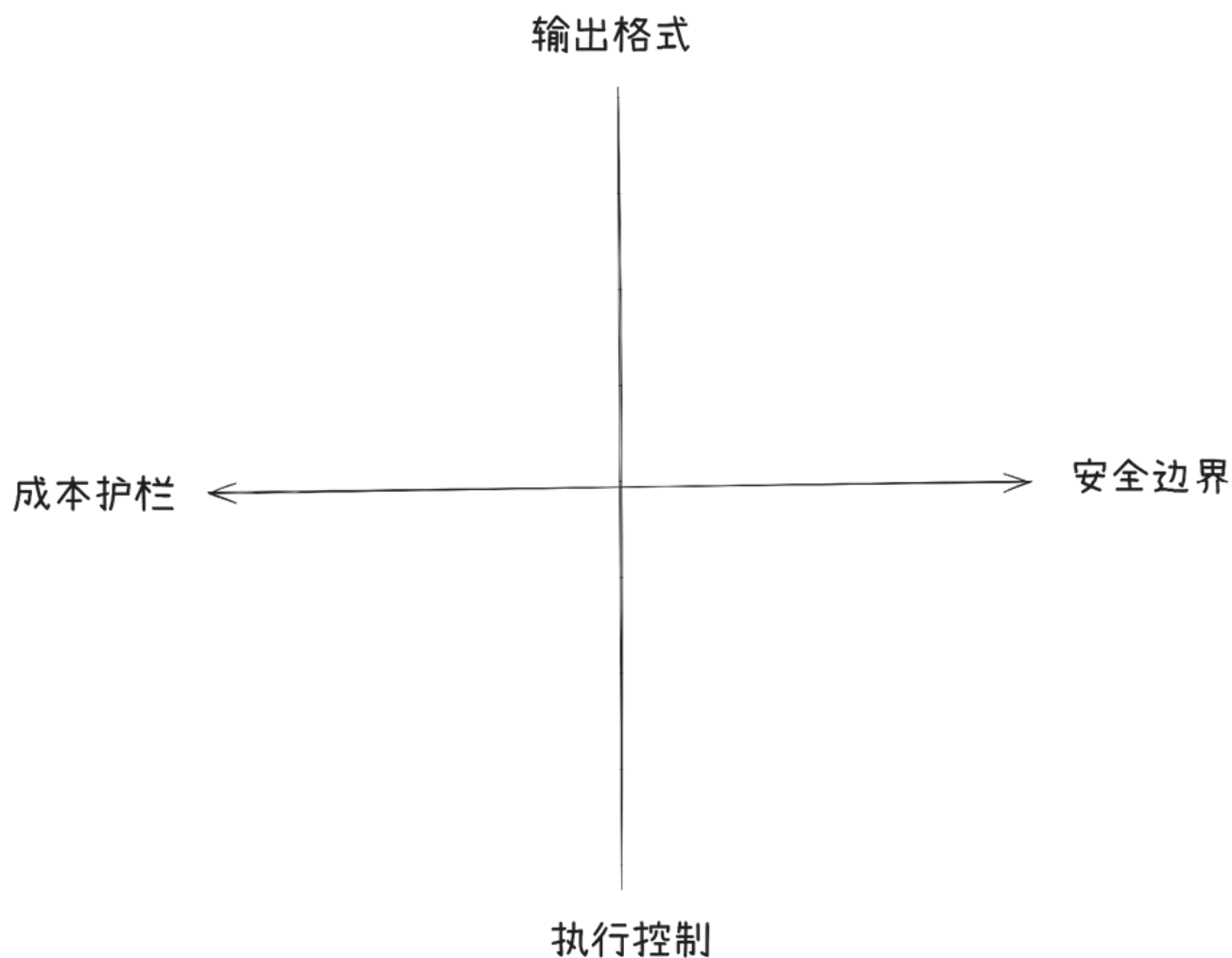


图7-1 Headless模式参数体系的4个维度

“看，”咖哥指着白板说道，“Headless模式的参数并非杂乱无章，而是可以归纳为这4个维度。任何一次生产级的无人值守调用，都必须在这4个维度上作出明确且坚定的决策。”

7.2.1 输出格式控制

output-format参数是连接Claude Code与下游自动化流水线的“协议转换器”。它明确了输出数据的结构，直接决定了后续脚本或系统如何消费这些结果。

该参数包含3种核心格式——text格式、json格式和stream-json格式。

1. text格式

最简单的纯文本输出，保留了自然的语言流，适用于直接阅读或简单的脚本处理。

claude -p "列出主要问题" --output-format text **2. json格式**

输出为一个完整的、严格结构化的JSON对象。它不仅包含任务结果，还封装了丰富的执行元数据，是构建健壮CI/CD流水线的基石。

claude -p "审查 PR" --output-format json 当使用json格式时，返回的对象包含审计和监控所需的全量信息。

```
{  
  
  "type": "result", "subtype": "success", "session_id": "550e8400-e29b-41d4-a716-  
446655440000", "is_error": false, "duration_ms": 12345, "duration_api_ms": 10000,  
  "num_turns": 5, "total_cost_usd": 0.0342, "usage": {  
  
    "input_tokens": 5000, "output_tokens": 1500, "cache_creation_input_tokens": 0,  
    "cache_read_input_tokens": 3000  
  
  }, "result": "发现2个 Critical问题: \n1. SQL注入风险...", "structured_output": null }
```

在自动化运维和成本优化中，以下几个字段至关重要。

- **total_cost_usd**可用于精确成本追踪，在CI/CD流水线中，用户可以将每次运行的成本记录到数据库，生成“每个PR的AI审查成本”报表，甚至对超出预算的部门进行自动告警或阻断。
- **num_turns**可用于执行轮次监控，反映模型实际交互的深度（对比设定的--max-turns限制）。
- **usage.cache_read_input_tokens**反映Prompt缓存的生效情况。在频繁调用相同System Prompt或基础代码库的场景中，cache_read_input_tokens越高，意味着跳过的计算越多，成本越低且速度越快。

3. stream-json格式

逐行输出JSON事件（即NDJSON格式），专为长任务的实时监控而设计。示例如下。

claude-p "分析代码" --output-format stream-json 输出中，每一行均为一个独立的JSON对象，并严格按时间顺序排列。

```
{"type": "system", "subtype": "init", "session_id": "...", "model": "claude-sonnet-4-6", "tools":  
["Read", "Grep", "Glob"]}
```

```
{"type":"assistant","message":{"role":"assistant","content":[{"type":"text","text":"正在分析..."}]}}
```

```
{"type":"user","message":{"role":"user","content":[{"type":"tool_result","tool_use_id":"...", "content":"..."}]}}
```

```
{"type":"assistant","message":{"role":"assistant","content":[{"type":"text","text":"分析完成。"}]}}
```

```
{"type":"result","subtype":"success","total_cost_usd":0.03,"result":"最终结果"}
```

请特别关注首行的system/init事件，它汇报了当前会话的模型版本、可用工具列表及MCP服务器状态等初始化信息。这一特性在调试CI配置问题时非常有用。

“3种格式的选择其实很简单，”咖哥总结道，“需要人工阅读就采用纯文本格式，需要机器解析就采用JSON格式，需要实时监控就采用stream-json格式。”

7.2.2 成本护栏

在交互模式下，你可以随时按下Ctrl+C组合键中断Claude的执行；但在Headless模式下，由于缺乏这种“人工熔断”机制，必须通过参数来实施控制。请看以下代码示例。

```
# 限制执行轮数（防止过度分析）
```

```
claude -p "审查代码" --max-turns 5
```

```
# 设置硬性成本上限（美元）
```

```
claude -p "分析日志" --max-budget-usd 0.50
```

```
# 双重保险：同时限制轮数与成本
```

```
claude -p "分析整个代码库" --max-turns 10 --max-budget-usd 2.00
```

--max-turns与--max-budget-usd分别从不同维度进行约束。

- **--max-turns**：限制Claude与工具之间的交互轮数。每调用一次工具计为一轮。例如，一个读取了5个文件后给出结论的任务，通常只需要6轮。

- **--max-budget-usd**: 限制API调用的总金额。即使交互轮数很少（如仅3轮），如果每轮涉及大量Token（如读取巨型日志文件），累计成本仍可能很高，此时该参数将起到关键的保护作用。

两者在触发时的行为机制截然不同。

- 当**--max-turns**耗尽时，Claude会在当前轮次输出其已得出的结论。尽管该结论可能尚不完整，但通常仍具备参考价值。在JSON输出中，此类事件的subtype标记为**error_max_turns**。
- 当**--max-budget-usd**触发时，执行过程会立即强制停止，不再输出任何后续结论。在JSON输出中，此类事件的subtype标记为**error_max_budget_usd**。

“在生产环境中，务必同时配置这两个参数。”咖哥强调，“**--max-turns**用于防止Claude在处理复杂任务时陷入无限递归，而**--max-budget-usd**则能避免因一次意外的大范围分析而耗尽API预算。这两道防线缺一不可。”

7.2.3 安全边界：工具权限

在Headless模式下，由于缺乏人工确认工具调用的环节，必须预先划定Claude的能力边界，这是CI/CD场景中最关键的安全设置。请看以下代码示例。

白名单模式：仅允许只读工具（最安全的审查配置）

```
claude -p "审查代码变更" --allowedTools "Read,Grep,Glob"
```

黑名单模式：禁止特定工具

```
claude -p "生成文档" --disallowedTools "Bash"
```

细粒度模式：允许特定的命令模式

```
claude -p "检查 Git 历史" --allowedTools "Read,Grep,Glob,Bash (git log ),Bash(git diff)"
```

--allowedTools（白名单）与**--disallowedTools**（黑名单）在安全语义上截然不同。

- **白名单基于“封闭世界假设”**：仅允许使用明确列出的工具，其余工具一律拒绝。
- **黑名单基于“开放世界假设”**：除明确禁止的工具以外，其余所有工具均可使用。

在自动化场景中，白名单的安全性永远高于黑名单。这是因为采用白名单时，你不需要预见所有潜在的危险工具，只需要列出任何确实需要的工具即可。

请注意，`--allowedTools`支持模式匹配语法。例如，`Bash(git log *)`表示“允许使用Bash工具，但仅限执行以`git log`为开头的命令”。这种细粒度的控制机制，在需要部分Shell能力（如Git操作）却又不敢赋予完整 Bash 权限的场景下尤为实用。

此外，还有一个更为激进的参数——`--dangerously-skip-permissions`，用于跳过所有权限检查。切勿忽视其名称中的`dangerously`一词，这绝非装饰。在CI环境中，严禁使用此参数，除非你在严格隔离的容器中运行，且确信 Claude的操作无法造成超出容器范围的影响。

7.2.4 执行控制：模型、Prompt、结构化输出

我们可以通过模型选择、System Prompt定制、结构化输出来控制Claude的“思考方式”。

1. 模型选择

不同任务适配不同模型，需要根据需求权衡。请看以下代码示例。

```
# 简单格式检查（快速且低成本）
```

```
claude -p "检查代码格式" --model claude-haiku-4-5
```

```
# 深度安全审查（使用最强模型）
```

```
claude -p "安全漏洞分析" --model claude-sonnet-4-6
```

```
# 指定备选模型（主模型过载时自动降级）
```

```
claude -p "代码审查" --model claude-sonnet-4-6 --fallback-model haiku --fallback-model
```

是生产中至关重要但极易被忽视的参数。在API高峰期，若主模型不可用导致CI流水阻塞，所有PR都将停滞。设置降级模型可确保流水线在牺牲少许质量的前提下依然保持运行，避免流程中断。

2. System Prompt定制

通过定制System Prompt来定义Claude的角色与行为边界。请看以下代码示例。

完全替换System Prompt（这将使Claude忘记其默认的代码助手身份，仅遵循新指令）

```
claude -p "分析代码" --system-prompt "你是一名专注于OWASP Top 10的应用安全专家。"
```

追加System Prompt（推荐，在保留Claude基础能力的同时，增加特定约束）

```
claude -p "审查 PR" --append-system-prompt "务必检查敏感信息硬编码。"
```

从文件加载追加Prompt（适用于包含复杂审查规范的大型项目）

```
claude -p "审查代码" --append-system-prompt-file ./review-guidelines.txt
```

参数`--system-prompt`与`--append-system-prompt`的核心区别在于它们对Claude默认行为的处理方式不同。

- **--system-prompt:** 彻底覆盖Claude默认的System Prompt。Claude将不再知道自己是一个代码助手，也不再理解如何正确使用Read、Grep、Bash等内置工具。它只会机械地遵循你提供的新指令，极易导致工具调用失败或行为不可控。
- **--append-system-prompt:** 在Claude默认的System Prompt之后追加你的自定义内容。Claude保留了所有基础能力，同时严格遵循你添加的额外指示。适用于绝大多数CI/CD自动化场景。

3.结构化输出

强制Claude输出符合特定JSON Schema的数据，从而实现程序化的无缝集成。请看以下代码示例。

```
claude -p "提取代码中所有安全问题" \ --output-format json \ --json-schema '{
  "type": "object", "properties": {
    "issues": {
      "type": "array", "items": {
        "type": "object", "properties": {
          "severity": {"type": "string", "enum": ["critical", "high", "medium", "low"]}, "file":
{"type": "string"}, "line": {"type": "number"}, "description": {"type": "string"}}
```



```

    }, "required": ["severity", "file", "description"]
  },
  },
  },
  }, "required": ["issues"]
},
}'

```

使用`--json-schema`参数后，系统会自动验证Claude的输出。如果生成的内容不符合定义的Schema，Claude会自动重试生成过程，直到产出合法有效数据为止。最终验证通过的结构化数据将出现在JSON输出的`structured_output`字段中。该功能在需要程序化处理Claude输出的场景中极为有用——无需编写复杂的正则表达式或启发式规则来解析自由文本，即可直接获取标准的JSON对象，供下游脚本、数据库或报警系统调用。

7.3 Unix管道：将Claude融入命令行 workflow

“我注意到一个有趣的设计，”小雪说，“Claude Code似乎支持标准输入（stdin）？”

“没错，”咖哥回答道，“这正是Headless模式最优雅的特性之一——Claude Code能够无缝接入Unix管道，成为你工具链中灵活的一环。”

Unix哲学的核心是“每个工具只做好一件事，并通过管道将它们组合起来”。Claude Code完全契合了这一设计理念。请看以下代码示例。

分析日志文件

```
cat server.log | claude -p "找出所有500错误并总结根本原因"
```

分析git变更

```
git diff HEAD~1 | claude -p "总结这次提交的变更，并按Conventional Commits规范格式化"
```

解析API响应

```
curl -s https://api.example.com/health | claude -p "判断服务是否健康"
```

更有趣的是多段管道组合——Claude既能接收上游数据，也能将处理结果传递给下游工具。请看以下代码示例。

```
# Claude分析 → jq提取 → 发送通知
```

```
claude -p "检查是否存在安全漏洞" --output-format json | \ jq -r '.result' | \ mail -s "安全扫描报告" security@company.com
```

```
# Grep预过滤 → Claude深度分析
```

```
grep -r "TODO" src/ | claude -p "将这些待办事项按优先级分类"
```

```
# Claude 生成代码 → 直接写入文件
```

claude -p "生成一个Express健康检查路由" --output-format text > routes/health.js 在批量处理场景中，这种管道组合能力使Claude化身为一个“智能过滤器”。传统的文本处理工具（如Grep、Awk、Sed）擅长基于模式的精确匹配，却缺乏语义理解能力；而Claude虽精通语义分析，但高效处理大规模数据并非其强项。通过管道将两者结合，可实现优势互补：先利用Grep筛选出关键内容，再交由Claude进行深度的语义分析。

7.4 实战一：GitHub Actions自动代码审查

理论阐述至此已经足够，接下来通过一个完整的工程实例进行演示。

Anthropic提供了两种GitHub Actions集成方案：一是采用官方Action（anthropics/claude-code-action@v1），其封装程度高、配置简便；二是直接调用CLI，其灵活性更强、控制更为精细。

7.4.1 采用官方Action

Claude Code内置了便捷的安装命令：在交互模式下执行/install-github-app，即可引导用户完成GitHub App安装、Secret配置及Workflow文件创建的全流程。

若选择手动配置，步骤亦不烦琐。首先，进入GitHub仓库，然后单击Settings→Secrets→Actions命令，添加ANTHROPIC_API_KEY，随后创建如下Workflow文件。

```
# .github/workflows/claude-review.yml name: Claude PR Review
```

```

on:

  pull_request: types: [opened, synchronize, reopened]

  issue_comment: types: [created]

# 针对同一PR的新提交自动取消正在运行的旧任务，以节省资源

concurrency:

  group: ${{ github.workflow }}-${{ github.event.pull_request.number }}

  cancel-in-progress: true

jobs:

  review: runs-on: ubuntu-latest # 仅在触发PR相关事件或评论中包含“@claude”时运行

  if: |

    github.event_name == 'pull_request' ||

    contains(github.event.comment.body, '@claude')

  permissions: contents: read pull-requests: write issues: write

  steps: - uses: actions/checkout@v4

  with: fetch-depth: 0 # 拉取完整历史，以便进行差异分析

  - uses: anthropics/claude-code-action@v1

  with: anthropic_api_key: ${{ secrets.ANTHROPIC_API_KEY }}

  prompt: |

    请审查此PR的代码变更，重点检查以下内容。

    1. 安全漏洞（如SQL注入、XSS攻击、敏感信息硬编码等）。

    2. 未处理的异常情况及边界条件。

```

3. 性能隐患（如N+1查询、内存泄漏风险）。

请按以下格式输出报告。

```
## 审查报告
```

```
### Critical（必须修复）
```

```
### Warning（建议修复）
```

```
### Suggestion（可选优化）
```

```
claude_args: >-
```

```
--allowedTools "Read,Grep,Glob"
```

```
--max-turns 10
```

```
--model claude-sonnet-4-6
```

这段配置有以下3点值得特别说明。

1.两重触发模式

官方Action v1版本具备自动检测触发模式的能力。

- **Agent Mode（代理模式）**：当配置中提供了prompt参数时触发。每次PR创建或更新，Claude都会自动执行预设的审查任务。
- **Tag Mode（标签模式）**：当用户在PR评论中提及@claude时触发。此时Claude将像团队成员一样，针对具体提问进行交互式响应。

这两种模式可在同一个Workflow文件中共存，互不冲突。

2.参数透传机制

官方Action v1版本大幅简化了配置结构。所有Claude Code CLI的参数现统一通过claude_args字段传递。此前Beta版本中独立的max_turns、allowed_tools等字段已被废弃，不再支持。

3.并发控制策略

通过设置`cancel-in-progress:true`，可实现高效的并发控制。当同一PR连续推送并多次提交时，系统自动取消正在运行的旧任务，仅保留最新提交所触发的审查流程。这一机制能有效避免重复执行，从而节省API调用费用。

7.4.2 直接调用CLI

当你需要实现更复杂的逻辑，例如，根据审查结果决定是否阻塞PR合并、将审查报告发布为正式的PR评论而非普通评论，或针对不同类型的文件应用差异化的审查策略时，直接调用Claude CLI将有更大的灵活性。

```
name: Advanced AI Review
```

```
on:
```

```
  pull_request: types: [opened, synchronize]
```

```
  paths: - 'src/**' # 仅在src目录发生变更时触发
```

```
concurrency:
```

```
  group: ${{ github.workflow }}-${{ github.event.pull_request.number }}
```

```
  cancel-in-progress: true
```

```
jobs:
```

```
  review: runs-on: ubuntu-latest permissions: contents: read pull-requests: write
```

```
    steps: - uses: actions/checkout@v4
```

```
            with: fetch-depth: 0
```

```
            - uses: actions/setup-node@v4
```

```
              with: node-version: "20"
```

```
            - name: Install Claude Code run: npm install -g @anthropic-ai/claude-code
```

```
            - name: Get changed files id: changed run: |
```

```

FILES=$(git diff --name-only origin/${{ github.base_ref }}...HEAD)

echo "files=$(echo "$FILES" | tr '\n' ' ')" >> $GITHUB_OUTPUT

echo "count=$(echo "$FILES" | wc -l)" >> $GITHUB_OUTPUT

- name: Run Claude Review id: review env: ANTHROPIC_API_KEY: ${secrets.ANTHROPIC_API_KEY}

  CLAUDE_CODE_DISABLE_NONESSENTIAL_TRAFFIC: "1"

  run: |

    claude -p "请审查以下PR变更文件：

    ${ steps.changed.outputs.files }

    重点检查安全漏洞、逻辑错误及性能问题。

    针对每个问题，请明确指出文件名和行号。" \ --output-format json \ --max-turns 10 \ --max-budget-usd 0.50 \ --model claude-sonnet-4-6 \ --allowedTools "Read,Grep,Glob" > review.json

    # 提取审查结果与成本信息

    RESULT=$(jq -r '.result' review.json) COST=$(jq -r '.total_cost_usd' review.json)
    IS_ERROR=$(jq -r '.is_error' review.json)

    echo "cost=$COST" >> $GITHUB_OUTPUT

    echo "is_error=$IS_ERROR" >> $GITHUB_OUTPUT

    echo "result<<EOF" >> $GITHUB_OUTPUT

    echo "$RESULT" >> $GITHUB_OUTPUT

    echo "EOF" >> $GITHUB_OUTPUT

```

```

- name: Post Review Comment uses: actions/github-script@v7

with: script: |

  const cost = `${{ steps.review.outputs.cost }}`; const result = `${{
steps.review.outputs.result }}`;

  await github.rest.issues.createComment({

    issue_number: context.issue.number, owner: context.repo.owner, repo: context.repo.repo,
body: `## Claude Code Review\n\n${result}\n\n---\n*Cost: ${cost} | Automated by Claude
Code*`

  });

- name: Gate on critical issues run: |

RESULT="${{ steps.review.outputs.result }}"

if echo "$RESULT" | grep -qi "critical"; then echo "::error::Critical issues found"

exit 1

fi

```

上述配置虽然较官方Action复杂，但赋予了用户3项关键能力：**成本透明化**（每次审查的具体费用将直接显示在评论中）、**质量门禁**（当检测到Critical级别的严重问题时，Workflow会自动失败并阻塞PR合并）、**路径过滤**（通过paths限定，仅在src/目录发生变更时触发审查）。

此外，需要注意CLAUDE_CODE_DISABLE_NONESSENTIAL_TRAFFIC环境变量。这是CI/CD流水线中的标准最佳实践，用于一次性禁用自动更新检查、错误上报及遥测数据发送等在自动化场景中非必要的功能。

7.5 实战二：多阶段CI管道^{7.4}

节介绍的案例属于单阶段审查。在实际项目中，可能需要让Claude参与多个阶段，例如，先审查代码质量，再扫描安全漏洞，最后检查测试覆盖率。每个阶段往往具有不同的关注点、工具权限限制以及成本预算。

以下是一个多阶段AI流水线的配置示例。

```
name: Multi-Stage AI Pipeline

on:

  pull_request: types: [opened, synchronize]

jobs:

  # 阶段1: 快速格式检查（使用低成本模型）

  lint-check: runs-on: ubuntu-latest steps: - uses: actions/checkout@v4

    - name: Format Check env: ANTHROPIC_API_KEY: ${ secrets.ANTHROPIC_API_KEY }

      CLAUDE_CODE_DISABLE_NONESSENTIAL_TRAFFIC: "1"

      run: |

        npx @anthropic-ai/claude-code -p "检查src/下的代码是否符合项目编码规范" \ --model claude-
haiku-4-5 \ --max-turns 3 \ --max-budget-usd 0.05 \ --allowedTools "Read,Grep,Glob" \ --
output-format text

  # 阶段2: 深度安全审查（使用高性能模型）

  security-scan: runs-on: ubuntu-latest needs: lint-check # 仅在格式检查通过后执行

  steps: - uses: actions/checkout@v4

    - name: Security Scan env: ANTHROPIC_API_KEY: ${ secrets.ANTHROPIC_API_KEY }

      CLAUDE_CODE_DISABLE_NONESSENTIAL_TRAFFIC: "1"

      run: |

        npx @anthropic-ai/claude-code -p "深度安全扫描，重点检查OWASP Top 10" \ --model claude-
sonnet-4-6 \ --append-system-prompt "你是一名应用安全专家，请仅关注安全漏洞问题。" \ --max-
turns 10 \ --max-budget-usd 1.00 \ --allowedTools "Read,Grep,Glob" \ --output-format json \
--json-schema '{"type":"object","properties":{"vulnerabilities":{"type":"array","items":
{"type":"object","properties":{"severity":{"type":"string"},"description":
```



```

{"type":"string"},"file":{"type":"string"}},{"required":
["severity","description"]}]}},"required":["vulnerabilities"]}]' \ > security-report.json

# 阶段3: 测试覆盖分析

coverage-check: runs-on: ubuntu-latest needs: lint-check # 依赖于格式检查, 但与安全检查并
行

steps: - uses: actions/checkout@v4

- name: Coverage Analysis env: ANTHROPIC_API_KEY: ${ secrets.ANTHROPIC_API_KEY }

CLAUDE_CODE_DISABLE_NONESSENTIAL_TRAFFIC: "1"

run: |

npx @anthropic-ai/claude-code -p "分析哪些代码路径缺少测试覆盖" \ --model claude-sonnet-
4-6 \ --max-turns 8 \ --max-budget-usd 0.50 \ --allowedTools "Read,Grep,Glob" \ --output-
format text 上述配置涉及3个设计要点。

```

- **模型分级**: 针对轻量级任务（如格式检查），选用**Haiku**（该模型响应速度快、成本低）；针对重量级任务（如安全扫描和测试覆盖分析），选用**Sonnet**（利用其更强的推理能力进行深度分析）。根据不同任务的复杂度匹配相应的“算力配比”，实现效率与效果的最优平衡。
- **预算分级**: 为简单任务设定严格上限（如0.05美元），为复杂任务预留充足空间（如1美元）。通过`--max-budget-usd`参数精确控制单次运行的最大支出，确保整体流水线成本可控且可预测。
- **并行执行**: 在任务`security-scan`和`coverage-check`中均设置了`needs: lint-check`，意味着它们必须等待第一阶段完成后才能启动。由于这两个任务之间没有相互依赖，GitHub Actions会自动将它们并行运行。

7.6 流式输出：实时监控长耗时任务

对于耗时较长的分析任务，`stream-json`格式允许用户实时观测Claude的执行进度，而不需要等待工作全部完成后才查看最终结果。请看以下代码示例。

```

claude -p "分析整个src/目录的架构问题" \ --output-format stream-json \ --allowedTools
"Read,Grep,Glob" | while read -r line; do TYPE=$(echo "$line" | jq -r '.type // ""')

    case "$TYPE" in "system") # 初始化阶段：输出模型信息

        echo "[INIT] 模型: $(echo "$line" | jq -r '.model // "unknown"')"

        ;; "assistant") # 提取并输出Claude生成的文本内容

            TEXT=$(echo "$line" | jq -r '.message.content[0].text // ""') [ -n "$TEXT" ] && echo "
[CLAUDE] $TEXT"

            ;; "result") # 任务结束：输出总轮数与费用

                COST=$(echo "$line" | jq -r '.total_cost_usd // 0') TURNS=$(echo "$line" | jq -r
'.num_turns // 0') echo "[DONE] 任务完成！共 $TURNS 轮，费用: \\\$COST"

                ;; esac done

```

stream-json格式定义了一套完整的事件协议。除上述案例中涉及的system（初始化）、assistant（助手回复）、user（用户输入）及result（结果汇总）以外，还有两类特殊事件值得注意。

- **stream_event**（需要添加`--include-partial-messages`参数）：提供Claude输出的增量数据，精确到每一个Token级别的文本片段。适用于需要实现“打字机效果”的用户界面，但在CI自动化流程中通常不需要启用。
- **system/compact_boundary**：上下文压缩边界事件。当对话历史长度接近模型的上下文窗口限制时，Claude会自动触发机制压缩早期内容以腾出空间。在长耗时分析任务中，若检测到此事件，表明当前任务已消耗大量上下文资源，需要注意潜在的上下文丢失风险。

7.7 会话管理：跨步骤维持上下文

在Headless模式下，系统默认不持久化会话状态——每次调用均视为独立的全新上下文。然而，在涉及多轮交互的复杂场景中，往往需要在多次调用之间保持上下文的连续性。请看以下代码示例。

```
# 第一步：执行初始分析
```

调用Claude分析代码结构，并捕获完整的JSON输出

```
RESULT=$(claude -p "分析 src/ 的模块依赖关系" \ --output-format json \ --allowedTools  
"Read,Grep,Glob")
```

第二步：提取会话标识符

从返回结果中解析session_id，用于后续恢复会话

```
SESSION_ID=$(echo "$RESULT" | jq -r '.session_id')
```

第三步：基于上下文继续对话

使用 --resume 参数载入上一轮的会话状态，实现连续推理

```
claude -p "基于刚才的分析，指出循环依赖问题" \ --resume "$SESSION_ID" \ --output-format text
```

--resume参数的核心价值在于让Claude完整“继承”上一轮对话的上下文状态，包括已读取的文件内容、执行过的分析步骤以及得出的中间结论。相较于在新一轮Prompt中手动复述上下文信息，这种方式不仅显著提升了交互效率，更确保了模型推理的准确性与连贯性。

除基础的会话恢复以外，CLI工具还提供了一系列高级参数以应对复杂的交互场景。请看以下代码示例。

自动续接：继续当前目录下最近的会话

不需要手动查找session_id，系统自动匹配并更新

```
claude -p "继续刚才的分析" --continue
```

会话分支：基于现有会话创建新分支

保留原始会话不变，从指定节点衍生出新的探索路径

```
claude -p "如果改用微服务架构呢?" --resume "$SESSION_ID" --fork-session
```

禁用持久化：强制无状态模式

适用于完全隔离的CI/CD流水线，确保不留下任何会话历史

`claude -p “一次性分析” --no-session-persistence --fork-session`是一个极具实用价值的参数，其应用逻辑类似于版本控制系统中的Git分支机制。它从指定会话的某个时间点“分叉”出一个全新的会话实例。原始会话（主干）保持原样，不受任何影响；新会话（分支）则从该分叉点开始独立演进。这一特性完美契合“假设性分析”的需求。例如，在完成同一份代码库的基础分析后，开发者可以通过该参数创建多个分支，分别探讨不同的重构方向。

7.8 CI 环境配置：生产级清单

在真实的CI/CD环境中部署Claude Code，除核心参数`-p`和`--allowedTools`以外，还需要设置一系列环境变量。以下是咖哥整理的“CI环境配置清单”。

7.8.1 必须设置的环境变量

必须设置的环境变量如下。

认证

`ANTHROPIC_API_KEY=sk-ant-...` # API密钥

功能禁用

方案一：使用聚合变量（推荐，可一次性禁用多项非必要功能）

`CLAUDE_CODE_DISABLE_NONESSENTIAL_TRAFFIC=1` # 禁用自动更新、遥测及错误上报

方案二：分别禁用（如需更细粒度的控制）

`DISABLE_AUTOUPDATER=1` # 禁用自动更新（CI环境中必需）

`DISABLE_TELEMETRY=1` # 禁用遥测

`DISABLE_ERROR_REPORTING=1` # 禁用错误上报

`DISABLE_AUTOUPDATER`是CI/CD环境中至关重要的环境变量。如果未设置该变量，Claude Code可能在执行过程中尝试自动更新，从而引发版本不一致或服务意外中断的风险。相比之下，`CLAUDE_CODE_DISABLE_NONESSENTIAL_TRAFFIC`则是一个便捷的“总开关”，能够一次性禁用所有非必要的后台流量（包括自动更新、遥测及错误上报），简化配置流程。

7.8.2 性能调优参数

涉及性能调优的参数如下。

超时控制

Bash 命令默认超时时间（单位：毫秒）

BASH_DEFAULT_TIMEOUT_MS=120000 # 对应2min

Bash 命令最大允许超时时间（单位：毫秒）

BASH_MAX_TIMEOUT_MS=600000 # 对应10min

输出控制

CLAUDE_CODE_MAX_OUTPUT_TOKENS=32000 # 模型单次响应最大Token数（默认值32 000，上限64 000）

MAX_MCP_OUTPUT_TOKENS=25000 # MCP工具响应最大Token数

#上下文管理

CLAUDE_CODE_AUTOCOMPACT_PCT_OVERRIDE=50 # 自动压缩触发阈值（取值范围1~100，代表上下文使用百分比；默认值50%）

7.8.3 GitHub Actions中的完整配置模板

以下是在GitHub Actions工作流中部署Claude Code的完整配置片段。

```
- name: Run Claude Analysis env: # 认证密钥（从仓库Secrets中获取）
```

```
  ANTHROPIC_API_KEY: ${{ secrets.ANTHROPIC_API_KEY }}
```

```
  # 性能与环境控制
```

```
  CLAUDE_CODE_DISABLE_NONESSENTIAL_TRAFFIC: "1" # 禁用非必要后台流量
```

```
  BASH_DEFAULT_TIMEOUT_MS: "300000" # Bash命令默认超时：5 min
```

```
run: |
```

```
npx @anthropic-ai/claude-code -p "$PROMPT" \ --output-format json \ --max-turns 10 \ --max-budget-usd 1.00 \ --allowedTools "Read,Grep,Glob" \ --model claude-sonnet-4-6 \ --fallback-model haiku \ --append-system-prompt "遵循项目根目录CLAUDE.md中的审查规范。"
```

7.8.4 MCP服务器在CI中的配置

如果CI任务需要借助MCP服务器扩展能力（如数据库查询、外部API调用等），可通过`--mcp-config`参数加载专用配置文件，并配合`--strict-mcp-config`确保环境隔离。请看以下代码示例。

```
npx @anthropic-ai/claude-code \ -p "分析数据库Schema的优化空间" \ --mcp-config ./ci-mcp-config.json \ --strict-mcp-config \ --allowedTools "Read,Grep,Glob,mcp__database__query" \ --max-turns 10
```

`--strict-mcp-config`参数表示启用严格模式，强制Claude仅使用`--mcp-config`指定的配置文件。忽略用户主目录和项目根目录中的其他MCP配置。这在CI环境中很重要——可以防止开发者本地残留的MCP配置被意外加载。

7.9 跨平台CI/CD集成

尽管GitHub Actions拥有官方Action支持，但Claude Code的Headless模式本质上是与平台无关的。只要运行环境支持Node.js，即可在任何CI/CD平台上部署。

7.9.1 GitLab CI/CD配置

以下配置展示了如何在GitLab Runner中执行代码审查任务，并将结果结构化输出。

```
# .gitlab-ci.yml

claude-review:

  image: node:20

  variables: ANTHROPIC_API_KEY: $ANTHROPIC_API_KEY

  CLAUDE_CODE_DISABLE_NONESSENTIAL_TRAFFIC: "1"
```

```

script: - npm install -g @anthropic-ai/claude-code - |

claude -p "审查MR的代码变更，重点关注潜在的安全漏洞和性能问题" \ --output-format json \ -
-max-turns 10 \ --max-budget-usd 0.50 \ --allowedTools "Read,Grep,Glob" > review.json - cat
review.json | jq -r '.result'

rules: - if: $CI_PIPELINE_SOURCE == "merge_request_event"

```

值得注意的是，Anthropic与GitLab的合作正在深化。目前GitLab已推出官方AI功能（Beta阶段），引入了AI_FLOW_INPUT和AI_FLOW_CONTEXT等专用变量，不需要手动拼接Prompt上下文；通过gitlab-mcp-server直接集成，允许AI模型安全地访问GitLab API。

7.9.2 JenkinsPipeline集成配置

在Jenkins中，可以通过Declarative Pipeline（声明式流水线）轻松集成Claude Code。以下配置展示了如何安全地管理凭据、执行Headless模式下的代码审查，并解析结构化输出。

```

pipeline {

    agent any environment {

        ANTHROPIC_API_KEY = credentials('anthropic-api-key')
        CLAUDE_CODE_DISABLE_NONESSENTIAL_TRAFFIC = '1'

    }

    stages {

        stage('AI Code Review') {

            steps {

                sh 'npm install -g @anthropic-ai/claude-code'

                sh '''

                    claude -p "审查最近提交的代码变更，重点关注逻辑错误和代码规范" \ --output-format json \ -
                    -max-turns 10 \ --max-budget-usd 0.50 \ --allowedTools "Read,Grep,Glob" > review.json '''

                script {

```

```

def review = readJSON file: 'review.json'

echo "审查结果: ${review.result}"

echo "费用: \${review.total_cost_usd}"

    }

    }

    }

    }

    }

```

7.9.3 本地自动化脚本

除CI/CD平台以外，Claude Code的Headless模式也非常适合封装成本地脚本，供团队成员在提交代码前进行自检，或作为本地自动化工具链的一部分。

以下是一个增强版的review.sh脚本，具备环境检查、动态文件筛选、结构化报告生成及错误处理能力。

```

#!/bin/bash

# review.sh — 团队共享的本地代码审查脚本

set -e

TARGET=${1:-.}

# 前置检查

[ -z "$ANTHROPIC_API_KEY" ] && echo "Error: ANTHROPIC_API_KEY not set" && exit 1

command -v claude &> devnull || { echo "Error: Claude Code not installed"; exit 1; }

echo "Starting review for: $TARGET"

# 构建文件列表

```



```

if [ -d "$TARGET" ]; then FILES=$(find "$TARGET" -type f \( -name ".ts" -o -name ".js" -o -
name "*.py" \) | head -20) else

FILES="$TARGET"

fi

# 运行审查

RESULT=$(claude -p "审查以下文件的代码质量和安全问题：

$FILES

输出格式：使用Markdown格式，每个问题必须指明文件名和行号。" \ --output-format text \ --max-
turns 15 \ --max-budget-usd 0.50 \ --allowedTools Read,Grep,Glob)

# 保存报告

REPORT="review-$(date +%Y%m%d-%H%M%S).md"

echo -e "# Code Review Report\n\n**Date**： $(date)\n\n**Target**： $TARGET\n\n---\n\n$RESULT" >
"$REPORT"

echo "Report saved: $REPORT"

```

7.10 安全原则与最佳实践

“在CI/CD中运行Agent是一件需要认真对待的安全事项。”咖哥的语气严肃了一些。这绝非杞人忧天。一个配置不当的Headless模式任务可能引发以下严重风险。

- 机密泄露：读取并输出.env、config.json或硬编码的Secrets文件。
- 破坏性操作：执行恶意的Shell命令，破坏构建环境或覆盖代码。
- 费用失控：陷入死循环或被诱导生成大量无意义内容，消耗大量API调用费用。
- 内容污染：在PR/MR评论中输出幻觉内容、不当言论甚至被注入攻击载荷。

防御策略必须从以下 5 个层面构建纵深防御体系。

7.10.1 最小权限原则

在实际应用场景中，可采取以下措施。

代码审查任务：只读

```
--allowedTools "Read,Grep,Glob"
```

格式修复任务：读写

```
--allowedTools "Read,Grep,Glob>Edit,Write"
```

Git操作任务：细粒度Bash白名单

```
--allowedTools "Read,Grep,Glob,Bash(git diff ),Bash(git log )"
```

严禁授予Claude无限制的Bash执行权限。即便在必须使用Shell能力的场景中，也必须通过模式匹配严格限制可执行命令的范围。

7.10.2 Secrets管理

针对Secrets管理，这里通过正确与错误的做法对比说明。

错误做法：硬编码密钥（密钥将直接明文出现在 Actions 运行日志中）

```
env:
```

```
  ANTHROPIC_API_KEY: "sk-ant-xxx"
```

正确做法：引用仓库密钥

```
env:
```

```
  ANTHROPIC_API_KEY: "${{ secrets.ANTHROPIC_API_KEY }}"
```

GitHub Actions会自动在日志中对secrets引用的值进行遮蔽（显示为 ***）。

尽管GitHub具备日志遮蔽机制，但若在Prompt中指示模型“列出所有环境变量”或“打印配置详情”，模型可能会在输出文本中复述密钥值。这种由模型生成的文本

无法被自动遮蔽机制捕获，从而导致密钥泄露。

严禁在Prompt中要求模型访问、读取或输出任何环境变量及密钥值。

7.10.3 容器隔离

以下展示了用于构建高度安全的“沙箱”环境以运行代码审查任务的GitHub Actions workflow配置片段。

jobs:

```
review: runs-on: ubuntu-latest container: image: node:20
```

options: --read-only --tmpfs /tmp --network none --read-only: 将文件系统设为只读，防止写入操作（配合使用--tmpfs /tmp为临时文件提供可写空间）。

--network none: 在不需要外部访问的审查任务中完全切断网络连接。在此模式下，Claude仅能读取代码仓库内的文件，无法与外界通信。

7.10.4 成本防护

以下展示了GitHub Actions workflow中用于优化资源使用并降低运行成本的配置片段。

路径过滤：仅在src目录变更时触发

on:

```
pull_request: paths: - 'src/**'
```

```
- '!src/**/*test.*' # 排除测试文件
```

并发限制：同一PR仅保留最新一次运行，自动取消中间版本

concurrency:

```
group: claude-${{ github.event.pull_request.number }}
```

cancel-in-progress: true 路径过滤与并发控制是成本优化的首道防线。相比于限制单次运行的资源消耗，从源头避免不必要的触发能更有效地降低总成本。

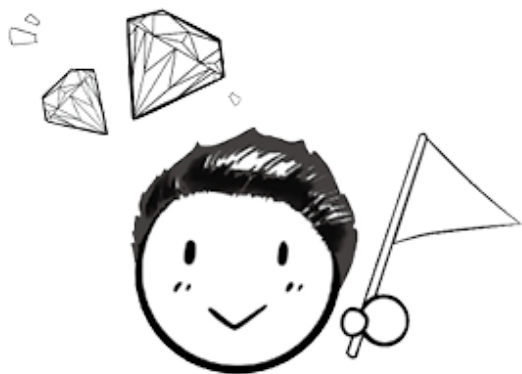
7.10.5 审计日志

以下展示了执行一个AI任务并将包含完整审计信息的运行日志持久化保存的配置片段。

JSON输出格式天然包含完整的审计信息

```
claude -p "任务" --output-format json | tee -a varlog/claude-audit.jsonl
```

每次Headless模式调用的JSON输出均自动包含session_id、total_cost_usd、num_turns及usage等关键审计字段。为严格遵循合规要求，建议将这些日志实时推送至集中式日志系统（如ELK、Datadog等），以实现长期归档与深度分析。以上五层防御绝非“选做题”，而是生产环境中的“必答题”：最小权限原则杜绝Claude“做不该做的事”，Secrets管理严防凭据泄露，容器隔离阻断逃逸风险，成本防护避免账单失控，审计日志确保全程可追溯。缺失任何一层，都意味着安全防线出现了致命缺口。



咖哥
发言

7.11 从CLI到Agent SDK: Headless模式的编程接口

前面介绍的示例均是通过CLI来调用Claude Code，但在实际工程实践中，Claude Code还提供了Agent SDK（支持TypeScript和Python），允许开发者直接使用编程语言而非命令行参数来控制Headless模式执行流程。

以下是TypeScript Agent SDK和Python Agent SDK示例。

```
// TypeScript Agent SDK示例
```

```

import { query } from "@anthropic-ai/claude-agent-sdk";

async function reviewPR(changedFiles: string[]) {

    let sessionId: string | undefined; let result: string = "";

    for await (const message of query({

        prompt: `审查以下文件的代码变更: ${changedFiles.join(", ")}`, options: {

            allowedTools: ["Read", "Grep", "Glob"], maxTurns: 10, maxBudgetUsd: 0.50, model: "claude-sonnet-4-6", outputFormat: { type: "json_schema", schema: reviewSchema }, }, )) {

        if (message.type === "system" && message.subtype === "init") {

            sessionId = message.session_id; }

        if (message.type === "result" && message.structured_output) {

            result = message.structured_output; }

        }

    return { sessionId, result }; }

```

Python Agent SDK示例

```

from claude_agent_sdk import query, ClaudeAgentOptions, ResultMessage

async def review_pr(changed_files: list[str]): options = ClaudeAgentOptions(

    allowed_tools=["Read", "Grep", "Glob"], max_turns=10, max_budget_usd=0.50, model="claude-sonnet-4-6", )

    async for message in query(

        prompt=f"审查以下文件: {', '.join(changed_files)}", options=options, ): if

```

isinstance(message, ResultMessage): return message.result

CLI与Agent SDK的关系，恰如curl与HTTP客户端库之别：CLI胜在轻量便捷，适用于简单的自动化脚本及CI/CD流水线；而Agent SDK则专为复杂逻辑场景而生——无论是基于上一轮审查结果动态调整后续Prompt，还是在多次Claude调用时进行精细的流程编排，抑或是要将

Claude深度集成至大型应用系统中，Agent SDK都能提供开发者所需的编程灵活性。在第8章中，我们将深入剖析Agent SDK的具体用法。

7.12 渐进式落地策略

“说了这么多技术细节，”小冰问，“如果我们团队想引入Headless模式进行PR审查，该如何分步推进？”

咖哥建议采取四阶段渐进式落地路径。

阶段一：观察者模式。Claude仅执行只读分析，输出审查报告但不阻塞任何流程。审查报告以评论形式发布在PR中，供团队成员参考或忽略。这个阶段的核心目标是建立信任——让团队直观感受Claude的审查质量，同时识别其局限性（如易误报场景），据此迭代优化Prompt与参数配置。

阶段二：顾问模式。在积累充足的正面反馈后，将Claude的审查结果纳入CI状态监控。发现严重（Critical）问题时，将CI状态标记为“警告”（黄色），但仍不阻塞合并。人工审查员需要结合Claude的报告做出最终判断。这个阶段的核心目标是人机协作。

阶段三：门禁模式。Claude的审查成为代码合并的必要条件之一。一旦检测到严重安全问题，CI状态立即变红并阻塞合并。但在此阶段必须配套“逃生通道”，允许拥有特定权限的人员手动覆盖Claude的阻塞决定（类似于跳过不稳定的测试用例）。

阶段四：主动修复模式。Claude不仅能发现问题，还能自动修复格式错误、补充缺失的异常处理逻辑或生成单元测试。此阶段需要开放Write工具权限，但所有修复必须以新Commit的形式提交，严禁直接修改 PR作者的原始代码。

“切忌一步到位，”咖哥强调，“每个阶段至少运行2到4周，密切监控误报率、成本趋势及团队接受度。AI辅助审查的核心价值不在于‘完美替代人工’，而在于‘让人类审查员将有限的精力聚焦于最具价值的决策上’。”

本章小结

Headless模式推动了Claude Code的角色蜕变——从局限于终端交互的智能助手，进化为可无缝嵌入各类工程流程的可编程组件。这一转变由4个维度的参数定义。

- **-p**（入口）：将Claude Code的运行模式从交互式会话切换为非交互式执行，奠定自动化基础。
- **--output-format**（数据接口）：规范输出结构，确保下游系统能高效解析与调用审查结果。
- **--allowedTools**（安全边界）：严格限定工具权限，确保自动化环境中的操作合规、可控。
- **--max-turns**与**--max-budget-usd**（成本护栏）：设定执行轮次与预算上限，防止无人值守场景下的资源失控。

在工程架构层面，Headless模式的¹最大价值在于赋予Claude Code“CI/CD生态平等公民”的身份。如同单元测试、静态分析或容器镜像构建，AI代码审查正式成为流水线中的标准环节：它需要具备明确的输入定义、确定性的输出格式、可量化的成本预算，以及可纳入版本控制的配置文件。

在落地策略上，建议遵循“观察者模式”起步，逐步演进至“门禁模式”的路径。这一渐进过程旨在为团队预留充足的时间以建立信任、积累实战数据并优化配置。切记不可急于求成——AI辅助审查的核心价值并非追求对人工的“完美替代”，而是通过智能过滤，让人类审查员将宝贵的精力聚焦于最具关键价值的决策环节。

在第8章中，我们将深入从命令行参数迈向编程接口（Agent SDK）的世界。你将掌握如何使用TypeScript或Python代码取代烦琐的CLI参数，灵活控制Claude的行为逻辑，从而构建真正定制化、高度集成的 AI应用系统。

思考题¹

将Claude Code的PR审查集成至现有CI/CD流水线，不仅需要攻克技术配置难关，更需要妥善解决团队协作流程中的核心议题：如何确立审查结果的权威性？如何厘清其与现有Linter工具的职能边界？如何构建高效的误报处理机制？

2. `--max-turns 5`与`--max-budget-usd 0.50`分别在何种场景下会率先触发限制？在设计生产环境的Headless模式任务时，你应优先关注哪一指标？请阐述理由。

3. 请明确你的团队所使用的CI平台（如GitHub Actions、GitLab CI、Jenkins或其他）。基于此，请为项目量身定制一套Claude Code集成方案，涵盖触发条件、工具权限、成本预算及安全策略，并输出完整的配置文件。

第8章 器以载道：Agent SDK智能体开发套件

借舟者随波而行，造舟者定其去向。

自从小雪将Claude Code集成至公司的CI/CD流水线，实现了自动审查PR与生成报告后，团队反响热烈。受此启发，产品经理提出了一个更具前瞻性的构想。

“我们能否开发一款内部工具，让非技术背景的同事也能利用Claude Code分析代码？例如，产品经理只需要输入用户故事，系统便能自动定位相关代码模块、评估实现复杂度，并列待修改的文件清单。”

小冰稍作沉吟。虽然Headless模式能部分满足需求，但它本质上是一个命令行工具，对不熟悉终端操作的产品经理而言门槛过高。他们真正需要的是一款具备表单输入、进度反馈及历史记录功能的Web界面。这意味着，这已不再是单纯的“工具使用”，而是需要“构建一款产品”。

“Headless模式通过参数驱动Claude，而Agent SDK则通过代码编排Claude，”咖哥补充道，“两者的区别，好比‘使用计算器’与‘为计算器编程’：前者只能依赖预设的功能按钮，后者则能自定义其行为逻辑。”

8.1 Agent SDK的定位：从工具到组件

本书前面介绍的机制（包括CLAUDE.md、子智能体、Skills、Hooks、MCP以及Headless模式）均以“使用Claude Code”为核心视角。在这一范式下，开发者作为用户，在Claude Code既定的框架内通过配置来扩展其行为边界。

Agent SDK的出发点则截然不同。它预设开发者是[应用构建者](#)，旨在将Claude Code的核心能力深度嵌入自有应用中。何时启动Claude、传入何种参数、如何编排中间执行步骤，以及如何管理多轮对话的状态，这一切完全由开发者的代码自主控制。此时，你不再是在“使用”一个现成的产品，而是在“构建”一个以Claude为智能引擎的全新应用。

这一演进路径如图8-1所示，该图清晰地展示了控制粒度从“配置”向“代码”的跨越。

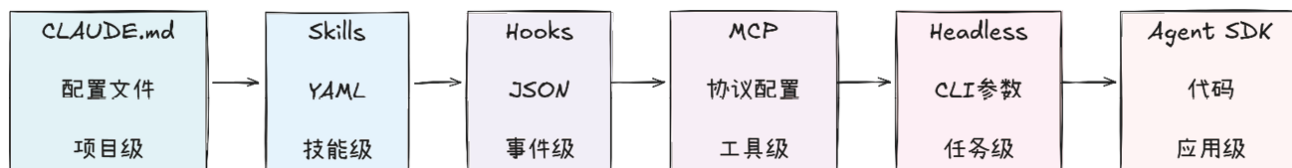


图8-1 从CLAUDE.md到Agent SDK的控制粒度演进路径

每深入一层，控制粒度便愈发精细，定制化程度也随之提升。Agent SDK处于这一演进链的末端：它在赋予开发者最大灵活性的同时，也带来了最高的工程复杂度。

Agent SDK提供Python与TypeScript/JavaScript两个版本，两者功能完全对等。值得注意的是，自2025年末起，其包名已由claude-code-sdk正式更名为claude-agent-sdk，以更准确地反映其定位。安装方式如下。

```
# Python
```

```
pip install claude-agent-sdk
```

```
# TypeScript/Node.js npm install @anthropic-ai/claude-agent-sdk
```

8.2 核心API: query函数

Agent SDK的入口是query函数。如果说Headless模式的入口是claude -p "prompt"，那么query函数便是其在编程层面的等价物——不过，它返回的并非最终结果，而是一个[异步消息流](#)。

8.2.1 Python版本

以下代码展示了如何使用Agent SDK通过Python以编程方式调用Agent，执行分析代码库架构的任务。

```
import asyncio

from claude_agent_sdk import query, ClaudeAgentOptions

async def analyze_code(): options = ClaudeAgentOptions(

max_turns=5, allowed_tools=["Read", "Grep", "Glob"], system_prompt="你是一名代码架构分析师",
)
```

```

async for message in query(

prompt="分析 src/auth/ 目录的实现架构", options=options ):

if message.type == "assistant": for block in message.content: if hasattr(block, 'text'):
print(block.text, end="", flush=True) elif message.type == "result": print(f"\n\n完成。费用: ${message.total_cost_usd:.4f}")

```

8.2.2 TypeScript版本

以下代码展示了如何使用Agent SDK通过TypeScript以编程方式调用Agent，执行分析代码库架构的任务。

```

import { query, ClaudeAgentOptions } from '@anthropic-ai/claude-agent-sdk';

const options: ClaudeAgentOptions = {

maxTurns: 5,

allowedTools: ['Read', 'Grep', 'Glob'], systemPrompt: '你是一名代码架构分析师。', };

async function analyzeCode() {

for await (const message of query({

prompt: '分析src/auth/目录的实现架构', options,

}))) {

if (message.type === 'assistant') {

for (const block of message.content) {

if ('text' in block) {

process.stdout.write(block.text); }

}

} else if (message.type === 'result') {

```

```
console.log(`\n\n完成。费用：${message.totalCostUsd}`); }

    }

}
```

```
analyzeCode();
```

调用query函数会返回一个[异步生成器](#)（async generator）。它无需等待Claude完成全部任务才返回，而是每当Claude生成一条消息时，就即时返回该消息。在Web应用中，这意味着你可以将Claude的思考过程以流式形式推送至前端，复现类似Claude.ai的打字机效果；而在CI/CD场景中，它允许你在Claude仍在运行时就可以着手处理已生成的输出。

8.3 消息类型：解读Claude的输出流

“理解消息类型是使用Agent SDK的基石，”咖哥指出，“正如HTTP开发者必须精通状态码，Agent SDK开发者也需要透彻掌握每类消息的含义。”

query函数生成的消息流是一个有序序列，其中每条消息均通过type字段标识其具体类型。

8.3.1 system/init——会话初始化

以下代码展示了会话初始化的系统消息示例。

```
{

  "type": "system", "subtype": "init", "session_id": "550e8400-...", "model": "claude-sonnet-4-6", "tools": ["Read", "Grep", "Glob"], "mcp_servers": []

}
```

消息流中的首条消息用于汇报当前会话的配置详情，涵盖模型版本、可用工具集及MCP服务器状态。这在调试过程中尤为关键：如果发现Claude未按预期调用某项工具，应首先核查init消息中是否已包含该工具的定义。

8.3.2 assistant——Claude的响应

以下代码展示了Claude每一轮响应的消息示例。

```
{  
  
  "type": "assistant", "message": {  
  
    "role": "assistant", "content": [  
  
      {"type": "text", "text": "让我先看看目录结构……"}, {"type": "tool_use", "id": "toolu_xxx",  
      "name": "Glob", "input": {"pattern": "src/auth/**/*"}}  
  
    ]  
  
  }  
  
}
```

在上述代码中，content字段是一个数组，可混合包含文本块（即Claude的“思考”过程或最终输出）与工具调用块（即Claude 决定执行的操作）。值得注意的是，单条assistant 消息中可能包含多个工具调用，这意味着Claude能够并发请求读取多个文件或执行其他并行操作。

8.3.3 user——工具执行结果

以下代码展示了工具执行完成后的结果示例。

```
{  
  
  "type": "user",  
  
  "message": {  
  
    "role": "user", "content": [  
  
      {"type": "tool_result", "tool_use_id": "toolu_xxx", "content": "src/auth/\n ├──  
login.ts\n ├── session.ts\n └── middleware.ts"}  
  
    ]  
  
  }  
  
}
```

请注意，tool_use_id需要与assistant消息中的工具调用id严格对应。该映射关系由Agent SDK自动维护，开发者通常不需要手动处理。

8.3.4 result——任务完成

以下代码展示了任务完成时的终止消息示例。

```
{  
  
"type": "result", "subtype": "success", "session_id": "550e8400-...", "num_turns": 5,  
  
"total_cost_usd": 0.0342, "duration_ms": 12345, "duration_api_ms": 10000, "usage": {  
  
"input_tokens": 5000, "output_tokens": 1500, "cache_read_input_tokens": 3000  
  
},  
  
"result": "架构分析: src/auth/ 采用分层设计...", "structured_output": null }
```

在上述代码中，subtype是判断任务结束状态的关键字段（详见表8-1）。表8-1

result消息的subtype类型与含义
subtype含义
success任务正常完成
error_max_turns达到最大轮次限制（--max-turns）
error_max_budget_usd达到费用上限
error_during_execution执行过程中发生错误
error_max_structured_output_retries结构化输出验证失败（超过最大重试次数）

以下代码展示了一个实用的消息处理模式，用于在异步消息流中收集文本输出、记录工具调用详情并存储会话元数据。

```
async def process_query(prompt: str, options: ClaudeAgentOptions) -> dict: # 初始化结果容器  
  
result = {"text": [], "tools": [], "metadata": {}, "error": None}  
  
# 异步遍历消息流  
  
async for message in query(prompt=prompt, options=options): if message.type == "assistant":  
# 处理助手响应中的内容块  
  
for block in message.content: if hasattr(block, 'text'): # 收集文本输出（思考或回答）  
  
result["text"].append(block.text) elif hasattr(block, 'name'): # 记录工具调用（名称与输入参数）
```

```

result["tools"].append({"tool": block.name, "input": block.input}) elif message.type ==
"result": # 提取终止消息中的元数据

result["metadata"] = {

"session_id": message.session_id, "cost": message.total_cost_usd, "turns":
message.num_turns, "duration_ms": message.duration_ms, }

# 检查是否因错误终止

if message.is_error: result["error"] = message.subtype

return result

```

8.4 ClaudeAgentOptions: 精细的行为控制

ClaudeAgentOptions是控制ClaudeAgent行为的核心配置对象（该类名已由旧版本的Claude CodeOptions更新而来）。它将Headless CLI的命令行参数转化为编程接口，使得动态配置和精细化控制成为可能。

以下代码展示了对该配置对象的结构解析。

```

from claude_agent_sdk import ClaudeAgentOptions

options = ClaudeAgentOptions(

# === 模型与执行控制 ===

model="claude-sonnet-4-6", # 指定使用的模型版本

max_turns=10, # 限制最大交互轮数，防止死循环

max_budget_usd=1.0, # 设置单次任务的费用上限（美元），超出即停止

# === 工具权限管理 ===

# 白名单：仅允许使用列出的工具

allowed_tools=["Read", "Grep", "Glob", "Write"], # 黑名单：明确禁止使用的工具（优先级通常高于白名单或作为补充）

```

```

disallowed_tools=["Bash"],

# === 权限模式 ===

# default: 标准模式，需用户确认危险操作

# acceptEdits: 自动接受文件编辑

# plan: 仅规划不执行

# bypassPermissions: 跳过所有确认（高风险，仅限受信任环境）

permission_mode="default",

# === 提示工程 ===

system_prompt="你是一名高级代码审查员", # 设定核心系统指令

append_system_prompt="务必检查SQL注入漏洞", # 在核心指令

后追加特定要求

# === 工作环境配置 ===

cwd="path/to/project", # 设置Agent的工作目录

env={"PROJECT_NAME": "MyApp"}, # 注入自定义环境变量

# === 会话管理 ===

resume="session-id-to-resume", # 从指定的session_id恢复上下文

no_session_persistence=False, # 若为True，则任务结束后

不保存会话历史

# === 结构化输出 ===

# 强制模型输出符合特定JSON Schema格式的数据

output_format={"type": "json_schema", "schema": my_

```

```

schema}, # 结构化输出

# === MCP集成 ===

# 动态启动并连接外部MCP服务器

mcp_servers=[

{"name": "db", "command": "python", "args": [". /db_

server.py"]}

],

)

```

8.4.1 权限模式详解

权限模式是Agent SDK应用安全防线的核心。它决定了Claude在遇到潜在风险操作（如写文件、执行命令）时的行为逻辑（详见表8-2）。[表8-2 Agent SDK权限模式与适用场景](#)模式行为逻辑适用场景default标准交互。遇到危险操作（如写文件、运行Shell）时，等待外部确认（通常通过回调或人工干预）面向用户的Web应用、交互式CLI工具acceptEdits半自动。自动接受文件编辑类操作，但对于执行命令（Bash）、网络请求等仍会询问自动化修复脚本、代码格式化流水线plan只读/规划模式。禁止所有修改。模型只能读取文件、分析代码并输出计划，无法执行写操作或命令代码审查、架构分析、漏洞扫描bypassPermissions完全信任。跳过所有权限检查，自动执行所有操作（包括rm -rf、curl等高危命令）严格隔离的CI容器、沙箱环境

“在CI/CD流水线中，bypassPermissions看似便捷——毕竟不需要人工确认嘛，”咖啡哥说，“但请务必牢记，这意味着赋予Claude执行任意命令的权限，包括删除文件、修改配置，甚至发起网络请求。因此，仅在确保执行环境完全隔离（如容器化部署、文件系统只读且无网络连接）时，方可启用该选项。”

8.4.2 工具权限的模式匹配

allowed_tools支持与Headless CLI相同的模式匹配语法。请看以下代码示例。

```

options = ClaudeAgentOptions(

allowed_tools=[

"Read",

```



```

"Grep",

"Glob",

"Bash(git diff *)", # 仅允许执行git diff命令

"Bash(npm test *)", # 仅允许执行npm test命令

"mcp__database__query", # 仅允许使用MCP数据库查询工具

]

)

```

8.5 会话管理：跨调用的上下文延续

query函数适用于单次任务场景。然而，许多实际应用需要多轮交互：用户首先指示Claude分析问题，待结果返回后再指定修复方向，最后在修复完成后要求Claude进行验证。

8.5.1 基于session_id的会话延续

以下代码展示了如何使用一个支持会话延续的异步Agent接口，分两步完成一个复杂的安全分析任务。

第一次调用：分析问题

```
session_id = None
```

```
async for message in query(prompt="分析 src/auth 的安全问题", options=options): if
message.type == "system" and message.subtype == "init": session_id = message.session_id #
```

【关键】获取会话ID

```
if message.type == "result": print(message.result)
```

第二次调用：在上一轮上下文的基础上继续深入

```
resume_options = ClaudeAgentOptions(**options.__dict__, resume=session_id) async for message
in query(
```

```
prompt="重点分析你发现的第一个SQL注入风险", options=resume_options ):
```

if message.type == "result": print(message.result) resume参数使Claude能够“记忆”上一轮对话的完整上下文（包括已读取的文件、执行的分析步骤以及得出的结论）。相较于在第二个Prompt中重复描述上下文，这种方式不仅效率高，而且能确保分析的准确性与连贯性。

8.5.2 会话分叉

以下代码展示了如何利用会话分叉技术，基于同一个初始分析结果，探索多个不同的解决方案或假设场景且确保各分支互不干扰。

```
# 基于同一个分析结果，探索不同的方向
```

```
# 创建分叉配置：复用原会话状态，但开启新分支
```

```
options_fork = ClaudeAgentOptions(**options.__dict__, resume=session_id, fork_session=True)
```

```
# 方向 A：探讨微服务重构方案
```

```
async for message in query(
```

```
prompt="如果重构为微服务架构，需要修改哪些部分？", options=options_fork ):
```

```
...
```

```
# 方向 B：探讨现有架构的安全加固方案
```

```
# 基于同一个 session_id 再次分叉，确保两个方向完全独立
```

```
async for message in query(
```

```
prompt="如果保持现有架构，应如何加固安全性？", options=options_fork ):
```

```
...
```

设置fork_session=True可从指定会话的特定时间点“分叉”出一个全新的会话副本，而原始会话的状态保持不变。这一机制在需要进行“假设性分析”或对比多种技术路线的场景中极具价值。

8.6 自定义工具：扩展Claude的能力边界

Claude原生内置了Read、Write、Bash、Grep等基础工具，但在实际应用场景中，开发者往往需要集成特定领域的能力，如查询数据库、调用内部API、发送通知或执行复杂的业务逻辑。

Agent SDK中的自定义工具实质上是[进程内MCP服务器](#)。与第6章介绍的独立部署的MCP服务器不同，它不需要启动单独的进程，而是直接在应用进程中运行。这种架构消除了进程间通信的开销，从而显著提升了工具调用的效率与响应速度。

8.6.1 使用@tool装饰器定义工具

以下代码展示了如何使用Agent SDK构建一台进程内MCP服务器，将两个具体的业务功能（数据库查询与发送通知）封装为安全的工具，供Agent调用。

```
from claude_agent_sdk import tool, create_sdk_mcp_server import json

# 工具1：数据库查询（只读）

@tool(

name="query_database", description="Execute a read-only SQL query on the application
database", parameters={

"query": str, "limit": int }

)

async def query_database(args): sql = args["query"]

limit = args.get("limit", 100)

# 【安全关键】强制校验：仅允许SELECT语句

if not sql.strip().upper().startswith("SELECT"): return {

"content": [{"type": "text", "text": "Error: Only SELECT queries allowed"}], "isError": True
}
```

```

results = await db.execute(f"{sql} LIMIT {limit}") return {

"content": [{"type": "text", "text": json.dumps(results, indent=2)}]

}

# 工具2: 发送通知

@tool(

name="send_notification", description="Send a notification to the team Slack channel",

parameters={

"channel": str, "message": str }

)

async def send_notification(args): await slack.post_message(args["channel"],

args["message"])

return {

"content": [{"type": "text", "text": f"Notification sent to #{args['channel']}"}]

}

# 创建MCP服务器承载这些工具

tools_server = create_sdk_mcp_server(

name="app-tools", version="1.0.0", tools=[query_database, send_notification]

)

```

8.6.2 在Agent配置中注册并启用自定义工具

以下代码展示了如何在Agent配置中注册并启用自定义工具。

```

options = ClaudeAgentOptions(

mcp_servers={

```

```

"app-tools": tools_server },

allowed_tools=[

"Read", "Grep", "Glob", # 内置工具

"mcp__app-tools__query_database", # 自定义工具

"mcp__app-tools__send_notification", ]

)

```

自定义工具的名称严格遵循mcp__{服务器名}__{工具名}格式。这种带前缀的命名方式不仅避免了与内置工具（如Read、Bash）的名称冲突，更在日志审计和监控仪表盘中提供了天然的分类标签。

8.6.3 使用Pydantic模型进行参数验证

在处理复杂参数结构时，推荐使用Pydantic模型来替代简单的字典定义。请看以下代码示例。

```

from pydantic import BaseModel, Field from claude_agent_sdk import tool

# 定义结构化参数模型

class DatabaseQueryParams(BaseModel): table: str = Field(

... ,

description="Table name"

)

columns: list[str] = Field(

default=["*"], description="Columns to select"

)

where: str | None = Field(

default=None, description="SQL WHERE clause condition (optional)"

```

```

)

limit: int = Field(

default=100, ge=1,

le=1000,

description="Maximum number of rows to return"

)

# 将模型直接作为工具参数定义

@tool(

name="safe_query", description="Execute a safe, parameterized database query",

parameters=DatabaseQueryParams )

async def safe_query(args: DatabaseQueryParams): # args已经通过Pydantic模型验证，类型安全

```

… 自定义工具的设计务必慎之又慎。Claude可能会以出乎意料的方式调用工具，如传入极端参数值、在短时间内连续发起数十次调用，或利用前一次工具的输出构造新的输入。为此，请遵循以下三大原则：单一职责（确保每个工具仅专注于完成一个特定任务）、描述清晰（description字段是写给Claude看的“使用指南”，必须明确界定“什么时候可用”以及“什么时候禁用”）、零信任验证（永远不要信任Claude传入的任何参数，必须在工具内部对所有输入进行严格校验）。



**咖哥
发言**

8.7 Agent SDK中的Hooks：程序化的拦截

第5章介绍的Hooks机制依赖于JSON配置与Shell脚本；而在Agent SDK中，Hooks已全面升级为[原生编程接口](#)。开发者可以直接使用Python或TypeScript函数定义Hook逻辑，彻底摒弃了对Shell脚本和JSON配置文件的依赖。

8.7.1 PreToolUse：执行前拦截

以下代码展示了一个细粒度的运行时安全拦截系统。该系统利用Hooks机制，在Agent实际执行工具（如运行Bash命令或修改文件）之前，对操作进行深度检查与过滤。一旦检测到危险行为，系统将直接拒绝执行，从而有效防止AI造成破坏。

```
from claude_agent_sdk import ClaudeAgentOptions, HookMatcher

async def block_dangerous_bash(input_data, tool_use_id, context): """拦截危险的Bash命令"""

# 仅处理Bash工具调用

if input_data["tool_name"] != "Bash": return {}

command = input_data["tool_input"].get("command", "") # 定义危险命令特征列表

dangerous = ["rm -rf", "sudo", "chmod 777", "> dev", "mkfs", "dd if="]

for pattern in dangerous: if pattern in command: return {

    "hookSpecificOutput": {

        "hookEventName": "PreToolUse", "permissionDecision": "deny", "permissionDecisionReason":
        f"Blocked: {pattern}"

    }

}

return {}

async def protect_config_files(input_data, tool_use_id, context): """保护关键配置文件不被修
改"""
```

```

tool_name = input_data["tool_name"]

# 仅处理文件写入类工具

if tool_name not in ["Write", "Edit"]: return {}

file_path = input_data["tool_input"].get("file_path", "") # 定义受保护的路径或文件名特征

protected = [".env", "secrets", "production.yaml", "database/migrations"]

for p in protected: if p in file_path: return {

    "hookSpecificOutput": {

        "hookEventName": "PreToolUse", "permissionDecision": "deny", "permissionDecisionReason":
        f"Protected file: {file_path}"

        }

    }

return {}

# 配置Agent选项, 注册安全拦截Hook options = ClaudeAgentOptions(

hooks={

    "PreToolUse": [

        HookMatcher(matcher="Bash", hooks=[block_dangerous_

bash]),

        HookMatcher(matcher="Write", hooks=[protect_config_

files]),

        HookMatcher(matcher="Edit", hooks=[protect_config_

```



```

files]),

    ]

    }

)

```

8.7.2 PostToolUse: 执行后处理

以下代码展示了如何利用PostToolUse（工具调用后）Hook实现自动化 workflow 增强与全链路审计。

```

import subprocess

import json

from datetime import datetime from claude_agent_sdk import ClaudeAgentOptions, HookMatcher

async def auto_format_on_write(input_data, tool_use_id, context): """写入文件后自动触发代码格式化"""

# 仅处理文件写入类工具

if input_data["tool_name"] not in ["Write", "Edit"]: return {}

file_path = input_data["tool_input"].get("file_path", "")

# 根据文件扩展名选择对应的格式化工具

if file_path.endswith(".py"): # 使用 Black 格式化 Python 代码

subprocess.run(["black", file_path], capture_output=True) elif file_path.endswith((".ts",
".js", ".tsx", ".jsx")): # 使用 Prettier 格式化前端代码

subprocess.run(["prettier", "--write", file_path], capture_output=True)

return {}

async def audit_all_tools(input_data, tool_use_id, context): """记录所有工具调用的审计日志"""

```

```

audit_entry = {

    "timestamp": datetime.now().isoformat(), "tool": input_data["tool_name"], "input":
    input_data["tool_input"], "tool_use_id": tool_use_id, }

# 将审计日志追加写入 JSONL 文件

with open("agent-audit.jsonl", "a") as f: f.write(json.dumps(entry) + "\n")

return {}

# 配置Agent选项，注册后置Hook options = ClaudeAgentOptions(

hooks={

    "PreToolUse": [...], # 此处可接前面定义的安全拦截Hook

    "PostToolUse": [

        HookMatcher(matcher="Write", hooks=[auto_format_

on_write]),

        HookMatcher(matcher="Edit", hooks=[auto_format_

on_write]),

        HookMatcher(matcher="*", hooks=[audit_all_tools]), ]

    }

)

```

8.7.3 Agent SDK Hooks与Shell Hooks的关系

表8-3对比了Agent SDK Hooks与Shell Hooks的核心差异，清晰地展示了从配置化脚本到原生编程接口的演进。表8-3 [Shell Hooks与Agent SDK Hooks对比](#) Hook机制定义方式执行环境数据交换异步支持调试体验适用场景Shell Hookssettings.json配置+Shell脚本文件独立子进程通过stdin/stdout传递JSON字符串不支持（同步阻塞）困难（依赖echo打印或写入日志文件）Claude Code交互模式/Headless CLI Agent SDK HooksPython/TypeScript

原生函数当前进程内直接通过函数参数和返回值对象原生支持async/await高效（支持IDE断点调试、单步跟踪）基于Agent SDK构建的嵌入式应用

如果希望直接使用Claude Code（无论是终端交互模式还是Headless CLI模式），建议采用Shell Hooks；如果正在使用Agent SDK开发自定义应用、集成服务或构建复杂的自动化工作流，建议采用Agent SDK Hooks。

8.8 4道安全防线

Agent SDK构建了由4道安全防线组成的纵深防御体系，如图8-2所示。

纵深防御体系

第一道：权限模式

`permission_mode = 'plan'`

全局开关



第二道：工具白名单

`allowed_tools = ['Read', ...]`

工具级别



第三道：canUseTool 回调

在每次工具调用前进行动态逻辑判断

运行时动态



第四道：PreToolUse Hooks

深度检查具体参数、修改输入拦截危险指令

最细粒度

图8-2 Agent SDK的纵深防御体系

canUseTool是Agent SDK特有的运行时权限回调机制。与功能丰富但复杂的Hooks不同，CanUseTool强调以二元判断（允许/拒绝）作为实现动态访问控制的最轻量级手段。

以下代码展示了如何构建一个智能的“守门员”，在工具执行前根据具体内容动态决定是否放行。

```
async def can_use_tool(tool_name: str, tool_input: dict) -> dict: """运行时权限检查回调"""
```

```
# 场景1：文件写入保护
```

```
# 防止敏感配置文件被篡改
```

```
if tool_name in ["Write", "Edit"]: file_path = tool_input.get("file_path", "") if ".env" in file_path or "secrets" in file_path: return {
```

```
"allowed": False, "reason": "Access to sensitive files denied"
```

```
}
```

```
# 场景2：网络命令封锁
```

```
# 即使白名单允许了Bash，也要禁止特定的网络操作
```

```
if tool_name == "Bash": command = tool_input.get("command", "") if any(cmd in command for cmd in ["curl", "wget", "ssh"]):
```

```
return {
```

```
"allowed": False, "reason": "Network commands not allowed"
```

```
}
```

```
return {"allowed": True}
```

```
options = ClaudeAgentOptions(
```

```
permission_mode="acceptEdits", allowed_tools=["Read", "Write", "Edit", "Grep", "Glob"],  
can_use_tool=can_use_tool, hooks={
```

```

"PreToolUse": [HookMatcher(matcher="*", hooks=[audit_all_tools])]

    }

)

```

4道安全防线的协同逻辑：permission_mode确立全局基调，allowed_tools剔除冗余工具，canUseTool在运行时对每次调用实施动态校验，而PreToolUse Hooks则负责最细粒度的参数审查、输入修正及日志记录审计。

8.9 结构化输出：强制JSON Schema

在构建应用时，你通常需要Claude输出结构化数据而非自由文本。Agent SDK通过output_format参数支持基于JSON Schema的验证。请看以下代码示例。

```

from pydantic import BaseModel

class SecurityReport(BaseModel):
    summary: str

    issues: list[dict] # 结构示例: [{severity, file, line, description}]

    risk_score: float # 范围: 0.0~10.0

options = ClaudeAgentOptions(

    output_format={

        "type": "json_schema", "schema": SecurityReport.model_json_schema() },

    max_turns=10,

    allowed_tools=["Read", "Grep", "Glob"], )

async for message in query(prompt="对src/ 进行安全审查", options=options):
    if message.type == "result" and message.structured_

    output:

    report = SecurityReport.model_validate(message.structured_output)

```

report 是类型安全的Pydantic对象

```
print(f"风险评分: {report.risk_score}") for issue in report.issues: print(f"
[{issue['severity']}] {issue['file']}: {issue.get('line', '?')} - {issue['description']}")
```

在TypeScript环境中，你可以利用Zod库实现同等功能。请看以下代码示例。

```
import { z } from 'zod';

const SecurityReport = z.object({

summary: z.string(), issues: z.array(z.object({

severity: z.enum(['critical', 'high', 'medium', 'low']),

file: z.string(), description: z.string(), })),

riskScore: z.number().min(0).max(10), });

const options: ClaudeAgentOptions = {

outputFormat: {

type: 'json_schema', schema: z.toJSONSchema(SecurityReport), },

};
```

Claude会自动将输出格式化为符合上述Schema的JSON。如果首次生成的内容未通过验证，Agent SDK将自动触发重试机制（最大重试次数由内部策略控制）；如果超过限制仍未成功，系统将返回error_max_structured_output_retries错误。

8.10 实战：构建代码分析Web服务

以下是整合了前面所有知识的完整运行示例——代码分析Web服务。该脚本展示了如何配置Claude Agent，通过流式形式接收分析结果，在Web服务场景中（以命令行模拟）输出实时反馈。

```
#!/usr/bin/env python3
```

```
"""代码分析Web服务——完整运行示例"""
```

```

import asyncio

import json

from datetime import datetime from claude_agent_sdk import query, ClaudeAgentOptions

async def analyze_codebase(directory: str, focus: str = "general"): """
    分析指定目录的代码库。

    Args:
        directory: 要分析的本地目录路径
        focus: 分析重点领域 - "security" "performance" "quality" / "general"
    """

    # 定义不同关注点的System Prompt
    focus_prompts = {

        "security": "专注于安全漏洞：SQL 注入、XSS、敏感信息硬编码、权限控制缺失等。",
        "performance": "专注于性能问题：N+1 查询、内存泄漏、冗余计算、缓存策略缺失等。",
        "quality": "专注于代码质量：命名规范、DRY原则、圈复杂度、测试覆盖率建议等。",
        "general": "进行全面代码审查：涵盖安全性、性能、代码质量及架构合理性。"

    }

    options = ClaudeAgentOptions(

        model="claude-sonnet-4-6", max_turns=15, max_budget_usd=0.50, allowed_tools=["Read",
        "Grep", "Glob"], permission_mode="plan", # 只读模式

        cwd=directory, append_system_prompt=focus_prompts.get(focus, focus_prompts["general"]), )

    # 初始化收集器

    output_text = []

    tools_used = []

    metadata = {}

```



```

# 执行流式查询

async for message in query(

prompt=f"分析当前项目的代码，{focus_prompts.get(focus, '')}。请直接输出Markdown格式的分析报
告。", options=options, ):

    if message.type == "assistant": for block in message.content: if hasattr(block, 'text'):
        output_text.append(block.text) # 【关键点】流式输出：在实际 Web 服务中，此处应为 yield SSE
        事件

    print(block.text, end="", flush=True) elif hasattr(block, 'name'):
        tools_used.append(block.name)

    elif message.type == "result": metadata = {

        "session_id": message.session_id, "cost_usd": message.total_cost_usd, "turns":
        message.num_turns, "duration_ms": message.duration_ms, "success": not message.is_error, }

# 返回结构化结果

return {

    "report": "\n".join(output_text), "tools_used": tools_used, "metadata": metadata, }

async def main():

    import sys

    directory = sys.argv[1] if len(sys.argv) > 1 else "."

    focus = sys.argv[2] if len(sys.argv) > 2 else "general"

    print(f"分析 {directory} (重点: {focus}) ... \n") result = await analyze_codebase(directory,
    focus)

    print(f"\n\n--- 会话统计 ---") print(f"状态: {'成功' if meta['success'] else '失败'}")
    print(f"费用: ${result['metadata'].get('cost_usd', 0):.4f}")

    print(f"耗时: {result['metadata'].get('duration_ms', 0) /

```

```
1000:.1f}s")
```

```
print(f"交互轮数: {meta.get('turns', 0)}") print(f"使用工具: {'',  
'}.join(result['tools_used'])}")
```

if __name__ == "__main__": asyncio.run(main()) 在实际的Web服务架构中，调用query函数返回的异步生成器可直接对接SSE（ServerSent Events），实现低延迟的实时数据推送。

以下是一个基于FastAPI的完整示例。该架构允许前端实时渲染Claude的“思考”过程，将传统的“加载动画”替换为动态生成的文本流，显著提升用户体验。

```
from fastapi import FastAPI from fastapi.responses import StreamingResponse
```

```
app = FastAPI()
```

```
@app.post("/api/analyze") async def analyze(request: AnalyzeRequest): async def  
event_stream(): async for message in query(prompt=request.prompt, options=options): if  
message.type == "assistant": for block in message.content: if hasattr(block, 'text'): yield  
f"data: {json.dumps({'type': 'text', 'content': block.text})}\n\n"
```

```
elif message.type == "result": yield f"data: {json.dumps({'type': 'done', 'cost':  
message.total_cost_usd})}\n\n"
```

```
return StreamingResponse(event_stream(), media_type="text/event-stream")
```

8.11 Agent SDK与Headless CLI：如何选型

虽然Agent SDK与Headless CLI都能让Claude在无人值守的环境下运行，但它们的适用场景有着明确的分界（详见表8-4）。选择错误的工具往往会导致架构过度复杂或功能受限。表8-4 Agent SDK与Headless CLI的场景选择对照

场景推荐方式原因CI/CD流水线中的单步任务Headless CLI零依赖部署，直接嵌入YAML配置，不需要安装Python/Node运行时Shell脚本自动化Headless CLI完美契合管道组合，简单直接构建Web/桌面AI应用Agent SDK支持流式输出（SSE）、精细的状态管理及前端交互需要自定义工具Agent SDK支持@tool装饰器注册本地函数，实现复杂业务逻辑闭环多轮对话/持续会话Agent SDK内置session管理，自动维护上下文历史，不需要手动拼接消息需要精细控制消息流Agent SDK基于异步生成器，可逐块拦截、修改或路由消息并发管理多个实例Agent SDK编程化并发控制（如asyncio.gather），轻松处理高并发请求

一条有意思的经验法则：**如果你的逻辑可以用一行Shell命令描述，请选用CLI；如果需要写if/else判断或循环，请选用Agent SDK。**

Agent SDK降低了构建Agent的门槛，但“容易”绝不等于“简单”。将Claude嵌入实际产品时，开发者必须妥善解决以下关键问题：错误重试（应对网络波动与API限流）、成本追踪（监控每次调用的Token消耗）、用户隔离（确保不同用户的对话互不干扰）、响应超时（制定长任务的中断策略）以及熔断机制（在连续失败时自动停止调用）。掌握这些工程细节，是将Agent SDK演示原型升级为生产级Agent的必修课。



咖哥
发言

本章小结

Agent SDK将Claude Code从单一的终端工具，转型为可嵌入任意应用的AI组件。这一转变依托于两大核心API——query函数和ClaudeAgentOptions。

query函数是Agent SDK的入口，其功能对标Headless模式的`claude -p`命令，但以异步生成器的形式返回消息流，支持实时处理Claude的每一步输出。该消息流包含5种类型：`system/init`（初始化信号）、`assistant`（Claude的响应，可能包含文本内容及工具调用请求）、`user`（工具执行后的反馈结果）和`result`（任务终止信号，携带成本、轮数、耗时等元数据）。

ClaudeAgentOptions则提供了与Headless CLI参数对等的编程接口，涵盖模型选择、工具权限、权限模式、Prompt定制、会话管理及结构化输出等功能。相较于命令行，编程接口的核心优势在于动态配置能力：开发者可根据用户角色动态调整权限模式，依据任务类型灵活切换模型，或基于上一轮的执行结果实时优化下一轮的Prompt。

自定义工具通过@tool装饰器与进程内MCP服务器实现，赋予Claude调用任意自定义函数的能力，无论是查询数据库、调用内部API，还是发送通知。与此同时，4道安全防线（权限模式、工具白名单、canUseTool回调、PreToolUse Hooks）确保Claude在应用中只做该做的事。

从CLAUDE.md配置、Skills定义，到Hooks拦截、MCP集成，再到Headless模式与Agent SDK的全面掌控，每深入一层，意味着我们距离构建成熟的“产品”而非单一的“工具”更近一步。在第9章中，我们将探讨如何将这些能力打包分发，并深入解析Plugins插件生态。

思考题₁

如果需要为公司内部构建一个“代码影响分析”工具（输入需求描述，输出受影响的代码模块列表及修改难度评估），你将如何设计其系统架构？在此场景中，应重点利用Agent SDK的哪些核心特性？

2. 自定义工具的description字段是供Claude理解的“操作指南”。一个优质的description应清晰包含哪些信息？一个差的description可能导致什么问题？

3. 4道安全防线（权限模式、工具白名单、canUseTool回调、PreToolUseHooks）之间是什么关系？在你的应用场景中，哪些防线是必需的，哪些可以省略？

第9章 集腋成裘：Plugins生态

独行快，众行远。

小雪最近在开源社区发现了一个名为“React全栈团队的Claude Code最佳实践”的GitHub仓库，其标题极具吸引力。深入查看后，她发现其中包含一套精心设计的Skills，涵盖代码审查、组件文档生成及测试用例编写等功能；此外，该仓库还提供了用于自动格式化和PR格式的Hooks脚本，并附带了一份完整的.mcp.json配置文件，已预先集成GitHub API与Linear项目管理工具。

“要是这套方案能直接投入使用就好了。”小雪感叹道。

然而，现实并不顺利。在该仓库中，Skills位于一个目录，Hooks脚本在另一个目录，.mcp.json是独立文件，还有若干自定义命令的定义文件散落在各处。若要应用这套配置，她必须手动将所有文件复制到指定位置，理清每一部分的安装方式，并祈祷没有遗漏任何依赖。最终，她选择了放弃——仅弄清楚安装顺序就耗费了整个下午。

“这正是Plugins（插件）旨在解决的问题，”咖哥说，“它将一套完整的Claude Code能力打包成可一键安装的单元。就像npm之于JavaScript生态、pip之于Python生态一样，Plugins是Claude Code能力的标准化分发机制（见图9-1）。 ”

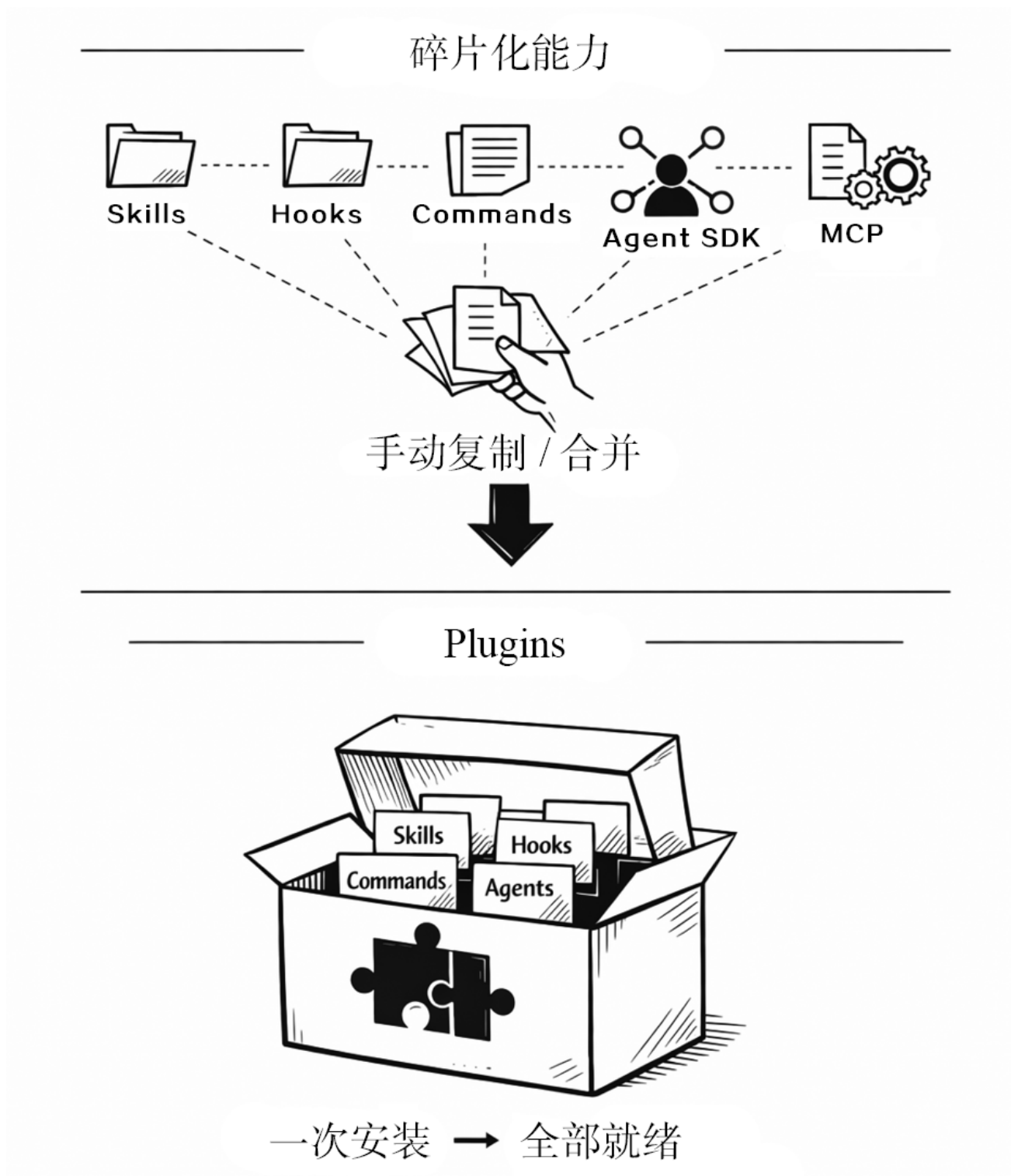


图9-1 Plugins是Claude Code能力的标准化分发机制

9.1 Plugins的定位：能力的封装与分发

在深入理解Plugins之前，有必要先回顾我们已构建的工具栈。截至目前，我们探讨了6种独立的扩展机制：利用CLAUDE.md管理项目记忆，通过子智能体实现任务委派，借助Skills封装领域工作流，依靠Hooks提供事件驱动控制，使用MCP连接外部服务，以及基于Headless模式/Agent SDK实现编程调用。尽管这些机制各司其职，但它们存在一个共同的短板——**彼此割裂，难以作为整体进行分发**。

当你耗费数周为团队打磨出一套完整的开发工作流，并试图将其分享给另一团队时，现实却令人却步：Skills需要复制至`.claude/skills/`目录，Hooks配置需要合并入`settings.json`，MCP服务器需要单独配置，自定义命令需要移至`.claude/commands/`，而子智能体定义又散落在其他位置。接收方不仅必须逐一理清每个组件的安装步骤，还需要应对潜在的命名冲突。这已不再是简单的经验分享，而是一场烦琐的“迁移作业”。

Plugins本身并不引入新的功能，而是提供了一套**统一的打包格式和安装机制**。一个Plugin能够同时囊括Skills、Hooks、Commands、Agents以及MCP配置，只需要一行命令即可让所有组件就位。

`/plugin install team-toolkit@our-company` 正如表9-1所示，一个Plugin可以包含多种组件类型，涵盖了我们在前几章所学的所有扩展机制。表9-1 一个

Plugin可包含的组件类型

组件作用对应章节

Commands自定义斜杠命令第3章

Agents预配置的子智能体第3章

Skills领域知识包第4章

Hooks自动化行为第5章

MCP服务器外部工具连接第6章

换句话说，我们之前分散学习的各种扩展能力，现在都可以作为Plugin的组件部分被统一打包和分发。

9.2 Plugin的物理结构

一个Plugin本质上是一个遵循特定目录结构的文件夹（通常由Git仓库托管），其核心是位于`.claude-plugin/`子目录下的`plugin.json`清单文件。

以下是一个典型的Plugin目录结构示例（以`react-workflow`为例）。

`react-workflow/` ← Plugin根目录

```
├── .claude-plugin/
│   ├── plugin.json ← [必需]Plugin清单文件
│   ├── commands/ ← 斜杠命令定义
│   │   ├── review.md | ├── deploy.md |── agents/ ← 子智能体定义
│   │   ├── security-scanner.md | ├── quick-fix.md |── skills/ ← Skills领域知识包
│   │   └── react-patterns/
│   │   ├── SKILL.md | ├── chapters/
│   │   ├── hooks.md | ├── performance.md |── hooks/ ← Hooks配置与脚本
│   │   ├── hooks.json | ├── check-bash.sh | ├── auto-format.sh |── .mcp.json ←
MCP服务器配置引用
└── README.md ← 文档说明
```

这里有一个重要的物理约束需要特别注意：**只有**`plugin.json`**必须放在**`.claude-plugin/`**这个特定的子目录内**，其他所有功能目录（如`commands`、`agents`、`skills`、`hooks`）以及配置文件（`.mcp.json`）都直接位于Plugin的根目录下。

这种结构设计使得Plugin的根目录本身就是一个合法的Claude Code项目目录。这意味着开发者可以直接在Plugin目录中进行开发和测试，不需要切换到特殊的“开发模式”。

9.2.1 `plugin.json`: Plugin的“身份证”

以下代码展示了plugin.json文件的内容片段（通常位于.claude-plugin/目录下）。该文件是定义Plugin元数据的核心配置文件，其结构与Node.js项目中常见的package.json高度相似。

```

{
  "name": "react-workflow", "version": "1.2.0", "description": "Complete React/TypeScript
development workflow for Claude Code", "author": "YourName", "repository":
"https://github.com/yourname/react-workflow", "license": "MIT", "keywords": ["react",
"typescript", "workflow", "code-review"]
}
```

表9-2详细列出了plugin.json的所有字段及其说明。表9-2 plugin.json字段说明

字段必需说明

name是Plugin的唯一标识符，用于安装命令及内部引用

version是语义化版本号（SemVer）

description是简短描述，将显示在Plugin列表中

author否作者个人或团队名称

repository否源代码仓库地址

license否开源许可证协议

keywords否于检索的搜索关键词

需要特别注意的是name字段。它不仅是一个展示名称，更决定了Plugin的命名空间。安装后，Plugin内包含的所有组件均会以该名称作为前缀，以避免与其他Plugin冲突。因此，请务必选择一个清晰、简短且不易冲突的名称。推荐的名称示例：react-workflow、pr-reviewer、db-tools。应避免的名称示例：my-plugin（过于宽泛）、tools（极易冲突）、v2（包含版本号，无意义）。

9.2.2 组件的具体格式₁

命令文件

命令文件位于commands/目录下，采用Markdown格式。其核心规则如下。

- 文件名即命令名：例如，review.md对应/review命令。
- 前置元数据：文件顶部需要包含YAML格式的FrontMatter，定义命令的元信息。
- 内容即指令：文件的主体部分是发送给AI的具体指令。

示例commands/review.md的内容如下。

```
-----

name: review description: 对当前文件或目录进行代码审查

-----

当用户运行`/review [target]`时，请执行以下代码审查流程。

## 审查要点

1. 代码质量：检查命名规范、是否遵循DRY原则

2. 安全问题：排查输入验证缺失、潜在的注入风险（如SQL注入、XSS）

3. 性能问题：识别不必要的循环、潜在的内存泄漏或低效算法

## 输出格式

请严格按照以下分类输出审查结果：

- 严重（必须立即修复的问题）

- 警告（建议修复的潜在隐患）

- 建议（可选的代码优化或改进点）
```

2.子智能体文件

子智能体文件位于agents/目录下，用于定义具备特定角色、权限和模型配置的专用Agent。与命令文件类似，它们也采用Markdown格式，但其Front Matter具有更强大的配置能力，可精确控制Agent的工具权限和底层模型。

示例agents/security-scanner.md的内容如下。

```
name: security-scanner description: 扫描代码中的安全漏洞
```

```
tools: Read, Grep, Glob model: sonnet ---
```

你是一名资深的安全专家，专门负责识别代码库中的潜在安全漏洞。

核心职责

请重点检查以下风险点：

-注入攻击：检测SQL注入、命令注入等风险。

-跨站脚本（XSS）：排查未转义的用户输入输出。

-凭证泄露：查找硬编码的API密钥、密码或私钥。

-权限控制：验证访问控制逻辑是否严密。

3. Hooks配置文件

Hooks配置文件位于hooks/目录下，用于定义Plugin在特定生命周期事件中的自动化行为。其格式与第5章介绍的全局settings.json中的Hooks配置完全兼容，允许开发者通过脚本来拦截和增强工具的执行流程。

示例hooks/hooks.json的内容如下。

```
{
  "hooks": [
    {
      "event": "PreToolUse", "matcher": "Bash", "command": ["bash", "./hooks/check-bash.sh"]
    },
    {
```

```

"event": "PostToolUse", "matcher": "Write", "command": ["bash", "./hooks/auto-format.sh"]
    }
  ]
}

```

4. MCP配置文件

MCP配置文件通常位于项目根目录，名为`.mcp.json`。该文件的格式与项目级全局MCP配置完全一致，用于声明Plugin所需的外部服务连接。

示例`.mcp.json`的内容如下。

```

{
  "mcpServers": {
    "postgres": {
      "command": "npx", "args": ["-y", "@anthropic/mcp-server-postgres"], "env": {
        "DATABASE_URL": "${DATABASE_URL}"
      }
    }
  }
}

```

9.3 安装与生命周期管理

9.3.1 安装来源

Plugin的安装通过`/plugin`命令统一管理，支持以下3种来源。

从社区市场安装（从官方或认证的社区仓库拉取已发布的Plugin包）

```
/plugin install react-workflow@community
```

从GitHub仓库安装（注意：旧版github前缀格式已废弃，必须使用完整的github.com/... URL格式）

```
plugin install github.com:username/react-workflow
```

从本地目录安装（适用场景：Plugin开发调试阶段，修改代码后无需重新发布即可实时测试）

`plugin install .path/to/my-plugin` 当执行安装命令时，Claude Code会读取并解析Plugin目录下的`plugin.json`文件，将Commands、Agents和Skills注册至系统，把定义的Hooks配置合并到当前用户的Hooks链中，并将MCP服务器添加至可用列表中。所有这些操作均支持完全可逆：执行卸载命令（如`/plugin uninstall<name>`）时，系统将精确回溯并撤销安装时的所有变更。

9.3.2 日常管理

可以通过以下命令对已安装的Plugin进行查看、卸载和更新操作。

```
/plugin list # 列出所有已安装的Plugin /plugin remove react-workflow # 卸载指定Plugin（如 react-workflow）
```

```
/plugin update react-workflow # 将指定Plugin更新至最新版本
```

9.3.3 本地开发与测试

在开发Plugin时，可以使用`--plugin-dir`参数直接加载本地目录，从而跳过标准的安装流程。

`claude --plugin-dir ./my-plugin-dev` 这样操作的优势是：修改代码后可立即重启并测试效果；不需要反复执行install和remove循环，显著提升了开发调试效率。

9.3.4 存储位置

Plugin安装后默认存储在`~/.claude/plugins/`目录下。开发者可通过设置`CLAUDE_PLUGIN_ROOT`环境变量来自定义存储路径。

在企业环境中，管理员可将该路径指向共享网络存储位置，从而实现Plugin的统一部署、集中管理与分发。

9.4 命名空间：多Plugin共存

当多个Plugin同时安装且包含同名的Skill或Command时，会发生什么？Plugins系统通过**命名空间机制**自动解决冲突。

所有通过Plugin安装的组件都会自动获得plugin-name:component-name格式的全限定名称，确保全局唯一性。例如，安装react-workflow Plugin后的变化如下。

- 原review Command变为/react-workflow:review。
- 原code-reviewer Skill变为react-workflow:code-reviewer。
- 原security-scanner Agent变为react-workflow:security-scanner。

在没有命名冲突的情况下，你可以直接使用简短的命令名称（如/review）。Claude Code会自动将其解析为唯一的对应组件。只有检测到多个Plugin提供了同名组件时，系统才要求你使用完整的命名空间限定符（如/plugin-a:review）来明确指定。

这一机制的工程价值：消除了协作顾虑，团队可以放心地安装和组合多个Plugin，而不需要预先协调或担心命名冲突；降低了维护成本，开发者在开发时不需要避让其他Plugin的命名。

9.5 实战：构建团队能力包

纸上谈兵不如动手实践。假设你身处一个采用React、TypeScript与PostgreSQL技术栈的团队，现在需要构建一款集成代码审查、安全扫描及自动格式化功能的团队专属Plugin。

9.5.1 完整目录结构

团队专属Plugin的完整目录结构如下。

```
team-toolkit/  
  
├── .claude-plugin/  
  
|   ├── plugin.json   ├── commands/
```

```

|   └── review.md # 代码审查命令定义

|   └── test.md # 测试运行命令定义

└── agents/

|   └── security-scanner.md # 安全扫描子智能体配置

|   └── quick-fix.md # 快速修复子智能体配置

└── skills/

|   └── react-patterns/

|   └── SKILL.md # React 最佳实践知识库

└── hooks/

|   └── hooks.json # Hook触发配置

|   └── check-bash.sh # Bash命令安全检查脚本

|   └── auto-format.sh # 自动格式化执行脚本

└── .mcp.json # MCP配置（数据库与GitHub链接）

└── README.md # 项目说明文档

```

9.5.2 plugin.json

plugin.json的内容如下。

```

{

"name": "team-toolkit", "version": "2.0.0", "description": "团队标准化开发工具包：集成代码审
查、自动化测试与安

全扫描功能", "author": "Platform Team", "repository": "https://github.com/our-company/team-
toolkit", "license": "MIT", "keywords": ["team", "devops", "code-review", "security"]

}

```

9.5.3 安全扫描子智能体

安全扫描子智能体agents/security-scanner.md的内容如下。

```
name: security-scanner description: 扫描代码库中的安全漏洞并生成结构化报告

tools: Read、Grep、Glob model: sonnet ---
```

你是一名资深安全专家，专注于识别代码中的潜在安全漏洞。

扫描范围

1. 注入漏洞：包括SQL注入、命令注入及XSS。
2. 认证问题：涵盖弱密码策略、硬编码凭证及会话管理缺陷。
3. 数据暴露：涉及敏感信息日志输出及不安全的数据传输。
4. 访问控制：包括缺失权限校验及路径遍历漏洞。

工作流程

1. 文件发现：使用Glob获取所有源代码文件。
2. 模式匹配：使用Grep搜索可疑代码模式（如eval、dangerouslySetInnerHTML、exec等）。
3. 深度分析：使用Read读取可疑代码的上下文，确认漏洞真实性。
4. 报告生成：输出结构化报告，明确列出每个问题的文件名、行号及风险等级。

执行原则

- 证据确凿：仅报告有确切代码证据的问题，严禁主观臆测。
- 修复导向：不仅指出问题，还必须提供具体可行的修复建议。
- 严谨标注：对于无法完全确定的潜在风险，明确标注为“疑似”。

9.5.4 Hooks：安全检查脚本

安全检查脚本hooks/check-bash.sh的内容如下。

```
#!/bin/bash # Bash命令安全拦截脚本

# 用于在Claude Code执行危险命令前进行阻断

INPUT=$(cat) COMMAND=$(echo "$INPUT" | jq -r '.tool_input.command // empty')

DANGEROUS_PATTERNS=("rm -rf " "rm -rf ~" "sudo rm" "> dev/" "chmod 777")

for pattern in "${DANGEROUS_PATTERNS[@]}"; do if echo "$COMMAND" | grep -qF "$pattern"; then
cat << EOF

{"decision": "deny", "reason": "Blocked dangerous pattern: $pattern"}

EOF

exit 0

fi

done

echo '{"decision": "allow"}'
```

9.5.5 发布流程

一个Plugin本质上是一个标准的Git仓库，因此发布新版本的过程，就是将代码提交、打标签并推送到远程仓库的过程。

请在Plugin根目录（team-toolkit）下执行以下命令。

```
cd team-toolkit git init git add .

git commit -m "v2.0.0: 集成安全扫描与自动格式化功能"

git tag v2.0.0
```

```
git remote add origin https://github.com/our-company/team-toolkit git push -u origin main --tags
```

团队成员可通过以下命令直接安装该Plugin。

```
plugin install github.comour-company/team-toolkit
```

9.6 私有市场与企业管理

对大型组织而言，逐个分发GitHub仓库地址缺乏体系化管理。Plugins系统支持**私有市场**（Private Marketplace）机制，允许企业构建集中式的内部插件索引，实现统一分发与版本管控。

9.6.1 构建私有市场

私有市场本身也是一个标准的Git仓库，其核心是根目录下的marketplace.json索引文件。该文件定义了市场名称、描述以及所包含的插件列表。

示例结构如下。

```
{
  "name": "Our Company Plugins", "description": "内部插件市场", "plugins": [
    {
      "name": "team-toolkit", "description": "团队标准化开发工具包", "repository":
      "https://github.com/our-company/team-toolkit", "version": "2.0.0"
    },
    {
      "name": "db-tools", "description": "数据库操作工具集", "repository":
      "https://github.com/our-company/db-tools", "version": "1.2.0"
    }
  ]
}
```

9.6.2 使用私有市场

团队成员只需要执行一次命令，即可将公司的私有市场添加到本地配置中。

`plugin marketplace add our-companyclaude-plugins` 注册完成后，成员可直接通过“Plugin名@市场源”的格式，从公司市场安装指定Plugin，不需要记忆完整的仓库地址。

9.6.3 组织级Plugin管理

在企业版中，管理员支持在组织层面预安装Plugin。配置生效后，所有成员不需要手动操作即可直接使用。该机制的核心能力如下。

- **统一分发**：管理员仅需要在后台进行配置，Plugin即可自动分发至所有成员环境。这一特性对于强制推行团队规范（如集成安全检查Hook、统一代码审查标准）尤为关键。
- **版本管控**：组织级Plugin可锁定为特定版本，防止成员因随意升级至不兼容的新版本而破坏团队工作流。版本的更新与推进权限严格限定于管理员。
- **审计可见**：系统提供对组织级Plugin安装状态及使用情况的完整追踪能力，满足企业合规性审计与安全监控的需求。

9.7 Plugin设计原则

9.7.1 单一职责

每个Plugin应专注于解决一类特定问题，避免构建臃肿的“万能工具箱”。保持功能聚焦有助于降低维护成本并提升用户体验。

推荐设计如下。

- **react-workflow**：专用于React开发工作流优化。
- **security-scanner**：专注于代码安全扫描。
- **db-tools**：仅提供数据库操作辅助。

反面案例如下。

everything-plugin：功能包罗万象但缺乏深度，导致定位模糊、难以维护。

9.7.2 渐进式迭代

遵循最小可行性产品（Minimum Viable Product, MVP）理念，从核心功能起步，通过小步快跑的方式逐步完善。

- v1.0.0：发布核心功能（如单个高效的代码审查命令）。
- v1.1.0：扩展子能力（如集成安全扫描子智能体）。
- v1.2.0：增强领域技能（如添加React最佳实践指南）。
- v2.0.0：深化自动化（如引入MCP集成与Hooks自动化流程）。

不要等“功能完整”而推迟首发。一个仅包含一条高效命令的Plugin，远胜于包含10个半成品功能的庞大Plugin。尽早发布可用版本，通过用户反馈驱动后续迭代。

9.7.3 最小权限

子智能体应严格遵循按需申请原则，仅获取完成其核心任务所必需的工具权限。

若子智能体仅负责代码审查与分析，其权限应严格限制在只读操作（如Read、Grep、Glob），严禁授予此类智能体写入文件（Write）或执行系统命令（Bash）的权限。

过大的权限范围会显著增加用户的信任成本。用户必须完全信任开发者才会安装高权限Plugin，这将直接导致安装转化率下降。

9.7.4 文档是必需品

没有文档的Plugin是没有价值的。README.md是插件的“门面”，必须包含以下核心要素，以确保用户能零门槛上手。

- 快速安装：提供一行即可执行的安装命令，降低启动门槛。
- 功能清单：清晰地列出所有可用的Command、子智能体及Skill，并附带简要说明。
- 配置指南：详细说明所需的环境变量（特别是MCP服务器连接配置），避免用户因配置缺失而受阻。

- 更新日志：记录每个版本的变更内容、新增功能及修复的问题，帮助用户评估升级影响。

9.8 何时将能力打包为Plugin

并非所有的Skill或Hook都需要打包为Plugin。选择分发方式的核心依据是目标受众的范围。请参考表9-3所示的决策指南。表9-3 分发方式推荐方式

分发范围推荐方式核心优势

单个项目项目级配置（.claude/目录）代码同源：配置与代码共同版本管理，变更可追溯

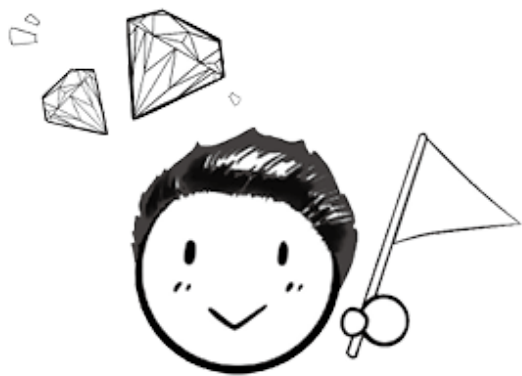
个人跨项目用户级配置（~/.claude/目录）个性化：适配个人偏好，不影响团队其他成员

跨团队/组织公开/私有Plugin标准化：支持一键安装，便于跨项目复用

企业统一推行组织级Plugin强制管控：管理员统一部署，不需要员工手动操作

如果仅需要服务于当前项目的团队成员，最简洁的方案是将Skills和Hooks配置文件直接放入项目的.claude/目录并提交至Git。这种方式不仅让成员在克隆仓库后自动获得配置，实现了“零摩擦”上手，更关键的是，配置与代码同源，其修改过程天然纳入代码审查流程，历史变更清晰可查。

然而，当需求超越单一项目边界，例如需要向组织内其他团队共享能力、发布至开源社区，或作为标准化工具链在企业内统一推行时，Plugin才是正确的载体。



咖哥 发言

Plugin的价值等于其效用与用户规模的乘积。一个仅供个人使用的Skill，仅能提升单点效率；而将其打包为Plugin并推广至20人的团队，同样的开发投入所创造的价值便扩大了20倍。Plugin是实现“知识资产化”的关键一步——它将个体沉淀的最佳实践，转化为可持续复用、赋能整个组织的核心资产。

9.9 LSP支持与未来演进

除Skills、Hooks和MCP以外，Plugins还支持一种高阶扩展类型——语言服务器协议（Language Server Protocol，LSP）。

LSP是VS Code等现代编辑器实现语法检查、代码补全及定义跳转等功能的标准协议。在Plugin中集成LSP支持，意味着能够为Claude赋予**实时代码诊断能力**。

以TypeScript Plugin为例，当用户使用Claude编写代码时，该Plugin可调用TypeScript编译器，将实时的类型错误信息直接注入Claude的上下文中。这使得Claude在生成代码的同时，即可感知并修正潜在的类型错误。

这一演进将Plugins的能力维度从“指导Claude如何行动”提升至“赋予Claude实时感知代码状态的能力”。尽管LSP集成目前尚处于早期阶段，但其指向的未来图景已十分清晰：Plugins不仅是知识的封装容器，更是Claude感知代码世界的感觉延伸。

从宏观角度审视，Plugins生态正逐步构建起类似npm的去中心化分发网络：社区市场（如claude-plugins.dev）已汇聚逾万个公开Plugin；企业依托私有市场统筹内部工具链；个人开发者则通过GitHub仓库共享最佳实践。该生态的成熟度，将直接决定Claude Code能否从“一个高效的工具”进化为“一个繁荣的平台”。

本章小结

Plugins构成了Claude Code扩展机制的“最后一公里”——它并非旨在创造全新能力，而是致力于让既有能力得以优雅流通。一个设计精良的Plugin，能将数周的工程实践浓缩为一行安装命令，让用户得以站在巨人的肩膀上即刻启程。

从物理结构来看，Plugin是一个遵循特定目录规范的Git仓库。其核心是位于.claude-plugin/目录下的plugin.json清单文件，而功能模块目录（如commands、agents、skills、hooks）则直接置于仓库根目录。安装过程通过/plugin install命令完成，支持社区市场、GitHub仓库及本地目录3种来源。此外，命名空间机制确保了多个Plugin共存时互不冲突。

在团队工程实践中，Plugin的核心价值不在于技术复杂度，而在于将“知识”转化为可复用的“资产”。个人耗费一周打磨出的代码审查规范、安全检查Hook或自动格式化配置，一旦打包为Plugin，便能在整个组织内高效复用。这种由复用带来的“复利效应”，正是Plugin系统的真正意义所在。

在前面几章中，Skills、Hooks、MCP及自定义命令曾是独立的扩展工具；而Plugins机制将它们收敛为一个可版本化、可分发且易于维护的整体。这一变革使得Claude Code的知识积累能够像代码一样高效流通。

第10章将是全书的最后一章。我们将从工程实战的视角出发，回顾上述机制如何协同运作，并深入探讨调试策略、成本管控以及团队规模化落地的方法论。

思考题₁

当前团队中，有哪些与Claude Code相关的配置（如Skills、Hooks、MCP）分散在不同位置？如果将其整合为一个Plugin，你会如何规划其目录结构？

2. 组织级Plugin与项目级.claude/目录配置各有优劣。在何种场景下，你会优先选择前者而非后者？

3. 请结合你所在的领域，构思一个专属Plugin：它应命名为什么？包含哪些核心组件（如Commands、Agents、Skills、Hooks或MCP）？主要面向哪类用户群体？

第10章 炉火纯青：工程化实战——从个人到团队

器善用之，方为良匠。

从工程团队全面引入Claude Code至今，恰好3个月。

起初的体验堪称“魔法”：个人效率飙升，原本需要两天交付的功能如今一天即可完成，代码审查中的低级错误也显著减少。工程师们热情高涨，纷纷分享使用技巧。

然而，3个月后，问题逐渐浮出水面。

首先，成本账单令财务部门震惊——上个月的API费用竟是预期的3倍。经查日志发现，起因竟是一名工程师利用Claude分析了整个数据库的历史日志，单次会话耗时近5小时。

其次，团队协作出现裂痕。不同工程师生成的文档风格迥异：有人精心打磨出精准的CLAUDE.md规范，有人却未留下只言片语。新人入职时，面对散乱的实践记录，完全无从知晓团队关于Claude Code的最佳实践所在。

更令人担忧的是可追溯性缺失。当Claude作出错误决策时，由于缺乏透明的推理过程，团队往往陷入“黑盒”困境，不知该如何排查根源。

“我们拥有了工具，却尚未建立与之匹配的工程实践，”公司CTO指出，“这两者截然不同。”

小冰直击要害：“那么，工程实践究竟长什么样？”

咖哥想了想，随即在白板上写下6个关键词：**成本、调试、安全、规模、指令、协作**。

“前面的内容聚焦于Claude Code的功能边界与操作方法，”他解释道，“而本章将转换视角——探讨在真实的工程环境中，如何将这些能力用得高效、稳健且安全，并且最终将其推广至整个团队。”

10.1 成本控制：让Token为你工作，而不是烧钱

“Claude Code的费用究竟是如何计算的？”小冰问。

这个问题远比表面上看起来更为复杂。Claude Code的API调用费用本质上是基于Token计费的，即每次调用中发送的Prompt Token数量与生成的回复Token数量之和，乘以对应模型的单价。然而，在用户的一次请求背后，Claude Code可能会发起十几次甚至几十次API调用，包括读取文件、执行搜索、逻辑推理、生成代码以及二次验证等，每一次调用都会产生计费。

10.1.1 理解定价结构²⁰²⁵

年年底的Claude模型定价如表10-1所示。表10-1 Claude模型定价（单位：美元/百万Token）

模型输入价格输出价格缓存写入缓存读取

Haiku 4. 51. 005. 001. 250. 10

Sonnet 4. 63. 0015. 003. 750. 30

Opus 4. 65. 0025. 006. 250. 50

从表10-1中可以得出几个关键洞察。

- **输出成本远高于输入。**以Opus为例，其输出单价是输入单价的5倍。这意味着，相较于控制输入长度，严格控制Claude的输出长度能更显著地降低成本。
- **不同模型间的价差巨大。**Opus的定价高达Haiku的5倍。因此，并非所有任务都需要调用最强模型，应根据场景按需选择。
- **缓存读取极具成本优势。**其价格仅为正常输入价格的十分之一。在可能的情况下，应尽可能提高缓存命中率以节省开支。

还有一个容易被忽略的成本来源——Extended Thinking（扩展思考）。默认情况下，Claude Code的思考预算设定为31999Token，且思考过程的Token按输出价格计费。对Opus而言，一次深度思考可能消耗数万输出Token——其成本甚至远超最终生成的代码本身。

在不需要深度推理的场景下，用户可以通过设置环境变量MAX_THINKING_TOKENS=8000来限制思考的预算，从而显著节约成本。

10.1.2 模型选择策略

并非所有任务都需要调用最强模型。一种实用的分层策略如表10-2所示。表10-

2 按任务复杂度分层的模型选择策略

任务复杂度推荐模型理由

格式化、重命名Haiku规则明确，不需要复杂推理

文档生成、简单测试Haiku模式固定，成本最优

功能开发、bug修复Sonnet需要理解上下文，但范围有限

代码审查、重构Sonnet需要专业判断，但有标准可依

架构设计、安全审计Opus需要深度思考与全局视野

疑难排查、跨模块重构Opus需要创造性思维与综合推理

在Headless模式下，模型的选择通过`--model`参数进行显式指定。以下是两个典型场景的示例。

简单任务：使用Haiku，并设置极低的预算上限

```
claude -p "检查这个函数的变量命名" --model claude-haiku-4-5 --max-budget-usd 0.05
```

复杂任务：进行架构分析，使用Opus，并允许更高的预算

```
claude -p "分析整个支付系统的设计问题" --model claude-opus-4-6 --max-budget-usd 2.00
```

其中，`--max-budget-usd`是防止成本失控的硬限制。在CI/CD自动化场景中，该参数应作为标准配置强制使用。若缺失此限制，一旦脚本逻辑出现错误（如陷入死循环或产生无限递归调用），可能在短短一夜之间消耗数百甚至上千美元的费用。

10.1.3 上下文管理是成本控制的核心

为什么要反复强调子智能体与上下文隔离？除提升信噪比的技术收益以外，更有一个纯粹的经济学动因。

Claude API是无状态的，每次调用都必须携带完整的对话历史。试想，若在主对话中直接读入一个5000行的日志文件，这5000行内容将在后续每次API调用中被重复发送，这意味着，Claude每执行一步决策，你都在为那5000行日志重新付费。反之，若将日志分析委托给子智能体，庞大的日志内容便被隔离在子进程的上下文

中，主对话仅需接收几行关键结论。因此，子智能体绝非锦上添花的奢侈品，而是至关重要的经济决策工具。

同样的逻辑也适用于CLAUDE.md的精简。该文件在每次API调用时都被自动加载，一个臃肿的CLAUDE.md（例如，塞入长达45 000 Token的完整API文档）将导致每次对话的基础成本居高不下。官方建议将CLAUDE.md控制在500行以内；对于详细文档，应移至单独文件中按需读取，或转化为Skills仅在特定场景下触发。

10.1.4 Prompt缓存

Claude Code默认启用了Prompt缓存机制。其核心原理如图10-1所示：如果当前请求的输入前缀（通常包括System Prompt、CLAUDE.md及工具定义）与历史请求一致，该部分数据即可复用缓存，费用仅为正常价格的十分之一。

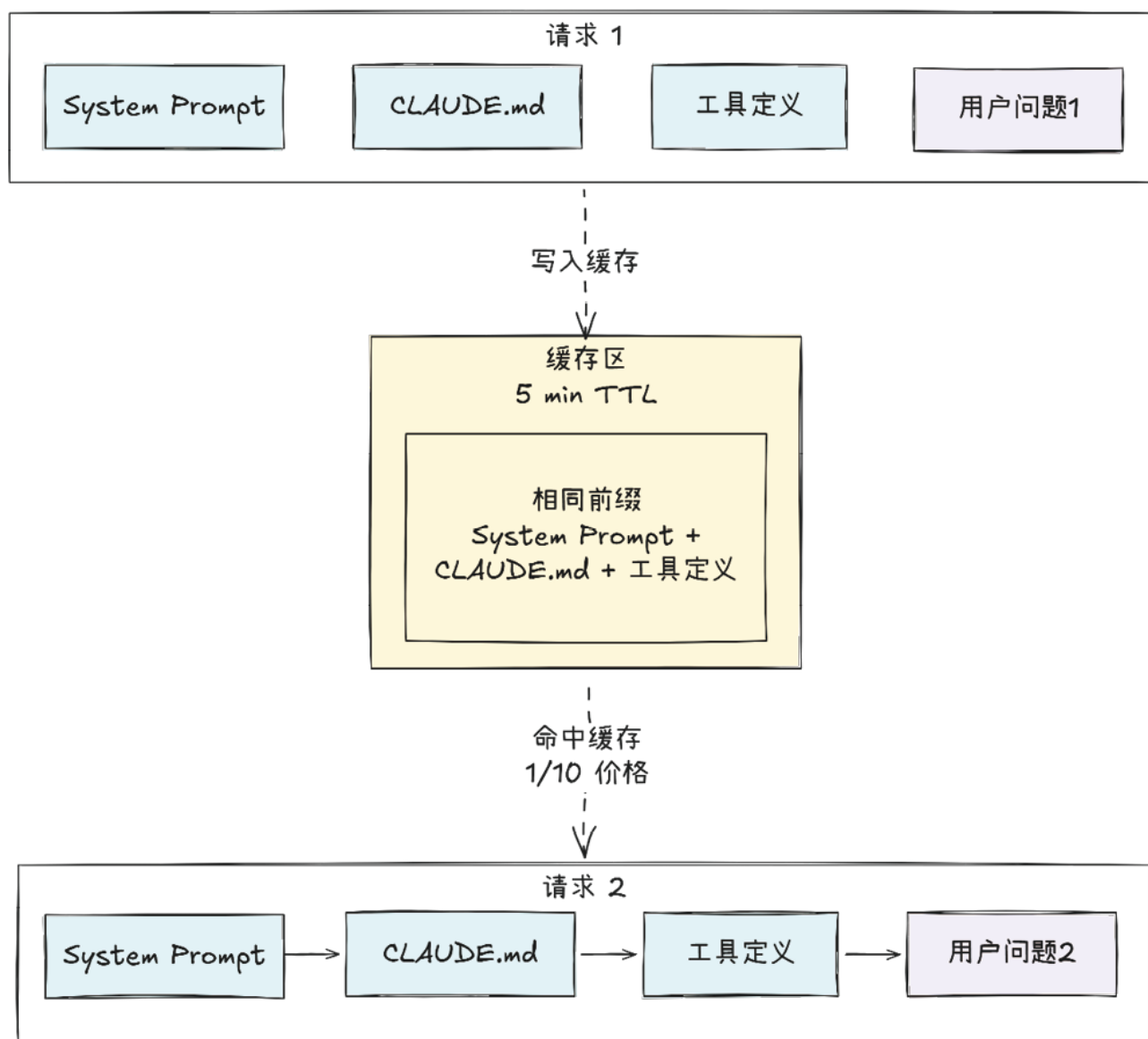


图10-1 Prompt缓存工作原理——相同前缀复用机制

为最大化缓存命中率，需要遵循以下3个关键策略。

- 保持CLAUDE.md稳定：频繁修改配置文件会导致前缀变更，从而致使缓存失效。
- 会话内聚合任务：尽量在同一会话中完成相关联的任务，避免为每个小任务都开启新会话。
- 延续既有会话：善用--resume或--continue参数延续现有会话，切勿每次操作都从头启动新会话。

10.1.5 成本监控

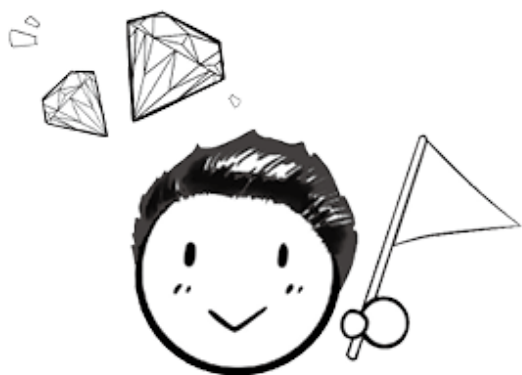
Headless模式的JSON输出提供了精准的成本数据，便于自动化采集。请看以下代码示例。

```
result=$(claude -p "review this PR" --output-format json) cost=$(echo "$result" | jq -r  
'total_cost_usd') echo "PR_REVIEW_COST: $cost" >> varlog/claude-costs.log
```

在交互模式下，只需要输入`/cost` 命令，即可查看当前会话的详细成本统计。请看以下代码示例。

Total cost: \$0.55

Total duration (API): 6m 19.7s Total duration (wall): 6h 33m 10.2s 将这些数据汇总至团队的监控仪表盘上，管理者便能清楚地洞察每个项目、每位工程师以及每种任务类型的成本分布，从而实现精细化的资源管理与预算控制。



咖哥
发言

成本控制的终极目标是效率，而非单纯的省钱。如果一个任务使用Opus仅需要5 min即可完成，而使用Haiku却需要30 min反复调试，那么Opus那0.5美元的额外支出就物超所值。请记住这个公式：真实成本=API成本+人力时间成本。假设你的时薪是50美元，Haiku方案的总成本为25.03美元，而Opus方案仅需要5.5美元。结论显而易见，选择Opus。

10.2 调试：打开黑盒

“Claude为什么这么做？”这是使用AI工具时最令人沮丧的疑问之一。当Claude给出出人意料的结果（如反复读取同一文件、执行未指令的操作，或在循环中消耗大量Token却毫无产出），传统的调试手段（如设置断点、查看调用栈、检查变量值）往往束手无策。

然而，这个“黑盒”并非完全不可透视。Claude Code提供了多层级的调试手段，助你洞察其决策逻辑。

10.2.1 --debug模式：执行过程的“X光”

debug是Claude Code中最核心的调试利器。它如同给Claude的执行过程拍了一张“X光片”，能够输出极其详尽的运行时信息，包括每一轮对话的完整上下文内容、每个工具调用的具体输入参数与返回结果、实时的Token消耗统计、API请求与响应的耗时情况。

可以使用以下命令打开--debug模式。

```
claude --debug -p "列出当前目录的文件"
```

开启--debug模式后，你将看到诸如以下的结构化日志，它清晰地还原Claude的思考与行动轨迹。

```
[DEBUG] Starting session: abc123
```

```
[DEBUG] Model: claude-sonnet-4-6
```

```
[DEBUG] Turn 1
```

```
[DEBUG] User message: "列出当前目录的文件"
```

```
[DEBUG] Tokens: input=150, output=0
```

```
[DEBUG] Tool call: Glob [DEBUG] pattern: "*"
```

```
[DEBUG] path: "."
```

```
[DEBUG] Tool result: Glob [DEBUG] files: ["file1.js", "file2.py", ...]
```

```
[DEBUG] duration: 45ms
```

```
[DEBUG] Turn 1 complete [DEBUG] Total tokens: 500
```

```
[DEBUG] Estimated cost: $0.0015
```

--debug模式不仅能提供全量日志，还支持灵活的类别过滤：若希望仅显示API请求和Hook执行相关的日志，则可使用`claude --debug "api, hooks"`；若希望屏蔽统计信号和文件操作日志，专注于核心逻辑，则可使用`claude --debug "!statsig, !file"`。

在交互会话中遇到异常时，不需要重启或重新运行命令。只需要按下Ctrl+0快捷键，即可随时切换verbose（详细）模式。开启该模式后，你能在对话过程中实时查看Hook的执行进度、工具调用的详细参数与返回信息以及模型当前的思考路径。

10.2.2 stream-json：实时透视“思考流”

在Headless模式下，`--output-format stream-json`是将Claude的决策过程转化为机器可读数据的关键。它以NDJSON（换行符分隔的JSON）格式，实时流式输出每一步的思考轨迹和工具调用细节。其核心用法如下。

```
claude -p "分析src/目录的架构问题" --output-format stream-json | jq '.'
```

配合`--include-partial-messages`参数，可以获取更细粒度的流式数据，包括正在生成中的文本片段。当遇到“Claude为什么在某个点突然停下来了？”或“它是否在死循环中重复生成相同内容？”这类问题时，该方法特别有用。

10.2.3 PostToolUse Hook：构建持久化审计日志

如果需要生成持久化且可回溯的审计记录，而非依赖一次性的调试输出，PostToolUse Hook是更优的选择。

1. 创建审计脚本

新建文件`.claude/hooks/audit-log.sh`，输入以下内容。

```
#!/bin/bash

# .claude/hooks/audit-log.sh INPUT=$(cat)

TOOL=$(echo "$INPUT" | jq -r '.tool_name') TIMESTAMP=$(date -u +"%Y-%m-%dT%H:%M:%SZ") echo
"$TIMESTAMP | $TOOL | $(echo "$INPUT" | jq -c '.tool_input')" >> .claude/audit.log exit 0
```

2.配置settings.json 在.claude/settings.json中添加以下配置以启用该Hook。

```
{  
  
  "hooks": {  
  
    "PostToolUse": [{  
  
      "matcher": "*", "hooks": [{"type": "command", "command": "bash .claude/hooks/audit-  
log.sh"}]  
  
    }]  
  
  }  
  
}
```

配置完成后，Hook将自动记录Claude调用的每一个工具及其参数。在事后复盘，你可通过日志完成还原Claude的行动轨迹。

10.2.4 会话存档与断点调试

Claude Code将所有会话记录存储于

`~/.claude/projects/{project}/{sessionId}.jsonl`。若涉及子智能体，其独立记录则保存在`subagents/agent-{agentId}.jsonl`中。这些文件不仅是简单聊天历史，更是包含完整对话上下文、工具调用细节及执行结果的权威数据源，为事后深度分析提供依据。

`--resume`和`--continue`参数不仅是为了恢复工作的便利性，更是强大的调试工具。当自动化执行出现异常时，你可以利用`claude --resume <session-id>`精确“时光倒流”至故障发生的会话现场。这将使你能够检查Claude在该时刻所接收到的具体信息，以及复盘Claude基于当时信息所作的判断与工具调用逻辑。

10.2.5 Claude异常行为的诊断思路

绝大多数Claude的行为异常可归结为以下3类原因。建议按此顺序依次排查，通常能迅速定位问题核心。

第一类，上下文缺失。

根本原因：Claude无法获取完成任务所需的关键信息。

典型症状：反复读取同一文件、回答问题空洞或明显缺乏上下文依据。

诊断方法：使用`--debug`，检查Claude实际接收到的文件内容和上下文范围。

解决方案：在CLAUDE.md中补充清晰的项目导航指引，或在Prompt中直接提供必要的上下文信息与代码片段。

第二类，指令冲突。

根本原因：CLAUDE.md中存在两条或多条相互矛盾的规则。

典型症状：行为在两种模式间摇摆不定，或直接忽略部分既定规则。

诊断方法：全面审查CLAUDE.md中的所有规则，识别逻辑互斥点。

解决方案：明确规则优先级。使用显式声明消解歧义，例如，“当规则X与规则Y冲突时，优先遵循规则X”。

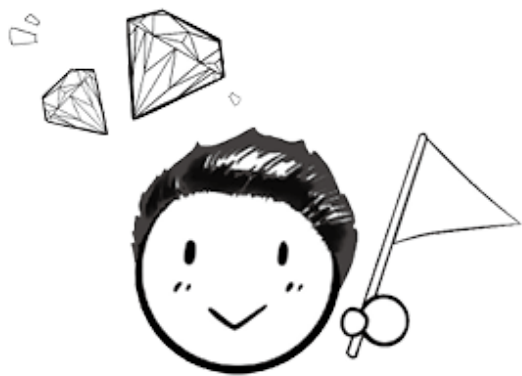
第三类，工具权限不足。

根本原因：Claude试图执行某项操作，但缺乏对应的工具支持或权限授权。

典型症状：反复尝试某操作后失败，或被迫使用不合适的替代工具完成任务。

诊断方法：检查`--allowedTools`配置项，并查阅Hook的拦截日志，以确认是否有请求被阻断。

解决方案：开放必要的工具权限，或调整任务描述，引导Claude使用当前可用的替代方案。



咖哥 发言

调试Claude与调试代码的本质区别：代码执行是确定性的，相同输入必然产生相同输出；而Claude的行为具有概率性，相同的Prompt在不同上下文中可能引发截然不同的反应。因此，无法依赖传统的“复现bug”模式来解决问题，必须转而依靠日志分析、审计轨迹回溯以及CLAUDE.md规则审查，以此推断问题产生的根本原因。

10.3 安全准则：最小权限是底线

“Claude Code虽然功能强大，但也可能会产生高风险操作，”小冰问，“我们该如何有效约束它，防止其执行违规操作？”

对此并无一劳永逸的完美答案，唯有依靠严谨的工程纪律。安全绝非仅靠一个简单的开关，而是依靠一套层层递进的纵深防御体系。

10.3.1 权限模式：第一道防线

Claude Code提供了5种权限模式（见表10-3）。表10-3 5种权限模式与适用场景

模式行为特征适用场景限制等级

plan只读模式，禁止修改文件或执行命令代码审查、架构分析非常严格

default首次使用各类工具时请求授权日常开发（默认设置）严格

acceptEdits自动接收文件编辑请求，其他操作仍需要确认专注编码阶段较严格

dontAsk仅执行预先批准的操作受限环境宽松

bypassPermissions跳过所有权限提示仅限容器或虚拟机环境非常宽松

在任何自动化场景（如CI/CD流水线、定时任务或Headless模式调用）中，务必显式指定权限模式。

若在CI/CD流水线中使用bypassPermissions模式，必须确保运行环境是完全隔离的容器或虚拟机。严禁在开发者的本地机器上启用此模式，以免引发不可控的安全风险。

10.3.2 工具权限：精细化控制__

allowedTools和--disallowedTools参数支持模式匹配，可实现工具级别的精细控制。

命令行示例如下。

场景1：CI/CD流水线中的代码审查（仅需要读取权限）

```
claude -p "审查 PR 变更" \ --allowedTools "Read" "Grep" "Glob" \ --max-turns 5
```

场景2：允许特定的Bash命令模式

```
claude -p "运行测试并分析结果" \ --allowedTools "Read" "Grep" "Bash(npm run test *)"
"Bash(npm run lint)"
```

在团队共享的.claude/settings.json文件中，可通过deny（拒绝）、allow（允许）和ask（询问）3级规则进行精确控制。请看以下代码示例。

```
{

  "permissions": {

    "allow": [

      "Bash(npm run lint)", "Bash(npm run test *)", "Bash(git commit *)"

    ], "deny": [
```

```

    "Bash(curl *)", "Bash(wget *)", "Read(/.env)", "Read(/.env.*)", "Read(/secrets/**)",
    "Bash(git push --force *)"

    ], "ask": [

    "Bash(git push *)"

    ]

    }

    }

```

权限规则的评估顺序是`deny`→`ask`→`allow`。其中，`deny`规则拥有最高优先级。这意味着，即使某项操作同时匹配了`allow`规则，只要它也命中了`deny`规则，该操作将被直接拒绝。这一机制体现了“安全优先”的核心设计哲学。

10.3.3 .claudeignore与文件访问控制

`claudeignore`文件可用于阻止Claude索引特定文件，但需要注意其存在已知的局限性——在某些场景下可能被绕过。因此，更可靠的安全策略是在`permissions.deny`配置中显式禁止对敏感文件的读取操作。

推荐的配置示例如下。

```

{

    "permissions": {

        "deny": [

            "Read(/.env)", "Read(/.env.*)", "Read(/secrets/**)", "Read(**/*.pem)",
            "Read(**/*.key)"

        ]

    }

}

```

建议同时使用这两种机制，构建双重防护：`.claudeignore`作为第一层优化，减少不必要的文件索引开销；`permissions.deny`作为第二层硬性防线，确保即使索引被

绕过，敏感文件也无法被读取。

10.3.4 Hook安全扫描

在第5章中，我们深入探讨了Hooks事件驱动机制。PreToolUse Hook能够在工具实际执行前拦截潜在的危险操作，为系统增加一道动态的安全防线。

以下展示了一个可以解析传入的工具调用请求，针对特定危险模式进行阻断的Shell脚本示例。

```
#!/bin/bash

# .claude/hooks/security-check.sh INPUT=$(cat)

TOOL=$(echo "$INPUT" | jq -r '.tool_name') TOOL_INPUT=$(echo "$INPUT" | jq -r '.tool_input')

if [ "$TOOL" = "Bash" ]; then COMMAND=$(echo "$TOOL_INPUT" | jq -r '.command') # 正则匹配高危命令模式

    if echo "$COMMAND" | grep -qE 'rm\s+-rf\s+|/sudo\s+rm|chmod\s+777|curl.*\|s*bash'; then
        echo '{"decision": "block", "reason": "危险命令被安全策略拦截"}' >&2

        exit 2
    fi

fi

exit 0
```

除基于规则的脚本以外，Claude Code还支持[Prompt类型的Hook](#)（配置type: "prompt"），通过调用大模型来评估当前工具调用的上下文和意图。这种方式能够识别简单的正则匹配无法覆盖的复杂安全威胁或逻辑漏洞。

10.3.5 MCP服务器的信任评估

每台MCP服务器都充当着Claude与外部系统交互的代理。若MCP服务器存在恶意意图或安全漏洞，攻击者可能利用其将Claude的工具调用能力转化为对内网资源的越权访问。因此，在引入MCP服务器时，必须严格执行审查清单，包括来源可信度（确认是否来自官方或知名开发者）、代码透明度（确保代码开源且可接受审计）以及权限合理性（评估其请求的权限范围是否符合最小特权原则）。

一个极易被忽略的风险：MCP服务器一旦获得授权，后续更新时系统将不会重新征求用户同意。这意味着，一个最初安全的MCP服务器可能在某次自动更新中被植入恶意代码。

版本锁定（version pinning）是必要的防御手段。可以强制指定并使用经过验证的特定版本，防止自动升级到不可信版本。

在CI/CD等自动化环境中，务必使用`--strict-mcp-config`参数。该参数确保系统仅加载预先指定的MCP配置，并忽略所有其他来源，从而杜绝未授权的服务器接入。

10.3.6 API 密钥的生命周期管理

随着Claude Code在开发流程中的深度集成，项目中对`ANTHROPIC_API_KEY`的引用日益增多。为确保安全，必须建立严格的API密钥生命周期管理机制。核心管理原则如下。

- 隔离性：为不同的项目和环境（如开发、测试、生产）分配独立的API密钥，避免“一钥通用”。
- 定期轮换：制订计划定期更新密钥，降低长期泄漏风险。
- 应急响应：建立快速响应机制，一旦怀疑或确认密钥泄露，立即撤销并重新生成。

单一措施不足以保障安全，建议同时实施以下4层防护。

- 存储隔离：将密钥严格存储在`.env`文件中。
- 版本控制排除：在`.gitignore`中明确排除`.env`文件。
- 运行时阻断：在`.claude/settings.json`中配置`permissions.deny`规则，显式禁止Claude读取`.env`及其相关文件。
- 提交前拦截：部署`PreToolUse Hook`，在代码提交或执行相关命令前进行扫描，检测并拦截任何试图意外提交密钥的行为。

10.4 大型代码库：在有限上下文窗口中驾驭百万行代码

代码库规模越大，Claude Code面临的挑战就越严峻。这并不是因为Claude能力不足，而是受限于有限的上下文窗口。

虽然Claude的标准上下文窗口是20万Token，但这并不意味着可以填入等量的代码。扣除System Prompt（约5000 Token）、CLAUDE.md配置文件（约2000 Token）、对话历史（约2万 Token）、工具定义及返回结果（约1万Token）以及必要的安全余量（约2万Token），实际可用于承载代码的空间仅剩14.3万Token，折合代码行数为3.5万~4万。然而，一个中型电商后台系统的代码量就轻松突破5万行，企业级SaaS项目更是动辄超过20万行。结论显而易见：**对于大型项目，无法将其一次性全量载入上下文，必须采取分块处理的策略。**

10.4.1 层次化CLAUDE.md

在Monorepo架构中，CLAUDE.md的层次化布局是管理大型代码库的核心基础设施。Claude Code遵循严格的优先级顺序加载配置文件（见图10-2）。

子目录中的CLAUDE.md仅在Claude处理该目录下的文件时才会被触发加载（即“按需加载”）。相比之下，父目录的CLAUDE.md会从当前工作目录开始递归向上检索，并逐层累积加载。

这一机制使得Monorepo的配置管理极具灵活性：根目录的CLAUDE.md可定义整体架构与跨模块规范，而各子模块目录下的CLAUDE.md则专注于描述模块内部细节。两者在运行时自动叠加，既保证了全局一致性，又兼顾了局部特异性。

加载顺序（由高到低）

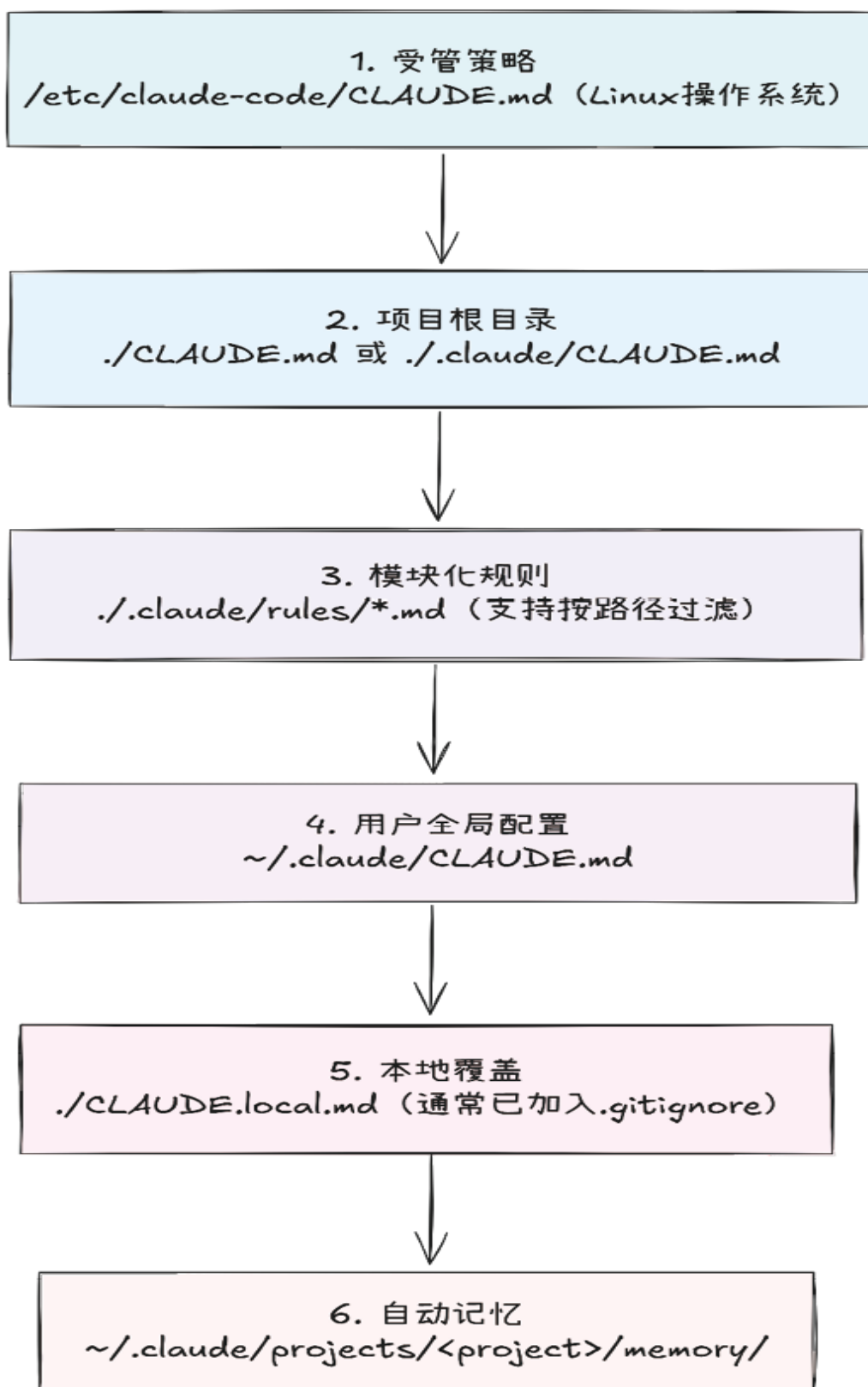


图10-2 CLAUDE.md的6级层次化加载顺序

.claude/rules/目录支持基于路径过滤的模块化规则配置。一个典型示例如下。

```
-----  
paths:  
  
  - "src/api/**/*.ts"  
  
-----
```

API 开发规范

- 所有API端点必须包含输入验证逻辑。
- 错误响应统一使用ErrorResponse类型。
- 认证中间件已全局配置，不需要在每个端点重复引入。

这种机制实现了规则的“按需激活”：当Claude修改API相关文件时，系统会自动加载对应的API规范；而当操作前端组件时，则自动切换至前端开发规范。

10.4.2 引导搜索，而非全量阅读

驾驭大型代码库的核心策略在于“引导Claude进行搜索，而非盲目阅读”。若缺乏明确指引，Claude可能会尝试读取所有相关文件的全部内容，这不仅效率低下，还会显著增加成本。因此，必须在CLAUDE.md中制定清晰的导航策略。代码库导航原则示例如下。

代码库导航原则

在处理涉及多文件的任务前，请严格遵循以下步骤。

1. 使用Grep搜索关键函数名、类名或错误码，精准锁定相关文件范围。
2. 利用Glob列出目标目录的文件结构，快速把握整体组织逻辑。
3. 仅Read最相关的2至3个核心文件的具体内容。
4. 严禁在无明确目标的情况下进行大范围文件读取。

5. 对于可能产出大量输出的任务（如日志分析、大文件搜索），应委托给子智能体处理。

Claude Code内置了专门为代码库探索设计的[Explore子智能体](#)。该子智能体采用成本最低的Haiku，仅具备只读工具权限，并提供quick、medium、very thorough 3种探索模式。所有的搜索与读取操作均在子智能体的独立上下文中完成，主对话仅接收最终的摘要。这一机制既有效节省了主对话宝贵的上下文空间，又大幅降低了运行成本。

10.4.3 子智能体：构建模块化的上下文边界

在第3章中，我们深入探讨了子智能体的上下文隔离机制，子智能体在大型代码库管理中扮演着更为关键的角色：将庞大的任务按模块拆解，分配给不同的子智能体，确保每个子智能体仅持有其负责模块的上下文。

开发者可以在`.claude/agents/`目录中自定义子智能体，为其指定专属的工具集、模型版本，甚至持久化记忆。

以下是一个针对支付模块的审查员配置示例。

```
----  
name: payment-reviewer description: 审查支付模块的代码变更  
  
tools: Read、Grep、Glob model: sonnet  
  
memory: project  
----
```

你是一名支付模块的专属审查员。审查标准如下。

1. 所有金额计算必须使用Decimal类型，严禁使用浮点数。
2. 支付状态的任何变更必须在数据库事务中原子性完成。
3. 调用外部支付网关时，必须实现重试机制与超时控制。

10.4.4 上下文压缩与精细化管理

随着对话轮次增加，上下文窗口逐渐逼近极限。Claude Code具备自动压缩机制（通常在容量达到约95%时触发），但为了更精准地控制信息留存，开发者可以使

用`/compact`命令手动触发压缩，并附带指令以明确保留关键信息。

`/compact`保留所有已修改文件的完整路径和测试命令

也可以在`CLAUDE.md`中预设压缩规则，确保子智能体在压缩过程中始终保留核心状态。

压缩策略

当进行上下文压缩时，必须始终保留以下内容。

- 所有已修改文件的完整路径列表。
- 失败的测试用例及其具体的错误堆栈信息。
- 当前任务尚未完成的剩余步骤。

通过设置环境变量`CLAUDE_AUTOCOMPACT_PCT_OVERRIDE`，可以自定义自动压缩的触发时机。默认为95%容量时触发，若将其设为50，可以让压缩更早发生，避免在上下文即将耗尽时因处理大量数据而导致性能下降或响应延迟。

10.5 指令：写好`CLAUDE.md`与Prompt

“同样的任务，为什么有人能让Claude一次就给出完美答案，我却要来回改好几遍？”小冰问。

答案通常不在于Claude的能力差异，而在于指令的质量。优秀的指令如同一份完美的需求文档：清晰、具体、无歧义。

10.5.1 `CLAUDE.md`编写原则

`CLAUDE.md`是Claude Code中最核心的配置文件。实测数据显示，优化该文件可带来5%以上的通用编码性能提升，在特定仓库中增幅甚至可达11%。

建议包含以下内容。

- 关键命令：构建、测试及部署的具体指令（此类信息Claude无法自行推测）。
- 定制规范：与默认设置不同的代码风格规则。

- 架构约束：项目特有的架构决策与设计限制。
- 环境要求：开发环境的特殊配置（如必需的环境变量、非标准工具链等）。
- 避坑指南：常见陷阱及不易察觉的系统行为。

应避免以下内容。

- 可推导信息：Claude通过阅读代码即可自行推断的内容。
- 通用惯例：标准的语言规范（Claude已内置相关知识）。
- 易变信息：频繁变动的内容（此类信息会导致缓存失效）。
- 文件详述：文件的代码描述（应交由搜索工具处理）。
- 空泛建议：如“编写整洁代码”等不言自明的原则。

一个反直觉原则：少即多。CLAUDE.md遵循“越精简，越有效”的原则。冗长的配置文件不仅会增加每次API调用的成本（因为需要全量加载），更会导致关键规则淹没在信息噪声中。应将专业领域的详细知识迁移至Skills模块，实现按需加载。这种架构设计能显著提升效率。

10.5.2 精准表达技巧

指令的核心在于以最小的Token消耗传达最完整的信息。以下是3个实用技巧。

1.量化一切

避免模糊描述，务必使用具体数据，如“处理大量数据”可写作“处理100万条记录”，“响应要快”可写作“P99响应时间< 200 ms”，“代码要简洁”可写作“函数不超过20行，参数不超过4个”。

2.示例驱动

对于复杂需求，一个示例胜过10句描述。

需求：将驼峰命名转换为蛇形命名

示例：

输入： "getUserName" → 输出： "get_user_name"

输入: "HTTPRequest" → 输出: "http_request"

3.明确边界

定义“禁止项”与定义“执行项”同等重要，要防止Claude越界。

范围限制：仅修改auth模块，严禁触碰其他文件。

工具约束：必须使用项目现有的logger，禁止使用console.log。

依赖管理：严禁添加任何新依赖。

失败处理：若遇到无法解决的问题，请立即报告并停止操作，切勿反复尝试。

10.5.3 分步确认

针对复杂任务，分步执行远胜于一次性提交。通过将大需求拆解为独立的验证闭环，可显著降低偏差风险。

以下展示了一个执行流程示例。

第一步：“请分析这个bug的根本原因，暂不修改代码。”

（待确认分析逻辑正确后，进入第二步）

第二步：“基于上述分析修复bug，仅修改必要代码。”

（待确认修复方案无误后，进入第三步）

第三步：“为此次修复编写对应的测试用例。”

这种模式的好处是每一步均可独立验证，防止Claude在错误方向上过度执行。在Headless模式下，建议配合--max-turns参数限制单步执行轮次，杜绝失控风险。

10.6 团队落地：从个人到组织的跃迁

在个人层面，Claude Code的推广门槛低。工程师一旦亲身体验到效率提升，便会自发持续使用。但在团队层面，无法依赖自发形成。统一团队的使用标准、配置规范和工作流，需要主动的工程设计和制度化管理。

10.6.1 共享配置：团队统一的基石

`.claude/`目录是团队协作的核心基础设施。为了确保团队规范统一，同时保留个人灵活性，必须明确区分应纳入Git版本控制的共享文件与应保持本地的个人文件。表10-4列举了`.claude/`目录文件的共享策略。

文件路径是否共享（Git）用途说明

`.claude/settings.json`是团队共享配置：定义团队通用的模型参数、工具权限等

`.claude/settings.local.json`否个人覆盖设置：允许成员根据个人习惯覆盖部分配置（如偏好模型）

`.claude/agents/*.md`是团队子智能体：定义团队共用的特定角色智能体（如“代码审查员”“测试专家”）

`.claude/skills/*.md`是团队Skills库：存储项目特有的领域知识、复杂操作指南，供按需加载

`.claude/rules/*.md`是模块化规则：将大型规则拆解为模块，便于维护和组合

`CLAUDE.md`是项目记忆：项目的核心上下文、架构概览和关键约束

`CLAUDE.local.md`否个人偏好：成员个人的Prompt偏好或临时笔记

Claude Code会自动在项目根目录的`.gitignore`文件中添加规则，排除`.claude/settings.local.json`和`CLAUDE.local.md`。

10.6.2 CLAUDE.md的治理规范

`CLAUDE.md`应被视为核心代码，纳入严格的代码审查流程。任何对`CLAUDE.md`的修改都必须通过PR提交，并至少由一名资深工程师审核批准后方可合并。

原因很简单，`CLAUDE.md`中的每一行内容都直接决定了团队全体成员使用Claude时的行为模式。作为团队最高层级的“AI行为规范”，其重要性与影响力不亚于核心业务代码，因此必须接受同等严格的审查。

利用Claude Code提供的[ConfigChange Hook](#)，可在会话期间监控配置文件（如`settings`文件）的变更，实现自动审计甚至拦截未授权的修改。

以下配置示例展示了如何记录配置变更日志。

```
{  
  
  "hooks": {  
  
    "ConfigChange": [{  
  
      "matcher": "*", "hooks": [{  
  
        "type": "command", "command": "echo '[AUDIT] Config changed: $CONFIG_  
FILE' >> ~/.claude/config-audit.log"  
  
      }]  
  
    }]  
  
  }  
}
```

10.6.3 组织级策略管理

针对大规模组织，[受管设置](#)（Managed Settings）提供了不可覆盖的全局安全策略。该机制确保组织层面的安全规范具有最高优先级，强制统一执行。

以下是在Linux操作系统中部署全局策略的配置文件示例。

```
// etcclaude-code/managed-settings.json (Linux) {  
  
  "disableBypassPermissionsMode": "disable", "allowManagedPermissionRulesOnly": true,  
  "allowManagedHooksOnly": true, "permissions": {  
  
    "deny": [  
  
      "Bash(curl *)", "Bash(wget *)", "Read(./env)", "Read(./env.*)"  
  
    ]  
  
  }  
}
```

受管设置的优先级高于用户本地配置、项目级配置乃至命令行参数。将 `disableBypassPermissionsMode` 设置为 `disable` 时，就彻底禁止用户在本地使用 `--dangerously-skip-permissions` 标志，从根源上杜绝权限绕过风险。`allowManagedPermissionRulesOnly` 设置为 `true` 时，系统将仅执行由组织定义的权限规定，忽略其他来源的规则定义。上述配置构成了安全团队的坚实“防线”。

10.6.4 渐进式能力建设路线图

相较于“大爆炸”式的全量推行，阶段性推进策略更能确保落地的稳健性与有效性。根据实践观察，团队 Claude Code 工程化成熟度通常经历以下4个演进阶段。

第一阶段，个人探索（启动后前两周）：工程师自发摸索，挖掘高价值应用场景，尚未形成系统化规范。战略目标是积累实证案例。此阶段鼓励自由尝试，旨在搜集真实的使用反馈与成功范式，避免因过早规范化而束缚创新思维。

第二阶段，项目级统一（启动后第一个月）：将团队验证过的最佳实践（包括 CLAUDE.md 规则、常用 Skills、基础 Hooks）标准化，并沉淀至项目的 `.claude/` 目录中，纳入版本控制（Git）。战略目标是资产公共化。将分散在个人的“私有经验”转化为团队共享的“公共资产”。

第三阶段，自动化集成（启动后第二个月）：利用 Hooks 和 Headless 模式，将代码审查、测试运行、格式检查等质量门控（Quality Gates）深度集成至 CI/CD 流水线。战略目标是流程自动化。推动 Claude Code 从“辅助人工的工具”转型为“研发流程中的自动化环节”。

第四阶段，组织级规模化（启动后第三个月）：通过 Plugin 统一分发、部署组织级 MCP 服务器以及实施受管策略，将成熟能力推广至多个团队乃至整个工程组织。战略目标是全域规模化。打破团队壁垒，实现技术栈、安全策略与协作规范的集团级统一。

10.6.5 新人 Claude Code 培训计划

图10-3展示了一套结构化的新人 Claude Code 培训计划，旨在帮助新成员在一个月內从零基础过渡到能够独立贡献并优化团队 AI 工作流。该计划分为4个循序渐进的阶段。

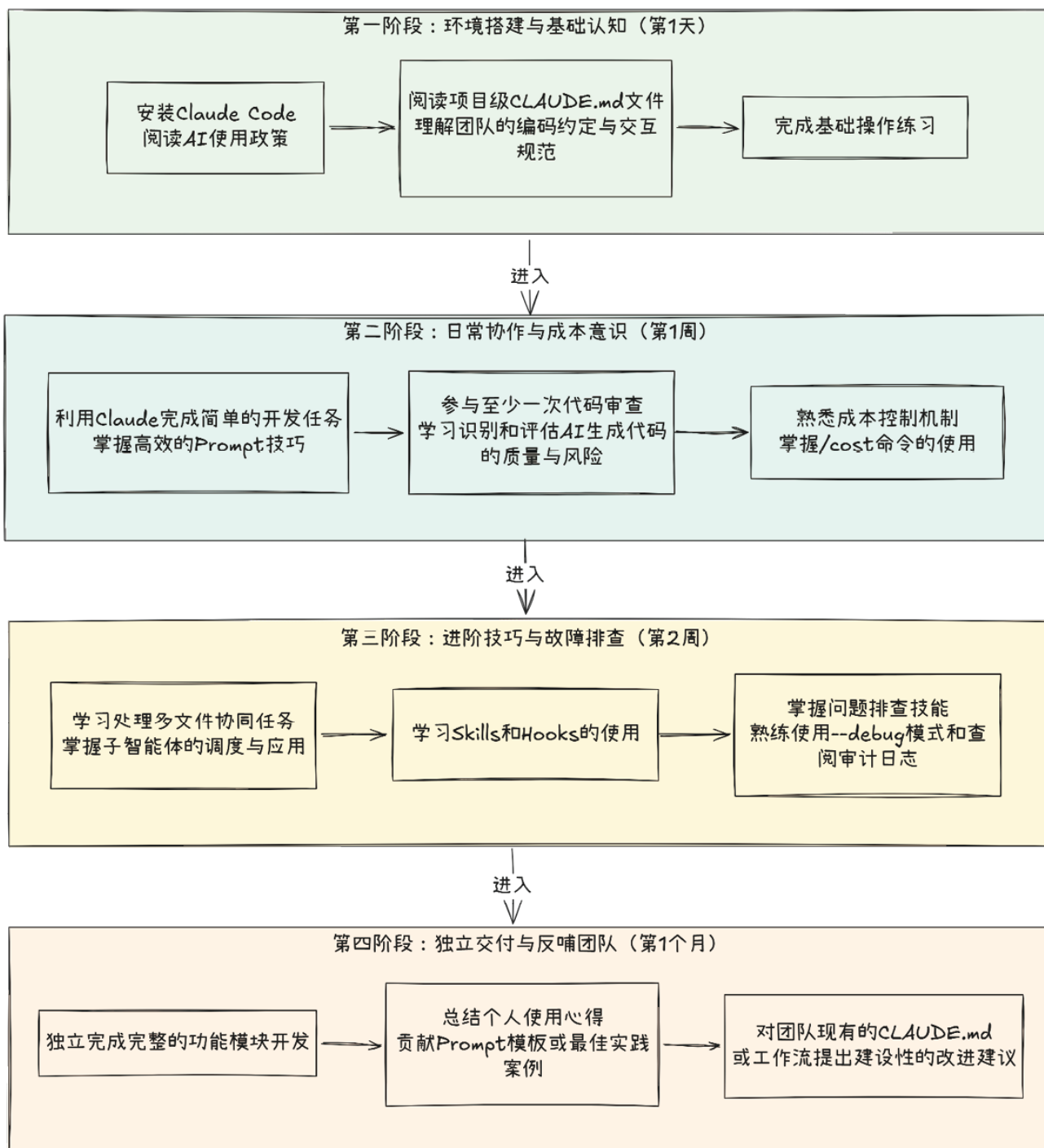


图10-3 新人Claude Code培训计划

10.7 SDD生态与工作流的融合

在深入探讨前面介绍的独立机制后，一个核心问题随之浮出水面：如何将这些分散的组件串联成一套完整的工作流？

当前，开源社区在工作流层面已呈现爆发式增长。GitHub上涌现出大量基于Claude Code的插件与框架，它们都在尝试回答同一个命题：AI时代的软件工程方法论应当是何种形态？尽管路径各异，答案却殊途同归，即SDD（Specification-Driven Development，规范驱动开发）：先定义规范，再编写代码。

10.7.1 SDD四层生态架构

这些工具被天然地划分为4个层级，每一层都致力于解决特定的核心问题。

第一层，原子技能。

以Matt Pocock开源的grill-me为典型代表。该项目仅由6行Markdown代码构成，却能将Claude从“唯命是从的执行者”转变为“穷追不舍的面试官”，迫使用户审视每一个未决事项与隐含假设。这是第3章所述Skills极致的力量体现：利用声明式的Markdown重塑AI的行为模式。

第二层，上下文持久化。

该层旨在解决长周期任务中的上下文丢失问题。Planning with Files采用了最朴素的策略：将计划写入文件（如task_plan.md、findings.md、progress.md）。在对话上下文被清空后，Claude可通过重新读取这些文件恢复进度。Trellis则更进一步，通过.trellis/目录结构实现了跨平台标准化，使得同一份规范在Claude Code、Cursor、OpenCode等十余个平台上通用。

第三层，规范生成框架。

此层提供从规范定义到代码生成的完整管线。Spec-Kit（GitHub官方项目）定义了从“宪法”到代码的全流程，其9个命令覆盖开发全生命周期，但对小项目而言略显繁重。相比之下，OpenSpec（Fission AI出品）更为轻量，仅需要4个核心命令即可一步到位；而cc-sdd则引入EARS需求语法，实现了极简化的SDD流程。

第四层，完整方法论。

该层集成了完整的工程实践体系。Superpowers（收录于Anthropic官方插件市场）打包了14个可组合的Skills，定义了7个标准阶段：头脑风暴→创建Git Worktree→制订计划→执行计划→测试驱动开发（Test-Driven Development，TDD）→代码审查→收尾合并。它强制执行TDD原则（如果用户先写代码后写测试，

系统将自动删除代码），并利用子智能体并行分发任务，这是第4章所述子智能体机制的实战应用。此外，Compound Engineering聚焦于其他框架往往忽视的“学习”维度：每次迭代后，它通过`/ce:compound`命令将经验沉淀至`docs/solutions/`目录；当下次遇到类似问题时，Agent会优先检索历史解法，实现持续进化。

10.7.2 workflows推荐组合方案

表10-5列举了3种工作流工具推荐组合方案。表10-5 workflows工具推荐组合方案

适用场景推荐插件组合核心工作流

`grill-me`+`OpenSpec`+需求拷问→规范生成→进度持久化→实现验证

个人开发者

Planning with Files（利用`grill-me`深挖需求，`OpenSpec`快速生成规范，文件记录确保长任务不丢失上下文）

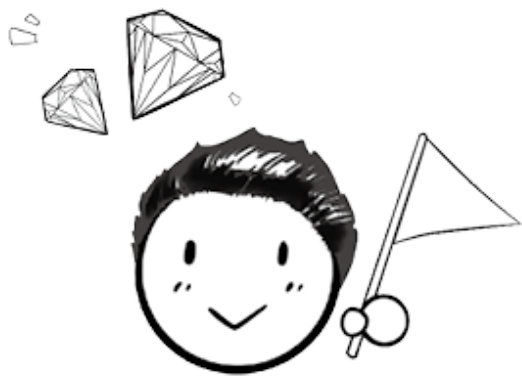
规范统一→全流程接管

小团队协作`Superpowers`+`Trellis`（`Trellis`确保跨平台规范一致性，`Superpowers`强制执行标准化开发流程）

规范执行+知识复利

长期演进项目`OpenSpec`+`Compound Engineering`+`grill-me`（在严格规范驱动下，通过`Compound Engineering`沉淀历史解法，实现团队能力的持续迭代）

不推荐的组合：`Superpowers`+`Spec-Kit`，两者均为重型框架，阶段定义存在冲突，混用会导致流程混乱；全量安装，试图安装所有工具不仅无法提升效率，反而会增加认知负担，这违背了引入工具以简化工作的初衷。



咖哥 发言

SDD的核心不在于依赖何种插件，而在于确立“先思后行”的开发习惯。如果能在启动Claude Code之前，仅用5 min厘清“目标范畴（做什么、不做什么）”与“验证标准”，即使不安装任何插件，其成效也已超越90%的凭感觉编程（如Vibe Coding）。插件的本质，仅是将这一关键习惯自动化而已。

10.8 全局视图：8种核心机制的协奏

至此，我们已经完整解析了Claude Code工程化的8种核心机制。为了从宏观视角把握其整体架构，我们将这些机制整合于表10-6中，以便一览全貌。表10-

6 8种核心机制总览

机制核心价值主要应用场景

CLAUDE.md项目记忆与上下文持久化全场景通用（建议始终配置）

子智能体并行执行与上下文隔离复杂多步骤任务拆解

Skills领域知识复用与 workflow 标准化高频重复的专业任务

Hooks事件驱动的安全门控与自动化质量保障与安全防护

MCP外部数据源与工具集成需要实时数据或外部API调用的场景

Headless模式CI/CD集成与自动化管道构建非交互式自动化流程

Agent SDK自定义智能体应用开发产品化开发（而非单纯工具使用）

Plugins能力打包与跨团队分发社区共享与企业级推广

没有一种机制是万能的，也没有哪种机制是多余的。CLAUDE.md构成了整个体系的基石，它定义了Claude的“工作上下文”；Skills与子智能体则是在此上下文中扩展能力的两个维度，前者深化领域知识，后者突破执行边界；Hooks充当执行过程中的质量与安全防线；MCP则是连接外部世界的桥梁；Headless模式与Agent SDK将Claude Code从单纯的交互工具升维为可编程的能力组件；Plugins最终将上述所有能力打包为可流通、可复用的知识资产。

这8种机制按需组合、协同运作，共同构建了一个完整且高效的AI辅助开发工程体系。

“那么，学完这一轮，我究竟能用Claude Code做什么？”小冰在最后提出了这个问题。

咖哥想了一会儿，答道：“你能用Claude Code做的事，与3个月前相比并无本质不同。它原本就具备阅读代码、编写代码、审查代码、生成文档以及运行测试的能力。你真正学到的，是如何将这些分散的能力[系统化地整合起来](#)。”

这一区别至关重要。工具的使用者仅在需要时调用工具，用毕即弃；而工程师则将工具深度融入工作流，使其在恰当的时机、以正确的方式、在既定的约束下自动介入。后者不仅要求对工具本身的熟练掌握，更依赖于对流程的深刻理解、对约束机制的精心设计，以及对潜在失败模式的预判。

这正是“工程化”的核心要义：[让强大却充满不确定性的工具，在秩序井然的系统中可靠地创造价值](#)。

尽管Claude Code的能力将持续进化——上下文窗口不断扩容、推理精度日益提升、新工具与API层出不穷，但工程化思维始终稳固：洞察机制原理，在恰当的层次上做出决策，以约束换取可靠性，并将个人的最佳实践转化为团队的公共资产。

小雪最后总结道：“从最初的架构全景到收尾的工程实战，这段旅程的本质，正是从‘认识工具’迈向‘驾驭工具’的过程。”

本章小结

作为全书的最后一章，本章并非单纯罗列技术清单，而是深度提炼工程方法论。

成本控制的核心在于把握3个关键维度：根据任务复杂度，灵活调度Haiku、Sonnet与Opus，实现性能与成本的最优平衡；通过子智能体隔离上下文、精简CLAUDE.md内容以及优化缓存策略，有效降低Token消耗；设置预算硬限制，利用`--max-budget-usd`参数设置不可逾越的成本红线。此外，需特别警惕Extended Thinking（扩展思考）模式，这往往是被忽视的隐性成本来源，应在必要时谨慎启用。

调试的首要原则是让执行过程透明化。为此，我们构建了多维度的可观测性体系：利用`--debug`标志获取实时日志；通过`stream-json`输出机器可读的执行轨迹；借助PostToolUse Hook生成持久化的审计记录；保留会话文件作为原始数据，为事后深度分析提供完整依据。对于常见的异常行为，其根源通常可归结为三大类：上下文信息缺失、指令逻辑冲突以及运行权限不足。

安全准则的核心在于严格遵循最小权限原则。我们构建了分层级的防御体系：提供5种权限模式，以精准匹配不同复杂度的使用场景；通过deny、allow、ask三级规则，实现对工具调用的原子级管控；利用PreToolUse Hook在操作执行前拦截潜在危险行为；借助受管设置下发不可覆盖的全局安全策略；针对MCP服务器，必须实施版本锁定与严格的代码审查。

针对**大型代码库**，核心策略在于以“引导式搜索”替代“全量阅读”。利用层次化的CLAUDE.md与`.claude/rules/`目录结构，实现规则的按需加载与场景化适配；派遣Explore子智能体在独立上下文中执行深度搜索与文件遍历；应用智能压缩技术提炼关键信息，显著延长有效工作时长。

指令的灵魂在于极致的精准度。为构建高可靠性的交互范式，需遵循以下准则：严格将CLAUDE.md控制在500行以内；将垂直领域的专业知识迁移至Skills；Prompt量化，拒绝模糊描述，提供高质量的示例，清晰界定任务的边界；复杂任务分步确认。

团队落地需要分阶段稳步推进：个人探索→项目级统一→自动化集成→组织级规模化。在此过程中，共享`.claude/`目录是协作基石，通过PR流程治理CLAUDE.md以确保规范质量，而受管设置则是守护安全的最终防线。

从今天起，你掌握的不仅是一套高效工具，更是一种全新的思维方式。

思考题₁

回想你在使用Claude Code时遇到的最大痛点，是成本失控、行为不一致、可靠性不足，还是其他挑战？结合本章介绍的工程实践，你计划优先从哪个机制入手进行

改进？

2. 若需要为一个初涉Claude Code的10人工程团队制定“首月”工程化路线图，你会优先推行哪3项举措？为何首选这3项，而非其他？
3. 本章提到“真实成本=API成本+时间成本”。在你的团队中，这一公式的平衡点何在？有哪些场景值得投入更高昂的模型费用，以换取开发者的宝贵时间？

后记 庖丁解牛，游刃有余

完稿之际，我再次审视本书的目录，竟发觉了一个有趣的巧合：本书的架构本身，便是一个关于Harness工程的隐喻（见图1）。



图1 Harness工程的隐喻

第1章和第2章是[上下文注入](#)——为读者加载“项目背景”，恰如CLAUDE.md在会话开始时为模型加载项目记忆。

第3章到第6章是[工具注册](#)——Skills、子智能体、Hooks、MCP四大组件，宛如四类工具，逐一交到读者手中。

第7章到第9章是[执行环境](#)——Headless模式、Agent SDK、Plugins，让能力从终端延伸至CI/CD、编程接口与生态分发。

第10章是[治理层](#)——成本、安全、指令、协作，确保整个系统在真实工程环境中可靠运转。

上下文、工具、执行环境、治理，这不正是Harness工程的四大组件？

这一结构并非刻意雕琢，而是自然而成。毕竟，[理解一个系统的最佳方式，便是按照系统本身的结构来组织知识](#)。

在本书的撰写过程中（2025年末至2026年初），我亲历了AI辅助开发领域一场不可逆转的范式转移：竞争的重心已从模型本身，彻底转向Harness工程。

2025年，业界关注的焦点尚在“哪个模型最强”；而步入2026年，顶尖工程师们开始追问一个全新的命题——“[如何让同一模型表现得更卓越](#)”。答案不再是指向更大的参数量或更大的上下文窗口，而是回归更精巧的Harness设计，即更精准的上下文管理、更合理的工具编排、更严苛的权限约束，以及更智能的自动压缩策略。

开源社区的爆发式增长有力地印证了这一趋势。[Superpowers](#)（4.2万星）仅凭14个Skills便定义了从头脑风暴到代码审查的完整开发方法论；[Compound Engineering](#)赋予了Agent从错误中汲取经验、自我进化的能力；[grill-me](#)仅用几行指令便重塑了AI的行为模式；[Trellis](#)则让一份规范跨在11个AI平台间实现了无缝通用。它们并未发明新的模型——它们都在致力于同一件事：设计更卓越的Harness。

而这些Harness的设计，无一不是建立在本书所阐述的基础机制之上——[CLAUDE.md](#)承载记忆，Skills注入知识，子智能体实现任务分治，Hooks充当约束守卫，MCP负责外部连接。[Superpowers](#)的七阶段工作流，正是这5种机制的精妙编排；[Compound Engineering](#)的知识沉淀闭环，则是CLAUDE.md与Skills的高级应用。

[零件是恒定的，组合是无穷的。](#)

本书赋予你的是零件，至于如何组合，则取决于你所面对的具体问题，以及你的工程判断力。

写作期间，一个概念反复在我的脑海中浮现——[Agentic Loop](#)，它宛如一个优雅的莫比乌斯环（见图2）。

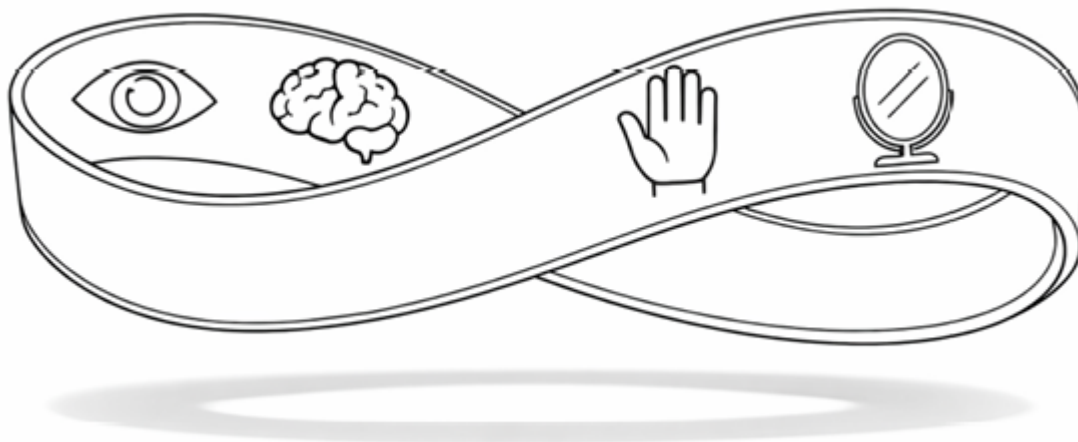


图2 一个优雅的莫比乌斯环

“推理→工具调用→结果回注→继续推理”这一循环看似简单得微不足道，实则是Claude Code一切复杂行为涌现的基石。当你提交一个bug时，它绝非简单地“看一眼便猜测答案”，而是经历了一个反复观察、假设与验证的过程：先查阅报错日志，继而搜索相关代码，深入理解上下文，最后才动手修复。这种复杂的行为模式并非被硬编码在程序中，而是由循环结构涌现出来的智能表现。

我始终认为，这一循环蕴含着一种哲学层面的美感。它揭示的其实是：**智能并非一次性的灵光乍现，而是持续的试探与修正**。模型在每一轮循环中都能回溯上一轮的结果，因此它自然会做出“再看看”“再试试”的决策。这与人类解决问题的方式一模一样——只不过人类的循环节奏更为迟缓，且极易在中途被外力打断。

作为工程师，你能做的，是为这个循环设计**约束**：它能感知什么（工具白名单）、它能记住什么（CLAUDE.md+上下文管理）、它能做什么（权限系统）、它何时应当停止（max-turns+Hooks）。你不需要干预循环内部的决策，那是模型的职责所在。你掌控的，是循环的边界条件。

这正是Harness工程的本质：并非编程行为，而是约束行为。

如果你在读完本书后仍感意犹未尽，渴望更深入地探索“规范驱动”的方法论，推荐你关注我即将出版的《SDD实战：规范驱动开发》。该书系统阐述了SDD的完整方法论体系：从需求分析到架构设计，从任务拆解到验证迭代，从Agent设计模式到团队协作。如果说这本书旨在教你如何**调教Harness**，那么《SDD实战：规范驱动开发》则专注于教你如何**定义规范**——两者相辅相成，共同勾勒出AI时代软件工程的完整图景。

倘若你对Agent设计模式的理论框架感兴趣，希望从更高的抽象维度理解感知、记忆、推理、行动、治理等认知模块，那么《Agent设计模式 图解可复用智能体架构》将是你的不二之选。该书以框架形式系统梳理了21个Agent设计模式，是目前该领域较为完备的架构参考。

这3本书分别锚定了3个维度：[Harness工程](#)、[规范方法论](#)、[设计模式理论](#)。自下而上，是构建的视角，教你如何落地；自上而下，是设计的视角，教你如何抽象。

庖丁解牛，游刃有余。

但庖丁并非只会用刀——他洞悉牛的骨骼，他知道刀该从何处切入、何处抽离，他能感知筋膜之间那一丝微不可察的空隙。

Claude Code是刀，你的项目是牛，而本书要教你的，是骨骼的走向、空隙的位置。

现在，该你动手了。

黄佳

2026年初夏

配套资源验证码260928

Copyright Copyright 2010, 2012 Adobe Systems Incorporated
 (<http://www.adobe.com/>), with Reserved Font Name ‘Source’ .
 License This Font Software is licensed under the SIL Open Font
 License, Version 1.1. This license is copied below, and is also
 available with a FAQ at: <http://scripts.sil.org/OFL> SIL OPEN FONT
 LICENSE Version 1.1 – 26 February 2007 PREAMBLE The goals of the
 Open Font License (OFL) are to stimulate worldwide development of
 collaborative font projects, to support the font creation efforts
 of academic and linguistic communities, and to provide a free and
 open framework in which fonts may be shared and improved in
 partnership with others. The OFL allows the licensed fonts to be
 used, studied, modified and redistributed freely as long as they
 are not sold by themselves. The fonts, including any derivative
 works, can be bundled, embedded, redistributed and/or sold with
 any software provided that any reserved names are not used by
 derivative works. The fonts and derivatives, however, cannot be
 released under any other type of license. The requirement for
 fonts to remain under this license does not apply to any document
 created using the fonts or their derivatives. DEFINITIONS “Font
 Software” refers to the set of files released by the Copyright
 Holder(s) under this license and clearly marked as such. This may
 include source files, build scripts and documentation. “Reserved
 Font Name” refers to any names specified as such after the
 copyright statement(s). “Original Version” refers to the
 collection of Font Software components as distributed by the
 Copyright Holder(s). “Modified Version” refers to any derivative
 made by adding to, deleting, or substituting “â€” in part or in
 whole “â€” any of the components of the Original Version, by
 changing formats or by porting the Font Software to a new
 environment. “Author” refers to any designer, engineer,
 programmer, technical writer or other person who contributed to
 the Font Software. PERMISSION & CONDITIONS Permission is hereby
 granted, free of charge, to any person obtaining a copy of the
 Font Software, to use, study, copy, merge, embed, modify,
 redistribute, and sell modified and unmodified copies of the Font
 Software, subject to the following conditions: 1) Neither the Font
 Software nor any of its individual components, in Original or

Modified Versions, may be sold by itself. 2) Original or Modified Versions of the Font Software may be bundled, redistributed and/or sold with any software, provided that each copy contains the above copyright notice and this license. These can be included either as stand-alone text files, human-readable headers or in the appropriate machine-readable metadata fields within text or binary files as long as those fields can be easily viewed by the user. 3) No Modified Version of the Font Software may use the Reserved Font Name(s) unless explicit written permission is granted by the corresponding Copyright Holder. This restriction only applies to the primary font name as presented to the users. 4) The name(s) of the Copyright Holder(s) or the Author(s) of the Font Software shall not be used to promote, endorse or advertise any Modified Version, except to acknowledge the contribution(s) of the Copyright Holder(s) and the Author(s) or with their explicit written permission. 5) The Font Software, modified or unmodified, in part or in whole, must be distributed entirely under this license, and must not be distributed under any other license. The requirement for fonts to remain under this license does not apply to any document created using the Font Software. TERMINATION This license becomes null and void if any of the above conditions are not met. DISCLAIMER THE FONT SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO ANY WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT OF COPYRIGHT, PATENT, TRADEMARK, OR OTHER RIGHT. IN NO EVENT SHALL THE COPYRIGHT HOLDER BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, INCLUDING ANY GENERAL, SPECIAL, INDIRECT, INCIDENTAL, OR CONSEQUENTIAL DAMAGES, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF THE USE OR INABILITY TO USE THE FONT SOFTWARE OR FROM OTHER DEALINGS IN THE FONT SOFTWARE.

*文中代码字体版权说明 